pandas

Knowledge-Base Reference

Version 3.1.0

DataFrames and Series, selection, missing data, group-by, merging, reshaping, time series and Copy-on-Write, plus an API quick reference for everyday functions and methods.

Source: official documentation, https://pandas.pydata.org/docs/
Content © 2008-2026 pandas Development Team and contributors (BSD-3-Clause)
Compiled October 29, 2026

Contents

About this reference **4**

10 minutes to pandas **5**
- Basic data structures in pandas . . . 5
- Object creation . . . 5
- Viewing data . . . 6
- Selection . . . 7
- Missing data . . . 9
- Operations . . . 9
- Merge . . . 10
- Grouping . . . 11
- Reshaping . . . 12
- Time series . . . 12
- Categoricals . . . 13
- Plotting . . . 13
- Importing and exporting data . . . 14
- Gotchas . . . 15

Copy-on-Write and migration guide **16**
- Copy-on-Write (CoW) . . . 16
- The new string data type . . . 21

Indexing and selecting data **28**
- Different choices for indexing . . . 28
- Basics . . . 29
- Attribute access . . . 31
- Slicing ranges . . . 31
- Selection by label . . . 32
- Selection by position . . . 34
- Selection by callable . . . 35
- Combining positional and label-based indexing . . . 36
- Selecting random samples . . . 37
- Setting with enlargement . . . 38
- Fast scalar value getting and setting . . . 38
- Boolean indexing . . . 39
- Indexing with isin . . . 40
- The `where` Method and Masking . . . 41
- Setting with enlargement conditionally using `numpy` . . . 43
- The `query` Method . . . 43
- Duplicate data . . . 47
- Dictionary-like `get` method . . . 48
- Looking up values by index/column labels . . . 48
- Index objects . . . 49
- Set / reset index . . . 50
- Series Assignment and Index Alignment . . . 52
- Column assignment vs. in-place setting . . . 52

Working with missing data **54**
- Values considered ”missing” . . . 54

NA semantics . . . 55
Inserting missing data . . . 57
Calculations with missing data . . . 58
Dropping missing data . . . 58
Filling missing data . . . 59

Group by: split-apply-combine **64**
Splitting an object into groups . . . 65
Iterating through groups . . . 69
Selecting a group . . . 70
Aggregation . . . 70
Transformation . . . 74
Filtration . . . 77
Flexible apply . . . 79
Numba accelerated routines . . . 80
Other useful features . . . 80
Examples . . . 85

Merge, join, concatenate and compare **87**
concat . . . 87
merge . . . 90
DataFrame.join . . . 93
merge_ordered . . . 96
merge_asof . . . 96
compare . . . 97

Reshaping and pivot tables **99**
pivot and pivot_table . . . 99
stack and unstack . . . 102
melt and wide_to_long . . . 106
get_dummies and from_dummies . . . 107
explode . . . 108
crosstab . . . 109
cut . . . 110
factorize . . . 110

Time series / date functionality **111**
Overview . . . 111
Timestamps vs. time spans . . . 112
Converting to timestamps . . . 113
Generating ranges of timestamps . . . 116
Timestamp limitations . . . 117
Indexing . . . 118
Time/date components . . . 122
DateOffset objects . . . 123
Time Series-related instance methods . . . 133
Resampling . . . 134
Time span representation . . . 139
Converting between representations . . . 142
Representing out-of-bounds spans . . . 143
Time zone handling . . . 143

API quick reference **148**
Input / output . . . 148
Core objects . . . 168
Inspecting and selecting . . . 170

Cleaning 179
Transforming 192
Grouping and combining 201
Time series and windows 212

About this reference

This knowledge-base reference collects core chapters of the official **pandas 3.1.0** documentation (https://pandas.pydata.org/docs/).

- **Chapters 2 onward** are converted from the project's own documentation source files (the reStructuredText files the website is built from), taken from the `v3.1.0` release tag of github.com/pandas-dev/pandas. Text and code are the project's; Sphinx-only elements such as cross-reference links, rendered plot output and interactive widgets were simplified for print.
- **The API quick reference** at the end is generated from the docstrings of the installed 3.1.0 package, the same text the official API pages are built from. Each entry is trimmed to signature, parameters, returns, notes and a short excerpt of the notes and examples.

Note on examples. Most pandas user-guide examples are run when the website is built, and their output is generated then. This PDF shows the example code; the generated output tables are not included. The API quick reference examples do include output.

Built October 29, 2026 for a personal knowledge base. Selection and assembly by Claude; no content was rewritten.

License. pandas and its documentation are distributed under the BSD 3-Clause License. Copyright (c) 2008-2011, AQR Capital Management, LLC, Lambda Foundry, Inc. and PyData Development Team; Copyright (c) 2011-2026, Open source contributors. Redistribution with this notice is permitted. Full text: https://github.com/pandas-dev/pandas/blob/main/LICENSE

Chapter sources

- 10 minutes to pandas: https://pandas.pydata.org/docs/user_guide/10min.html
- Copy-on-Write and migration guide: https://pandas.pydata.org/docs/user_guide/migration.html
- Indexing and selecting data: https://pandas.pydata.org/docs/user_guide/indexing.html
- Working with missing data: https://pandas.pydata.org/docs/user_guide/missing_data.html
- Group by: split-apply-combine: https://pandas.pydata.org/docs/user_guide/groupby.html
- Merge, join, concatenate and compare: https://pandas.pydata.org/docs/user_guide/merging.html
- Reshaping and pivot tables: https://pandas.pydata.org/docs/user_guide/reshaping.html
- Time series / date functionality: https://pandas.pydata.org/docs/user_guide/timeseries.html

10 minutes to pandas

Source: https://pandas.pydata.org/docs/user_guide/10min.html

This is a short introduction to pandas, geared mainly for new users. You can see more complex recipes in the Cookbook.

Customarily, we import as follows:

```
import numpy as np
import pandas as pd
```

Basic data structures in pandas

pandas provides two types of classes for handling data:

1. **`Series`: a one-dimensional labeled array holding data of any type** such as integers, strings, Python objects etc.
2. `DataFrame`: a two-dimensional data structure that holds data like a two-dimension array or a table with rows and columns.

Object creation

See the Intro to data structures section.

Creating a `Series` by passing a list of values, letting pandas create a default `RangeIndex`.

```
s = pd.Series([1, 3, 5, np.nan, 6, 8])
s
```

Creating a `DataFrame` by passing a NumPy array with a datetime index using `date_range` and labeled columns:

```
dates = pd.date_range("20130101", periods=6)
dates
df = pd.DataFrame(np.random.randn(6, 4), index=dates, columns=list("ABCD"))
df
```

Creating a `DataFrame` by passing a dictionary of objects where the keys are the column labels and the values are the column values.

```
df2 = pd.DataFrame(
    {
        "A": 1.0,
        "B": pd.Timestamp("20130102"),
        "C": pd.Series(1, index=list(range(4)), dtype="float32"),
        "D": np.array([3] * 4, dtype="int32"),
        "E": pd.Categorical(["test", "train", "test", "train"]),
        "F": "foo",
    }
)
df2
```

The columns of the resulting `DataFrame` have different dtypes:

```
df2.dtypes
```

If you're using IPython, tab completion for column names (as well as public attributes) is automatically enabled. Here's a subset of the attributes that will be completed:

```
@verbatim
In [1]: df2.<TAB>  # noqa: E225, E999
df2.A                  df2.bool
df2.abs                df2.boxplot
df2.add                df2.C
df2.add_prefix         df2.clip
df2.add_suffix         df2.columns
df2.align              df2.copy
df2.all                df2.count
df2.any                df2.combine
df2.apply              df2.D
df2.B                  df2.describe
df2.diff               df2.duplicated
```

As you can see, the columns `A`, `B`, `C`, and `D` are automatically tab completed. `E` and `F` are there as well; the rest of the attributes have been truncated for brevity.

Viewing data

See the Essential basic functionality section.

Use `DataFrame.head` and `DataFrame.tail` to view the top and bottom rows of the frame respectively:

```
df.head()
df.tail(3)
```

Display the `DataFrame.index` or `DataFrame.columns`:

```
df.index
df.columns
```

Return a NumPy representation of the underlying data with `DataFrame.to_numpy` without the index or column labels:

```
df.to_numpy()
```

> Note
> **NumPy arrays have one dtype for the entire array while pandas DataFrames have one dtype per column**. When you call `DataFrame.to_numpy`, pandas will find the NumPy dtype that can hold *all* of the dtypes in the DataFrame. If the common data type is `object`, `DataFrame.to_numpy` will require copying data.
>
> ```
> df2.dtypes
> df2.to_numpy()
> ```

`describe` shows a quick statistic summary of your data:

```
df.describe()
```

Transposing your data:

```
df.T
```

`DataFrame.sort_index` sorts by an axis:

```
df.sort_index(axis=1, ascending=False)
```

`DataFrame.sort_values` sorts by values:

```
df.sort_values(by="B")
```

Selection

> Note
> While standard Python / NumPy expressions for selecting and setting are intuitive and come in handy for interactive work, for production code, we recommend the optimized pandas data access methods, `DataFrame.at`, `DataFrame.iat`,

See the indexing documentation Indexing and Selecting Data and MultiIndex / Advanced Indexing.

Getitem (`[]`)

For a `DataFrame`, passing a single label selects a column and yields a `Series`:

```
df["A"]
```

If the label only contains letters, numbers, and underscores, you can alternatively use the column name attribute:

```
df.A
```

Passing a list of column labels selects multiple columns, which can be useful for getting a subset/rearranging:

```
df[["B", "A"]]
```

For a `DataFrame`, passing a slice `:` selects matching rows:

```
df[0:3]
df["20130102":"20130104"]
```

Selection by label

See more in Selection by Label using `DataFrame.loc` or `DataFrame.at`.

Selecting a row matching a label:

```
df.loc[dates[0]]
```

Selecting all rows (`:`) with a select column labels:

```
df.loc[:, ["A", "B"]]
```

For label slicing, both endpoints are *included*:

```
df.loc["20130102":"20130104", ["A", "B"]]
```

Selecting a single row and column label returns a scalar:

```
df.loc[dates[0], "A"]
```

For getting fast access to a scalar (equivalent to the prior method):

```
df.at[dates[0], "A"]
```

Selection by position

See more in Selection by Position using `DataFrame.iloc` or `DataFrame.iat`.

Select via the position of the passed integers:

```
df.iloc[3]
```

Integer slices act similar to NumPy/Python:

```
df.iloc[3:5, 0:2]
```

Lists of integer position locations:

```
df.iloc[[1, 2, 4], [0, 2]]
```

For slicing rows explicitly:

```
df.iloc[1:3, :]
```

For slicing columns explicitly:

```
df.iloc[:, 1:3]
```

For getting a value explicitly:

```
df.iloc[1, 1]
```

For getting fast access to a scalar (equivalent to the prior method):

```
df.iat[1, 1]
```

Boolean indexing

Select rows where `df.A` is greater than `0`.

```
df[df["A"] > 0]
```

Selecting values from a `DataFrame` where a boolean condition is met:

```
df[df > 0]
```

Using `isin` method for filtering:

```
df2 = df.copy()
df2["E"] = ["one", "one", "two", "three", "four", "three"]
df2
df2[df2["E"].isin(["two", "four"])]
```

Setting

Setting a new column automatically aligns the data by the indexes:

```
s1 = pd.Series(
   [1, 2, 3, 4, 5, 6],
   index=pd.date_range("20130102", periods=6))
```

```
s1
df["F"] = s1
```

Setting values by label:

```
df.at[dates[0], "A"] = 0
```

Setting values by position:

```
df.iat[0, 1] = 0
```

Setting by assigning with a NumPy array:

```
df.loc[:, "D"] = np.array([5] * len(df))
```

The result of the prior setting operations:

```
df
```

A `where` operation with setting:

```
df2 = df.copy()
df2[df2 > 0] = -df2
df2
```

Missing data

For NumPy data types, `np.nan` represents missing data. It is by default not included in computations. See the `Missing Data section <missing_data>`.

Reindexing allows you to change/add/delete the index on a specified axis. This returns a copy of the data:

```
df1 = df.reindex(index=dates[0:4], columns=list(df.columns) + ["E"])
df1.loc[dates[0] : dates[1], "E"] = 1
df1
```

`DataFrame.dropna` drops any rows that have missing data:

```
df1.dropna(how="any")
```

`DataFrame.fillna` fills missing data:

```
df1.fillna(value=5)
```

`isna` gets the boolean mask where values are `nan`:

```
pd.isna(df1)
```

Operations

See the Basic section on Binary Ops.

Stats

Operations in general *exclude* missing data.

Calculate the mean value for each column:

```
df.mean()
```

Calculate the mean value for each row:

```
df.mean(axis=1)
```

Operating with another `Series` or `DataFrame` with a different index or column will align the result with the union of the index or column labels. In addition, pandas automatically broadcasts along the specified dimension and will fill unaligned labels with `np.nan`.

```
s = pd.Series([1, 3, 5, np.nan, 6, 8], index=dates).shift(2)
s
df.sub(s, axis="index")
```

User defined functions

`DataFrame.agg` and `DataFrame.transform` applies a user defined function that reduces or broadcasts its result respectively.

```
df.agg(lambda x: np.mean(x) * 5.6)
df.transform(lambda x: x * 101.2)
```

Value Counts

See more at Histogramming and Discretization.

```
s = pd.Series(np.random.randint(0, 7, size=10))
s
s.value_counts()
```

String Methods

`Series` is equipped with a set of string processing methods in the `str` attribute that make it easy to operate on each element of the array, as in the code snippet below. See more at `Vectorized String Methods <text.string_methods>`.

```
s = pd.Series(["A", "B", "C", "Aaba", "Baca", np.nan, "CABA", "dog", "cat"])
s.str.lower()
```

Merge

Concat

pandas provides various facilities for easily combining together `Series` and `DataFrame` objects with various kinds of set logic for the indexes and relational algebra functionality in the case of join / merge-type operations.

See the Merging section.

Concatenating pandas objects together row-wise with `concat`:

```
df = pd.DataFrame(np.random.randn(10, 4))
df

# break it into pieces
pieces = [df[:3], df[3:7], df[7:]]
```

```
pd.concat(pieces)
```

> Note
> Adding a column to a `DataFrame` is relatively fast. However, adding a row requires a copy, and may be expensive. We recommend passing a pre-built list of records to the `DataFrame` constructor instead of building a `DataFrame` by iteratively appending records to it.

Join

`merge` enables SQL style join types along specific columns. See the Database style joining section.

```
left = pd.DataFrame({"key": ["foo", "foo"], "lval": [1, 2]})
right = pd.DataFrame({"key": ["foo", "foo"], "rval": [4, 5]})
left
right
pd.merge(left, right, on="key")
```

`merge` on unique keys:

```
left = pd.DataFrame({"key": ["foo", "bar"], "lval": [1, 2]})
right = pd.DataFrame({"key": ["foo", "bar"], "rval": [4, 5]})
left
right
pd.merge(left, right, on="key")
```

Grouping

By ”group by” we are referring to a process involving one or more of the following steps:

- **Splitting** the data into groups based on some criteria
- **Applying** a function to each group independently
- **Combining** the results into a data structure

See the Grouping section.

```
df = pd.DataFrame(
    {
        "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],
        "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
        "C": np.random.randn(8),
        "D": np.random.randn(8),
    }
)
df
```

Grouping by a column label, selecting column labels, and then applying the `.DataFrameGroupBy.sum` function to the resulting groups:

```
df.groupby("A")[["C", "D"]].sum()
```

The unique values of the grouped column now form the index of the result.

Grouping by multiple columns label forms `MultiIndex`.

```
df.groupby(["A", "B"]).sum()
```

Reshaping

See the sections on Hierarchical Indexing and Reshaping.

Stack

```
arrays = [
   ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
   ["one", "two", "one", "two", "one", "two", "one", "two"],
]
index = pd.MultiIndex.from_arrays(arrays, names=["first", "second"])
df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=["A", "B"])
df2 = df[:4]
df2
```

The `stack` method "compresses" a level in the DataFrame's columns:

```
stacked = df2.stack()
stacked
```

With a "stacked" DataFrame or Series (having a `MultiIndex` as the `index`), the inverse operation of `stack` is `unstack`, which by default unstacks the **last level**:

```
stacked.unstack()
stacked.unstack(1)
stacked.unstack(0)
```

Pivot tables

See the section on Pivot Tables.

```
df = pd.DataFrame(
    {
        "A": ["one", "one", "two", "three"] * 3,
        "B": ["A", "B", "C"] * 4,
        "C": ["foo", "foo", "foo", "bar", "bar", "bar"] * 2,
        "D": np.random.randn(12),
        "E": np.random.randn(12),
    }
)
df
```

`pivot_table` pivots a `DataFrame` specifying the `values`, `index` and `columns`

```
pd.pivot_table(df, values="D", index=["A", "B"], columns=["C"])
```

Time series

pandas has simple, powerful, and efficient functionality for performing resampling operations during frequency conversion (e.g., converting secondly data into 5-minutely data). This is extremely common in, but not limited to, financial applications. See the Time Series section.

```
rng = pd.date_range("1/1/2012", periods=100, freq="s")
ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)
ts.resample("5Min").sum()
```

`Series.tz_localize` localizes a time series to a time zone:

```
rng = pd.date_range("3/6/2012 00:00", periods=5, freq="D")
ts = pd.Series(np.random.randn(len(rng)), rng)
ts
ts_utc = ts.tz_localize("UTC")
ts_utc
```

`Series.tz_convert` converts a timezone aware time series to another time zone:

```
ts_utc.tz_convert("US/Eastern")
```

Adding a non-fixed duration (`BusinessDay`) to a time series:

```
rng
rng + pd.offsets.BusinessDay(5)
```

Categoricals

pandas can include categorical data in a `DataFrame`. For full docs, see the categorical introduction and the API documentation.

```
df = pd.DataFrame(
    {"id": [1, 2, 3, 4, 5, 6], "raw_grade": ["a", "b", "b", "a", "a", "e"]}
)
```

Converting the raw grades to a categorical data type:

```
df["grade"] = df["raw_grade"].astype("category")
df["grade"]
```

Rename the categories to more meaningful names:

```
new_categories = ["very good", "good", "very bad"]
df["grade"] = df["grade"].cat.rename_categories(new_categories)
```

Reorder the categories and simultaneously add the missing categories (methods under `Series.cat` return a new `Series` by default):

```
df["grade"] = df["grade"].cat.set_categories(
    ["very bad", "bad", "medium", "good", "very good"]
)
df["grade"]
```

Sorting is per order in the categories, not lexical order:

```
df.sort_values(by="grade")
```

Grouping by a categorical column with `observed=False` also shows empty categories:

```
df.groupby("grade", observed=False).size()
```

Plotting

See the Plotting docs.

We use the standard convention for referencing the matplotlib API:

```
import matplotlib.pyplot as plt

plt.close("all")
```

The `plt.close` method is used to close a figure window:

```
ts = pd.Series(np.random.randn(1000), index=pd.date_range("1/1/2000", periods=1000))
ts = ts.cumsum()

@savefig series_plot.png
ts.plot();
```

> Note
> When using Jupyter, the plot will appear using `plot`. Otherwise use matplotlib.pyplot.show to show it or matplotlib.pyplot.savefig to write it to a file.

`plot` plots all columns:

```
df = pd.DataFrame(
    np.random.randn(1000, 4), index=ts.index, columns=["A", "B", "C", "D"]
)

df = df.cumsum()

plt.figure();
df.plot();
@savefig frame_plot.png
plt.legend(loc='best');
```

Importing and exporting data

See the IO Tools section.

CSV

Writing to a csv file: using `DataFrame.to_csv`

```
df = pd.DataFrame(np.random.randint(0, 5, (10, 5)))
df.to_csv("foo.csv")
```

Reading from a csv file: using `read_csv`

```
pd.read_csv("foo.csv")
```

Parquet

Writing to a Parquet file:

```
df.to_parquet("foo.parquet")
```

Reading from a Parquet file Store using `read_parquet`:

```
pd.read_parquet("foo.parquet")
```

Excel

Reading and writing to Excel.

Writing to an excel file using `DataFrame.to_excel`:

```
df.to_excel("foo.xlsx", sheet_name="Sheet1")
```

Reading from an excel file using `read_excel`:

```
pd.read_excel("foo.xlsx", "Sheet1", index_col=None, na_values=["NA"], engine="openpyxl")
```

Gotchas

If you are attempting to perform a boolean operation on a `Series` or `DataFrame` you might see an exception like:

```
if pd.Series([False, True, False]):
    print("I was true")
```

See Comparisons and Gotchas for an explanation and what to do.

Copy-on-Write and migration guide

Source: https://pandas.pydata.org/docs/user_guide/migration.html

For large changes that are difficult or impossible to deprecate in a user-friendly manner, pandas will implement the changes under the `future` configuration. This section goes into detail for each of these changes.

Copy-on-Write (CoW)

> Note
> Copy-on-Write is now the default with pandas 3.0.

Copy-on-Write was first introduced in version 1.5.0. Starting from version 2.0 most of the optimizations that become possible through CoW are implemented and supported. All possible optimizations are supported starting from pandas 2.1.

CoW will lead to more predictable behavior since it is not possible to update more than one object with one statement, e.g. indexing operations or methods won't have side-effects. Additionally, through delaying copies as long as possible, the average performance and memory usage will improve.

Previous behavior

pandas indexing behavior is tricky to understand. Some operations return views while other return copies. Depending on the result of the operation, mutating one object might accidentally mutate another:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: subset = df["foo"]
In [3]: subset.iloc[0] = 100
In [4]: df
Out[4]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

Mutating `subset`, e.g. updating its values, also updated `df`. The exact behavior was hard to predict. Copy-on-Write solves accidentally modifying more than one object, it explicitly disallows this. `df` is unchanged:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
subset = df["foo"]
subset.iloc[0] = 100
df
```

The following sections will explain what this means and how it impacts existing applications.

Migrating to Copy-on-Write

Copy-on-Write is the default and only mode in pandas 3.0. This means that users need to migrate their code to be compliant with CoW rules.

Note
In pandas 2.x, a warning mode was available via `pd.options.mode.copy_on_write = "warn"`. This option is deprecated in pandas 3.0 and has no effect, as Copy-on-Write is now always enabled.

The following items describe the user visible changes:

Chained assignment will never work

`loc` should be used as an alternative. Check the chained assignment section for more details.

Accessing the underlying array of a pandas object will return a read-only view

```
ser = pd.Series([1, 2, 3])
ser.to_numpy()
```

This example returns a NumPy array that is a view of the Series object. This view can be modified and thus also modify the pandas object. This is not compliant with CoW rules. The returned array is set to non-writeable to protect against this behavior. Creating a copy of this array allows modification. You can also make the array writeable again if you don't care about the pandas object anymore.

See the section about read-only NumPy arrays for more details.

Only one pandas object is updated at once

The following code snippet updated both `df` and `subset` without CoW:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: subset = df["foo"]
In [3]: subset.iloc[0] = 100
In [4]: df
Out[4]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

This is not possible anymore with CoW, since the CoW rules explicitly forbid this. This includes updating a single column as a `Series` and relying on the change propagating back to the parent `DataFrame`. See the chained assignment section on how to perform these kinds of operations.

Updating a column selected from a `DataFrame` with an inplace method will also not work anymore.

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df["foo"].replace(1, 5, inplace=True)
df
```

This is another form of chained assignment. This can generally be rewritten in 2 different forms:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df.replace({"foo": {1: 5}}, inplace=True)
df
```

A different alternative would be to not use `inplace`:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df["foo"] = df["foo"].replace(1, 5)
df
```

Attributes set on a selected column do not persist

Selecting a column now returns a new `Series` object each time rather than a cached one, so setting `Series.attrs` on a selected column (for example `df["foo"].attrs[...] = ...`) is no longer

visible on subsequent selections of that column. Store such metadata on the parent `DataFrame`'s `attrs` instead.

Constructors now copy NumPy arrays by default

The Series and DataFrame constructors now copies a NumPy array by default when not otherwise specified. This was changed to avoid mutating a pandas object when the NumPy array is changed inplace outside of pandas. You can set `copy=False` to avoid this copy.

Description

CoW means that any DataFrame or Series derived from another in any way always behaves as a copy. As a consequence, we can only change the values of an object through modifying the object itself. CoW disallows updating a DataFrame or a Series that shares data with another DataFrame or Series object inplace.

This avoids side-effects when modifying values and hence, most methods can avoid actually copying the data and only trigger a copy when necessary.

The following example will operate inplace:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df.iloc[0, 0] = 100
df
```

The object `df` does not share any data with any other object and hence no copy is triggered when updating the values. In contrast, the following operation triggers a copy of the data under CoW:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df2 = df.reset_index(drop=True)
df2.iloc[0, 0] = 100

df
df2
```

`reset_index` returns a lazy copy with CoW while it copies the data without CoW. Since both objects, `df` and `df2` share the same data, a copy is triggered when modifying `df2`. The object `df` still has the same values as initially while `df2` was modified.

If the object `df` isn't needed anymore after performing the `reset_index` operation, you can emulate an inplace-like operation through assigning the output of `reset_index` to the same variable:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df = df.reset_index(drop=True)
df.iloc[0, 0] = 100
df
```

The initial object gets out of scope as soon as the result of `reset_index` is reassigned and hence `df` does not share data with any other object. No copy is necessary when modifying the object. This is generally true for all methods listed in Copy-on-Write optimizations.

Previously, when operating on views, the view and the parent object was modified:

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: subset = df["foo"]
In [3]: subset.iloc[0] = 100
In [4]: df
Out[4]:
   foo  bar
0  100    4
1    2    5
2    3    6
```

CoW triggers a copy when `df` is changed to avoid mutating `view` as well:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
view = df[:]
df.iloc[0, 0] = 100

df
view
```

Chained Assignment

Chained assignment references a technique where an object is updated through two subsequent indexing operations, e.g.

```
In [1]: df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
In [2]: df["foo"][df["bar"] > 5] = 100
In [3]: df
Out[3]:
   foo  bar
0    1    4
1    2    5
2  100    6
```

The column `foo` was updated where the column `bar` is greater than 5. This violated the CoW principles though, because it would have to modify the view `df["foo"]` and `df` in one step. Hence, chained assignment will consistently never work and raise a `ChainedAssignmentError` warning with CoW enabled:

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df["foo"][df["bar"] > 5] = 100
```

With copy on write this can be done by using `loc`.

```
df.loc[df["bar"] > 5, "foo"] = 100
```

When the column is given by name (`loc`) and the row by position (`iloc`), you either need to convert the column name to its position using `Index.get_loc` or you need to convert the row position to its index. Both variants are shown in the following example.

```
df = pd.DataFrame({"foo": [1, 2, 3], "bar": [4, 5, 6]})
df.iloc[0, df.columns.get_loc("foo")] = 100
df.loc[df.index[1], "bar"] = 200
df
```

The `iloc` variant works as a direct replacement of the chained assignment `df["foo"].iloc[0] = 100` while the `loc` variant first translates the position to the index and then finds all positions with that index. It does more work and is only equivalent if the DataFrame has a unique row index.

Note that many such statements in the code can potentially hurt performance. If possible, prefer to update the whole column at once. If you have a boolean mask, `DataFrame.where` is another suitable alternative for this case.

Read-only NumPy arrays

Accessing the underlying NumPy array of a DataFrame will return a read-only array if the array shares data with the initial DataFrame:

The array is a copy if the initial DataFrame consists of more than one array:

```
df = pd.DataFrame({"a": [1, 2], "b": [1.5, 2.5]})
df.to_numpy()
```

The array shares data with the DataFrame if the DataFrame consists of only one NumPy array:

```
df = pd.DataFrame({"a": [1, 2], "b": [3, 4]})
df.to_numpy()
```

This array is read-only, which means that it can't be modified inplace:

```
arr = df.to_numpy()
arr[0, 0] = 100
```

The same holds true for a Series, since a Series always consists of a single array.

There are two potential solutions to this:

- Trigger a copy manually if you want to avoid updating DataFrames that share memory with your array.
- Make the array writeable. This is a more performant solution but circumvents Copy-on-Write rules, so it should be used with caution.

```
arr = df.to_numpy()
arr.flags.writeable = True
arr[0, 0] = 100
arr
```

Patterns to avoid

No defensive copy will be performed if two objects share the same data while you are modifying one object inplace.

```
df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df2 = df.reset_index(drop=True)
df2.iloc[0, 0] = 100
```

This creates two objects that share data and thus the setitem operation will trigger a copy. This is not necessary if the initial object `df` isn't needed anymore. Simply reassigning to the same variable will invalidate the reference that is held by the object.

```
df = pd.DataFrame({"a": [1, 2, 3], "b": [4, 5, 6]})
df = df.reset_index(drop=True)
df.iloc[0, 0] = 100
```

No copy is necessary in this example. Creating multiple references keeps unnecessary references alive and thus will hurt performance with Copy-on-Write.

Copy-on-Write optimizations

A new lazy copy mechanism that defers the copy until the object in question is modified and only if this object shares data with another object. This mechanism was added to methods that don't require a copy of the underlying data. Popular examples are `DataFrame.drop` for `axis=1` and `DataFrame.rename`.

These methods return views when Copy-on-Write is enabled, which provides a significant performance improvement compared to the regular execution.

The new string data type

The upcoming pandas 3.0 release introduces a new, default string data type. This will most likely cause some work when upgrading to pandas 3.0, and this page provides an overview of the issues you might run into and gives guidance on how to address them.

This new dtype is already available in the pandas 2.3 release, and you can enable it with:

```
pd.options.future.infer_string = True
```

This allows you to test your code before the final 3.0 release.

> Note
> This migration guide focuses on the changes and migration steps needed when you are currently using `object` dtype for string data, which is used by default in pandas < 3.0. If you are already using one of the opt-in string dtypes, you can continue to do so without change. See string_migration_guide-for_existing_users for more details.

Background

Historically, pandas has always used the NumPy `object` dtype as the default to store text data. This has two primary drawbacks. First, `object` dtype is not specific to strings: any Python object can be stored in an `object`-dtype array, not just strings, and seeing `object` as the dtype for a column with strings is confusing for users. Second, this is not always very efficient (both performance wise and for memory usage).

Since pandas 1.0, an opt-in string data type has been available, but this has not yet been made the default, and uses the `pd.NA` scalar to represent missing values.

Pandas 3.0 changes the default dtype for strings to a new string data type, a variant of the existing optional string data type but using `NaN` as the missing value indicator, to be consistent with the other default data types.

To improve performance, the new string data type will use the `pyarrow` package by default, if installed (and otherwise it uses object dtype under the hood as a fallback).

See PDEP-14: Dedicated string data type for pandas 3.0 for more background and details.

Brief introduction to the new default string dtype

By default, pandas will infer this new string dtype instead of object dtype for string data (when creating pandas objects, such as in constructors or IO functions).

Being a default dtype means that the string dtype will be used in IO methods or constructors when the dtype is being inferred and the input is inferred to be string data:

```
>>> pd.Series(["a", "b", None])
0      a
1      b
2    NaN
dtype: str
```

It can also be specified explicitly using the `"str"` alias:

```
>>> pd.Series(["a", "b", None], dtype="str")
0      a
1      b
2    NaN
dtype: str
```

Similarly, functions like `read_csv`, `read_parquet`, and others will now use the new string dtype when reading string data.

In contrast to the current object dtype, the new string dtype will only store strings. This also means that it will raise an error if you try to store a non-string value in it (see below for more details).

Missing values with the new string dtype are always represented as `NaN` (`np.nan`), and the missing value behavior is similar to other default dtypes.

This new string dtype should otherwise behave the same as the existing `object` dtype users are used to. For example, all string-specific methods through the `str` accessor will work the same:

```
>>> ser = pd.Series(["a", "b", None], dtype="str")
>>> ser.str.upper()
0    A
1    B
2  NaN
dtype: str
```

Note
The new default string dtype is an instance of the `pandas.StringDtype` class. The dtype can be constructed as `pd.StringDtype(na_value=np.nan)`, but for general usage we recommend to use the shorter `"str"` alias.

Overview of behavior differences and how to address them

The dtype is no longer a numpy ”object” dtype

When inferring or reading string data, the data type of the resulting DataFrame column or Series will silently start being the new `"str"` dtype instead of the numpy `"object"` dtype, and this can have some impact on your code.

The new string dtype is a pandas data type (”extension dtype”), and no longer a numpy `np.dtype` instance. Therefore, passing the dtype of a string column to numpy functions will no longer work (e.g. passing it to a `dtype=` argument of a numpy function, or using `np.issubdtype` to check the dtype).

Checking the dtype

When checking the dtype, code might currently do something like:

```
>>> ser = pd.Series(["a", "b", "c"])
>>> ser.dtype == "object"
```

to check for columns with string data (by checking for the dtype being `"object"`). This will no longer work in pandas 3+, since `ser.dtype` will now be `"str"` with the new default string dtype, and the above check will return `False`.

To check for columns with string data, you should instead use:

```
>>> ser.dtype == "str"
```

How to write compatible code

For code that should work on both pandas 2.x and 3.x, you can use the `pandas.api.types.is_string_dtype` function:

```
>>> pd.api.types.is_string_dtype(ser.dtype)
True
```

This will return `True` for both the object dtype and the string dtypes.

Hardcoded use of object dtype

If you have code where the dtype is hardcoded in constructors, like

```
>>> pd.Series(["a", "b", "c"], dtype="object")
```

this will keep using the object dtype. You will want to update this code to ensure you get the benefits of the new string dtype.

How to write compatible code?

First, in many cases it can be sufficient to remove the specific data type, and let pandas do the inference. But if you want to be specific, you can specify the `"str"` dtype:

```
>>> pd.Series(["a", "b", "c"], dtype="str")
```

This is actually compatible with pandas 2.x as well, since in pandas < 3, `dtype="str"` was essentially treated as an alias for object dtype.

> Attention
> While using `dtype="str"` in constructors is compatible with pandas 2.x, specifying it as the dtype in `astype` runs into the issue of also stringifying missing values in pandas 2.x. See the section

For selecting string columns with `select_dtypes` in a pandas 2.x and 3.x compatible way, it is not possible to use `"str"`. While this works for pandas 3.x, it raises an error in pandas 2.x. As an alternative, you can select both `object` (for pandas 2.x) and `"string"` (for pandas 3.x; which will also select the default `str` dtype and does not error on pandas 2.x):

```
# can use ``include=["str"]`` for pandas >= 3
>>> df.select_dtypes(include=["object", "string"])
```

The missing value sentinel is now always NaN

When using object dtype, multiple possible missing value sentinels are supported, including `None` and `np.nan`. With the new default string dtype, the missing value sentinel is always NaN (`np.nan`):

```
# with object dtype, None is preserved as None and seen as missing
>>> ser = pd.Series(["a", "b", None], dtype="object")
>>> ser
0       a
1       b
2    None
dtype: object
>>> print(ser[2])
None

# with the new string dtype, any missing value like None is coerced to NaN
>>> ser = pd.Series(["a", "b", None], dtype="str")
>>> ser
0      a
1      b
2    NaN
dtype: str
>>> print(ser[2])
nan
```

Generally this should be no problem when relying on missing value behavior in pandas methods (for example, `ser.isna()` will give the same result as before). But when you relied on the exact value of `None` being present, that can impact your code.

How to write compatible code?

When checking for a missing value, instead of checking for the exact value of `None` or `np.nan`, you should use the `pandas.isna` function. This is the most robust way to check for missing values, as it will work regardless of the dtype and the exact missing value sentinel:

```
>>> pd.isna(ser[2])
True
```

One caveat: this function works both on scalars and on array-likes, and in the latter case it will return an array of bools. When using it in a Boolean context (for example, `if pd.isna(..): ..`) be sure to only pass a scalar to it.

"setitem" operations will now raise an error for non-string data

With the new string dtype, any attempt to set a non-string value in a Series or DataFrame will raise an error:

```
>>> ser = pd.Series(["a", "b", None], dtype="str")
>>> ser[1] = 2.5
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
...
TypeError: Invalid value '2.5' for dtype 'str'. Value should be a string or missing value, got 'float'
 ↪  instead.
```

If you relied on the flexible nature of object dtype being able to hold any Python object, but your initial data was inferred as strings, your code might be impacted by this change.

How to write compatible code?

You can update your code to ensure you only set string values in such columns, or otherwise you can explicitly ensure the column has object dtype first. This can be done by specifying the dtype explicitly in the constructor, or by using the `astype` method:

```
>>> ser = pd.Series(["a", "b", None], dtype="str")
>>> ser = ser.astype("object")
>>> ser[1] = 2.5
```

This `astype("object")` call will be redundant when using pandas 2.x, but this code will work for all versions.

Invalid unicode input

Python allows to have a built-in `str` object that represents invalid unicode data. And since the `object` dtype can hold any Python object, you can have a pandas Series with such invalid unicode data:

```
>>> ser = pd.Series(["\u2600", "\ud83d"], dtype=object)
>>> ser
0    ☀
1    \ud83d
dtype: object
```

However, when using the string dtype using `pyarrow` under the hood, this can only store valid unicode data, and otherwise it will raise an error:

```
>>> ser = pd.Series(["\u2600", "\ud83d"])
---------------------------------------------------------------------------
UnicodeEncodeError                        Traceback (most recent call last)
...
UnicodeEncodeError: 'utf-8' codec can't encode character '\ud83d' in position 0: surrogates not allowed
```

If you want to keep the previous behaviour, you can explicitly specify `dtype=object` to keep working with object dtype.

When you have byte data that you want to convert to strings using `decode()`, the `decode` method now has a `dtype` parameter to be able to specify object dtype instead of the default of string dtype for this use case.

`Series.values` now returns an `ExtensionArray`

With object dtype, using `.values` on a Series will return the underlying NumPy array.

```
>>> ser = pd.Series(["a", "b", np.nan], dtype="object")
>>> type(ser.values)
<class 'numpy.ndarray'>
```

However with the new string dtype, the underlying ExtensionArray is returned instead.

```
>>> ser = pd.Series(["a", "b", pd.NA], dtype="str")
>>> ser.values
<ArrowStringArray>
['a', 'b', nan]
Length: 3, dtype: str
```

If your code requires a NumPy array, you should use `Series.to_numpy`.

```
>>> ser = pd.Series(["a", "b", pd.NA], dtype="str")
>>> ser.to_numpy()
['a' 'b' nan]
```

In general, you should always prefer `Series.to_numpy` to get a NumPy array or `Series.array` to get an ExtensionArray over using `Series.values`.

Notable bug fixes

`astype(str)` preserving missing values

The stringifying of missing values is a long standing "bug" or misfeature, as discussed in https://github.com/pandas-dev/pandas/issues/25353, but fixing it introduces a significant behaviour change.

With pandas < 3, when using `astype(str)` or `astype("str")`, the operation would convert every element to a string, including the missing values:

```
# OLD behavior in pandas < 3
>>> ser = pd.Series([1.5, np.nan])
>>> ser
0    1.5
1    NaN
dtype: float64
>>> ser.astype("str")
0    1.5
1    nan
dtype: object
```

```
>>> ser.astype("str").to_numpy()
array(['1.5', 'nan'], dtype=object)
```

Note how `NaN` (`np.nan`) was converted to the string `"nan"`. This was not the intended behavior, and it was inconsistent with how other dtypes handled missing values.

With pandas 3, this behavior has been fixed, and now `astype("str")` will cast to the new string dtype, which preserves the missing values:

```
# NEW behavior in pandas 3
>>> pd.options.future.infer_string = True
>>> ser = pd.Series([1.5, np.nan])
>>> ser.astype("str")
0    1.5
1    NaN
dtype: str
>>> ser.astype("str").to_numpy()
array(['1.5', nan], dtype=object)
```

If you want to preserve the old behaviour of converting every object to a string, you can use `ser.map(str)` instead. If you want do such conversion while preserving the missing values in a way that works with both pandas 2.x and 3.x, you can use `ser.map(str, na_action="ignore")` (for pandas 3.x only, you can do `ser.astype("str")`).

If you want to convert to object or string dtype for pandas 2.x and 3.x, respectively, without needing to stringify each individual element, you will have to use a conditional check on the pandas version. For example, to convert a categorical Series with string categories to its dense non-categorical version with object or string dtype:

```
>>> import pandas as pd
>>> ser = pd.Series(["a", np.nan], dtype="category")
>>> ser.astype(object if pd.__version__ < "3" else "str")
```

`prod()` raising for string data

In pandas < 3, calling the `prod` method on a Series with string data would generally raise an error, except when the Series was empty or contained only a single string (potentially with missing values):

```
>>> ser = pd.Series(["a", None], dtype=object)
>>> ser.prod()
'a'
```

When the Series contains multiple strings, it will raise a `TypeError`. This behaviour stays the same in pandas 3 when using the flexible `object` dtype. But by virtue of using the new string dtype, this will generally consistently raise an error regardless of the number of strings:

```
>>> ser = pd.Series(["a", None], dtype="str")
>>> ser.prod()
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
...
TypeError: Cannot perform reduction 'prod' with string dtype
```

For existing users of the nullable `StringDtype`

While pandas 3.0 introduces a new _default string data type, pandas had an opt-in nullable string data type since pandas 1.0, which can be specified using `dtype="string"`. This nullable string

dtype uses `pd.NA` as the missing value indicator. In addition, also through `ArrowDtype` (by using `dtypes_backend="pyarrow"`) since pandas 1.5, one could already make use of a dedicated string dtype.

If you are already using one of the nullable string dtypes, for example by specifying `dtype="string"`, by using `convert_dtypes`, or by specifying the `dtype_backend` argument in IO functions, you can continue to do so without change.

The migration guide above applies to code that is currently (< 3.0) using object dtype for string data.

Indexing and selecting data

Source: https://pandas.pydata.org/docs/user_guide/indexing.html

The axis labeling information in pandas objects serves many purposes:

- Identifies data (i.e. provides *metadata*) using known indicators, important for analysis, visualization, and interactive console display.
- Enables automatic and explicit data alignment.
- Allows intuitive getting and setting of subsets of the data set.

In this section, we will focus on the final point: namely, how to slice, dice, and generally get and set subsets of pandas objects. The primary focus will be on Series and DataFrame as they have received more development attention in this area.

> Note
> The Python and NumPy indexing operators `[]` and attribute operator `.` provide quick and easy access to pandas data structures across a wide range of use cases. This makes interactive work intuitive, as there's little new to learn if you already know how to deal with Python dictionaries and NumPy arrays. However, since the type of the data to be accessed isn't known in advance, directly using standard operators has some optimization limits. For production code, we recommended that you take advantage of the optimized pandas data access methods exposed in this chapter.

See the MultiIndex / Advanced Indexing for `MultiIndex` and more advanced indexing documentation.

See the cookbook for some advanced strategies.

Different choices for indexing

Object selection has had a number of user-requested additions in order to support more explicit location based indexing. pandas now supports three types of multi-axis indexing.

- `.loc` is primarily label based, but may also be used with a boolean array. `.loc` will raise `KeyError` when the items are not found. Allowed inputs are:

  > - A single label, e.g. `5` or `'a'` (Note that `5` is interpreted as a *label* of the index. This use is **not** an integer position along the index.).
  > - A list or array of labels `['a', 'b', 'c']`.
  > - A slice object with labels `'a':'f'` (Note that contrary to usual Python slices, **both** the start and the stop are included, when present in the index! See Slicing with labels and Endpoints are inclusive.)
  > - A boolean array (any `NA` values will be treated as `False`).
  > - A `callable` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above).
  > - A tuple of row (and column) indices whose elements are one of the above inputs.

  See more at Selection by Label.

- `.iloc` is primarily integer position based (from `0` to `length-1` of the axis), but may also be used with a boolean array. `.iloc` will raise `IndexError` if a requested indexer is out-of-bounds, ex-

cept *slice* indexers which allow out-of-bounds indexing. (this conforms with Python/NumPy *slice* semantics). Allowed inputs are:

- An integer e.g. `5`.
- A list or array of integers `[4, 3, 0]`.
- A slice object with ints `1:7`.
- A boolean array (any `NA` values will be treated as `False`).
- A `callable` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above).
- A tuple of row (and column) indices whose elements are one of the above inputs.

See more at Selection by Position, Advanced Indexing and `Advanced Hierarchical <advanced.advanced_hierarchical>`.

- `.loc`, `.iloc`, and also `[]` indexing can accept a `callable` as indexer. See more at Selection By Callable.

> Note
> Destructuring tuple keys into row (and column) indexes occurs *before* callables are applied, so you cannot return a tuple from a callable to index both rows and columns.

Getting values from an object with multi-axes selection uses the following notation (using `.loc` as an example, but the following applies to `.iloc` as well). Any of the axes accessors may be the null slice `:`. Axes left out of the specification are assumed to be `:`, e.g. `p.loc['a']` is equivalent to `p.loc['a', :]`.

```
ser = pd.Series(range(5), index=list("abcde"))
ser.loc[["a", "c", "e"]]

df = pd.DataFrame(np.arange(25).reshape(5, 5), index=list("abcde"), columns=list("abcde"))
df.loc[["a", "c", "e"], ["b", "d"]]
```

Basics

As mentioned when introducing the data structures in the `last section <basics>`, the primary function of indexing with `[]` (a.k.a. `__getitem__` for those familiar with implementing class behavior in Python) is selecting out lower-dimensional slices. The following table shows return type values when indexing pandas objects with `[]`:

Object Type	Selection	Return Value Type
Series	`series[label]`	scalar value
DataFrame	`frame[colname]`	`Series` corresponding to colname
DataFrame	`frame[[colname]]`	`DataFrame` with columns corresponding to `[colname]`
DataFrame	`frame[list_of_colnames]`	`DataFrame` with columns corresponding to `list_of_colnames`

The same principle applies to label-based selection with `.loc`: a list-like of labels preserves the corresponding axis (so `df.loc[:, ["A"]]` returns a `DataFrame`, even when the list contains a single label), while a scalar label reduces the axis when labels on that axis are unique (so `df.loc[:, "A"]` typically returns a `Series`; if `"A"` is duplicated on the axis, a `DataFrame` is returned instead). See indexing.label for details.

Here we construct a simple time series data set to use for illustrating the indexing functionality:

```
dates = pd.date_range('1/1/2000', periods=8)
df = pd.DataFrame(np.random.randn(8, 4),
                  index=dates, columns=['A', 'B', 'C', 'D'])
df
```

Note
None of the indexing functionality is time series specific unless specifically stated.

Thus, as per above, we have the most basic indexing using `[]`:

```
s = df['A']
s[dates[5]]
```

You can pass a list of columns to `[]` to select columns in that order. If a column is not contained in the DataFrame, an exception will be raised. Multiple columns can also be set in this manner:

```
df
df[['B', 'A']] = df[['A', 'B']]
df
```

You may find this useful for applying a transform (in-place) to a subset of the columns.

Warning
pandas aligns all AXES when setting `Series` and `DataFrame` from `.loc`.
This will **not** modify `df` because the column alignment is before value assignment.

```
df[['A', 'B']]
df.loc[:, ['B', 'A']] = df[['A', 'B']]
df[['A', 'B']]
```

The correct way to swap column values is by using raw values:

```
df.loc[:, ['B', 'A']] = df[['A', 'B']].to_numpy()
df[['A', 'B']]
```

However, pandas does not align AXES when setting `Series` and `DataFrame` from `.iloc` because `.iloc` operates by position.
This will modify `df` because the column alignment is not done before value assignment.

```
df[['A', 'B']]
df.iloc[:, [1, 0]] = df[['A', 'B']]
df[['A','B']]
```

Selecting columns with `select`

`DataFrame.select` returns a DataFrame with the requested columns in the given order, similar to passing a list of columns to `[]`. As a method, it can be used within a chain of operations. Columns can be given as individual arguments or as a single list:

```
df.select('B', 'A')
df.select(['B', 'A'])
```

In addition to existing columns, computed columns can be included via `pandas.col` expressions or callables. A positional expression keeps the name of the underlying column, while a keyword argument names the resulting column. Later arguments can refer to columns computed earlier in the same call:

```
df.select('A', pd.col('B') * 2, C=pd.col('A') + pd.col('B'), D=pd.col('C') > 0)
```

A computed column with the same name as an existing column replaces it for later arguments. It does not replace a column that was already selected, so the result can contain duplicate column labels:

```
df.select('A', A=pd.col('A') * 2, E=pd.col('A'))
```

Attribute access

You may access an index on a `Series` or column on a `DataFrame` directly as an attribute:

```
sa = pd.Series([1, 2, 3], index=list('abc'))
dfa = df.copy()
```

```
sa.b
dfa.A
```

```
sa.a = 5
sa
dfa.A = list(range(len(dfa.index)))  # ok if A already exists
dfa
dfa['A'] = list(range(len(dfa.index)))  # use this form to create a new column
dfa
```

Warning

- You can use this access only if the index element is a valid Python identifier, e.g. `s.1` is not allowed. See here for an explanation of valid identifiers.
- The attribute will not be available if it conflicts with an existing method name, e.g. `s.min` is not allowed, but `s['min']` is possible.
- Similarly, the attribute will not be available if it conflicts with any of the following list: `index`, `major_axis`, `minor_axis`, `items`.
- In any of these cases, standard indexing will still work, e.g. `s['1']`, `s['min']`, and `s['index']` will access the corresponding element or column.

If you are using the IPython environment, you may also use tab-completion to see these accessible attributes.

You can also assign a `dict` to a row of a `DataFrame`:

```
x = pd.DataFrame({'x': [1, 2, 3], 'y': [3, 4, 5]})
x.iloc[1] = {'x': 9, 'y': 99}
x
```

You can use attribute access to modify an existing element of a Series or column of a DataFrame, but be careful; if you try to use attribute access to create a new column, it creates a new attribute rather than a new column and will this raise a `UserWarning`:

```
df_new = pd.DataFrame({'one': [1., 2., 3.]})
df_new.two = [4, 5, 6]
df_new
```

Slicing ranges

The most robust and consistent way of slicing ranges along arbitrary axes is described in the Selection by Position section detailing the `.iloc` method. For now, we explain the semantics of slicing using the `[]` operator.

> Note
> When the `Series` has float indices, slicing will select by position.

With Series, the syntax works exactly as with an ndarray, returning a slice of the values and the corresponding labels:

```
s[:5]
s[::2]
s[::-1]
```

Note that setting works as well:

```
s2 = s.copy()
s2[:5] = 0
s2
```

With DataFrame, slicing inside of `[]` **slices the rows**. This is provided largely as a convenience since it is such a common operation.

```
df[:3]
df[::-1]
```

Selection by label

> Warning
> `.loc` is strict when you present slicers that are not compatible (or convertible) with the index type. For example using integers in a `DatetimeIndex`. These will raise a `TypeError`.
>
> ```
> dfl = pd.DataFrame(np.random.randn(5, 4),
>                    columns=list('ABCD'),
>                    index=pd.date_range('20130101', periods=5))
> dfl
> dfl.loc[2:3]
> ```
>
> String likes in slicing *can* be convertible to the type of the index and lead to natural slicing.
>
> ```
> dfl.loc['20130102':'20130104']
> ```

pandas provides a suite of methods in order to have **purely label based indexing**. This is a strict inclusion based protocol. Every label asked for must be in the index, or a `KeyError` will be raised. When slicing, both the start bound **AND** the stop bound are *included*, if present in the index. Integers are valid labels, but they refer to the label **and not the position**.

The `.loc` attribute is the primary access method. The following are valid inputs:

- A single label, e.g. `5` or `'a'` (Note that `5` is interpreted as a *label* of the index. This use is **not** an integer position along the index.).
- A list or array of labels `['a', 'b', 'c']`.
- A slice object with labels `'a':'f'`. Note that contrary to usual Python slices, **both** the start and the stop are included, when present in the index! See Slicing with labels.
- A boolean array.
- A `callable`, see Selection By Callable.

```
s1 = pd.Series(np.random.randn(6), index=list('abcdef'))
s1
s1.loc['c':]
s1.loc['b']
```

Note that setting works as well:

```
s1.loc['c':] = 0
s1
```

With a DataFrame:

```
df1 = pd.DataFrame(np.random.randn(6, 4),
                   index=list('abcdef'),
                   columns=list('ABCD'))
df1
df1.loc[['a', 'b', 'd'], :]
```

Accessing via label slices:

```
df1.loc['d':, 'A':'C']
```

For getting a cross section using a label (equivalent to `df.xs('a')`):

```
df1.loc['a']
```

For getting values with a boolean array:

```
df1.loc['a'] > 0
df1.loc[:, df1.loc['a'] > 0]
```

NA values in a boolean array propagate as `False`:

```
mask = pd.array([True, False, True, False, pd.NA, False], dtype="boolean")
mask
df1[mask]
```

For getting a value explicitly:

```
# this is also equivalent to ``df1.at['a','A']``
df1.loc['a', 'A']
```

Slicing with labels

When using `.loc` with slices, if both the start and the stop labels are present in the index, then elements *located* between the two (including them) are returned:

```
s = pd.Series(list('abcde'), index=[0, 3, 2, 5, 4])
s.loc[3:5]
```

If the index is sorted, and can be compared against start and stop labels, then slicing will still work as expected, by selecting labels which *rank* between the two:

```
s.sort_index()
s.sort_index().loc[1:6]
```

However, if at least one of the two is absent *and* the index is not sorted, an error will be raised (since doing otherwise would be computationally expensive, as well as potentially ambiguous for mixed type indexes). For instance, in the above example, `s.loc[1:6]` would raise `KeyError`.

For the rationale behind this behavior, see Endpoints are inclusive.

```
s = pd.Series(list('abcdef'), index=[0, 3, 2, 5, 4, 2])
s.loc[3:5]
```

Also, if the index has duplicate labels *and* either the start or the stop label is duplicated, an error will be raised. For instance, in the above example, `s.loc[2:5]` would raise a `KeyError`.

For more information about duplicate labels, see Duplicate Labels.

When using a slice with a step, such as `.loc[start:stop:step]`, note that *start* and *stop* are interpreted as **labels**, while *step* is applied over the **positional index** within that label range. This means a stepped slice will behave differently than using the labels `range(start, stop, step)` when the index is not contiguous integers.

For example, in a `Series` with a non-contiguous integer index:

```
s = pd.Series(range(10), index=[0, 5, 10, 15, 20, 25, 30, 35, 40, 45])
s.loc[10:50:5]              # (10), then skip 5 positions → 35 only
s.loc[[10, 15, 20, 25]]     # explicit label selection
```

The first applies *step* across **positional locations** between the start/stop labels. The second selects each label directly.

Similarly, with a string-based index, the behavior is identical:

```
s = pd.Series(range(10), index=['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j'])
s.loc['b':'i':2]            # Start at 'b' (position 1), stop at 'i' (position 8), step 2 positions → 'b',
 ↪  'd', 'f', 'h'
s.loc[['b', 'd', 'f', 'h']] # explicit label selection
```

In both cases, *start* and *stop* determine the label boundaries (inclusive), while *step* skips positions within that range, regardless of the index type.

Selection by position

pandas provides a suite of methods in order to get **purely integer based indexing**. The semantics follow closely Python and NumPy slicing. These are `0-based` indexing. When slicing, the start bound is *included*, while the upper bound is *excluded*. Trying to use a non-integer, even a **valid** label will raise an `IndexError`.

The `.iloc` attribute is the primary access method. The following are valid inputs:

- An integer e.g. `5`.
- A list or array of integers `[4, 3, 0]`.
- A slice object with ints `1:7`.
- A boolean array.
- A `callable`, see Selection By Callable.
- A tuple of row (and column) indexes, whose elements are one of the above types.

```
s1 = pd.Series(np.random.randn(5), index=list(range(0, 10, 2)))
s1
s1.iloc[:3]
s1.iloc[3]
```

Note that setting works as well:

```
s1.iloc[:3] = 0
s1
```

With a DataFrame:

```
df1 = pd.DataFrame(np.random.randn(6, 4),
                   index=list(range(0, 12, 2)),
                   columns=list(range(0, 8, 2)))
df1
```

Select via integer slicing:

```
df1.iloc[:3]
df1.iloc[1:5, 2:4]
```

Select via integer list:

```
df1.iloc[[1, 3, 5], [1, 3]]
```

```
df1.iloc[1:3, :]
```

```
df1.iloc[:, 1:3]
```

```
# this is also equivalent to ``df1.iat[1,1]``
df1.iloc[1, 1]
```

For getting a cross section using an integer position (equiv to `df.xs(1)`):

```
df1.iloc[1]
```

Out of range slice indexes are handled gracefully just as in Python/NumPy.

```
# these are allowed in Python/NumPy.
x = list('abcdef')
x
x[4:10]
x[8:10]
s = pd.Series(x)
s
s.iloc[4:10]
s.iloc[8:10]
```

Note that using slices that go out of bounds can result in an empty axis (e.g. an empty DataFrame being returned).

```
dfl = pd.DataFrame(np.random.randn(5, 2), columns=list('AB'))
dfl
dfl.iloc[:, 2:3]
dfl.iloc[:, 1:3]
dfl.iloc[4:6]
```

A single indexer that is out of bounds will raise an `IndexError`. A list of indexers where any element is out of bounds will raise an `IndexError`.

```
dfl.iloc[[4, 5, 6]]
```

```
dfl.iloc[:, 4]
```

Selection by callable

`.loc`, `.iloc`, and also `[]` indexing can accept a `callable` as indexer. The `callable` must be a function with one argument (the calling Series or DataFrame) that returns valid output for indexing.

> Note
> For `.iloc` indexing, returning a tuple from the callable is not supported, since tuple destructuring for row and column indexes occurs *before* applying callables.

```
df1 = pd.DataFrame(np.random.randn(6, 4),
                   index=list('abcdef'),
                   columns=list('ABCD'))
df1

df1.loc[lambda df: df['A'] > 0, :]
df1.loc[:, lambda df: ['A', 'B']]

df1.iloc[:, lambda df: [0, 1]]

df1[lambda df: df.columns[0]]
```

You can use callable indexing in `Series`.

```
df1['A'].loc[lambda s: s > 0]
```

Using these methods / indexers, you can chain data selection operations without using a temporary variable.

```
bb = pd.read_csv('data/baseball.csv', index_col='id')
(bb.groupby(['year', 'team']).sum(numeric_only=True)
   .loc[lambda df: df['r'] > 100])
```

Combining positional and label-based indexing

If you wish to get the 0th and the 2nd elements from the index in the 'A' column, you can do:

```
dfd = pd.DataFrame({'A': [1, 2, 3],
                    'B': [4, 5, 6]},
                   index=list('abc'))
dfd
dfd.loc[dfd.index[[0, 2]], 'A']
```

This can also be expressed using `.iloc`, by explicitly getting locations on the indexers, and using *positional* indexing to select things.

```
dfd.iloc[[0, 2], dfd.columns.get_loc('A')]
```

For getting *multiple* indexers, using `.get_indexer`:

```
dfd.iloc[[0, 2], dfd.columns.get_indexer(['A', 'B'])]
```

Reindexing

The idiomatic way to achieve selecting potentially not-found elements is via `.reindex()`. See also the section on reindexing.

```
s = pd.Series([1, 2, 3])
s.reindex([1, 2, 3])
```

Alternatively, if you want to select only *valid* keys, the following is idiomatic and efficient; it is guaranteed to preserve the dtype of the selection.

```
labels = [1, 2, 3]
s.loc[s.index.intersection(labels)]
```

Having a duplicated index will raise for a `.reindex()`:

```
s = pd.Series(np.arange(4), index=['a', 'a', 'b', 'c'])
labels = ['c', 'd']
s.reindex(labels)
```

Generally, you can intersect the desired labels with the current axis, and then reindex.

```
s.loc[s.index.intersection(labels)].reindex(labels)
```

However, this would *still* raise if your resulting index is duplicated.

```
labels = ['a', 'd']
s.loc[s.index.intersection(labels)].reindex(labels)
```

Selecting random samples

A random selection of rows or columns from a Series or DataFrame with the `sample` method. The method will sample rows by default, and accepts a specific number of rows/columns to return, or a fraction of rows.

```
s = pd.Series([0, 1, 2, 3, 4, 5])

# When no arguments are passed, returns 1 row.
s.sample()

# One may specify either a number of rows:
s.sample(n=3)

# Or a fraction of the rows:
s.sample(frac=0.5)
```

By default, `sample` will return each row at most once, but one can also sample with replacement using the `replace` option:

```
s = pd.Series([0, 1, 2, 3, 4, 5])

# Without replacement (default):
s.sample(n=6, replace=False)

# With replacement:
s.sample(n=6, replace=True)
```

By default, each row has an equal probability of being selected, but if you want rows to have different probabilities, you can pass the `sample` function sampling weights as `weights`. These weights can be a list, a NumPy array, or a Series, but they must be of the same length as the object you are sampling. Missing values will be treated as a weight of zero, and inf values are not allowed. If weights do not sum to 1, they will be re-normalized by dividing all weights by the sum of the weights. For example:

```
s = pd.Series([0, 1, 2, 3, 4, 5])
example_weights = [0, 0, 0.2, 0.2, 0.2, 0.4]
s.sample(n=2, weights=example_weights)

# Weights will be re-normalized automatically
example_weights2 = [0.5, 0, 0, 0, 0, 0]
s.sample(n=1, weights=example_weights2)
```

When applied to a DataFrame, you can use a column of the DataFrame as sampling weights (provided you are sampling rows and not columns) by simply passing the name of the column as

a string.

```
df2 = pd.DataFrame({'col1': [9, 8, 7, 6],
                    'weight_column': [0.5, 0.4, 0.1, 0]})
df2.sample(n=2, weights='weight_column')
```

`sample` also allows users to sample columns instead of rows using the `axis` argument.

```
df3 = pd.DataFrame({'col1': [1, 2, 3], 'col2': [2, 3, 4]})
df3.sample(n=1, axis=1)
```

Finally, one can also set a seed for `sample`'s random number generator using the `random_state` argument, which will accept either an integer (as a seed) or a NumPy RandomState object.

```
df4 = pd.DataFrame({'col1': [1, 2, 3], 'col2': [2, 3, 4]})

# With a given seed, the sample will always draw the same rows.
df4.sample(n=2, random_state=2)
df4.sample(n=2, random_state=2)
```

Setting with enlargement

The `.loc/[]` operations can perform enlargement when setting a non-existent key for that axis.

In the `Series` case this is effectively an appending operation.

```
se = pd.Series([1, 2, 3])
se
se[5] = 5.
se
```

A `DataFrame` can be enlarged on either axis via `.loc`.

```
dfi = pd.DataFrame(np.arange(6).reshape(3, 2),
                   columns=['A', 'B'])
dfi
dfi.loc[:, 'C'] = dfi.loc[:, 'A']
dfi
```

This is like an `append` operation on the `DataFrame`.

```
dfi.loc[3] = 5
dfi
```

Fast scalar value getting and setting

Since indexing with `[]` must handle a lot of cases (single-label access, slicing, boolean indexing, etc.), it has a bit of overhead in order to figure out what you're asking for. If you only want to access a scalar value, the fastest way is to use the `at` and `iat` methods, which are implemented on all of the data structures.

Similarly to `loc`, `at` provides **label** based scalar lookups, while, `iat` provides **integer** based lookups analogously to `iloc`

```
s.iat[5]
df.at[dates[5], 'A']
df.iat[3, 0]
```

You can also set using these same indexers.

```
df.at[dates[5], 'A'] = 7
df.iat[3, 0] = 7
```

`at` previously could enlarge the object in-place if the indexer was missing, but this behavior is deprecated. Use `.loc` instead for setting values with new keys:

```
df.loc[dates[-1] + pd.Timedelta('1 day'), 0] = 7
df
```

Boolean indexing

Another common operation is the use of boolean vectors to filter the data. The operators are: `|` for `or`, `&` for `and`, and `~` for `not`. These **must** be grouped by using parentheses, since by default Python will evaluate an expression such as `df['A'] > 2 & df['B'] < 3` as `df['A'] > (2 & df['B']) < 3`, while the desired evaluation order is `(df['A'] > 2) & (df['B'] < 3)`.

Using a boolean vector to index a Series works exactly as in a NumPy ndarray:

```
s = pd.Series(range(-3, 4))
s
s[s > 0]
s[(s < -1) | (s > 0.5)]
s[~(s < 0)]
```

You may select rows from a DataFrame using a boolean vector the same length as the DataFrame's index (for example, something derived from one of the columns of the DataFrame):

```
df[df['A'] > 0]
```

List comprehensions and the `map` method of Series can also be used to produce more complex criteria:

```
df2 = pd.DataFrame({'a': ['one', 'one', 'two', 'three', 'two', 'one', 'six'],
                    'b': ['x', 'y', 'y', 'x', 'y', 'x', 'x'],
                    'c': np.random.randn(7)})

# only want 'two' or 'three'
criterion = df2['a'].map(lambda x: x.startswith('t'))

df2[criterion]

# equivalent but slower
df2[[x.startswith('t') for x in df2['a']]]

# Multiple criteria
df2[criterion & (df2['b'] == 'x')]
```

With the choice methods Selection by Label, Selection by Position, and Advanced Indexing you may select along more than one axis using boolean vectors combined with other indexing expressions.

```
df2.loc[criterion & (df2['b'] == 'x'), 'b':'c']
```

> Warning
> While `loc` supports two kinds of boolean indexing, `iloc` only supports indexing with a boolean array. If the indexer is a boolean `Series`, an error will be raised. For instance, in the following

example, `df.iloc[s.values, 1]` is ok. The boolean indexer is an array. But `df.iloc[s, 1]` would raise `ValueError`.

```
df = pd.DataFrame([[1, 2], [3, 4], [5, 6]],
                  index=list('abc'),
                  columns=['A', 'B'])
s = (df['A'] > 2)
s

df.loc[s, 'B']

df.iloc[s.values, 1]
```

Filtering with `DataFrame.filter`

New in version 3.1.0.

`DataFrame.filter` and `Series.filter` also accept a boolean mask through the `cond` keyword, keeping the rows where the mask is True. A `Series` mask is aligned on the index; any other array-like must have the same length as the index.

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=list('abc'))
df.filter(cond=df['A'] > 1)
```

`cond` may also be a callable that returns a boolean mask, or an expression created with `pandas.col`. These two forms may be passed positionally, which is convenient in a method chain where the intermediate object has no name:

```
df.filter(pd.col('A') > 1)
df.assign(C=pd.col('A') + pd.col('B')).filter(lambda df: df['C'] > 5)
```

Pass `axis=1` to filter columns instead of rows:

```
df.filter(cond=df.columns.str.startswith('A'), axis=1)
```

Missing values in the mask are treated as False by default, matching `df[mask]` for a mask with nullable boolean dtype. Pass `na=True` to keep the corresponding rows instead, or `na="raise"` to raise an error:

```
mask = pd.array([True, None, False], dtype="boolean")
df.filter(cond=mask)
df.filter(cond=mask, na=True)
```

Indexing with isin

Consider the `isin` method of `Series`, which returns a boolean vector that is true wherever the `Series` elements exist in the passed list. This allows you to select rows where one or more columns have values you want:

```
s = pd.Series(np.arange(5), index=np.arange(5)[::-1], dtype='int64')
s
s.isin([2, 4, 6])
s[s.isin([2, 4, 6])]
```

The same method is available for `Index` objects and is useful for the cases when you don't know which of the sought labels are in fact present:

```
s[s.index.isin([2, 4, 6])]

# compare it to the following
s.reindex([2, 4, 6])
```

In addition to that, `MultiIndex` allows selecting a separate level to use in the membership check:

```
s_mi = pd.Series(np.arange(6),
                 index=pd.MultiIndex.from_product([[0, 1], ['a', 'b', 'c']]))
s_mi
s_mi.iloc[s_mi.index.isin([(1, 'a'), (2, 'b'), (0, 'c')])]
s_mi.iloc[s_mi.index.isin(['a', 'c', 'e'], level=1)]
```

DataFrame also has an `isin` method. When calling `isin`, pass a set of values as either an array or dict. If values is an array, `isin` returns a DataFrame of booleans that is the same shape as the original DataFrame, with True wherever the element is in the sequence of values.

```
df = pd.DataFrame({'vals': [1, 2, 3, 4], 'ids': ['a', 'b', 'f', 'n'],
                   'ids2': ['a', 'n', 'c', 'n']})

values = ['a', 'b', 1, 3]

df.isin(values)
```

Oftentimes you'll want to match certain values with certain columns. Just make values a `dict` where the key is the column, and the value is a list of items you want to check for.

```
values = {'ids': ['a', 'b'], 'vals': [1, 3]}

df.isin(values)
```

To return the DataFrame of booleans where the values are *not* in the original DataFrame, use the ~ operator:

```
values = {'ids': ['a', 'b'], 'vals': [1, 3]}

~df.isin(values)
```

Combine DataFrame's `isin` with the `any()` and `all()` methods to quickly select subsets of your data that meet a given criteria. To select a row where each column meets its own criterion:

```
values = {'ids': ['a', 'b'], 'ids2': ['a', 'c'], 'vals': [1, 3]}

row_mask = df.isin(values).all(axis=1)

df[row_mask]
```

The `where` Method and Masking

Selecting values from a Series with a boolean vector generally returns a subset of the data. To guarantee that selection output has the same shape as the original data, you can use the `where` method in `Series` and `DataFrame`.

To return only the selected rows:

```
s[s > 0]
```

To return a Series of the same shape as the original:

```
s.where(s > 0)
```

Selecting values from a DataFrame with a boolean criterion now also preserves input data shape. `where` is used under the hood as the implementation. The code below is equivalent to `df.where(df < 0)`.

```
dates = pd.date_range('1/1/2000', periods=8)
df = pd.DataFrame(np.random.randn(8, 4),
                  index=dates, columns=['A', 'B', 'C', 'D'])
df[df < 0]
```

In addition, `where` takes an optional `other` argument for replacement of values where the condition is False, in the returned copy.

```
df.where(df < 0, -df)
```

You may wish to set values based on some boolean criteria. This can be done intuitively like so:

```
s2 = s.copy()
s2[s2 < 0] = 0
s2

df2 = df.copy()
df2[df2 < 0] = 0
df2
```

`where` returns a modified copy of the data.

> Note
> The signature for `DataFrame.where` differs from `numpy.where`. Roughly `df1.where(m, df2)` is equivalent to `np.where(m, df1, df2)`.
>
> ```
> df.where(df < 0, -df) == np.where(df < 0, df, -df)
> ```

Alignment

Furthermore, `where` aligns the input boolean condition (ndarray or DataFrame), such that partial selection with setting is possible. This is analogous to partial setting via `.loc` (but on the contents rather than the axis labels).

```
df2 = df.copy()
df2[df2[1:4] > 0] = 3
df2
```

Where can also accept `axis` and `level` parameters to align the input when performing the `where`.

```
df2 = df.copy()
df2.where(df2 > 0, df2['A'], axis='index')
```

This is equivalent to (but faster than) the following.

```
df2 = df.copy()
df.apply(lambda x, y: x.where(x > 0, y), y=df['A'])
```

`where` can accept a callable as condition and `other` arguments. The function must be with one argument (the calling Series or DataFrame) and that returns valid output as condition and `other` argument.

```
df3 = pd.DataFrame({'A': [1, 2, 3],
                    'B': [4, 5, 6],
```

```
                    'C': [7, 8, 9]})
df3.where(lambda x: x > 4, lambda x: x + 10)
```

Mask

`mask` is the inverse boolean operation of `where`.

```
s.mask(s >= 0)
df.mask(df >= 0)
```

Setting with enlargement conditionally using `numpy`

An alternative to `where` is to use `numpy.where`. Combined with setting a new column, you can use it to enlarge a DataFrame where the values are determined conditionally.

Consider you have two choices to choose from in the following DataFrame. And you want to set a new column color to 'green' when the second column has 'Z'. You can do the following:

```
df = pd.DataFrame({'col1': list('ABBC'), 'col2': list('ZZXY')})
df['color'] = np.where(df['col2'] == 'Z', 'green', 'red')
df
```

If you have multiple conditions, you can use `numpy.select` to achieve that. Say corresponding to three conditions there are three choice of colors, with a fourth color as a fallback, you can do the following.

```
conditions = [
    (df['col2'] == 'Z') & (df['col1'] == 'A'),
    (df['col2'] == 'Z') & (df['col1'] == 'B'),
    (df['col1'] == 'B')
]
choices = ['yellow', 'blue', 'purple']
df['color'] = np.select(conditions, choices, default='black')
df
```

The `query` Method

`DataFrame` objects have a `query` method that allows selection using an expression.

You can get the value of the frame where column `b` has values between the values of columns `a` and `c`. For example:

```
n = 10
df = pd.DataFrame(np.random.rand(n, 3), columns=list('abc'))
df

# pure python
df[(df['a'] < df['b']) & (df['b'] < df['c'])]

# query
df.query('(a < b) & (b < c)')
```

Do the same thing but fall back on a named index if there is no column with the name `a`.

```
df = pd.DataFrame(np.random.randint(n / 2, size=(n, 2)), columns=list('bc'))
df.index.name = 'a'
```

```
df
df.query('a < b and b < c')
```

If instead you don't want to or cannot name your index, you can use the name `index` in your query expression:

```
df = pd.DataFrame(np.random.randint(n, size=(n, 2)), columns=list('bc'))
df
df.query('index < b < c')
```

Note
If the name of your index overlaps with a column name, the column name is given precedence. For example,

```
df = pd.DataFrame({'a': np.random.randint(5, size=5)})
df.index.name = 'a'
df.query('a > 2')  # uses the column 'a', not the index
```

You can still use the index in a query expression by using the special identifier 'index':

```
df.query('index > 2')
```

If for some reason you have a column named `index`, then you can refer to the index as `ilevel_0` as well, but at this point you should consider renaming your columns to something less ambiguous.

`MultiIndex` `query` Syntax

You can also use the levels of a `DataFrame` with a `MultiIndex` as if they were columns in the frame:

```
n = 10
colors = np.random.choice(['red', 'green'], size=n)
foods = np.random.choice(['eggs', 'ham'], size=n)
colors
foods

index = pd.MultiIndex.from_arrays([colors, foods], names=['color', 'food'])
df = pd.DataFrame(np.random.randn(n, 2), index=index)
df
df.query('color == "red"')
```

If the levels of the `MultiIndex` are unnamed, you can refer to them using special names:

```
df.index.names = [None, None]
df
df.query('ilevel_0 == "red"')
```

The convention is `ilevel_0`, which means "index level 0" for the 0th level of the `index`.

`query` Use Cases

A use case for `query` is when you have a collection of `DataFrame` objects that have a subset of column names (or index levels/names) in common. You can pass the same query to both frames *without* having to specify which frame you're interested in querying

```
df = pd.DataFrame(np.random.rand(n, 3), columns=list('abc'))
df
df2 = pd.DataFrame(np.random.rand(n + 2, 3), columns=df.columns)
df2
```

```
expr = '0.0 <= a <= c <= 0.5'
map(lambda frame: frame.query(expr), [df, df2])
```

`query` Python versus pandas Syntax Comparison

Full numpy-like syntax:

```
df = pd.DataFrame(np.random.randint(n, size=(n, 3)), columns=list('abc'))
df
df.query('(a < b) & (b < c)')
df[(df['a'] < df['b']) & (df['b'] < df['c'])]
```

Slightly nicer by removing the parentheses (comparison operators bind tighter than `&` and `|`):

```
df.query('a < b & b < c')
```

Use English instead of symbols:

```
df.query('a < b and b < c')
```

Pretty close to how you might write it on paper:

```
df.query('a < b < c')
```

The `in` and `not in` operators

`query` also supports special use of Python's `in` and `not in` comparison operators, providing a succinct syntax for calling the `isin` method of a `Series` or `DataFrame`.

```
# get all rows where columns "a" and "b" have overlapping values
df = pd.DataFrame({'a': list('aabbccddeeff'), 'b': list('aaaabbbbcccc'),
                   'c': np.random.randint(5, size=12),
                   'd': np.random.randint(9, size=12)})
df
df.query('a in b')

# How you'd do it in pure Python
df[df['a'].isin(df['b'])]

df.query('a not in b')

# pure Python
df[~df['a'].isin(df['b'])]
```

You can combine this with other expressions for very succinct queries:

```
# rows where cols a and b have overlapping values
# and col c's values are less than col d's
df.query('a in b and c < d')

# pure Python
df[df['b'].isin(df['a']) & (df['c'] < df['d'])]
```

> Note
> Note that `in` and `not in` are evaluated in Python, since `numexpr` has no equivalent of this operation. However, **only the** `in`/`not in` **expression itself** is evaluated in vanilla Python. For example, in the expression

```
df.query('a in b + c + d')
```

`(b + c + d)` is evaluated by `numexpr` and *then* the `in` operation is evaluated in plain Python. In general, any operations that can be evaluated using `numexpr` will be.

Special use of the == operator with `list` objects

Comparing a `list` of values to a column using `==`/`!=` works similarly to `in`/`not in`.

```
df.query('b == ["a", "b", "c"]')

# pure Python
df[df['b'].isin(["a", "b", "c"])]

df.query('c == [1, 2]')

df.query('c != [1, 2]')

# using in/not in
df.query('[1, 2] in c')

df.query('[1, 2] not in c')

# pure Python
df[df['c'].isin([1, 2])]
```

Boolean operators

You can negate boolean expressions with the word `not` or the `~` operator.

```
df = pd.DataFrame(np.random.rand(n, 3), columns=list('abc'))
df['bools'] = np.random.rand(len(df)) > 0.5
df.query('~bools')
df.query('not bools')
df.query('not bools') == df[~df['bools']]
```

Of course, expressions can be arbitrarily complex too:

```
# short query syntax
shorter = df.query('a < b < c and (not bools) or bools > 2')

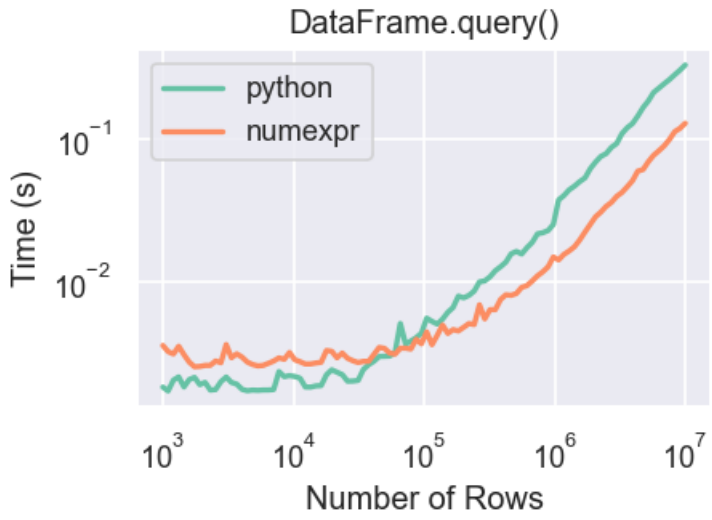
# equivalent in pure Python
longer = df[(df['a'] < df['b'])
            & (df['b'] < df['c'])
            & (~df['bools'])
            | (df['bools'] > 2)]

shorter
longer

shorter == longer
```

Performance of `query`

`DataFrame.query()` using `numexpr` is slightly faster than Python for large frames.



You will only see the performance benefits of using the `numexpr` engine with `DataFrame.query()` if your frame has more than approximately 100,000 rows.

This plot was created using a `DataFrame` with 3 columns each containing floating point values generated using `numpy.random.randn()`.

```
df = pd.DataFrame(np.random.randn(8, 4),
                  index=dates, columns=['A', 'B', 'C', 'D'])
df2 = df.copy()
```

Duplicate data

If you want to identify and remove duplicate rows in a DataFrame, there are two methods that will help: `duplicated` and `drop_duplicates`. Each takes as an argument the columns to use to identify duplicated rows.

- `duplicated` returns a boolean vector whose length is the number of rows, and which indicates whether a row is duplicated.
- `drop_duplicates` removes duplicate rows.

By default, the first observed row of a duplicate set is considered unique, but each method has a `keep` parameter to specify targets to be kept.

- `keep='first'` (default): mark / drop duplicates except for the first occurrence.
- `keep='last'`: mark / drop duplicates except for the last occurrence.
- `keep=False`: mark / drop all duplicates.

```
df2 = pd.DataFrame({'a': ['one', 'one', 'two', 'two', 'two', 'three', 'four'],
                    'b': ['x', 'y', 'x', 'y', 'x', 'x', 'x'],
                    'c': np.random.randn(7)})
df2
df2.duplicated('a')
df2.duplicated('a', keep='last')
df2.duplicated('a', keep=False)
df2.drop_duplicates('a')
df2.drop_duplicates('a', keep='last')
df2.drop_duplicates('a', keep=False)
```

Also, you can pass a list of columns to identify duplications.

```
df2.duplicated(['a', 'b'])
df2.drop_duplicates(['a', 'b'])
```

To drop duplicates by index value, use `Index.duplicated` then perform slicing. The same set of options are available for the `keep` parameter.

```
df3 = pd.DataFrame({'a': np.arange(6),
                    'b': np.random.randn(6)},
                   index=['a', 'a', 'b', 'c', 'b', 'a'])
df3
df3.index.duplicated()
df3[~df3.index.duplicated()]
df3[~df3.index.duplicated(keep='last')]
df3[~df3.index.duplicated(keep=False)]
```

Dictionary-like `get` method

Each of Series or DataFrame have a `get` method which can return a default value.

```
s = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s.get('a')  # equivalent to s['a']
s.get('x', default=-1)
```

Looking up values by index/column labels

Sometimes you want to extract a set of values given a sequence of row labels and column labels, this can be achieved by `pandas.factorize` and NumPy indexing.

For heterogeneous column types, we subset columns to avoid unnecessary NumPy conversions:

```
def pd_lookup_het(df, row_labels, col_labels):
   rows = df.index.get_indexer(row_labels)
   cols = df.columns.get_indexer(col_labels)
   sub = df.take(np.unique(cols), axis=1)
   sub = sub.take(np.unique(rows), axis=0)
   rows = sub.index.get_indexer(row_labels)
   values = sub.melt()["value"]
   cols = sub.columns.get_indexer(col_labels)
   flat_index = rows + cols * len(sub)
   result = values[flat_index]
   return result
```

For homogeneous column types, it is fastest to skip column subsetting and go directly to NumPy:

```
def pd_lookup_hom(df, row_labels, col_labels):
    rows = df.index.get_indexer(row_labels)
    df = df.loc[:, sorted(set(col_labels))]
    cols = df.columns.get_indexer(col_labels)
    result = df.to_numpy()[rows, cols]
    return result
```

Formerly this could be achieved with the dedicated `DataFrame.lookup` method which was deprecated in version 1.2.0 and removed in version 2.0.0.

Index objects

The pandas `Index` class and its subclasses can be viewed as implementing an *ordered multiset*. Duplicates are allowed.

`Index` also provides the infrastructure necessary for lookups, data alignment, and reindexing. The easiest way to create an `Index` directly is to pass a `list` or other sequence to `Index`:

```
index = pd.Index(['e', 'd', 'a', 'b'])
index
'd' in index
```

or using numbers:

```
index = pd.Index([1, 5, 12])
index
5 in index
```

If no dtype is given, `Index` tries to infer the dtype from the data. It is also possible to give an explicit dtype when instantiating an `Index`:

```
index = pd.Index(['e', 'd', 'a', 'b'], dtype="string")
index
index = pd.Index([1, 5, 12], dtype="int8")
index
index = pd.Index([1, 5, 12], dtype="float32")
index
```

You can also pass a `name` to be stored in the index:

```
index = pd.Index(['e', 'd', 'a', 'b'], name='something')
index.name
```

The name, if set, will be shown in the console display:

```
index = pd.Index(list(range(5)), name='rows')
columns = pd.Index(['A', 'B', 'C'], name='cols')
df = pd.DataFrame(np.random.randn(5, 3), index=index, columns=columns)
df
df['A']
```

Setting metadata

Indexes are ”mostly immutable”, but it is possible to set and change their `name` attribute. You can use the `rename`, `set_names` to set these attributes directly, and they default to returning a copy.

See Advanced Indexing for usage of MultiIndexes.

```
ind = pd.Index([1, 2, 3])
ind.rename("apple")
ind
ind = ind.set_names(["apple"])
ind.name = "bob"
ind
```

`set_names`, `set_levels`, and `set_codes` also take an optional `level` argument

```
index = pd.MultiIndex.from_product([range(3), ['one', 'two']], names=['first', 'second'])
index
index.levels[1]
index.set_levels(["a", "b"], level=1)
```

Set operations on Index objects

The two main operations are `union` and `intersection`. Difference is provided via the `.difference()` method.

```
a = pd.Index(['c', 'b', 'a'])
b = pd.Index(['c', 'e', 'd'])
a.difference(b)
```

Also available is the `symmetric_difference` operation, which returns elements that appear in either `idx1` or `idx2`, but not in both. This is equivalent to the Index created by `idx1.difference(idx2).union(idx2.difference(idx1))`, with duplicates dropped.

```
idx1 = pd.Index([1, 2, 3, 4])
idx2 = pd.Index([2, 3, 4, 5])
idx1.symmetric_difference(idx2)
```

> Note
> The resulting index from a set operation will be sorted in ascending order.

When performing `Index.union` between indexes with different dtypes, the indexes must be cast to a common dtype. Typically, though not always, this is object dtype. The exception is when performing a union between integer and float data. In this case, the integer values are converted to float

```
idx1 = pd.Index([0, 1, 2])
idx2 = pd.Index([0.5, 1.5])
idx1.union(idx2)
```

Missing values

> Important
> Even though `Index` can hold missing values (`NaN`), it should be avoided if you do not want any unexpected results. For example, some operations exclude missing values implicitly.

`Index.fillna` fills missing values with specified scalar value.

```
idx1 = pd.Index([1, np.nan, 3, 4])
idx1
idx1.fillna(2)

idx2 = pd.DatetimeIndex([pd.Timestamp('2011-01-01'),
                         pd.NaT,
                         pd.Timestamp('2011-01-03')])
idx2
idx2.fillna(pd.Timestamp('2011-01-02'))
```

Set / reset index

Occasionally you will load or create a data set into a DataFrame and want to add an index after you've already done so. There are a couple of different ways.

Set an index

DataFrame has a `set_index` method which takes a column name (for a regular `Index`) or a list of column names (for a `MultiIndex`). To create a new, re-indexed DataFrame:

```
data = pd.DataFrame({'a': ['bar', 'bar', 'foo', 'foo'],
                     'b': ['one', 'two', 'one', 'two'],
                     'c': ['z', 'y', 'x', 'w'],
                     'd': [1., 2., 3, 4]})
data
indexed1 = data.set_index('c')
indexed1
indexed2 = data.set_index(['a', 'b'])
indexed2
```

The `append` keyword option allow you to keep the existing index and append the given columns to a MultiIndex:

```
frame = data.set_index('c', drop=False)
frame = frame.set_index(['a', 'b'], append=True)
frame
```

Other options in `set_index` allow you not drop the index columns.

```
data.set_index('c', drop=False)
```

Reset the index

As a convenience, there is a new function on DataFrame called `reset_index` which transfers the index values into the DataFrame's columns and sets a simple integer index. This is the inverse operation of `set_index`.

```
data
data.reset_index()
```

The output is more similar to a SQL table or a record array. The names for the columns derived from the index are the ones stored in the `names` attribute.

You can use the `level` keyword to remove only a portion of the index:

```
frame
frame.reset_index(level=1)
```

`reset_index` takes an optional parameter `drop` which if true simply discards the index, instead of putting index values in the DataFrame's columns.

Adding an ad hoc index

You can assign a custom index to the `index` attribute:

```
df_idx = pd.DataFrame(range(4))
df_idx.index = pd.Index([10, 20, 30, 40], name="a")
df_idx
```

Why does assignment fail when using chained indexing?

Copy-on-Write is the new default with pandas 3.0. This means that chained indexing will never work. See this section for more context.

Series Assignment and Index Alignment

When assigning a Series to a DataFrame column, pandas performs automatic alignment based on index labels. This is a fundamental behavior that can be surprising to new users who might expect positional assignment.

Key Points:

- Series values are matched to DataFrame rows by index label
- Position/order in the Series doesn't matter
- Missing index labels result in NaN values
- This behavior is consistent across df[col] = series and df.loc[:, col] = series

You can think of this as reindexing the Series to the DataFrame index before assignment (for example, `df[col] = series.reindex(df.index)`).

Examples:

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'values': [1, 2, 3]}, index=['x', 'y', 'z'])

# Series with matching indices (different order)
s1 = pd.Series([10, 20, 30], index=['z', 'x', 'y'])
df['aligned'] = s1  # Aligns by index, not position
print(df)

# Series with partial index match
s2 = pd.Series([100, 200], index=['x', 'z'])
df['partial'] = s2  # Missing 'y' gets NaN
print(df)

# Series with non-matching indices
s3 = pd.Series([1000, 2000], index=['a', 'b'])
df['nomatch'] = s3  # All values become NaN
print(df)


#Avoiding Confusion:
#If you want positional assignment instead of index alignment:
# reset the Series index to match DataFrame index
df['s1_values'] = s1.reindex(df.index)
```

Column assignment vs. in-place setting

While index alignment is consistent between `df[col] = value` and `df.loc[:, col] = value` (see the previous section), the two forms differ in how they treat the column's existing dtype:

- `df[col] = value` **replaces** the column. The new column's dtype is inferred from `value`, regardless of what dtype the old column had.
- `df.loc[:, col] = value` (and `df.iloc[:, i] = value`) **sets the values in place** in the existing column. The original dtype is preserved, and the assignment will raise `TypeError` if `value` is not compatible with it.

This is most visible for `ExtensionDtype` columns such as `category`, `Int64`, `boolean`, or `string`, where the replacing form silently drops the extension dtype:

```
df = pd.DataFrame({"a": pd.Categorical(["x", "y"], categories=["x", "y", "z"])})
df["a"] = "z"          # replaces the column
df.dtypes

df = pd.DataFrame({"a": pd.Categorical(["x", "y"], categories=["x", "y", "z"])})
df.loc[:, "a"] = "z"   # sets values in place
df.dtypes
```

The same distinction applies to nullable numeric and boolean dtypes:

```
df = pd.DataFrame({"a": pd.array([1, 2], dtype="Int64")})
df["a"] = 5
df.dtypes

df = pd.DataFrame({"a": pd.array([1, 2], dtype="Int64")})
df.loc[:, "a"] = 5
df.dtypes
```

Use `df.loc[:, col] = value` (or `.iloc`) when you want to update every value of an existing column while keeping its dtype.

Working with missing data

Source: https://pandas.pydata.org/docs/user_guide/missing_data.html

Values considered "missing"

pandas uses different sentinel values to represent a missing (also referred to as NA) depending on the data type.

`numpy.nan` for NumPy data types. The disadvantage of using NumPy data types is that the original data type will be coerced to `np.float64` or `object`.

```
pd.Series([1, 2], dtype=np.int64).reindex([0, 1, 2])
pd.Series([True, False], dtype=np.bool_).reindex([0, 1, 2])
```

`NaT` for NumPy `np.datetime64`, `np.timedelta64`, and `PeriodDtype`. For typing applications, use `api.typing.NaTType`.

```
pd.Series([1, 2], dtype=np.dtype("timedelta64[ns]")).reindex([0, 1, 2])
pd.Series([1, 2], dtype=np.dtype("datetime64[ns]")).reindex([0, 1, 2])
pd.Series(["2020", "2020"], dtype=pd.PeriodDtype("D")).reindex([0, 1, 2])
```

`NA` for `StringDtype`, `Int64Dtype` (and other bit widths), `Float64Dtype` (and other bit widths), `BooleanDtype` and `ArrowDtype`. These types will maintain the original data type of the data. For typing applications, use `api.typing.NAType`.

```
pd.Series([1, 2], dtype="Int64").reindex([0, 1, 2])
pd.Series([True, False], dtype="boolean[pyarrow]").reindex([0, 1, 2])
```

To detect these missing value, use the `isna` or `notna` methods.

```
ser = pd.Series([pd.Timestamp("2020-01-01"), pd.NaT])
ser
pd.isna(ser)
```

> Note
>
> ```
> ser = pd.Series([1, None], dtype=object)
> ser
> pd.isna(ser)
> ```

> Warning
> Equality comparisons between `np.nan`, `NaT`, and `NA` do not act like `None`
>
> ```
> None == None  # noqa: E711
> np.nan == np.nan
> pd.NaT == pd.NaT
> pd.NA == pd.NA
> ```
>
> Therefore, an equality comparison between a `DataFrame` or `Series` with one of these missing values does not provide the same information as

```
ser = pd.Series([True, None], dtype="boolean[pyarrow]")
ser == pd.NA
pd.isna(ser)
```

NA semantics

> Note
> behaviour in some edge cases may still change in future releases.

NA is a scalar (singleton) used to represent missing values for nullable dtypes. The goal of NA is to provide a "missing" indicator that can be used consistently across data types (instead of np.nan, None or pd.NaT depending on the data type).

For example, when having missing values in a Series with the nullable integer dtype, it will use NA:

```
s = pd.Series([1, 2, None], dtype="Int64")
s
s[2]
s[2] is pd.NA
```

Currently, pandas does not use those data types using NA by default in a DataFrame or Series, so you need to specify the dtype explicitly. An easy way to convert to those dtypes is explained in the conversion section.

Propagation in arithmetic and comparison operations

In general, missing values *propagate* in operations involving NA. When one of the operands is unknown, the outcome of the operation is also unknown.

For example, NA propagates in arithmetic operations, similarly to np.nan:

```
pd.NA + 1
"a" * pd.NA
```

There are a few special cases when the result is known, even when one of the operands is NA.

```
pd.NA ** 0
1 ** pd.NA
```

In equality and comparison operations, NA also propagates. This deviates from the behaviour of np.nan, where comparisons with np.nan always return False.

```
pd.NA == 1
pd.NA == pd.NA
pd.NA < 2.5
```

To check if a value is equal to NA, use isna

```
pd.isna(pd.NA)
```

> Note
> An exception on this basic propagation rule are *reductions* (such as the mean or the minimum), where pandas defaults to skipping missing values. See the

Logical operations

For logical operations, NA follows the rules of the three-valued logic (or *Kleene logic*, similarly to R, SQL and Julia). This logic means to only propagate missing values when it is logically required.

For example, for the logical "or" operation (|), if one of the operands is True, we already know the result will be True, regardless of the other value (so regardless the missing value would be True or False). In this case, NA does not propagate:

```
True | False
True | pd.NA
pd.NA | True
```

On the other hand, if one of the operands is False, the result depends on the value of the other operand. Therefore, in this case NA propagates:

```
False | True
False | False
False | pd.NA
```

The behaviour of the logical "and" operation (&) can be derived using similar logic (where now NA will not propagate if one of the operands is already False):

```
False & True
False & False
False & pd.NA
```

```
True & True
True & False
True & pd.NA
```

NA in a boolean context

Since the actual value of an NA is unknown, it is ambiguous to convert NA to a boolean value.

```
bool(pd.NA)
```

This also means that NA cannot be used in a context where it is evaluated to a boolean, such as if condition: ... where condition can potentially be NA. In such cases, isna can be used to check for NA or condition being NA can be avoided, for example by filling missing values beforehand.

For the same reason, the Python keywords and, or, and not cannot be used with NA: these keywords always call bool() on their operands and cannot be overridden, so they will raise the same TypeError. This manifests as an asymmetry depending on operand position, since and and or only call bool() on the left operand:

```
True and pd.NA
pd.NA and True
```

Use the bitwise operators &, |, and ~ instead, which dispatch to __and__, __or__, and __invert__ and follow Kleene logic as described above.

A similar situation occurs when using Series or DataFrame objects in if statements, see gotchas.truth.

Converting a nullable array to NumPy keeps NA in the resulting object-dtype array, so NumPy operations that must produce a boolean result hit the same TypeError. Elementwise comparison is the common case: NumPy compares each pair of elements, gets NA back for the missing entries, and then fails when casting the result to bool dtype.

```
arr = pd.array(["a", "b", None], dtype="string")
np.asarray(arr)
np.asarray(arr) == np.array(["a", "z"])[:, None]
```

Pass an explicit `na_value` to `to_numpy` to substitute a value NumPy can compare, such as `None` or `np.nan`:

```
arr.to_numpy(dtype=object, na_value=None) == np.array(["a", "z"])[:, None]
```

NumPy ufuncs

`pandas.NA` implements NumPy's `__array_ufunc__` protocol. Most ufuncs work with `NA`, and generally return `NA`:

```
np.log(pd.NA)
np.add(pd.NA, 1)
```

> Warning
> Currently, ufuncs involving an ndarray and `NA` will return an object-dtype filled with NA values.
>
> ```
> a = np.array([1, 2, 3])
> np.greater(a, pd.NA)
> ```
>
> The return type here may change to return a different array type in the future.

See dsintro.numpy_interop for more on ufuncs.

Conversion

If you have a `DataFrame` or `Series` using `np.nan`, `DataFrame.convert_dtypes` and `Series.convert_dtypes`, respectively, will convert your data to use the nullable data types supporting `NA`, such as `Int64Dtype` or `ArrowDtype`. This is especially helpful after reading in data sets from IO methods where data types were inferred.

```
import io
data = io.StringIO("a,b\n,True\n2,")
df = pd.read_csv(data)
df.dtypes
df_conv = df.convert_dtypes()
df_conv
df_conv.dtypes
```

Inserting missing data

You can insert missing values by simply assigning to a `Series` or `DataFrame`. The missing value sentinel used will be chosen based on the dtype.

```
ser = pd.Series([1., 2., 3.])
ser.loc[0] = None
ser

ser = pd.Series([pd.Timestamp("2021"), pd.Timestamp("2021")])
ser.iloc[0] = np.nan
ser
```

```
ser = pd.Series([True, False], dtype="boolean[pyarrow]")
ser.iloc[0] = None
ser
```

For `object` types, pandas will use the value given:

```
s = pd.Series(["a", "b", "c"], dtype=object)
s.loc[0] = None
s.loc[1] = np.nan
s
```

Calculations with missing data

Missing values propagate through arithmetic operations between pandas objects.

```
ser1 = pd.Series([np.nan, np.nan, 2, 3])
ser2 = pd.Series([np.nan, 1, np.nan, 4])
ser1
ser2
ser1 + ser2
```

The descriptive statistics and computational methods discussed in the data structure overview (and listed `here <api.series.stats>` and here) all account for missing data.

The default behavior differs from many NumPy reduction functions. For example, pandas reductions such as `Series.std` and `DataFrame.std` skip missing values by default, while `numpy.std` returns `nan` when the input contains `nan` values. To include missing values in pandas reductions, pass `skipna=False`.

```
ser = pd.Series([1.0, np.nan, 3.0])
ser.std()
ser.std(skipna=False)
np.std(ser.to_numpy(), ddof=1)
```

When summing data, NA values or empty data will be treated as zero.

```
pd.Series([np.nan]).sum()
pd.Series([], dtype="float64").sum()
```

When taking the product, NA values or empty data will be treated as 1.

```
pd.Series([np.nan]).prod()
pd.Series([], dtype="float64").prod()
```

Cumulative methods like `cumsum` and `cumprod` ignore NA values by default, but preserve them in the resulting array. To override this behaviour and include NA values in the calculation, use `skipna=False`.

```
ser = pd.Series([1, np.nan, 3, np.nan])
ser
ser.cumsum()
ser.cumsum(skipna=False)
```

Dropping missing data

`dropna` drops rows or columns with missing data.

```
df = pd.DataFrame([[np.nan, 1, 2], [1, 2, np.nan], [1, 2, 3]])
df
df.dropna()
df.dropna(axis=1)

ser = pd.Series([1, pd.NA], dtype="int64[pyarrow]")
ser.dropna()
```

Filling missing data

Filling by value

`fillna` replaces NA values with non-NA data.

Replace NA with a scalar value

```
data = {"np": [1.0, np.nan, np.nan, 2], "arrow": pd.array([1.0, pd.NA, pd.NA, 2],
↪  dtype="float64[pyarrow]")}
df = pd.DataFrame(data)
df
df.fillna(0)
```

Replace NA with column-specific values using a dict

```
df.fillna({"np": 0, "arrow": 1})
```

When the data has object dtype, you can control what type of NA values are present.

```
df = pd.DataFrame({"a": [pd.NA, np.nan, None]}, dtype=object)
df
df.fillna(None)
df.fillna(np.nan)
df.fillna(pd.NA)
```

However when the dtype is not object, these will all be replaced with the proper NA value for the dtype.

```
data = {"np": [1.0, np.nan, np.nan, 2], "arrow": pd.array([1.0, pd.NA, pd.NA, 2],
↪  dtype="float64[pyarrow]")}
df = pd.DataFrame(data)
df
df.fillna(None)
df.fillna(np.nan)
df.fillna(pd.NA)
```

Fill gaps forward or backward

```
df.ffill()
df.bfill()
```

Limit the number of NA values filled

```
df.ffill(limit=1)
```

NA values can be replaced with corresponding value from a `Series` or `DataFrame` where the index and column aligns between the original object and the filled object.

```
dff = pd.DataFrame(np.arange(30, dtype=np.float64).reshape(10, 3), columns=list("ABC"))
dff.iloc[3:5, 0] = np.nan
```

```
dff.iloc[4:6, 1] = np.nan
dff.iloc[5:8, 2] = np.nan
dff
dff.fillna(dff.mean())
```

Note

```
dff.where(pd.notna(dff), dff.mean(), axis="columns")
```

Interpolation

`DataFrame.interpolate` and `Series.interpolate` fills NA values using various interpolation methods.

```
df = pd.DataFrame(
    {
        "A": [1, 2.1, np.nan, 4.7, 5.6, 6.8],
        "B": [0.25, np.nan, np.nan, 4, 12.2, 14.4],
    }
)
df
df.interpolate()

idx = pd.date_range("2020-01-01", periods=10, freq="D")
data = np.random.default_rng(2).integers(0, 10, 10).astype(np.float64)
ts = pd.Series(data, index=idx)
ts.iloc[[1, 2, 5, 6, 9]] = np.nan

ts
@savefig series_before_interpolate.png
ts.plot()
```

```
ts.interpolate()
@savefig series_interpolate.png
ts.interpolate().plot()
```

Interpolation relative to a `Timestamp` in the `DatetimeIndex` is available by setting `method="time"`

```
ts2 = ts.iloc[[0, 1, 3, 7, 9]]
ts2
ts2.interpolate()
ts2.interpolate(method="time")
```

For a floating-point index, use `method='values'`:

```
idx = [0.0, 1.0, 10.0]
ser = pd.Series([0.0, np.nan, 10.0], idx)
ser
ser.interpolate()
ser.interpolate(method="values")
```

If you have scipy installed, you can pass the name of a 1-d interpolation routine to `method`. as specified in the scipy interpolation documentation and reference guide. The appropriate interpolation method will depend on the data type.

Tip
If you are dealing with a time series that is growing at an increasing rate, use `method='barycentric'`.

If you have values approximating a cumulative distribution function, use `method='pchip'`. To fill missing values with goal of smooth plotting use `method='akima'`.

```
df = pd.DataFrame(
   {
      "A": [1, 2.1, np.nan, 4.7, 5.6, 6.8],
      "B": [0.25, np.nan, np.nan, 4, 12.2, 14.4],
   }
)
df
df.interpolate(method="barycentric")
df.interpolate(method="pchip")
df.interpolate(method="akima")
```

When interpolating via a polynomial or spline approximation, you must also specify the degree or order of the approximation:

```
df.interpolate(method="spline", order=2)
df.interpolate(method="polynomial", order=2)
```

Comparing several methods.

```
np.random.seed(2)

ser = pd.Series(np.arange(1, 10.1, 0.25) ** 2 + np.random.randn(37))
missing = np.array([4, 13, 14, 15, 16, 17, 18, 20, 29])
ser.iloc[missing] = np.nan
methods = ["linear", "quadratic", "cubic"]

df = pd.DataFrame({m: ser.interpolate(method=m) for m in methods})
@savefig compare_interpolations.png
df.plot()
```

Interpolating new observations from expanding data with `Series.reindex`.

```
ser = pd.Series(np.sort(np.random.uniform(size=100)))

# interpolate at new_index
new_index = ser.index.union(pd.Index([49.25, 49.5, 49.75, 50.25, 50.5, 50.75]))
interp_s = ser.reindex(new_index).interpolate(method="pchip")
interp_s.loc[49:51]
```

Interpolation limits

`interpolate` accepts a `limit` keyword argument to limit the number of consecutive `NaN` values filled since the last valid observation

```
ser = pd.Series([np.nan, np.nan, 5, np.nan, np.nan, np.nan, 13, np.nan, np.nan])
ser
ser.interpolate()
ser.interpolate(limit=1)
```

Note that `limit` caps how many consecutive `NaN` values are filled within each gap; it does not skip gaps that are longer than the limit. A gap with more consecutive `NaN` values than `limit` is filled *partially*, as with the three `NaN` values between `5` and `13` above. To leave such gaps untouched entirely, mask them out afterwards.

By default, `NaN` values are filled in a `forward` direction. Use `limit_direction` parameter to fill `backward` or from `both` directions.

```
ser.interpolate(limit=1, limit_direction="backward")
ser.interpolate(limit=1, limit_direction="both")
ser.interpolate(limit_direction="both")
```

By default, `NaN` values are filled whether they are surrounded by existing valid values or outside existing valid values. The `limit_area` parameter restricts filling to either inside or outside values.

```
# fill one consecutive inside value in both directions
ser.interpolate(limit_direction="both", limit_area="inside", limit=1)

# fill all consecutive outside values backward
ser.interpolate(limit_direction="backward", limit_area="outside")

# fill all consecutive outside values in both directions
ser.interpolate(limit_direction="both", limit_area="outside")
```

Replacing values

`Series.replace` and `DataFrame.replace` can be used similar to `Series.fillna` and `DataFrame.fillna` to replace or insert missing values.

```
df = pd.DataFrame(np.eye(3))
df
df_missing = df.replace(0, np.nan)
df_missing
df_filled = df_missing.replace(np.nan, 2)
df_filled
```

Replacing more than one value is possible by passing a list.

```
df_filled.replace([1, 44], [2, 28])
```

Replacing using a mapping dict.

```
df_filled.replace({1: 44, 2: 28})
```

Regular expression replacement

> Note
> Python strings prefixed with the `r` character such as `r'hello world'` are "raw" strings. They have different semantics regarding backslashes than strings without this prefix. Backslashes in raw strings will be interpreted as an escaped backslash, e.g., `r'\' == '\\'`.

Replace the '.' with `NaN`

```
d = {"a": list(range(4)), "b": list("ab.."), "c": ["a", "b", np.nan, "d"]}
df = pd.DataFrame(d)
df.replace(".", np.nan)
```

Replace the '.' with `NaN` with regular expression that removes surrounding whitespace

```
df.replace(r"\s*\.\s*", np.nan, regex=True)
```

Replace with a list of regexes.

```
df.replace([r"\.", r"(a)"], ["dot", r"\1stuff"], regex=True)
```

Replace with a regex in a mapping dict.

```
df.replace({"b": r"\s*\.\s*"}, {"b": np.nan}, regex=True)
```

Pass nested dictionaries of regular expressions that use the `regex` keyword.

```
df.replace({"b": {"b": r""}}, regex=True)
df.replace(regex={"b": {r"\s*\.\s*": np.nan}})
df.replace({"b": r"\s*(\.)\s*"}, {"b": r"\1ty"}, regex=True)
```

Pass a list of regular expressions that will replace matches with a scalar.

```
df.replace([r"\s*\.\s*", r"a|b"], "placeholder", regex=True)
```

All of the regular expression examples can also be passed with the `to_replace` argument as the `regex` argument. In this case the `value` argument must be passed explicitly by name or `regex` must be a nested dictionary.

```
df.replace(regex=[r"\s*\.\s*", r"a|b"], value="placeholder")
```

> Note
> A regular expression object from `re.compile` is a valid input as well.

Group by: split-apply-combine

Source: https://pandas.pydata.org/docs/user_guide/groupby.html

By ”group by” we are referring to a process involving one or more of the following steps:

- **Splitting** the data into groups based on some criteria.
- **Applying** a function to each group independently.
- **Combining** the results into a data structure.

Out of these, the split step is the most straightforward. In the apply step, we might wish to do one of the following:

- **Aggregation**: compute a summary statistic (or statistics) for each group. Some examples:
  - Compute group sums or means.
  - Compute group sizes / counts.
- **Transformation**: perform some group-specific computations and return a like-indexed object. Some examples:
  - Standardize data (zscore) within a group.
  - Filling NAs within groups with a value derived from each group.
- **Filtration**: discard some groups, according to a group-wise computation that evaluates to True or False. Some examples:
  - Discard data that belong to groups with only a few members.
  - Filter out data based on the group sum or mean.

Many of these operations are defined on GroupBy objects. These operations are similar to those of the aggregating API, window API, and resample API.

It is possible that a given operation does not fall into one of these categories or is some combination of them. In such a case, it may be possible to compute the operation using GroupBy's `apply` method. This method will examine the results of the apply step and try to sensibly combine them into a single result if it doesn't fit into either of the above three categories.

> Note
> An operation that is split into multiple steps using built-in GroupBy operations will be more efficient than using the `apply` method with a user-defined Python function.

The name GroupBy should be quite familiar to those who have used a SQL-based tool (or `itertools`), in which you can write code like:

```
SELECT Column1, Column2, mean(Column3), sum(Column4)
FROM SomeTable
GROUP BY Column1, Column2
```

We aim to make operations like this natural and easy to express using pandas. We'll address each area of GroupBy functionality, then provide some non-trivial examples / use cases.

See the cookbook for some advanced strategies.

Splitting an object into groups

The abstract definition of grouping is to provide a mapping of labels to group names. To create a GroupBy object (more on what the GroupBy object is later), you may do the following:

```
speeds = pd.DataFrame(
    [
        ("bird", "Falconiformes", 389.0),
        ("bird", "Psittaciformes", 24.0),
        ("mammal", "Carnivora", 80.2),
        ("mammal", "Primates", np.nan),
        ("mammal", "Carnivora", 58),
    ],
    index=["falcon", "parrot", "lion", "monkey", "leopard"],
    columns=("class", "order", "max_speed"),
)
speeds

grouped = speeds.groupby("class")
grouped = speeds.groupby(["class", "order"])
```

The mapping can be specified many different ways:

- A Python function, to be called on each of the index labels.
- A list or NumPy array of the same length as the index.
- A dict or `Series`, providing a `label -> group name` mapping.
- For `DataFrame` objects, a string indicating either a column name or an index level name to be used to group.
- A list of any of the above things.

Collectively we refer to the grouping objects as the **keys**. For example, consider the following `DataFrame`:

> Note
> A string passed to `groupby` may refer to either a column or an index level. If a string matches both a column name and an index level name, a `ValueError` will be raised.

```
df = pd.DataFrame(
    {
        "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],
        "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
        "C": np.random.randn(8),
        "D": np.random.randn(8),
    }
)
df
```

On a DataFrame, we obtain a GroupBy object by calling `groupby`. This method returns a `pandas.api.typing.DataFrameGroupBy` instance. We could naturally group by either the `A` or `B` columns, or both:

```
grouped = df.groupby("A")
grouped = df.groupby("B")
grouped = df.groupby(["A", "B"])
```

> Note
> `df.groupby('A')` is just syntactic sugar for `df.groupby(df['A'])`.

DataFrame groupby always operates along axis 0 (rows). To split by columns, first do a transpose:

```
In [4]: def get_letter_type(letter):
   ...:     if letter.lower() in 'aeiou':
   ...:         return 'vowel'
   ...:     else:
   ...:         return 'consonant'
   ...:

In [5]: grouped = df.T.groupby(get_letter_type)
```

pandas `Index` objects support duplicate values. If a non-unique index is used as the group key in a groupby operation, all values for the same index value will be considered to be in one group and thus the output of aggregation functions will only contain unique index values:

```
index = [1, 2, 3, 1, 2, 3]

s = pd.Series([1, 2, 3, 10, 20, 30], index=index)

s

grouped = s.groupby(level=0)

grouped.first()

grouped.last()

grouped.sum()
```

Note that **no splitting occurs** until it's needed. Creating the GroupBy object only verifies that you've passed a valid mapping.

> Note
> Many kinds of complicated data manipulations can be expressed in terms of GroupBy operations (though it can't be guaranteed to be the most efficient implementation). You can get quite creative with the label mapping functions.

GroupBy sorting

By default the group keys are sorted during the `groupby` operation. You may however pass `sort=False` for potential speedups. With `sort=False` the order among group-keys follows the order of appearance of the keys in the original dataframe:

```
df2 = pd.DataFrame({"X": ["B", "B", "A", "A"], "Y": [1, 2, 3, 4]})
df2.groupby(["X"]).sum()
df2.groupby(["X"], sort=False).sum()
```

Note that `groupby` will preserve the order in which *observations* are sorted *within* each group. For example, the groups created by `groupby()` below are in the order they appeared in the original `DataFrame`:

```
df3 = pd.DataFrame({"X": ["A", "B", "A", "B"], "Y": [1, 4, 3, 2]})
df3.groupby("X").get_group("A")

df3.groupby(["X"]).get_group(("B",))
```

GroupBy dropna

By default `NA` values are excluded from group keys during the `groupby` operation. However, in case you want to include `NA` values in group keys, you could pass `dropna=False` to achieve it.

```
df_list = [[1, 2, 3], [1, None, 4], [2, 1, 3], [1, 2, 2]]
df_dropna = pd.DataFrame(df_list, columns=["a", "b", "c"])

df_dropna
```

```
# Default ``dropna`` is set to True, which will exclude NaNs in keys
df_dropna.groupby(by=["b"], dropna=True).sum()

# In order to allow NaN in keys, set ``dropna`` to False
df_dropna.groupby(by=["b"], dropna=False).sum()
```

The default setting of `dropna` argument is `True` which means `NA` are not included in group keys.

GroupBy object attributes

The `groups` attribute of a GroupBy object is a dictionary that maps each unique group key to the index labels belonging to that group. In the above example:

```
df.groupby("A").groups
df.T.groupby(get_letter_type).groups
```

Calling the standard Python `len` function on the GroupBy object returns the number of groups, which is the same as the length of the `groups` dictionary:

```
grouped = df.groupby(["A", "B"])
grouped.groups
len(grouped)
```

`GroupBy` will tab complete column names, GroupBy operations, and other attributes:

```
n = 10
weight = np.random.normal(166, 20, size=n)
height = np.random.normal(60, 10, size=n)
time = pd.date_range("1/1/2000", periods=n)
gender = np.random.choice(["male", "female"], size=n)
df = pd.DataFrame(
    {"height": height, "weight": weight, "gender": gender}, index=time
)
df
gb = df.groupby("gender")
```

```
@verbatim
In [1]: gb.<TAB>  # noqa: E225, E999
gb.agg        gb.boxplot    gb.cummin     gb.describe   gb.filter     gb.get_group  gb.height     gb.last
↪  gb.median     gb.ngroups    gb.plot       gb.rank       gb.std        gb.transform
gb.aggregate  gb.count      gb.cumprod    gb.dtype      gb.first      gb.groups     gb.hist       gb.max
↪  gb.min        gb.nth        gb.prod       gb.resample   gb.sum        gb.var
gb.apply      gb.cummax     gb.cumsum     gb.gender     gb.head       gb.indices    gb.mean       gb.name
↪  gb.ohlc       gb.quantile   gb.size       gb.tail       gb.weight
```

GroupBy with MultiIndex

With hierarchically-indexed data, it's quite natural to group by one of the levels of the hierarchy.

Let's create a Series with a two-level `MultiIndex`.

```
arrays = [
    ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
    ["one", "two", "one", "two", "one", "two", "one", "two"],
]
index = pd.MultiIndex.from_arrays(arrays, names=["first", "second"])
s = pd.Series(np.random.randn(8), index=index)
s
```

We can then group by one of the levels in `s`.

```
grouped = s.groupby(level=0)
grouped.sum()
```

If the MultiIndex has names specified, these can be passed instead of the level number:

```
s.groupby(level="second").sum()
```

Grouping with multiple levels is supported.

```
arrays = [
    ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
    ["doo", "doo", "bee", "bee", "bop", "bop", "bop", "bop"],
    ["one", "two", "one", "two", "one", "two", "one", "two"],
]
index = pd.MultiIndex.from_arrays(arrays, names=["first", "second", "third"])
s = pd.Series(np.random.randn(8), index=index)
s
s.groupby(level=["first", "second"]).sum()
```

Index level names may be supplied as keys.

```
s.groupby(["first", "second"]).sum()
```

More on the `sum` function and aggregation later.

Grouping DataFrame with Index levels and columns

A DataFrame may be grouped by a combination of columns and index levels. You can specify both column and index names, or use a `Grouper`.

Let's first create a DataFrame with a MultiIndex:

```
arrays = [
    ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
    ["one", "two", "one", "two", "one", "two", "one", "two"],
]

index = pd.MultiIndex.from_arrays(arrays, names=["first", "second"])

df = pd.DataFrame({"A": [1, 1, 1, 1, 2, 2, 3, 3], "B": np.arange(8)}, index=index)

df
```

Then we group `df` by the `second` index level and the `A` column.

```
df.groupby([pd.Grouper(level=1), "A"]).sum()
```

Index levels may also be specified by name.

```
df.groupby([pd.Grouper(level="second"), "A"]).sum()
```

Index level names may be specified as keys directly to `groupby`.

```
df.groupby(["second", "A"]).sum()
```

DataFrame column selection in GroupBy

Once you have created the GroupBy object from a DataFrame, you might want to do something different for each of the columns. Thus, by using `[]` on the GroupBy object in a similar way as the one used to get a column from a DataFrame, you can do:

```
df = pd.DataFrame(
    {
        "A": ["foo", "bar", "foo", "bar", "foo", "bar", "foo", "foo"],
        "B": ["one", "one", "two", "three", "two", "two", "one", "three"],
        "C": np.random.randn(8),
        "D": np.random.randn(8),
    }
)

df

grouped = df.groupby(["A"])
grouped_C = grouped["C"]
grouped_D = grouped["D"]
```

This is mainly syntactic sugar for the alternative, which is much more verbose:

```
df["C"].groupby(df["A"])
```

Additionally, this method avoids recomputing the internal grouping information derived from the passed key.

You can also include the grouping columns if you want to operate on them.

```
grouped[["A", "B"]].sum()
```

> Note
> The `groupby` operation in pandas drops the `name` field of the columns Index object after the operation. This change ensures consistency in syntax between different column selection methods within groupby operations.

Iterating through groups

With the GroupBy object in hand, iterating through the grouped data is very natural and functions similarly to `itertools.groupby`:

```
In [4]: grouped = df.groupby('A')

In [5]: for name, group in grouped:
   ...:     print(name)
   ...:     print(group)
   ...:
```

In the case of grouping by multiple keys, the group name will be a tuple:

```
In [5]: for name, group in df.groupby(['A', 'B']):
   ...:     print(name)
```

```
...:     print(group)
...:
```

See timeseries.iterating-label.

Selecting a group

A single group can be selected using `.DataFrameGroupBy.get_group`:

```
grouped.get_group("bar")
```

Or for an object grouped on multiple columns:

```
df.groupby(["A", "B"]).get_group(("bar", "one"))
```

Aggregation

An aggregation is a GroupBy operation that reduces the dimension of the grouping object. The result of an aggregation is, or at least is treated as, a scalar value for each column in a group. For example, producing the sum of each column in a group of values.

```
animals = pd.DataFrame(
    {
        "kind": ["cat", "dog", "cat", "dog"],
        "height": [9.1, 6.0, 9.5, 34.0],
        "weight": [7.9, 7.5, 9.9, 198.0],
    }
)
animals
animals.groupby("kind").sum()
```

In the result, the keys of the groups appear in the index by default. They can be instead included in the columns by passing `as_index=False`.

```
animals.groupby("kind", as_index=False).sum()
```

Built-in aggregation methods

Many common aggregations are built-in to GroupBy objects as methods. Of the methods listed below, those with a * do *not* have an efficient, GroupBy-specific, implementation.

Method Description ——– ————-

Some examples:

```
df.groupby("A")[["C", "D"]].max()
df.groupby(["A", "B"]).mean()
```

Another aggregation example is to compute the size of each group. This is included in GroupBy as the `size` method. It returns a Series whose index consists of the group names and the values are the sizes of each group.

```
grouped = df.groupby(["A", "B"])
grouped.size()
```

While the `.DataFrameGroupBy.describe` method is not itself a reducer, it can be used to conveniently produce a collection of summary statistics about each of the groups.

```
grouped.describe()
```

Another aggregation example is to compute the number of unique values of each group. This is similar to the `.DataFrameGroupBy.value_counts` function, except that it only counts the number of unique values.

```
ll = [['foo', 1], ['foo', 2], ['foo', 2], ['bar', 1], ['bar', 1]]
df4 = pd.DataFrame(ll, columns=["A", "B"])
df4
df4.groupby("A")["B"].nunique()
```

Note
Aggregation functions **will not** return the groups that you are aggregating over as named *columns* when `as_index=True`, the default. The grouped columns will be the **indices** of the returned object.
Passing `as_index=False` **will** return the groups that you are aggregating over as named columns, regardless if they are named **indices** or *columns* in the inputs.

The `aggregate` method

Note
The `aggregate` method can accept many different types of inputs. This section details using string aliases for various GroupBy methods; other inputs are detailed in the sections below.

Any reduction method that pandas implements can be passed as a string to `aggregate`. Users are encouraged to use the shorthand, `agg`. It will operate as if the corresponding method was called.

```
grouped = df.groupby("A")
grouped[["C", "D"]].aggregate("sum")

grouped = df.groupby(["A", "B"])
grouped.agg("sum")
```

The result of the aggregation will have the group names as the new index. In the case of multiple keys, the result is a MultiIndex by default. As mentioned above, this can be changed by using the `as_index` option:

```
grouped = df.groupby(["A", "B"], as_index=False)
grouped.agg("sum")

df.groupby("A", as_index=False)[["C", "D"]].agg("sum")
```

Note that you could use the `DataFrame.reset_index` DataFrame function to achieve the same result as the column names are stored in the resulting `MultiIndex`, although this will make an extra copy.

```
df.groupby(["A", "B"]).agg("sum").reset_index()
```

Aggregation with user-defined functions

Users can also provide their own User-Defined Functions (UDFs) for custom aggregations.

Warning
When aggregating with a UDF, the UDF should not mutate the provided `Series`. See gotchas.udf-mutation for more information.

Note
Aggregating with a UDF is often less performant than using the pandas built-in methods on GroupBy. Consider breaking up a complex operation into a chain of operations that utilize the built-in methods.

```
animals
animals.groupby("kind")[["height"]].agg(lambda x: set(x))
```

The resulting dtype will reflect that of the aggregating function. If the results from different groups have different dtypes, then a common dtype will be determined in the same way as `DataFrame` construction.

```
animals.groupby("kind")[["height"]].agg(lambda x: x.astype(int).sum())
```

Applying multiple functions at once

On a grouped `Series`, you can pass a list or dict of functions to `SeriesGroupBy.agg`, outputting a DataFrame:

```
grouped = df.groupby("A")
grouped["C"].agg(["sum", "mean", "std"])
```

On a grouped `DataFrame`, you can pass a list of functions to `DataFrameGroupBy.agg` to aggregate each column, which produces an aggregated result with a hierarchical column index:

```
grouped[["C", "D"]].agg(["sum", "mean", "std"])
```

The resulting aggregations are named after the functions themselves.

For a `Series`, if you need to rename, you can add in a chained operation like this:

```
(
    grouped["C"]
    .agg(["sum", "mean", "std"])
    .rename(columns={"sum": "foo", "mean": "bar", "std": "baz"})
)
```

Or, you can simply pass a list of tuples each with the name of the new column and the aggregate function:

```
(
   grouped["C"]
   .agg([("foo", "sum"), ("bar", "mean"), ("baz", "std")])
)
```

For a grouped `DataFrame`, you can rename in a similar manner:

By chaining `rename` operation,

```
(
    grouped[["C", "D"]].agg(["sum", "mean", "std"]).rename(
        columns={"sum": "foo", "mean": "bar", "std": "baz"}
    )
)
```

Or, passing a list of tuples,

```
(
   grouped[["C", "D"]].agg(
      [("foo", "sum"), ("bar", "mean"), ("baz", "std")]
```

```
    )
)
```

Note
In general, the output column names should be unique, but pandas will allow you apply to the same function (or two functions with the same name) to the same column.

```
grouped["C"].agg(["sum", "sum"])
```

pandas also allows you to provide multiple lambdas. In this case, pandas will mangle the name of the (nameless) lambda functions, appending _<i> to each subsequent lambda.

```
grouped["C"].agg([lambda x: x.max() - x.min(), lambda x: x.median() - x.mean()])
```

Named aggregation

To support column-specific aggregation *with control over the output column names*, pandas accepts the special syntax in `.DataFrameGroupBy.agg` and `.SeriesGroupBy.agg`, known as "named aggregation", where

- The keywords are the *output* column names
- The values are tuples whose first element is the column to select and the second element is the aggregation to apply to that column. pandas provides the `NamedAgg` namedtuple with the fields `['column', 'aggfunc']` to make it clearer what the arguments are. As usual, the aggregation can be a callable or a string alias.

```
animals

animals.groupby("kind").agg(
    min_height=pd.NamedAgg(column="height", aggfunc="min"),
    max_height=pd.NamedAgg(column="height", aggfunc="max"),
    average_weight=pd.NamedAgg(column="weight", aggfunc="mean"),
)
```

`NamedAgg` is just a `namedtuple`. Plain tuples are allowed as well.

```
animals.groupby("kind").agg(
    min_height=("height", "min"),
    max_height=("height", "max"),
    average_weight=("weight", "mean"),
)
```

If the column names you want are not valid Python keywords, construct a dictionary and unpack the keyword arguments

```
animals.groupby("kind").agg(
    **{
        "total weight": pd.NamedAgg(column="weight", aggfunc="sum")
    }
)
```

When using named aggregation, additional keyword arguments are not passed through to the aggregation functions; only pairs of `(column, aggfunc)` should be passed as `**kwargs`. If your aggregation functions require additional arguments, apply them partially with `functools.partial`.

Named aggregation is also valid for Series groupby aggregations. In this case there's no column selection, so the values are just the functions.

```
animals.groupby("kind").height.agg(
    min_height="min",
    max_height="max",
)
```

Applying different functions to DataFrame columns

By passing a dict to `aggregate` you can apply a different aggregation to the columns of a DataFrame:

```
grouped.agg({"C": "sum", "D": lambda x: np.std(x, ddof=1)})
```

The function names can also be strings. In order for a string to be valid it must be implemented on GroupBy:

```
grouped.agg({"C": "sum", "D": "std"})
```

Transformation

A transformation is a GroupBy operation whose result is indexed the same as the one being grouped. Common examples include `cumsum` and `diff`.

```
speeds
grouped = speeds.groupby("class")["max_speed"]
grouped.cumsum()
grouped.diff()
```

Unlike aggregations, the groupings that are used to split the original object are not included in the result.

> Note
> Since transformations do not include the groupings that are used to split the result, the arguments `as_index` and `sort` in `DataFrame.groupby` and

A common use of a transformation is to add the result back into the original DataFrame.

```
result = speeds.copy()
result["cumsum"] = grouped.cumsum()
result["diff"] = grouped.diff()
result
```

Built-in transformation methods

The following methods on GroupBy act as transformations.

Method Description ——– ———–-

In addition, passing any built-in aggregation method as a string to `transform` (see the next section) will broadcast the result across the group, producing a transformed result. If the aggregation method has an efficient implementation, this will be performant as well.

The `transform` method

Similar to the aggregation method, the `transform` method can accept string aliases to the built-in transformation methods in the previous section. It can *also* accept string aliases to the built-in

aggregation methods. When an aggregation method is provided, the result will be broadcast across the group.

```
speeds
grouped = speeds.groupby("class")[["max_speed"]]
grouped.transform("cumsum")
grouped.transform("sum")
```

In addition to string aliases, the `transform` method can also accept User-Defined Functions (UDFs). The UDF must:

- Return a result that is either the same size as the group chunk or broadcastable to the size of the group chunk (e.g., a scalar, `grouped.transform(lambda x: x.iloc[-1])`).
- Operate column-by-column on the group chunk. The transform is applied to the first group chunk using chunk.apply.
- Not perform in-place operations on the group chunk. Group chunks should be treated as immutable, and changes to a group chunk may produce unexpected results. See gotchas.udf-mutation for more information.
- (Optionally) operates on all columns of the entire group chunk at once. If this is supported, a fast path is used starting from the *second* chunk.

> Note
> Transforming by supplying `transform` with a UDF is often less performant than using the built-in methods on GroupBy. Consider breaking up a complex operation into a chain of operations that utilize the built-in methods.
> All of the examples in this section can be made more performant by calling built-in methods instead of using UDFs. See below for examples.

Changed in version 2.0.0. When using `.transform` on a grouped DataFrame and the transformation function returns a DataFrame, pandas now aligns the result's index with the input's index. You can call `.to_numpy()` within the transformation function to avoid alignment.

Similar to groupby.aggregate.agg, the resulting dtype will reflect that of the transformation function. If the results from different groups have different dtypes, then a common dtype will be determined in the same way as `DataFrame` construction.

Suppose we wish to standardize the data within each group:

```
index = pd.date_range("10/1/1999", periods=1100)
ts = pd.Series(np.random.normal(0.5, 2, 1100), index)
ts = ts.rolling(window=100, min_periods=100).mean().dropna()

ts.head()
ts.tail()

transformed = ts.groupby(lambda x: x.year).transform(
    lambda x: (x - x.mean()) / x.std()
)
```

We would expect the result to now have mean 0 and standard deviation 1 within each group (up to floating-point error), which we can easily check:

```
# Original Data
grouped = ts.groupby(lambda x: x.year)
grouped.mean()
grouped.std()

# Transformed Data
grouped_trans = transformed.groupby(lambda x: x.year)
```

```
grouped_trans.mean()
grouped_trans.std()
```

We can also visually compare the original and transformed data sets.

```
compare = pd.DataFrame({"Original": ts, "Transformed": transformed})

@savefig groupby_transform_plot.png
compare.plot()
```

Transformation functions that have lower dimension outputs are broadcast to match the shape of the input array.

```
ts.groupby(lambda x: x.year).transform(lambda x: x.max() - x.min())
```

Another common data transform is to replace missing data with the group mean.

```
cols = ["A", "B", "C"]
values = np.random.randn(1000, 3)
values[np.random.randint(0, 1000, 100), 0] = np.nan
values[np.random.randint(0, 1000, 50), 1] = np.nan
values[np.random.randint(0, 1000, 200), 2] = np.nan
data_df = pd.DataFrame(values, columns=cols)
data_df

countries = np.array(["US", "UK", "GR", "JP"])
key = countries[np.random.randint(0, 4, 1000)]

grouped = data_df.groupby(key)

# Non-NA count in each group
grouped.count()

transformed = grouped.transform(lambda x: x.fillna(x.mean()))
```

We can verify that the group means have not changed in the transformed data, and that the transformed data contains no NAs.

```
grouped_trans = transformed.groupby(key)

grouped.mean()  # original group means
grouped_trans.mean()  # transformation did not change group means

grouped.count()  # original has some missing data points
grouped_trans.count()  # counts after transformation
grouped_trans.size()  # Verify non-NA count equals group size
```

As mentioned in the note above, each of the examples in this section can be computed more efficiently using built-in methods. In the code below, the inefficient way using a UDF is commented out and the faster alternative appears below.

```
# result = ts.groupby(lambda x: x.year).transform(
#     lambda x: (x - x.mean()) / x.std()
# )
grouped = ts.groupby(lambda x: x.year)
result = (ts - grouped.transform("mean")) / grouped.transform("std")

# result = ts.groupby(lambda x: x.year).transform(lambda x: x.max() - x.min())
grouped = ts.groupby(lambda x: x.year)
result = grouped.transform("max") - grouped.transform("min")
```

```
# grouped = data_df.groupby(key)
# result = grouped.transform(lambda x: x.fillna(x.mean()))
grouped = data_df.groupby(key)
result = data_df.fillna(grouped.transform("mean"))
```

Window and resample operations

It is possible to use `resample()`, `expanding()` and `rolling()` as methods on groupbys.

The example below will apply the `rolling()` method on the samples of the column B, based on the groups of column A.

```
df_re = pd.DataFrame({"A": [1] * 10 + [5] * 10, "B": np.arange(20)})
df_re

df_re.groupby("A").rolling(4).B.mean()
```

The `expanding()` method will accumulate a given operation (`sum()` in the example) for all the members of each particular group.

```
df_re.groupby("A").expanding().sum()
```

Suppose you want to use the `resample()` method to get a daily frequency in each group of your dataframe, and wish to complete the missing values with the `ffill()` method.

```
df_re = pd.DataFrame(
    {
        "date": pd.date_range(start="2016-01-01", periods=4, freq="W"),
        "group": [1, 1, 2, 2],
        "val": [5, 6, 7, 8],
    }
).set_index("date")
df_re

df_re.groupby("group").resample("1D").ffill()
```

Filtration

A filtration is a GroupBy operation that subsets the original grouping object. It may either filter out entire groups, part of groups, or both. Filtrations return a filtered version of the calling object, including the grouping columns when provided. In the following example, `class` is included in the result.

```
speeds
speeds.groupby("class").nth(1)
```

> Note
> Unlike aggregations, filtrations do not add the group keys to the index of the result. Because of this, passing `as_index=False` or `sort=True` will not affect these methods.

Filtrations will respect subsetting the columns of the GroupBy object.

```
speeds.groupby("class")[["order", "max_speed"]].nth(1)
```

Built-in filtrations

The following methods on GroupBy act as filtrations. All these methods have an efficient, GroupBy-specific, implementation.

Method Description ——– ———-

Users can also use transformations along with Boolean indexing to construct complex filtrations within groups. For example, suppose we are given groups of products and their volumes, and we wish to subset the data to only the largest products capturing no more than 90% of the total volume within each group.

```
product_volumes = pd.DataFrame(
    {
        "group": list("xxxxyyy"),
        "product": list("abcdefg"),
        "volume": [10, 30, 20, 15, 40, 10, 20],
    }
)
product_volumes

# Sort by volume to select the largest products first
product_volumes = product_volumes.sort_values("volume", ascending=False)
grouped = product_volumes.groupby("group")["volume"]
cumpct = grouped.cumsum() / grouped.transform("sum")
cumpct
significant_products = product_volumes[cumpct <= 0.9]
significant_products.sort_values(["group", "product"])
```

The `filter` method

> Note
> Filtering by supplying `filter` with a User-Defined Function (UDF) is often less performant than using the built-in methods on GroupBy. Consider breaking up a complex operation into a chain of operations that utilize the built-in methods.

The `filter` method takes a User-Defined Function (UDF) that, when applied to an entire group, returns either `True` or `False`. The result of the `filter` method is then the subset of groups for which the UDF returned `True`.

Suppose we want to take only elements that belong to groups with a group sum greater than 2.

```
sf = pd.Series([1, 1, 2, 3, 3, 3])
sf.groupby(sf).filter(lambda x: x.sum() > 2)
```

Another useful operation is filtering out elements that belong to groups with only a couple members.

```
dff = pd.DataFrame({"A": np.arange(8), "B": list("aabbbbcc")})
dff.groupby("B").filter(lambda x: len(x) > 2)
```

Alternatively, instead of dropping the offending groups, we can return a like-indexed objects where the groups that do not pass the filter are filled with NaNs.

```
dff.groupby("B").filter(lambda x: len(x) > 2, dropna=False)
```

For DataFrames with multiple columns, filters should explicitly specify a column as the filter criterion.

```
dff["C"] = np.arange(8)
dff.groupby("B").filter(lambda x: len(x["C"]) > 2)
```

Flexible `apply`

Some operations on the grouped data might not fit into the aggregation, transformation, or filtration categories. For these, you can use the `apply` function.

> Warning
> `apply` has to try to infer from the result whether it should act as a reducer, transformer, *or* filter, depending on exactly what is passed to it. Thus the grouped column(s) may be included in the output or not. While it tries to intelligently guess how to behave, it can sometimes guess wrong.

> Note
> All of the examples in this section can be more reliably, and more efficiently, computed using other pandas functionality.

```
df
grouped = df.groupby("A")

# could also just call .describe()
grouped["C"].apply(lambda x: x.describe())
```

The dimension of the returned result can also change:

```
grouped = df.groupby('A')['C']

def f(group):
    return pd.DataFrame({'original': group,
                         'demeaned': group - group.mean()})

grouped.apply(f)
```

`apply` on a Series can operate on a returned value from the applied function that is itself a series, and possibly upcast the result to a DataFrame:

```
def f(x):
    return pd.Series([x, x ** 2], index=["x", "x^2"])


s = pd.Series(np.random.rand(5))
s
s.apply(f)
```

Similar to groupby.aggregate.agg, the resulting dtype will reflect that of the apply function. If the results from different groups have different dtypes, then a common dtype will be determined in the same way as `DataFrame` construction.

Control grouped column(s) placement with `group_keys`

To control whether the grouped column(s) are included in the indices, you can use the argument `group_keys` which defaults to `True`. Compare

```
df.groupby("A", group_keys=True).apply(lambda x: x)
```

with

```
df.groupby("A", group_keys=False).apply(lambda x: x)
```

Numba accelerated routines

If Numba is installed as an optional dependency, the `transform` and `aggregate` methods support `engine='numba'` and `engine_kwargs` arguments. See enhancing performance with Numba for general usage of the arguments and performance considerations.

The function signature must start with `values, index` **exactly** as the data belonging to each group will be passed into `values`, and the group index will be passed into `index`.

> Warning
> When using `engine='numba'`, there will be no "fall back" behavior internally. The group data and group index will be passed as NumPy arrays to the JITed user defined function, and no alternative execution attempts will be tried.

Other useful features

Exclusion of non-numeric columns

Again consider the example DataFrame we've been looking at:

```
df
```

Suppose we wish to compute the standard deviation grouped by the `A` column. There is a slight problem, namely that we don't care about the data in column `B` because it is not numeric. You can avoid non-numeric columns by specifying `numeric_only=True`:

```
df.groupby("A").std(numeric_only=True)
```

Note that `df.groupby('A').colname.std().` is more efficient than `df.groupby('A').std().colname`. So if the result of an aggregation function is only needed over one column (here `colname`), it may be filtered *before* applying the aggregation function.

```
from decimal import Decimal

df_dec = pd.DataFrame(
    {
        "id": [1, 2, 1, 2],
        "int_column": [1, 2, 3, 4],
        "dec_column": [
            Decimal("0.50"),
            Decimal("0.15"),
            Decimal("0.25"),
            Decimal("0.40"),
        ],
    }
)
df_dec.groupby(["id"])[["dec_column"]].sum()
```

Handling of (un)observed Categorical values

When using a `Categorical` grouper (as a single grouper, or as part of multiple groupers), the `observed` keyword controls whether to return a cartesian product of all possible groupers values (`observed=False`) or only those that are observed groupers (`observed=True`).

Show all values:

```
pd.Series([1, 1, 1]).groupby(
    pd.Categorical(["a", "a", "a"], categories=["a", "b"]), observed=False
).count()
```

Show only the observed values:

```
pd.Series([1, 1, 1]).groupby(
    pd.Categorical(["a", "a", "a"], categories=["a", "b"]), observed=True
).count()
```

The returned dtype of the grouped will *always* include *all* of the categories that were grouped.

```
s = (
    pd.Series([1, 1, 1])
    .groupby(pd.Categorical(["a", "a", "a"], categories=["a", "b"]), observed=True)
    .count()
)
s.index.dtype
```

NA group handling

By `NA`, we are referring to any `NA` values, including `NA`, `NaN`, `NaT`, and `None`. If there are any `NA` values in the grouping key, by default these will be excluded. In other words, any "`NA` group" will be dropped. You can include NA groups by specifying `dropna=False`.

```
df = pd.DataFrame({"key": [1.0, 1.0, np.nan, 2.0, np.nan], "A": [1, 2, 3, 4, 5]})
df

df.groupby("key", dropna=True).sum()

df.groupby("key", dropna=False).sum()
```

Grouping with ordered factors

Categorical variables represented as instances of pandas's `Categorical` class can be used as group keys. If so, the order of the levels will be preserved. When `observed=False` and `sort=False`, any unobserved categories will be at the end of the result in order.

```
days = pd.Categorical(
    values=["Wed", "Mon", "Thu", "Mon", "Wed", "Sat"],
    categories=["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],
)
data = pd.DataFrame(
   {
       "day": days,
       "workers": [3, 4, 1, 4, 2, 2],
   }
)
data

data.groupby("day", observed=False, sort=True).sum()
```

```
data.groupby("day", observed=False, sort=False).sum()
```

Grouping with a grouper specification

You may need to specify a bit more data to properly group. You can use the `pd.Grouper` to provide this local control.

```
import datetime

df = pd.DataFrame(
    {
        "Branch": "A A A A A A A B".split(),
        "Buyer": "Carl Mark Carl Carl Joe Joe Joe Carl".split(),
        "Quantity": [1, 3, 5, 1, 8, 1, 9, 3],
        "Date": [
            datetime.datetime(2013, 1, 1, 13, 0),
            datetime.datetime(2013, 1, 1, 13, 5),
            datetime.datetime(2013, 10, 1, 20, 0),
            datetime.datetime(2013, 10, 2, 10, 0),
            datetime.datetime(2013, 10, 1, 20, 0),
            datetime.datetime(2013, 10, 2, 10, 0),
            datetime.datetime(2013, 12, 2, 12, 0),
            datetime.datetime(2013, 12, 2, 14, 0),
        ],
    }
)

df
```

Groupby a specific column with the desired frequency. This is like resampling.

```
df.groupby([pd.Grouper(freq="1ME", key="Date"), "Buyer"])[["Quantity"]].sum()
```

When `freq` is specified, the object returned by `pd.Grouper` will be an instance of `pandas.api.typing.TimeGrouper`. When there is a column and index with the same name, you can use `key` to group by the column and `level` to group by the index.

```
df = df.set_index("Date")
df["Date"] = df.index + pd.offsets.MonthEnd(2)
df.groupby([pd.Grouper(freq="6ME", key="Date"), "Buyer"])[["Quantity"]].sum()

df.groupby([pd.Grouper(freq="6ME", level="Date"), "Buyer"])[["Quantity"]].sum()
```

Taking the first rows of each group

Just like for a DataFrame or Series you can call head and tail on a groupby:

```
df = pd.DataFrame([[1, 2], [1, 4], [5, 6]], columns=["A", "B"])
df

g = df.groupby("A")
g.head(1)

g.tail(1)
```

This shows the first or last n rows from each group.

Taking the nth row of each group

To select the nth item from each group, use `.DataFrameGroupBy.nth` or `.SeriesGroupBy.nth`. Arguments supplied can be any integer, lists of integers, slices, or lists of slices; see below for examples. When the nth element of a group does not exist an error is *not* raised; instead no corresponding rows are returned.

In general this operation acts as a filtration. In certain cases it will also return one row per group, making it also a reduction. However because in general it can return zero or multiple rows per group, pandas treats it as a filtration in all cases.

```
df = pd.DataFrame([[1, np.nan], [1, 4], [5, 6]], columns=["A", "B"])
g = df.groupby("A")

g.nth(0)
g.nth(-1)
g.nth(1)
```

If the nth element of a group does not exist, then no corresponding row is included in the result. In particular, if the specified `n` is larger than any group, the result will be an empty DataFrame.

```
g.nth(5)
```

If you want to select the nth not-null item, use the `dropna` kwarg. For a DataFrame this should be either `'any'` or `'all'` just like you would pass to dropna:

```
# nth(0) is the same as g.first()
g.nth(0, dropna="any")
g.first()

# nth(-1) is the same as g.last()
g.nth(-1, dropna="any")
g.last()

g.B.nth(0, dropna="all")
```

You can also select multiple rows from each group by specifying multiple nth values as a list of ints.

```
business_dates = pd.date_range(start="4/1/2014", end="6/30/2014", freq="B")
df = pd.DataFrame(1, index=business_dates, columns=["a", "b"])
# get the first, 4th, and last date index for each month
df.groupby([df.index.year, df.index.month]).nth([0, 3, -1])
```

You may also use slices or lists of slices.

```
df.groupby([df.index.year, df.index.month]).nth(slice(1, None))
df.groupby([df.index.year, df.index.month]).nth([slice(1, None), slice(None, -1)])
```

Enumerate group items

To see the order in which each row appears within its group, use the `cumcount` method:

```
dfg = pd.DataFrame(list("aaabba"), columns=["A"])
dfg

dfg.groupby("A").cumcount()

dfg.groupby("A").cumcount(ascending=False)
```

Enumerate groups

To see the ordering of the groups (as opposed to the order of rows within a group given by `cumcount`) you can use `.DataFrameGroupBy.ngroup`.

Note that the numbers given to the groups match the order in which the groups would be seen when iterating over the groupby object, not the order they are first observed.

```
dfg = pd.DataFrame(list("aaabba"), columns=["A"])
dfg

dfg.groupby("A").ngroup()

dfg.groupby("A").ngroup(ascending=False)
```

Plotting

Groupby also works with some plotting methods. In this case, suppose we suspect that the values in column 1 are 3 times higher on average in group "B".

```
np.random.seed(1234)
df = pd.DataFrame(np.random.randn(50, 2))
df["g"] = np.random.choice(["A", "B"], size=50)
df.loc[df["g"] == "B", 1] += 3
```

We can easily visualize this with a boxplot:

```
@savefig groupby_boxplot.png
df.groupby("g").boxplot()
```

The result of calling `boxplot` is a dictionary whose keys are the values of our grouping column `g` ("A" and "B"). The values of the resulting dictionary can be controlled by the `return_type` keyword of `boxplot`. See the visualization documentation for more.

> Warning
> For historical reasons, `df.groupby("g").boxplot()` is not equivalent to `df.boxplot(by="g")`. See here for an explanation.

Piping function calls

Similar to the functionality provided by `DataFrame` and `Series`, functions that take `GroupBy` objects can be chained together using a `pipe` method to allow for a cleaner, more readable syntax. To read about `.pipe` in general terms, see here.

Combining `.groupby` and `.pipe` is often useful when you need to reuse GroupBy objects.

As an example, imagine having a DataFrame with columns for stores, products, revenue and quantity sold. We'd like to do a groupwise calculation of *prices* (i.e. revenue/quantity) per store and per product. We could do this in a multi-step operation, but expressing it in terms of piping can make the code more readable. First we set the data:

```
n = 1000
df = pd.DataFrame(
    {
        "Store": np.random.choice(["Store_1", "Store_2"], n),
        "Product": np.random.choice(["Product_1", "Product_2"], n),
        "Revenue": (np.random.random(n) * 50 + 10).round(2),
        "Quantity": np.random.randint(1, 10, size=n),
    }
```

```
)
df.head(2)
```

We now find the prices per store/product.

```
(
    df.groupby(["Store", "Product"])
    .pipe(lambda grp: grp.Revenue.sum() / grp.Quantity.sum())
    .unstack()
    .round(2)
)
```

Piping can also be expressive when you want to deliver a grouped object to some arbitrary function, for example:

```
def mean(groupby):
    return groupby.mean()


df.groupby(["Store", "Product"]).pipe(mean)
```

Here `mean` takes a GroupBy object and finds the mean of the Revenue and Quantity columns respectively for each Store-Product combination. The `mean` function can be any function that takes in a GroupBy object; the `.pipe` will pass the GroupBy object as a parameter into the function you specify.

Examples

Multi-column factorization

By using `.DataFrameGroupBy.ngroup`, we can extract information about the groups in a way similar to `factorize` (as described further in the reshaping API) but which applies naturally to multiple columns of mixed type and different sources. This can be useful as an intermediate categorical-like step in processing, when the relationships between the group rows are more important than their content, or as input to an algorithm which only accepts the integer encoding. (For more information about support in pandas for full categorical data, see the `Categorical introduction <categorical>` and the API documentation.)

```
dfg = pd.DataFrame({"A": [1, 1, 2, 3, 2], "B": list("aaaba")})

dfg

dfg.groupby(["A", "B"]).ngroup()

dfg.groupby(["A", [0, 0, 0, 1, 1]]).ngroup()
```

GroupBy by indexer to 'resample' data

Resampling produces new hypothetical samples (resamples) from already existing observed data or from a model that generates data. These new samples are similar to the pre-existing samples.

In order for resample to work on indices that are non-datetimelike, the following procedure can be utilized.

In the following examples, **df.index // 5** returns an integer array which is used to determine what gets selected for the groupby operation.

Note
The example below shows how we can downsample by consolidation of samples into fewer ones. Here by using **df.index // 5**, we are aggregating the samples in bins. By applying **std()** function, we aggregate the information contained in many samples into a small subset of values which is their standard deviation thereby reducing the number of samples.

```
df = pd.DataFrame(np.random.randn(10, 2))
df
df.index // 5
df.groupby(df.index // 5).std()
```

Returning a Series to propagate names

Group DataFrame columns, compute a set of metrics and return a named Series. The Series name is used as the name for the column index. This is especially useful in conjunction with reshaping operations such as stacking, in which the column index name will be used as the name of the inserted column:

```
df = pd.DataFrame(
    {
        "a": [0, 0, 0, 0, 1, 1, 1, 1, 2, 2, 2, 2],
        "b": [0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1],
        "c": [1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0],
        "d": [0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1],
    }
)

def compute_metrics(x):
    result = {"b_sum": x["b"].sum(), "c_mean": x["c"].mean()}
    return pd.Series(result, name="metrics")

result = df.groupby("a").apply(compute_metrics)

result

result.stack()
```

Merge, join, concatenate and compare

Source: https://pandas.pydata.org/docs/user_guide/merging.html

pandas provides various methods for combining and comparing `Series` or `DataFrame`.

- `concat`: Merge multiple `Series` or `DataFrame` objects along a shared index or column
- `DataFrame.join`: Merge multiple `DataFrame` objects along the columns
- `DataFrame.combine_first`: Update missing values with non-missing values in the same location
- `merge`: Combine two `Series` or `DataFrame` objects with SQL-style joining
- `merge_ordered`: Combine two `Series` or `DataFrame` objects along an ordered axis
- `merge_asof`: Combine two `Series` or `DataFrame` objects by near instead of exact matching keys
- `Series.compare` and `DataFrame.compare`: Show differences in values between two `Series` or `DataFrame` objects

concat

The `concat` function concatenates an arbitrary amount of `Series` or `DataFrame` objects along an axis while performing optional set logic (union or intersection) of the indexes on the other axes. Like `numpy.concatenate`, `concat` takes a list or dict of homogeneously-typed objects and concatenates them.

```
df1 = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    },
    index=[0, 1, 2, 3],
)

df2 = pd.DataFrame(
    {
        "A": ["A4", "A5", "A6", "A7"],
        "B": ["B4", "B5", "B6", "B7"],
        "C": ["C4", "C5", "C6", "C7"],
        "D": ["D4", "D5", "D6", "D7"],
    },
    index=[4, 5, 6, 7],
)

df3 = pd.DataFrame(
    {
        "A": ["A8", "A9", "A10", "A11"],
        "B": ["B8", "B9", "B10", "B11"],
        "C": ["C8", "C9", "C10", "C11"],
        "D": ["D8", "D9", "D10", "D11"],
    },
    index=[8, 9, 10, 11],
)

frames = [df1, df2, df3]
```

```
result = pd.concat(frames)
result
```

Note
reusing `concat` can create unnecessary copies. Collect all

```
frames = [process_your_file(f) for f in files]
result = pd.concat(frames)
```

Note
When concatenating `DataFrame` with named axes, pandas will attempt to preserve these index/column names whenever possible. In the case where all inputs share a common name, this name will be assigned to the result. When the input names do not all agree, the result will be unnamed. The same is true for `MultiIndex`, but the logic is applied separately on a level-by-level basis.

Joining logic of the resulting axis

The `join` keyword specifies how to handle axis values that don't exist in the first `DataFrame`.

`join='outer'` takes the union of all axis values.

```
df4 = pd.DataFrame(
    {
        "B": ["B2", "B3", "B6", "B7"],
        "D": ["D2", "D3", "D6", "D7"],
        "F": ["F2", "F3", "F6", "F7"],
    },
    index=[2, 3, 6, 7],
)
result = pd.concat([df1, df4], axis=1)
result
```

`join='inner'` takes the intersection of the axis values.

```
result = pd.concat([df1, df4], axis=1, join="inner")
result
```

To perform an effective "left" join using the *exact index* from the original `DataFrame`, result can be reindexed.

```
result = pd.concat([df1, df4], axis=1).reindex(df1.index)
result
```

Ignoring indexes on the concatenation axis

For `DataFrame` objects which don't have a meaningful index, the `ignore_index` ignores overlapping indexes.

```
result = pd.concat([df1, df4], ignore_index=True, sort=False)
result
```

Concatenating `Series` and `DataFrame` together

You can concatenate a mix of `Series` and `DataFrame` objects. The `Series` will be transformed to `DataFrame` with the column name as the name of the `Series`.

```
s1 = pd.Series(["X0", "X1", "X2", "X3"], name="X")
result = pd.concat([df1, s1], axis=1)
result
```

Unnamed `Series` will be numbered consecutively.

```
s2 = pd.Series(["_0", "_1", "_2", "_3"])
result = pd.concat([df1, s2, s2, s2], axis=1)
result
```

`ignore_index=True` will drop all name references.

```
result = pd.concat([df1, s1], axis=1, ignore_index=True)
result
```

Resulting keys

The `keys` argument adds another axis level to the resulting index or column (creating a `MultiIndex`) associate specific keys with each original `DataFrame`.

```
result = pd.concat(frames, keys=["x", "y", "z"])
result
result.loc["y"]
```

The `keys` argument can override the column names when creating a new `DataFrame` based on existing `Series`.

```
s3 = pd.Series([0, 1, 2, 3], name="foo")
s4 = pd.Series([0, 1, 2, 3])
s5 = pd.Series([0, 1, 4, 5])

pd.concat([s3, s4, s5], axis=1)
pd.concat([s3, s4, s5], axis=1, keys=["red", "blue", "yellow"])
```

You can also pass a dict to `concat` in which case the dict keys will be used for the `keys` argument unless other `keys` argument is specified:

```
pieces = {"x": df1, "y": df2, "z": df3}
result = pd.concat(pieces)
result
```

```
result = pd.concat(pieces, keys=["z", "y"])
result
```

The `MultiIndex` created has levels that are constructed from the passed keys and the index of the `DataFrame` pieces:

```
result.index.levels
```

`levels` argument allows specifying resulting levels associated with the `keys`.

```
result = pd.concat(
    pieces, keys=["x", "y", "z"], levels=[["z", "y", "x", "w"]], names=["group_key"]
)
result
```

```
result.index.levels
```

Appending rows to a DataFrame

If you have a Series that you want to append as a single row to a DataFrame, you can convert the row into a DataFrame and use concat.

```
s2 = pd.Series(["X0", "X1", "X2", "X3"], index=["A", "B", "C", "D"])
result = pd.concat([df1, s2.to_frame().T], ignore_index=True)
result
```

merge

merge performs join operations similar to relational databases like SQL. Users who are familiar with SQL but new to pandas can reference a comparison with SQL.

Merge types

merge implements common SQL style joining operations.

- **one-to-one**: joining two DataFrame objects on their indexes which must contain unique values.
- **many-to-one**: joining a unique index to one or more columns in a different DataFrame.
- **many-to-many**: joining columns on columns.

> Note
> When joining columns on columns, potentially a many-to-many join, any indexes on the passed DataFrame objects **will be discarded**.

For a **many-to-many** join, if a key combination appears more than once in both tables, the DataFrame will have the **Cartesian product** of the associated data.

```
left = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
    }
)

right = pd.DataFrame(
    {
        "key": ["K0", "K1", "K2", "K3"],
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
    }
)
result = pd.merge(left, right, on="key")
result
```

The how argument to merge specifies which keys are included in the resulting table. If a key combination **does not appear** in either the left or right tables, the values in the joined table will be NA. Here is a summary of the how options and their SQL equivalent names:

Merge method	SQL Join Name	Description
left	LEFT OUTER JOIN	Use keys from left frame only
right	RIGHT OUTER JOIN	Use keys from right frame only
outer	FULL OUTER JOIN	Use union of keys from both frames

Merge method	SQL Join Name	Description
inner	INNER JOIN	Use intersection of keys from both frames
cross	CROSS JOIN	Create the cartesian product of rows of both frames

```
left = pd.DataFrame(
   {
      "key1": ["K0", "K0", "K1", "K2"],
      "key2": ["K0", "K1", "K0", "K1"],
      "A": ["A0", "A1", "A2", "A3"],
      "B": ["B0", "B1", "B2", "B3"],
   }
)
right = pd.DataFrame(
   {
      "key1": ["K0", "K1", "K1", "K2"],
      "key2": ["K0", "K0", "K0", "K0"],
      "C": ["C0", "C1", "C2", "C3"],
      "D": ["D0", "D1", "D2", "D3"],
   }
)
result = pd.merge(left, right, how="left", on=["key1", "key2"])
result
```

```
result = pd.merge(left, right, how="right", on=["key1", "key2"])
result
```

```
result = pd.merge(left, right, how="outer", on=["key1", "key2"])
result
```

```
result = pd.merge(left, right, how="inner", on=["key1", "key2"])
result
```

```
result = pd.merge(left, right, how="cross")
result
```

You can merge `Series` and a `DataFrame` with a `MultiIndex` if the names of the `MultiIndex` correspond to the columns from the `DataFrame`. You can also transform the `Series` to a `DataFrame` using `Series.reset_index` before merging:

```
df = pd.DataFrame({"Let": ["A", "B", "C"], "Num": [1, 2, 3]})
df

ser = pd.Series(
    ["a", "b", "c", "d", "e", "f"],
    index=pd.MultiIndex.from_arrays(
        [["A", "B", "C"] * 2, [1, 2, 3, 4, 5, 6]], names=["Let", "Num"]
    ),
)
ser

pd.merge(df, ser.reset_index(), on=["Let", "Num"])
```

Performing an outer join with duplicate join keys in `DataFrame`:

```
left = pd.DataFrame({"A": [1, 2], "B": [2, 2]})

right = pd.DataFrame({"A": [4, 5, 6], "B": [2, 2, 2]})
```

```
result = pd.merge(left, right, on="B", how="outer")
result
```

> Warning
> Merging on duplicate keys significantly increase the dimensions of the result and can cause a memory overflow.

Merge key uniqueness

The `validate` argument checks whether the uniqueness of merge keys. Key uniqueness is checked before merge operations and can protect against memory overflows and unexpected key duplication.

```
left = pd.DataFrame({"A": [1, 2], "B": [1, 2]})
right = pd.DataFrame({"A": [4, 5, 6], "B": [2, 2, 2]})
result = pd.merge(left, right, on="B", how="outer", validate="one_to_one")
```

If the user is aware of the duplicates in the right `DataFrame` but wants to ensure there are no duplicates in the left `DataFrame`, one can use the `validate='one_to_many'` argument instead, which will not raise an exception.

```
pd.merge(left, right, on="B", how="outer", validate="one_to_many")
```

Merge result indicator

`merge` accepts the argument `indicator`. If `True`, a Categorical-type column called `_merge` will be added to the output object that takes on values:

Observation Origin	`_merge` value
Merge key only in `'left'` frame	`left_only`
Merge key only in `'right'` frame	`right_only`
Merge key in both frames	`both`

```
df1 = pd.DataFrame({"col1": [0, 1], "col_left": ["a", "b"]})
df2 = pd.DataFrame({"col1": [1, 2, 2], "col_right": [2, 2, 2]})
pd.merge(df1, df2, on="col1", how="outer", indicator=True)
```

A string argument to `indicator` will use the value as the name for the indicator column.

```
pd.merge(df1, df2, on="col1", how="outer", indicator="indicator_column")
```

Overlapping value columns

The merge `suffixes` argument takes a tuple or list of strings to append to overlapping column names in the input `DataFrame` to disambiguate the result columns:

```
left = pd.DataFrame({"k": ["K0", "K1", "K2"], "v": [1, 2, 3]})
right = pd.DataFrame({"k": ["K0", "K0", "K3"], "v": [4, 5, 6]})

result = pd.merge(left, right, on="k")
result
```

```
result = pd.merge(left, right, on="k", suffixes=("_l", "_r"))
result
```

DataFrame.join

`DataFrame.join` combines the columns of multiple, potentially differently-indexed `DataFrame` into a single result `DataFrame`.

```
left = pd.DataFrame(
    {"A": ["A0", "A1", "A2"], "B": ["B0", "B1", "B2"]}, index=["K0", "K1", "K2"]
)

right = pd.DataFrame(
    {"C": ["C0", "C2", "C3"], "D": ["D0", "D2", "D3"]}, index=["K0", "K2", "K3"]
)

result = left.join(right)
result
```

```
result = left.join(right, how="outer")
result
```

```
result = left.join(right, how="inner")
result
```

`DataFrame.join` takes an optional `on` argument which may be a column or multiple column names that the passed `DataFrame` is to be aligned.

```
left = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
        "key": ["K0", "K1", "K0", "K1"],
    }
)

right = pd.DataFrame({"C": ["C0", "C1"], "D": ["D0", "D1"]}, index=["K0", "K1"])

result = left.join(right, on="key")
result
```

```
result = pd.merge(
    left, right, left_on="key", right_index=True, how="left", sort=False
)
result
```

To join on multiple keys, the passed `DataFrame` must have a `MultiIndex`:

```
left = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
        "key1": ["K0", "K0", "K1", "K2"],
        "key2": ["K0", "K1", "K0", "K1"],
    }
)

index = pd.MultiIndex.from_tuples(
```

```
    [("K0", "K0"), ("K1", "K0"), ("K2", "K0"), ("K2", "K1")]
)
right = pd.DataFrame(
    {"C": ["C0", "C1", "C2", "C3"], "D": ["D0", "D1", "D2", "D3"]}, index=index
)
result = left.join(right, on=["key1", "key2"])
result
```

The default for `DataFrame.join` is to perform a left join which uses only the keys found in the calling `DataFrame`. Other join types can be specified with `how`.

```
result = left.join(right, on=["key1", "key2"], how="inner")
result
```

Joining a single Index to a MultiIndex

You can join a `DataFrame` with a `Index` to a `DataFrame` with a `MultiIndex` on a level. The `name` of the `Index` will match the level name of the `MultiIndex`.

python

left = pd.DataFrame({”A”: [”A0”, ”A1”, ”A2”], ”B”: [”B0”, ”B1”, ”B2”]}, index=pd.Index([”K0”, ”K1”, ”K2”], name=”key”),

)

index = pd.MultiIndex.from_tuples([(”K0”, ”Y0”), (”K1”, ”Y1”), (”K2”, ”Y2”), (”K2”, ”Y3”)], names=[”key”, ”Y”],

) right = pd.DataFrame({”C”: [”C0”, ”C1”, ”C2”, ”C3”], ”D”: [”D0”, ”D1”, ”D2”, ”D3”]}, index=index,)

result = left.join(right, how=”inner”) result

Joining with two `MultiIndex`

The `MultiIndex` of the input argument must be completely used in the join and is a subset of the indices in the left argument.

```
leftindex = pd.MultiIndex.from_product(
    [list("abc"), list("xy"), [1, 2]], names=["abc", "xy", "num"]
)
left = pd.DataFrame({"v1": range(12)}, index=leftindex)
left

rightindex = pd.MultiIndex.from_product(
    [list("abc"), list("xy")], names=["abc", "xy"]
)
right = pd.DataFrame({"v2": [100 * i for i in range(1, 7)]}, index=rightindex)
right

left.join(right, on=["abc", "xy"], how="inner")
```

```
leftindex = pd.MultiIndex.from_tuples(
    [("K0", "X0"), ("K0", "X1"), ("K1", "X2")], names=["key", "X"]
)
left = pd.DataFrame(
    {"A": ["A0", "A1", "A2"], "B": ["B0", "B1", "B2"]}, index=leftindex
)
```

```
rightindex = pd.MultiIndex.from_tuples(
    [("K0", "Y0"), ("K1", "Y1"), ("K2", "Y2"), ("K2", "Y3")], names=["key", "Y"]
)
right = pd.DataFrame(
    {"C": ["C0", "C1", "C2", "C3"], "D": ["D0", "D1", "D2", "D3"]}, index=rightindex
)

result = pd.merge(
    left.reset_index(), right.reset_index(), on=["key"], how="inner"
).set_index(["key", "X", "Y"])
result
```

Merging on a combination of columns and index levels

Strings passed as the `on`, `left_on`, and `right_on` parameters may refer to either column names or index level names. This enables merging `DataFrame` instances on a combination of index levels and columns without resetting indexes.

```
left_index = pd.Index(["K0", "K0", "K1", "K2"], name="key1")

left = pd.DataFrame(
    {
        "A": ["A0", "A1", "A2", "A3"],
        "B": ["B0", "B1", "B2", "B3"],
        "key2": ["K0", "K1", "K0", "K1"],
    },
    index=left_index,
)

right_index = pd.Index(["K0", "K1", "K2", "K2"], name="key1")

right = pd.DataFrame(
    {
        "C": ["C0", "C1", "C2", "C3"],
        "D": ["D0", "D1", "D2", "D3"],
        "key2": ["K0", "K0", "K0", "K1"],
    },
    index=right_index,
)

result = left.merge(right, on=["key1", "key2"])
result
```

> Note
> When `DataFrame` are joined on a string that matches an index level in both arguments, the index level is preserved as an index level in the resulting

> Note
> When `DataFrame` are joined using only some of the levels of a `MultiIndex`, the extra levels will be dropped from the resulting join. To preserve those levels, use `DataFrame.reset_index` on those level names to move those levels to columns prior to the join.

Joining multiple `DataFrame`

A list or tuple of `DataFrame` can also be passed to `join` to join them together on their indexes.

```
right2 = pd.DataFrame({"v": [7, 8, 9]}, index=["K1", "K1", "K2"])
result = left.join([right, right2])
```

DataFrame.combine_first

`DataFrame.combine_first` update missing values from one `DataFrame` with the non-missing values in another `DataFrame` in the corresponding location.

```
df1 = pd.DataFrame(
    [[np.nan, 3.0, 5.0], [-4.6, np.nan, np.nan], [np.nan, 7.0, np.nan]]
)
df2 = pd.DataFrame([[-42.6, np.nan, -8.2], [-5.0, 1.6, 4]], index=[1, 2])
result = df1.combine_first(df2)
result
```

merge_ordered

`merge_ordered` combines ordered data such as numeric or time series data with optional filling of missing data with `fill_method`.

```
left = pd.DataFrame(
    {"k": ["K0", "K1", "K1", "K2"], "lv": [1, 2, 3, 4], "s": ["a", "b", "c", "d"]}
)

right = pd.DataFrame({"k": ["K1", "K2", "K4"], "rv": [1, 2, 3]})

pd.merge_ordered(left, right, fill_method="ffill", left_by="s")
```

merge_asof

`merge_asof` is similar to an ordered left-join except that matches are on the nearest key rather than equal keys. For each row in the `left` `DataFrame`, the last row in the `right` `DataFrame` is selected where the `on` key is less than or equal to the left's key. Both `DataFrame` must be sorted by the key.

Optionally `merge_asof` can perform a group-wise merge by matching the `by` key in addition to the nearest match on the `on` key.

```
trades = pd.DataFrame(
    {
        "time": pd.to_datetime(
            [
                "20160525 13:30:00.023",
                "20160525 13:30:00.038",
                "20160525 13:30:00.048",
                "20160525 13:30:00.048",
                "20160525 13:30:00.048",
            ]
        ),
        "ticker": ["MSFT", "MSFT", "GOOG", "GOOG", "AAPL"],
        "price": [51.95, 51.95, 720.77, 720.92, 98.00],
        "quantity": [75, 155, 100, 100, 100],
    },
    columns=["time", "ticker", "price", "quantity"],
)
```

```
quotes = pd.DataFrame(
    {
        "time": pd.to_datetime(
            [
                "20160525 13:30:00.023",
                "20160525 13:30:00.023",
                "20160525 13:30:00.030",
                "20160525 13:30:00.041",
                "20160525 13:30:00.048",
                "20160525 13:30:00.049",
                "20160525 13:30:00.072",
                "20160525 13:30:00.075",
            ]
        ),
        "ticker": ["GOOG", "MSFT", "MSFT", "MSFT", "GOOG", "AAPL", "GOOG", "MSFT"],
        "bid": [720.50, 51.95, 51.97, 51.99, 720.50, 97.99, 720.50, 52.01],
        "ask": [720.93, 51.96, 51.98, 52.00, 720.93, 98.01, 720.88, 52.03],
    },
    columns=["time", "ticker", "bid", "ask"],
)
trades
quotes
pd.merge_asof(trades, quotes, on="time", by="ticker")
```

`merge_asof` within `2ms` between the quote time and the trade time.

```
pd.merge_asof(trades, quotes, on="time", by="ticker", tolerance=pd.Timedelta("2ms"))
```

`merge_asof` within `10ms` between the quote time and the trade time and exclude exact matches on time. Note that though we exclude the exact matches (of the quotes), prior quotes **do** propagate to that point in time.

```
pd.merge_asof(
    trades,
    quotes,
    on="time",
    by="ticker",
    tolerance=pd.Timedelta("10ms"),
    allow_exact_matches=False,
)
```

compare

The `Series.compare` and `DataFrame.compare` methods allow you to compare two `DataFrame` or `Series`, respectively, and summarize their differences.

```
df = pd.DataFrame(
    {
        "col1": ["a", "a", "b", "b", "a"],
        "col2": [1.0, 2.0, 3.0, np.nan, 5.0],
        "col3": [1.0, 2.0, 3.0, 4.0, 5.0],
    },
    columns=["col1", "col2", "col3"],
)
df
df2 = df.copy()
df2.loc[0, "col1"] = "c"
df2.loc[2, "col3"] = 4.0
```

```
df2
df.compare(df2)
```

By default, if two corresponding values are equal, they will be shown as `NaN`. Furthermore, if all values in an entire row / column are equal, that row / column will be omitted from the result. The remaining differences will be aligned on columns.

Stack the differences on rows.

```
df.compare(df2, align_axis=0)
```

Keep all original rows and columns with `keep_shape=True`.

```
df.compare(df2, keep_shape=True)
```

Keep all the original values even if they are equal.

```
df.compare(df2, keep_shape=True, keep_equal=True)
```

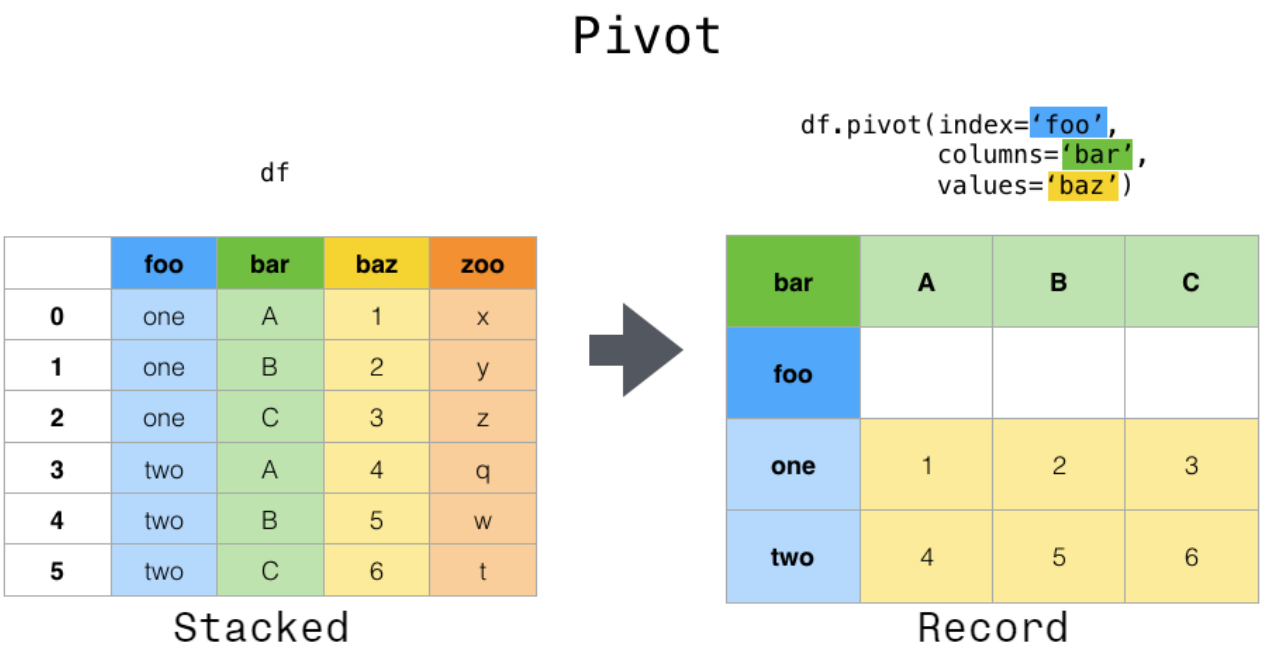
Reshaping and pivot tables

Source: https://pandas.pydata.org/docs/user_guide/reshaping.html

pandas provides methods for manipulating a `Series` and `DataFrame` to alter the representation of the data for further data processing or data summarization.

- `pivot` and `pivot_table`: Group unique values within one or more discrete categories.
- `stack` and `unstack`: Pivot a column or row level to the opposite axis respectively.
- `melt` and `wide_to_long`: Unpivot a wide `DataFrame` to a long format.
- `get_dummies` and `from_dummies`: Conversions with indicator variables.
- `explode`: Convert a column of list-like values to individual rows.
- `crosstab`: Calculate a cross-tabulation of multiple 1 dimensional factor arrays.
- `cut`: Transform continuous variables to discrete, categorical values
- `factorize`: Encode 1 dimensional variables into integer labels.

`pivot` and `pivot_table`



	foo	bar	baz	zoo
0	one	A	1	x
1	one	B	2	y
2	one	C	3	z
3	two	A	4	q
4	two	B	5	w
5	two	C	6	t

bar	A	B	C
foo			
one	1	2	3
two	4	5	6

`pivot`

Data is often stored in so-called ”stacked” or ”record” format. In a ”record” or ”wide” format, typically there is one row for each subject. In the ”stacked” or ”long” format there are multiple rows for each subject where applicable.

```
data = {
   "value": range(12),
   "variable": ["A"] * 3 + ["B"] * 3 + ["C"] * 3 + ["D"] * 3,
   "date": pd.to_datetime(["2020-01-03", "2020-01-04", "2020-01-05"] * 4)
```

```
}
df = pd.DataFrame(data)
```

To perform time series operations with each unique variable, a better representation would be where the `columns` are the unique variables and an `index` of dates identifies individual observations. To reshape the data into this form, we use the `DataFrame.pivot` method (also implemented as a top level function `pivot`):

```
pivoted = df.pivot(index="date", columns="variable", values="value")
pivoted
```

If the `values` argument is omitted, and the input `DataFrame` has more than one column of values which are not used as column or index inputs to `pivot`, then the resulting "pivoted" `DataFrame` will have `hierarchical columns <advanced.hierarchical>` whose topmost level indicates the respective value column:

```
df["value2"] = df["value"] * 2
pivoted = df.pivot(index="date", columns="variable")
pivoted
```

You can then select subsets from the pivoted `DataFrame`:

```
pivoted["value2"]
```

Note that this returns a view on the underlying data in the case where the data are homogeneously-typed.

> **Note**
> If your data contains duplicates, use `pivot_table`.

pivot_table

While `pivot` provides general purpose pivoting with various data types, pandas also provides `pivot_table` or `pivot_table` for pivoting with aggregation of numeric data.

The function `pivot_table` can be used to create spreadsheet-style pivot tables. See the cookbook for some advanced strategies.

```
import datetime

df = pd.DataFrame(
    {
        "A": ["one", "one", "two", "three"] * 6,
        "B": ["A", "B", "C"] * 8,
        "C": ["foo", "foo", "foo", "bar", "bar", "bar"] * 4,
        "D": np.random.randn(24),
        "E": np.random.randn(24),
        "F": [datetime.datetime(2013, i, 1) for i in range(1, 13)]
        + [datetime.datetime(2013, i, 15) for i in range(1, 13)],
    }
)
df
pd.pivot_table(df, values="D", index=["A", "B"], columns=["C"])
pd.pivot_table(
    df, values=["D", "E"],
    index=["B"],
    columns=["A", "C"],
    aggfunc="sum",
)
```

```
pd.pivot_table(
    df, values="E",
    index=["B", "C"],
    columns=["A"],
    aggfunc=["sum", "mean"],
)
```

The result is a `DataFrame` potentially having a `MultiIndex` on the index or column. If the `values` column name is not given, the pivot table will include all of the data in an additional level of hierarchy in the columns:

```
pd.pivot_table(df[["A", "B", "C", "D", "E"]], index=["A", "B"], columns=["C"])
```

Also, you can use `Grouper` for `index` and `columns` keywords. For detail of `Grouper`, see Grouping with a Grouper specification.

```
pd.pivot_table(df, values="D", index=pd.Grouper(freq="ME", key="F"), columns="C")
```

Adding margins

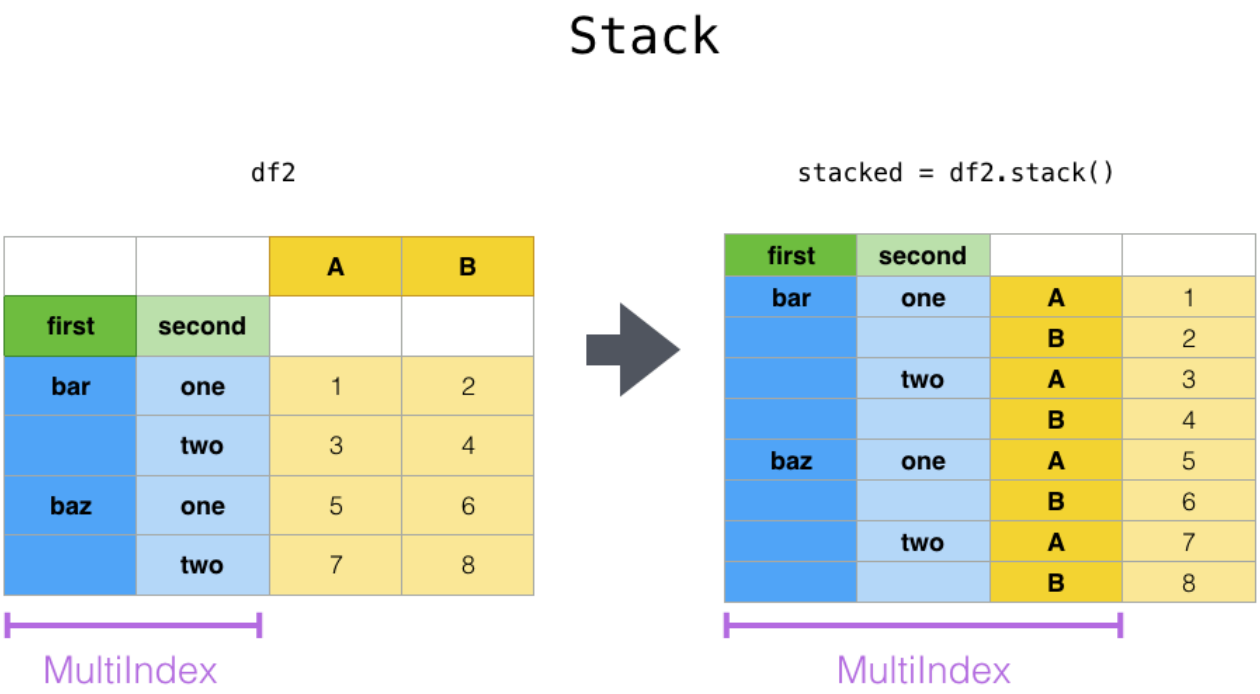
Passing `margins=True` to `pivot_table` will add a row and column with an `All` label with partial group aggregates across the categories on the rows and columns:

```
table = df.pivot_table(
    index=["A", "B"],
    columns="C",
    values=["D", "E"],
    margins=True,
    aggfunc="std"
)
table
```

Additionally, you can call `DataFrame.stack` to display a pivoted DataFrame as having a multi-level index:

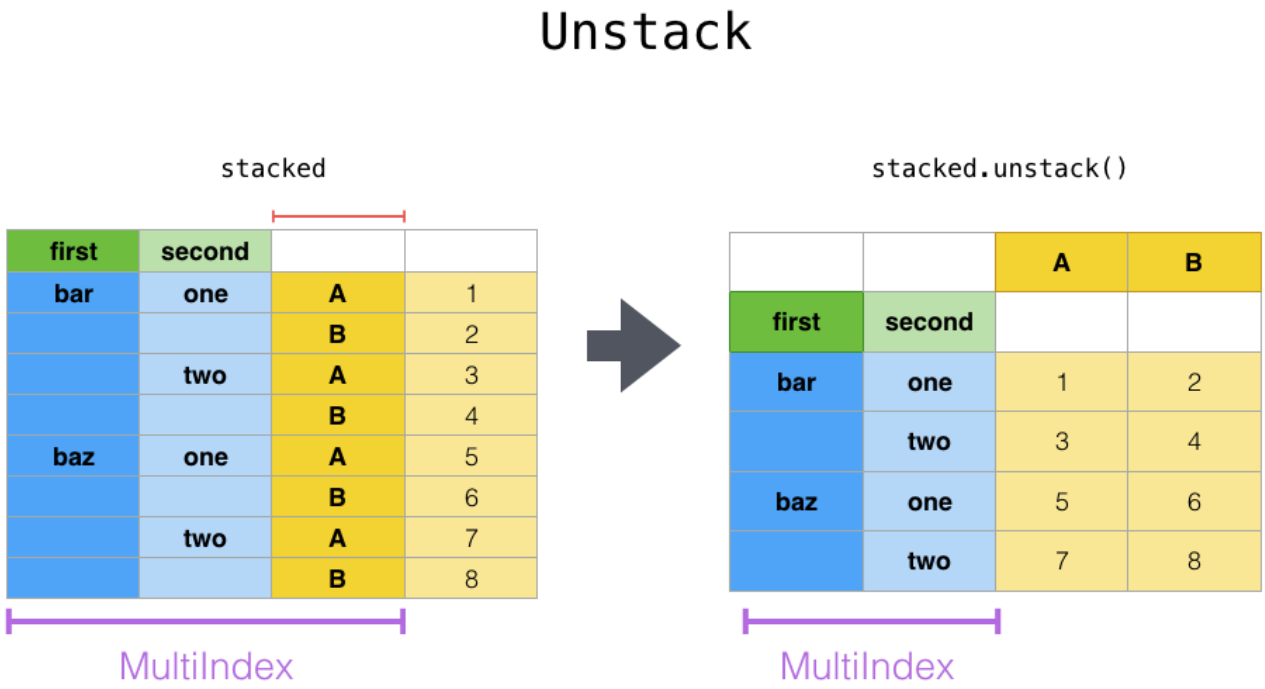
```
table.stack()
```

`stack` and `unstack`



Closely related to the `pivot` method are the related `stack` and `unstack` methods available on `Series` and `DataFrame`. These methods are designed to work together with `MultiIndex` objects (see the section on `hierarchical indexing <advanced.hierarchical>`).

- `stack`: "pivot" a level of the (possibly hierarchical) column labels, returning a `DataFrame` with an index with a new inner-most level of row labels.
- `unstack`: (inverse operation of `stack`) "pivot" a level of the (possibly hierarchical) row index to the column axis, producing a reshaped `DataFrame` with a new inner-most level of column labels.




```
tuples = [
   ["bar", "bar", "baz", "baz", "foo", "foo", "qux", "qux"],
   ["one", "two", "one", "two", "one", "two", "one", "two"],
]
index = pd.MultiIndex.from_arrays(tuples, names=["first", "second"])
df = pd.DataFrame(np.random.randn(8, 2), index=index, columns=["A", "B"])
df2 = df[:4]
df2
```

The `stack` function "compresses" a level in the `DataFrame` columns to produce either:

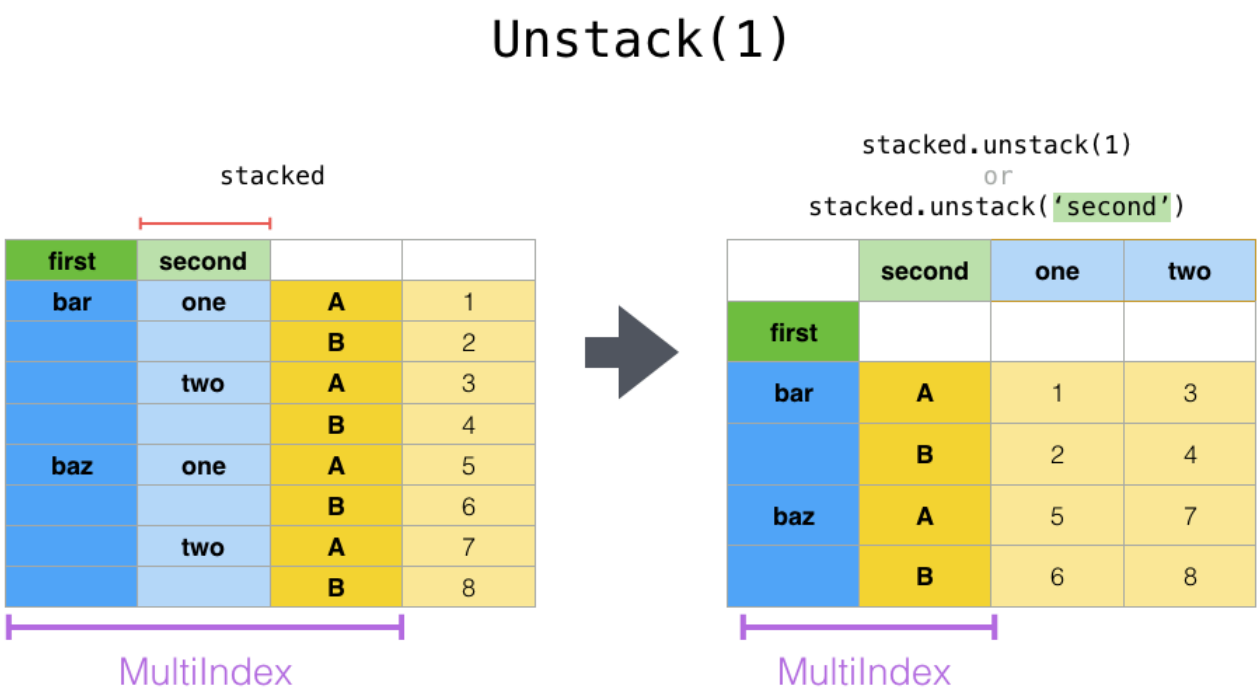
- A `Series`, in the case of a `Index` in the columns.
- A `DataFrame`, in the case of a `MultiIndex` in the columns.

If the columns have a `MultiIndex`, you can choose which level to stack. The stacked level becomes the new lowest level in a `MultiIndex` on the columns:

```
stacked = df2.stack()
stacked
```

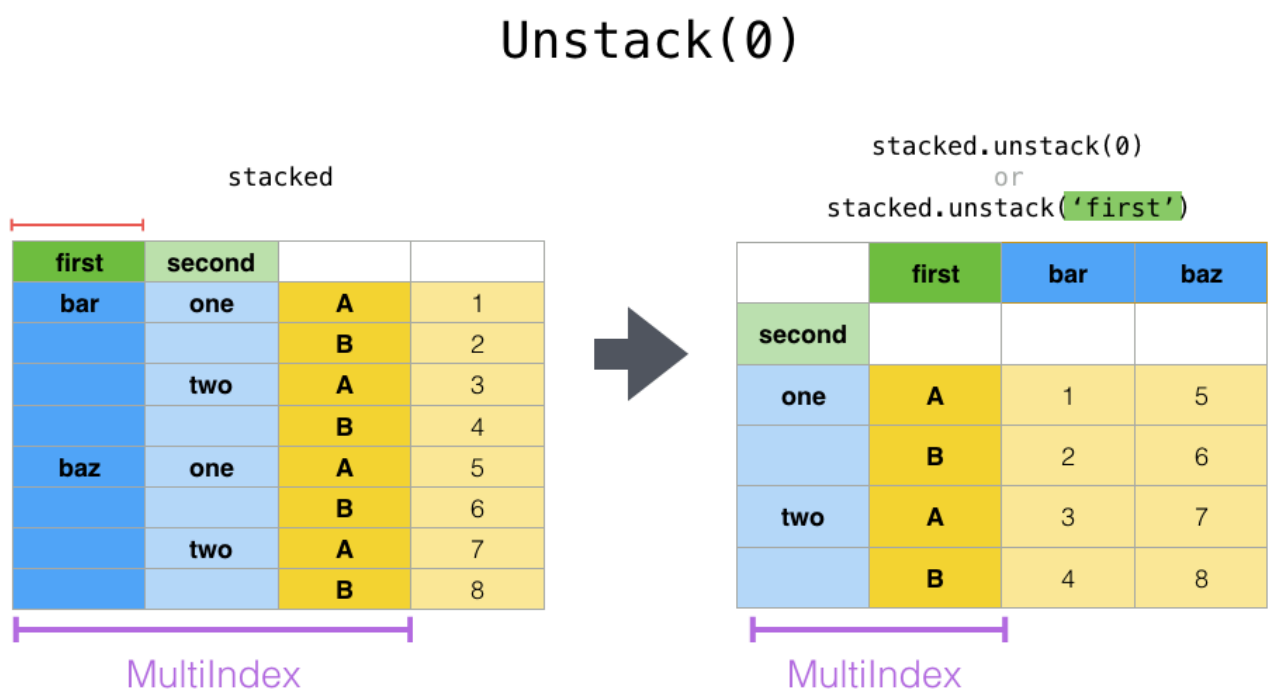
With a "stacked" `DataFrame` or `Series` (having a `MultiIndex` as the `index`), the inverse operation of `stack` is `unstack`, which by default unstacks the **last level**:

```
stacked.unstack()
stacked.unstack(1)
stacked.unstack(0)
```



If the indexes have names, you can use the level names instead of specifying the level numbers:

```
stacked.unstack("second")
```



Notice that the `unstack` method implicitly sorts the index levels involved, while `stack` preserves the order of the stacked column level. Because `unstack` sorts, a call to `stack` and then `unstack`, or vice versa, will result in a **sorted** copy of the original `DataFrame` or `Series`. Here "sorted" is with respect to the order of the `MultiIndex.levels`, which for a typical `MultiIndex` matches sorting by

value:

```
index = pd.MultiIndex.from_product([[2, 1], ["a", "b"]])
df = pd.DataFrame(np.random.randn(4), index=index, columns=["A"])
df
all(df.unstack().stack() == df.sort_index())
```

Multiple levels

You may also stack or unstack more than one level at a time by passing a list of levels, in which case the end result is as if each level in the list were processed individually.

```
columns = pd.MultiIndex.from_tuples(
    [
        ("A", "cat", "long"),
        ("B", "cat", "long"),
        ("A", "dog", "short"),
        ("B", "dog", "short"),
    ],
    names=["exp", "animal", "hair_length"],
)
df = pd.DataFrame(np.random.randn(4, 4), columns=columns)
df

df.stack(level=["animal", "hair_length"])
```

The list of levels can contain either level names or level numbers but not a mixture of the two.

```
# df.stack(level=['animal', 'hair_length'])
# from above is equivalent to:
df.stack(level=[1, 2])
```

Missing data

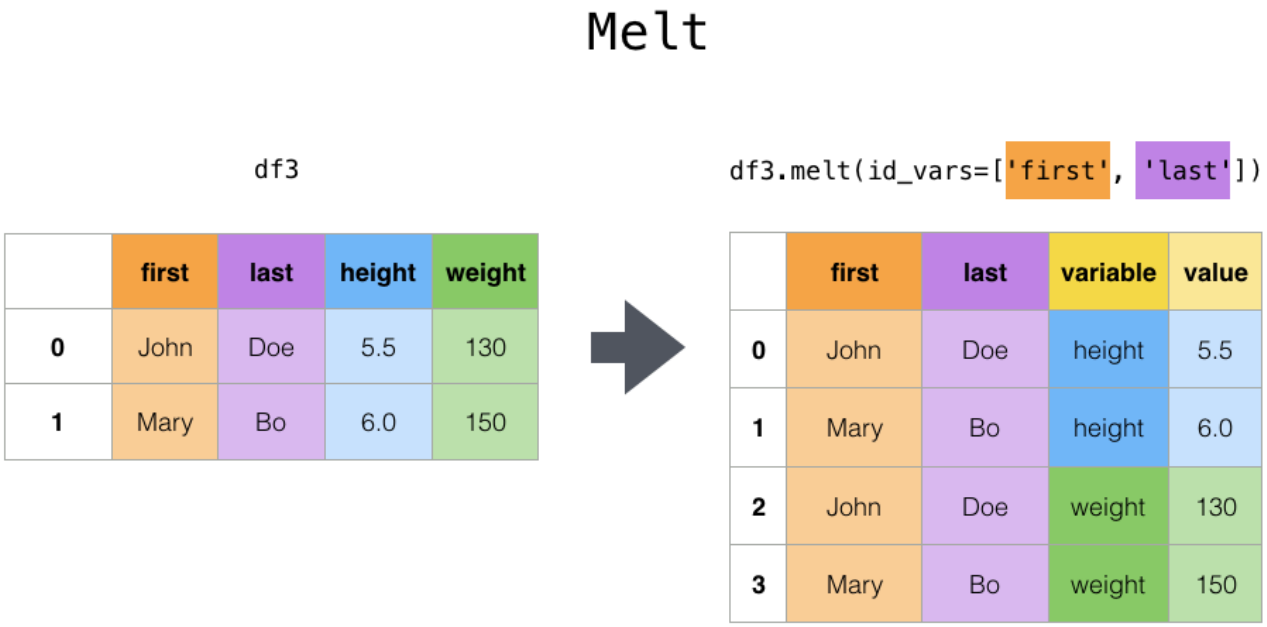
Unstacking can result in missing values if subgroups do not have the same set of labels. By default, missing values will be replaced with the default fill value for that data type.

```
columns = pd.MultiIndex.from_tuples(
    [
        ("A", "cat"),
        ("B", "dog"),
        ("B", "cat"),
        ("A", "dog"),
    ],
    names=["exp", "animal"],
)
index = pd.MultiIndex.from_product(
    [("bar", "baz", "foo", "qux"), ("one", "two")], names=["first", "second"]
)
df = pd.DataFrame(np.random.randn(8, 4), index=index, columns=columns)
df3 = df.iloc[[0, 1, 4, 7], [1, 2]]
df3
df3.unstack()
```

The missing value can be filled with a specific value with the `fill_value` argument.

```
df3.unstack(fill_value=-1e9)
```

`melt` and `wide_to_long`



The top-level `melt` function and the corresponding `DataFrame.melt` are useful to reshape a `DataFrame` into a format where one or more columns are *identifier variables*, while all other columns, considered *measured variables*, are "unpivoted" to the row axis, leaving just two non-identifier columns, "variable" and "value". The names of those columns can be customized by supplying the `var_name` and `value_name` parameters.

```
cheese = pd.DataFrame(
    {
        "first": ["John", "Mary"],
        "last": ["Doe", "Bo"],
        "height": [5.5, 6.0],
        "weight": [130, 150],
    }
)
cheese
cheese.melt(id_vars=["first", "last"])
cheese.melt(id_vars=["first", "last"], var_name="quantity")
```

When transforming a DataFrame using `melt`, the index will be ignored. The original index values can be kept by setting the `ignore_index` parameter to `False` (default is `True`). `ignore_index=False` will however duplicate index values.

```
index = pd.MultiIndex.from_tuples([("person", "A"), ("person", "B")])
cheese = pd.DataFrame(
    {
        "first": ["John", "Mary"],
        "last": ["Doe", "Bo"],
        "height": [5.5, 6.0],
        "weight": [130, 150],
    },
    index=index,
)
cheese
cheese.melt(id_vars=["first", "last"])
```

```
cheese.melt(id_vars=["first", "last"], ignore_index=False)
```

`wide_to_long` is similar to `melt` with more customization for column matching.

```
dft = pd.DataFrame(
    {
        "A1970": {0: "a", 1: "b", 2: "c"},
        "A1980": {0: "d", 1: "e", 2: "f"},
        "B1970": {0: 2.5, 1: 1.2, 2: 0.7},
        "B1980": {0: 3.2, 1: 1.3, 2: 0.1},
        "X": dict(zip(range(3), np.random.randn(3))),
    }
)
dft["id"] = dft.index
dft
pd.wide_to_long(dft, ["A", "B"], i="id", j="year")
```

`get_dummies` and `from_dummies`

To convert categorical variables of a `Series` into a ”dummy” or ”indicator”, `get_dummies` creates a new `DataFrame` with columns of the unique variables and the values representing the presence of those variables per row.

```
df = pd.DataFrame({"key": list("bbacab"), "data1": range(6)})

pd.get_dummies(df["key"])
df["key"].str.get_dummies()
```

`prefix` adds a prefix to the column names which is useful for merging the result with the original `DataFrame`:

```
dummies = pd.get_dummies(df["key"], prefix="key")
dummies

df[["data1"]].join(dummies)
```

This function is often used along with discretization functions like `cut`:

```
values = np.random.randn(10)
values

bins = [0, 0.2, 0.4, 0.6, 0.8, 1]

pd.get_dummies(pd.cut(values, bins))
```

`get_dummies` also accepts a `DataFrame`. By default, `object`, `string`, or `categorical` type columns are encoded as dummy variables with other columns unaltered.

```
df = pd.DataFrame({"A": ["a", "b", "a"], "B": ["c", "c", "b"], "C": [1, 2, 3]})
pd.get_dummies(df)
```

Specifying the `columns` keyword will encode a column of any type.

```
pd.get_dummies(df, columns=["A"])
```

As with the `Series` version, you can pass values for the `prefix` and `prefix_sep`. By default the column name is used as the prefix and _ as the prefix separator. You can specify `prefix` and `prefix_sep` in 3 ways:

- string: Use the same value for `prefix` or `prefix_sep` for each column to be encoded.
- list: Must be the same length as the number of columns being encoded.
- dict: Mapping column name to prefix.

```
simple = pd.get_dummies(df, prefix="new_prefix")
simple
from_list = pd.get_dummies(df, prefix=["from_A", "from_B"])
from_list
from_dict = pd.get_dummies(df, prefix={"B": "from_B", "A": "from_A"})
from_dict
```

To avoid collinearity when feeding the result to statistical models, specify `drop_first=True`.

```
s = pd.Series(list("abcaa"))

pd.get_dummies(s)

pd.get_dummies(s, drop_first=True)
```

When a column contains only one level, it will be omitted in the result.

```
df = pd.DataFrame({"A": list("aaaaa"), "B": list("ababc")})

pd.get_dummies(df)

pd.get_dummies(df, drop_first=True)
```

The values can be cast to a different type using the `dtype` argument.

```
df = pd.DataFrame({"A": list("abc"), "B": [1.1, 2.2, 3.3]})

pd.get_dummies(df, dtype=np.float32).dtypes
```

`from_dummies` converts the output of `get_dummies` back into a `Series` of categorical values from indicator values.

```
df = pd.DataFrame({"prefix_a": [0, 1, 0], "prefix_b": [1, 0, 1]})
df

pd.from_dummies(df, sep="_")
```

Dummy coded data only requires `k - 1` categories to be included, in this case the last category is the default category. The default category can be modified with `default_category`.

```
df = pd.DataFrame({"prefix_a": [0, 1, 0]})
df

pd.from_dummies(df, sep="_", default_category="b")
```

explode

For a `DataFrame` column with nested, list-like values, `explode` will transform each list-like value to a separate row. The resulting `Index` will be duplicated corresponding to the index label from the original row:

```
keys = ["panda1", "panda2", "panda3"]
values = [["eats", "shoots"], ["shoots", "leaves"], ["eats", "leaves"]]
df = pd.DataFrame({"keys": keys, "values": values})
df
```

```
df["values"].explode()
```

`DataFrame.explode` can also explode the column in the `DataFrame`.

```
df.explode("values")
```

`Series.explode` will replace empty lists with a missing value indicator and preserve scalar entries.

```
s = pd.Series([[1, 2, 3], "foo", [], ["a", "b"]])
s
s.explode()
```

A comma-separated string value can be split into individual values in a list and then exploded to a new row.

```
df = pd.DataFrame([{"var1": "a,b,c", "var2": 1}, {"var1": "d,e,f", "var2": 2}])
df.assign(var1=df.var1.str.split(",")).explode("var1")
```

crosstab

Use `crosstab` to compute a cross-tabulation of two (or more) factors. By default `crosstab` computes a frequency table of the factors unless an array of values and an aggregation function are passed.

Any `Series` passed will have their name attributes used unless row or column names for the cross-tabulation are specified

```
a = np.array(["foo", "foo", "bar", "bar", "foo", "foo"], dtype=object)
b = np.array(["one", "one", "two", "one", "two", "one"], dtype=object)
c = np.array(["dull", "dull", "shiny", "dull", "dull", "shiny"], dtype=object)
pd.crosstab(a, [b, c], rownames=["a"], colnames=["b", "c"])
```

If `crosstab` receives only two `Series`, it will provide a frequency table.

```
df = pd.DataFrame(
    {"A": [1, 2, 2, 2, 2], "B": [3, 3, 4, 4, 4], "C": [1, 1, np.nan, 1, 1]}
)
df

pd.crosstab(df["A"], df["B"])
```

`crosstab` can also summarize to `Categorical` data.

```
foo = pd.Categorical(["a", "b"], categories=["a", "b", "c"])
bar = pd.Categorical(["d", "e"], categories=["d", "e", "f"])
pd.crosstab(foo, bar)
```

For `Categorical` data, to include **all** of data categories even if the actual data does not contain any instances of a particular category, use `dropna=False`.

```
pd.crosstab(foo, bar, dropna=False)
```

Normalization

Frequency tables can also be normalized to show percentages rather than counts using the `normalize` argument:

```
pd.crosstab(df["A"], df["B"], normalize=True)
```

`normalize` can also normalize values within each row or within each column:

```
pd.crosstab(df["A"], df["B"], normalize="columns")
```

`crosstab` can also accept a third `Series` and an aggregation function (`aggfunc`) that will be applied to the values of the third `Series` within each group defined by the first two `Series`:

```
pd.crosstab(df["A"], df["B"], values=df["C"], aggfunc="sum")
```

Adding margins

`margins=True` will add a row and column with an `All` label with partial group aggregates across the categories on the rows and columns:

```
pd.crosstab(
    df["A"], df["B"], values=df["C"], aggfunc="sum", normalize=True, margins=True
)
```

cut

The `cut` function computes groupings for the values of the input array and is often used to transform continuous variables to discrete or categorical variables:

An integer `bins` will form equal-width bins.

```
ages = np.array([10, 15, 13, 12, 23, 25, 28, 59, 60])

pd.cut(ages, bins=3)
```

A list of ordered bin edges will assign an interval for each variable.

```
pd.cut(ages, bins=[0, 18, 35, 70])
```

If the `bins` keyword is an `IntervalIndex`, then these will be used to bin the passed data.

```
pd.cut(ages, bins=pd.IntervalIndex.from_breaks([0, 40, 70]))
```

factorize

`factorize` encodes 1 dimensional values into integer labels. Missing values are encoded as `-1`.

```
x = pd.Series(["A", "A", np.nan, "B", 3.14, np.inf])
x
labels, uniques = pd.factorize(x)
labels
uniques
```

`Categorical` will similarly encode 1 dimensional values for further categorical operations

```
pd.Categorical(x)
```

Time series / date functionality

Source: https://pandas.pydata.org/docs/user_guide/timeseries.html

pandas contains extensive capabilities and features for working with time series data for all domains. Using the NumPy `datetime64` and `timedelta64` dtypes, pandas has consolidated a large number of features from other Python libraries like `scikits.timeseries` as well as created a tremendous amount of new functionality for manipulating time series data.

For example, pandas supports:

Parsing time series information from various sources and formats

```
import datetime

dti = pd.to_datetime(
    ["1/1/2018", np.datetime64("2018-01-01"), datetime.datetime(2018, 1, 1)]
)
dti
```

Generate sequences of fixed-frequency dates and time spans

```
dti = pd.date_range("2018-01-01", periods=3, freq="h")
dti
```

Manipulating and converting date times with timezone information

```
dti = dti.tz_localize("UTC")
dti
dti.tz_convert("US/Pacific")
```

Resampling or converting a time series to a particular frequency

```
idx = pd.date_range("2018-01-01", periods=5, freq="h")
ts = pd.Series(range(len(idx)), index=idx)
ts
ts.resample("2h").mean()
```

Performing date and time arithmetic with absolute or relative time increments

```
friday = pd.Timestamp("2018-01-05")
friday.day_name()
# Add 1 day
saturday = friday + pd.Timedelta("1 day")
saturday.day_name()
# Add 1 business day (Friday --> Monday)
monday = friday + pd.offsets.BDay()
monday.day_name()
```

pandas provides a relatively compact and self-contained set of tools for performing the above tasks and more.

Overview

pandas captures 4 general time related concepts:

1. Date times: A specific date and time with timezone support. Similar to `datetime.datetime` from the standard library.
2. Time deltas: An absolute time duration. Similar to `datetime.timedelta` from the standard library.
3. Time spans: A span of time defined by a point in time and its associated frequency.
4. Date offsets: A relative time duration that respects calendar arithmetic. Similar to `dateutil.relativedelta.relativedelta` from the `dateutil` package.

Concept	Scalar Class	Array Class	pandas Data Type	Primary Creation Method
Date times	`Timestamp`	`DatetimeIndex`	`datetime64[us]` or `datetime64[us, tz]`	`to_datetime` or `date_range`
Time deltas	`Timedelta`	`TimedeltaIndex`	`timedelta64[us]`	`to_timedelta` or `timedelta_`
Time spans	`Period`	`PeriodIndex`	`period[freq]`	`Period` or `period_range`
Date offsets	`DateOffset`	None	None	`DateOffset`

The default resolution for date times and time deltas is microsecond (”us”), though second (”s”), millisecond (”ms”), and nanosecond (”ns”) are also supported. The resolution can be changed using `as_unit`. Arithmetic keeps the resolution of its operands, see Timedelta arithmetic.

For time series data, it's conventional to represent the time component in the index of a `Series` or `DataFrame` so manipulations can be performed with respect to the time element.

```
pd.Series(range(3), index=pd.date_range("2000", freq="D", periods=3))
```

However, `Series` and `DataFrame` can directly also support the time component as data itself.

```
pd.Series(pd.date_range("2000", freq="D", periods=3))
```

`Series` and `DataFrame` have extended data type support and functionality for `datetime`, `timedelta` and `Period` data when passed into those constructors. `DateOffset` data however will be stored as `object` data.

```
pd.Series(pd.period_range("1/1/2011", freq="M", periods=3))
pd.Series([pd.DateOffset(1), pd.DateOffset(2)])
pd.Series(pd.date_range("1/1/2011", freq="ME", periods=3))
```

Lastly, pandas represents null date times, time deltas, and time spans as `NaT` which is useful for representing missing or null date like values and behaves similar as `np.nan` does for float data.

```
pd.Timestamp(pd.NaT)
pd.Timedelta(pd.NaT)
pd.Period(pd.NaT)
# Equality acts as np.nan would
pd.NaT == pd.NaT
```

Timestamps vs. time spans

Timestamped data is the most basic type of time series data that associates values with points in time. For pandas objects it means using the points in time.

```
import datetime

pd.Timestamp(datetime.datetime(2012, 5, 1))
pd.Timestamp("2012-05-01")
pd.Timestamp(2012, 5, 1)
```

However, in many cases it is more natural to associate things like change variables with a time span instead. The span represented by `Period` can be specified explicitly, or inferred from datetime string format.

For example:

```
pd.Period("2011-01")

pd.Period("2012-05", freq="D")
```

`Timestamp` and `Period` can serve as an index. Lists of `Timestamp` and `Period` are automatically coerced to `DatetimeIndex` and `PeriodIndex` respectively.

```
dates = [
    pd.Timestamp("2012-05-01"),
    pd.Timestamp("2012-05-02"),
    pd.Timestamp("2012-05-03"),
]
ts = pd.Series(np.random.randn(3), dates)

type(ts.index)
ts.index

ts

periods = [pd.Period("2012-01"), pd.Period("2012-02"), pd.Period("2012-03")]

ts = pd.Series(np.random.randn(3), periods)

type(ts.index)
ts.index

ts
```

pandas allows you to capture both representations and convert between them. Under the hood, pandas represents timestamps using instances of `Timestamp` and sequences of timestamps using instances of `DatetimeIndex`. For regular time spans, pandas uses `Period` objects for scalar values and `PeriodIndex` for sequences of spans.

Converting to timestamps

To convert a `Series` or list-like object of date-like objects e.g. strings, epochs, or a mixture, you can use the `to_datetime` function. When passed a `Series`, this returns a `Series` (with the same index), while a list-like is converted to a `DatetimeIndex`:

```
pd.to_datetime(pd.Series(["Jul 31, 2009", "Jan 10, 2010", None]))

pd.to_datetime(["2005/11/23", "2010/12/31"])
```

If you use dates which start with the day first (i.e. European style), you can pass the `dayfirst` flag:

```
pd.to_datetime(["04-01-2012 10:00"], dayfirst=True)

pd.to_datetime(["04-14-2012 10:00"], dayfirst=True)
```

Warning
You see in the above example that `dayfirst` isn't strict. If a date can't be parsed with the day being first it will be parsed as if `dayfirst` were `False` and a warning will also be raised.

If you pass a single string to `to_datetime`, it returns a single `Timestamp`. `Timestamp` can also accept string input, but it doesn't accept string parsing options like `dayfirst` or `format`, so use `to_datetime` if these are required.

```
pd.to_datetime("2010/11/12")

pd.Timestamp("2010/11/12")
```

You can also use the `DatetimeIndex` constructor directly:

```
pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"])
```

The string 'infer' can be passed in order to set the frequency of the index as the inferred frequency upon creation:

```
pd.DatetimeIndex(["2018-01-01", "2018-01-03", "2018-01-05"], freq="infer")
```

In most cases, parsing strings to datetimes (with any of `to_datetime`, `DatetimeIndex`, or `Timestamp`) will produce objects with microsecond ("us") unit. The exception to this rule is if your strings have nanosecond precision, in which case the result will have "ns" unit:

```
pd.to_datetime(["2016-01-01 02:03:04"]).unit
pd.to_datetime(["2016-01-01 02:03:04.123"]).unit
pd.to_datetime(["2016-01-01 02:03:04.123456"]).unit
pd.to_datetime(["2016-01-01 02:03:04.123456789"]).unit
```

Changed in version 3.0.0. Previously, `to_datetime` and `DatetimeIndex` would always parse strings to "ns" unit. During pandas 2.x, `Timestamp` could give any of "s", "ms", "us", or "ns" depending on the specificity of the input string.

Providing a format argument

In addition to the required datetime string, a `format` argument can be passed to ensure specific parsing. This could also potentially speed up the conversion considerably.

```
pd.to_datetime("2010/11/12", format="%Y/%m/%d")

pd.to_datetime("12-11-2010 00:00", format="%d-%m-%Y %H:%M")
```

For more information on the choices available when specifying the `format` option, see the Python datetime documentation.

Assembling datetime from multiple DataFrame columns

You can also pass a `DataFrame` of integer or string columns to assemble into a `Series` of `Timestamps`.

```
df = pd.DataFrame(
    {"year": [2015, 2016], "month": [2, 3], "day": [4, 5], "hour": [2, 3]}
)
pd.to_datetime(df)
```

You can pass only the columns that you need to assemble.

```
pd.to_datetime(df[["year", "month", "day"]])
```

`pd.to_datetime` looks for standard designations of the datetime component in the column names, including:

- required: `year`, `month`, `day`
- optional: `hour`, `minute`, `second`, `millisecond`, `microsecond`, `nanosecond`

Invalid data

The default behavior, `errors='raise'`, is to raise when unparsable:

```
pd.to_datetime(['2009/07/31', 'asd'], errors='raise')
```

Pass `errors='coerce'` to convert unparsable data to `NaT` (not a time):

```
pd.to_datetime(["2009/07/31", "asd"], errors="coerce")
```

Epoch timestamps

pandas supports converting integer or float epoch times to `Timestamp` and `DatetimeIndex`. The default unit is nanoseconds when no `unit` is specified. However, epochs are often stored in another `unit` which can be specified. These are computed from the starting point specified by the `origin` parameter.

```
pd.to_datetime(
    [1349720105, 1349806505, 1349892905, 1349979305, 1350065705], unit="s"
)

pd.to_datetime(
    [1349720105100, 1349720105200, 1349720105300, 1349720105400, 1349720105500],
    unit="ms",
)
```

> Note
> The `unit` parameter does not use the same strings as the `format` parameter that was discussed above. The available units are listed on the documentation for `pandas.to_datetime`.

Constructing a `Timestamp` or `DatetimeIndex` with an epoch timestamp with the `tz` argument specified will raise a ValueError. If you have epochs in wall time in another timezone, you can read the epochs as timezone-naive timestamps and then localize to the appropriate timezone:

```
pd.Timestamp(1262347200000000000).tz_localize("US/Pacific")
pd.DatetimeIndex([1262347200000000000]).tz_localize("US/Pacific")
```

> Note
> Epoch times will be rounded to the nearest nanosecond.

> Warning
> Conversion of float epoch times can lead to inaccurate and unexpected results. decimal. Rounding during conversion from float to high precision `Timestamp` is unavoidable. The only way to achieve exact precision is to use a fixed-width types (e.g. an int64).
>
> ```
> pd.to_datetime([1490195805.433, 1490195805.433502912], unit="s")
> pd.to_datetime(1490195805433502912, unit="ns")
> ```

From timestamps to epoch

To invert the operation from above, namely, to convert from a `Timestamp` to a 'unix' epoch:

```
stamps = pd.date_range("2012-10-08 18:15:05", periods=4, freq="D")
stamps
```

We subtract the epoch (midnight at January 1, 1970 UTC) and then floor divide by the "unit" (1 second).

```
(stamps - pd.Timestamp("1970-01-01")) // pd.Timedelta("1s")
```

> Note
> You may also see `Timestamp.timestamp` used for this conversion. The explicit subtraction above is recommended because it is unambiguous about treating a timezone-naive value as UTC: `Timestamp.timestamp` treats a naive `Timestamp` as UTC, which does *not* match the standard library `datetime.datetime.timestamp`, where a naive value is interpreted as local time.

Another common way to perform this conversion is to convert directly to an integer dtype. Note that the exact integers this produces will depend on the specific unit or resolution of the datetime64 dtype:

```
stamps.astype(np.int64)
stamps.as_unit("s").astype(np.int64)
stamps.as_unit("ns").astype(np.int64)
```

Using the `origin` parameter

Using the `origin` parameter, one can specify an alternative starting point for creation of a `DatetimeIndex`. For example, to use 1960-01-01 as the starting date:

```
pd.to_datetime([1, 2, 3], unit="D", origin=pd.Timestamp("1960-01-01"))
```

The default is set at `origin='unix'`, which defaults to `1970-01-01 00:00:00`. Commonly called 'unix epoch' or POSIX time.

```
pd.to_datetime([1, 2, 3], unit="D")
```

Generating ranges of timestamps

To generate an index with timestamps, you can use either the `DatetimeIndex` or `Index` constructor and pass in a list of datetime objects:

```
dates = [
    datetime.datetime(2012, 5, 1),
    datetime.datetime(2012, 5, 2),
    datetime.datetime(2012, 5, 3),
]

# Note the frequency information
index = pd.DatetimeIndex(dates)
index

# Automatically converted to DatetimeIndex
index = pd.Index(dates)
index
```

In practice this becomes very cumbersome because we often need a very long index with a large number of timestamps. If we need timestamps on a regular frequency, we can use the `date_range` and `bdate_range` functions to create a `DatetimeIndex`. The default frequency for `date_range` is a **calendar day** while the default for `bdate_range` is a **business day**:

```
start = datetime.datetime(2011, 1, 1)
end = datetime.datetime(2012, 1, 1)

index = pd.date_range(start, end)
index

index = pd.bdate_range(start, end)
index
```

Convenience functions like `date_range` and `bdate_range` can utilize a variety of frequency aliases:

```
pd.date_range(start, periods=1000, freq="ME")

pd.bdate_range(start, periods=250, freq="BQS")
```

`date_range` and `bdate_range` make it easy to generate a range of dates using various combinations of parameters like `start`, `end`, `periods`, and `freq`. The start and end dates are strictly inclusive, so dates outside of those specified will not be generated:

```
pd.date_range(start, end, freq="BME")

pd.date_range(start, end, freq="W")

pd.bdate_range(end=end, periods=20)

pd.bdate_range(start=start, periods=20)
```

Specifying `start`, `end`, and `periods` will generate a range of evenly spaced dates from `start` to `end` inclusively, with `periods` number of elements in the resulting `DatetimeIndex`:

```
pd.date_range("2018-01-01", "2018-01-05", periods=5)

pd.date_range("2018-01-01", "2018-01-05", periods=10)
```

Custom frequency ranges

`bdate_range` can also generate a range of custom frequency dates by using the `weekmask` and `holidays` parameters. These parameters will only be used if a custom frequency string is passed.

```
weekmask = "Mon Wed Fri"

holidays = [datetime.datetime(2011, 1, 5), datetime.datetime(2011, 3, 14)]

pd.bdate_range(start, end, freq="C", weekmask=weekmask, holidays=holidays)

pd.bdate_range(start, end, freq="CBMS", weekmask=weekmask)
```

Timestamp limitations

`Timestamp` uses a 64-bit integer to represent time, so the representable range depends on the chosen resolution (`unit`). For the default nanosecond resolution the span is limited to approximately 584 years; coarser resolutions such as second extend the range to roughly `+/- 2.9e11` years.

The class-level attributes `Timestamp.min`, `Timestamp.max`, and `Timestamp.resolution` default to nanosecond limits:

```
pd.Timestamp.min
pd.Timestamp.max
pd.Timestamp.resolution
```

On a `Timestamp` *instance*, the same attributes reflect the bounds and step size of that instance's resolution:

```
ts = pd.Timestamp("2262-04-12").as_unit("s")
ts.min
ts.max
ts.resolution
```

Because `Timestamp` construction automatically picks a resolution wide enough to represent the input, a value outside the nanosecond range (such as `"3000-06-10"`) is parsed using a coarser unit rather than raising `OutOfBoundsDatetime`:

```
far_future = pd.Timestamp("3000-06-10")
far_future
far_future.unit
```

Different resolutions can be converted to each other through `Timestamp.as_unit`.

Indexing

One of the main uses for `DatetimeIndex` is as an index for pandas objects. The `DatetimeIndex` class contains many time series related optimizations:

- A large range of dates for various offsets are pre-computed and cached under the hood in order to make generating subsequent date ranges very fast (just have to grab a slice).
- Fast shifting using the `shift` method on pandas objects.
- Unioning of overlapping `DatetimeIndex` objects with the same frequency is very fast (important for fast data alignment).
- Quick access to date fields via properties such as `year`, `month`, etc.
- Regularization functions like `snap` and very fast `asof` logic.

`DatetimeIndex` objects have all the basic functionality of regular `Index` objects, and a smorgasbord of advanced time series specific methods for easy frequency processing.

> Note
> While pandas does not force you to have a sorted date index, some of these methods may have unexpected or incorrect behavior if the dates are unsorted.

`DatetimeIndex` can be used like a regular index and offers all of its intelligent functionality like selection, slicing, etc.

```
rng = pd.date_range(start, end, freq="BME")
ts = pd.Series(np.random.randn(len(rng)), index=rng)
ts.index
ts[:5].index
ts[::2].index
```

Partial string indexing

Dates and strings that parse to timestamps can be passed as indexing parameters:

```
ts["1/31/2011"]

ts[datetime.datetime(2011, 12, 25):]

ts["10/31/2011":"12/31/2011"]
```

To provide convenience for accessing longer time series, you can also pass in the year or year and month as strings:

```
ts["2011"]

ts["2011-6"]
```

This type of slicing will work on a `DataFrame` with a `DatetimeIndex` as well. Since the partial string selection is a form of label slicing, the endpoints **will be** included. This would include matching times on an included date:

> Note
> Indexing `DataFrame` rows with a *single* string with getitem (e.g. `frame[dtstring]`) is no longer supported. Use `.loc` instead (e.g. `frame.loc[dtstring]`).

```
dft = pd.DataFrame(
    np.random.randn(100000, 1),
    columns=["A"],
    index=pd.date_range("20130101", periods=100000, freq="min"),
)
dft
dft.loc["2013"]
```

This starts on the very first time in the month, and includes the last date and time for the month:

```
dft["2013-1":"2013-2"]
```

This specifies a stop time **that includes all of the times on the last day**:

```
dft["2013-1":"2013-2-28"]
```

This specifies an **exact** stop time (and is not the same as the above):

```
dft["2013-1":"2013-2-28 00:00:00"]
```

We are stopping on the included end-point as it is part of the index:

```
dft["2013-1-15":"2013-1-15 12:30:00"]
```

`DatetimeIndex` partial string indexing also works on a `DataFrame` with a `MultiIndex`:

```
dft2 = pd.DataFrame(
    np.random.randn(20, 1),
    columns=["A"],
    index=pd.MultiIndex.from_product(
        [pd.date_range("20130101", periods=10, freq="12h"), ["a", "b"]]
    ),
)
dft2
dft2.loc["2013-01-05"]
idx = pd.IndexSlice
dft2 = dft2.swaplevel(0, 1).sort_index()
dft2.loc[idx[:, "2013-01-05"], :]
```

Slicing with string indexing also honors UTC offset.

```
df = pd.DataFrame([0], index=pd.DatetimeIndex(["2019-01-01"], tz="US/Pacific"))
df
df["2019-01-01 12:00:00+04:00":"2019-01-01 13:00:00+04:00"]
```

Slice vs. exact match

The same string used as an indexing parameter can be treated either as a slice or as an exact match depending on the resolution of the index. If the string is less accurate than the index, it will be treated as a slice, otherwise as an exact match.

Consider a `Series` object with a minute resolution index:

```
series_minute = pd.Series(
    [1, 2, 3],
    pd.DatetimeIndex(
        ["2011-12-31 23:59:00", "2012-01-01 00:00:00", "2012-01-01 00:02:00"]
    ),
)
series_minute.index.resolution
```

A timestamp string less accurate than a minute gives a `Series` object.

```
series_minute["2011-12-31 23"]
```

A timestamp string with minute resolution (or more accurate), gives a scalar instead, i.e. it is not casted to a slice.

```
series_minute["2011-12-31 23:59"]
series_minute["2011-12-31 23:59:00"]
```

If index resolution is second, then the minute-accurate timestamp gives a `Series`.

```
series_second = pd.Series(
    [1, 2, 3],
    pd.DatetimeIndex(
        ["2011-12-31 23:59:59", "2012-01-01 00:00:00", "2012-01-01 00:00:01"]
    ),
)
series_second.index.resolution
series_second["2011-12-31 23:59"]
```

If the timestamp string is treated as a slice, it can be used to index `DataFrame` with `.loc[]` as well.

```
dft_minute = pd.DataFrame(
    {"a": [1, 2, 3], "b": [4, 5, 6]}, index=series_minute.index
)
dft_minute.loc["2011-12-31 23"]
```

> Note
> As noted above, indexing `DataFrame` rows with a single string via `[]` is no longer supported; use `.loc` instead.
>
> ```
> dft_minute.loc["2011-12-31 23:59"]
> ```

Note also that `DatetimeIndex` resolution cannot be less precise than day.

```
series_monthly = pd.Series(
    [1, 2, 3], pd.DatetimeIndex(["2011-12", "2012-01", "2012-02"])
```

```
)
series_monthly.index.resolution
series_monthly["2011-12"]  # returns Series
```

Exact indexing

As discussed in previous section, indexing a `DatetimeIndex` with a partial string depends on the "accuracy" of the period, in other words how specific the interval is in relation to the resolution of the index. In contrast, indexing with `Timestamp` or `datetime` objects is exact, because the objects have exact meaning. These also follow the semantics of *including both endpoints*.

These `Timestamp` and `datetime` objects have exact `hours, minutes,` and `seconds`, even though they were not explicitly specified (they are `0`).

```
dft[datetime.datetime(2013, 1, 1): datetime.datetime(2013, 2, 28)]
```

With no defaults.

```
dft[
    datetime.datetime(2013, 1, 1, 10, 12, 0): datetime.datetime(
        2013, 2, 28, 10, 12, 0
    )
]
```

Non-inclusive slicing

As noted in the sections above, label-based slicing with a `DatetimeIndex` **includes both endpoints**. This differs from standard Python sequence slicing where the stop endpoint is excluded.

If you need to select data using open or half-open intervals (for example, `start < t < end` or `start <= t < end`), you can use boolean indexing:

```
ts_example = pd.Series(
    range(5),
    index=pd.date_range("2000-01-01", periods=5, freq="D"),
)
ts_example
```

Select all entries strictly between `"2000-01-02"` and `"2000-01-04"` (excluding both endpoints):

```
ts_example[(ts_example.index > "2000-01-02") & (ts_example.index < "2000-01-04")]
```

Select entries with a half-open interval (include left, exclude right):

```
ts_example[(ts_example.index >= "2000-01-02") & (ts_example.index < "2000-01-04")]
```

For large time series where boolean indexing may be less efficient, you can use `get_slice_bound` together with `.iloc` to achieve non-inclusive slicing on a sorted index:

```
lo = ts_example.index.get_slice_bound("2000-01-02", "right")
hi = ts_example.index.get_slice_bound("2000-01-04", "left")
ts_example.iloc[lo:hi]
```

Here, `side="right"` returns the position just past `"2000-01-02"`, and `side="left"` returns the position of `"2000-01-04"` itself. Since `.iloc` uses standard Python half-open slicing, the result excludes both endpoints.

Selecting a time of day

Like partial string indexing, the methods `at_time` and `between_time` operate on the object's index, selecting rows by their time of day regardless of the date. `at_time` selects all timestamps at a particular time:

```
rng_time = pd.date_range("2018-01-01", periods=10, freq="6h")
ts_time = pd.Series(range(10), index=rng_time)
ts_time
ts_time.at_time("12:00")
```

`between_time` selects the rows whose time of day falls between two times, including both endpoints by default; the `inclusive` argument controls whether each endpoint is included:

```
ts_time.between_time("00:00", "12:00")
ts_time.between_time("00:00", "12:00", inclusive="left")
```

Both methods also accept `datetime.time` objects:

```
import datetime

ts_time.at_time(datetime.time(12, 0))
```

Truncating & fancy indexing

A `truncate` convenience function is provided that is similar to slicing. Note that `truncate` assumes a 0 value for any unspecified date component in a `DatetimeIndex` in contrast to slicing which returns any partially matching dates:

```
rng2 = pd.date_range("2011-01-01", "2012-01-01", freq="W")
ts2 = pd.Series(np.random.randn(len(rng2)), index=rng2)

ts2.truncate(before="2011-11", after="2011-12")
ts2["2011-11":"2011-12"]
```

Even complicated fancy indexing that breaks the `DatetimeIndex` frequency regularity will result in a `DatetimeIndex`, although frequency is lost:

```
ts2.iloc[[0, 2, 6]].index
```

Time/date components

There are several time/date properties that one can access from `Timestamp` or a collection of timestamps like a `DatetimeIndex`.

Property	Description
year	The year of the datetime
month	The month of the datetime
day	The days of the datetime
hour	The hour of the datetime
minute	The minutes of the datetime
second	The seconds of the datetime
microsecond	The microseconds of the datetime
nanosecond	The nanoseconds of the datetime
date	Returns datetime.date (does not contain timezone information)
time	Returns datetime.time (does not contain timezone information)

Property	Description
timetz	Returns datetime.time as local time with timezone information
dayofyear	The ordinal day of year
day_of_year	The ordinal day of year
dayofweek	The number of the day of the week with Monday=0, Sunday=6
day_of_week	The number of the day of the week with Monday=0, Sunday=6
weekday	The number of the day of the week with Monday=0, Sunday=6
quarter	Quarter of the date: Jan-Mar = 1, Apr-Jun = 2, etc.
days_in_month	The number of days in the month of the datetime
is_month_start	Logical indicating if first day of month (defined by frequency)
is_month_end	Logical indicating if last day of month (defined by frequency)
is_quarter_start	Logical indicating if first day of quarter (defined by frequency)
is_quarter_end	Logical indicating if last day of quarter (defined by frequency)
is_year_start	Logical indicating if first day of year (defined by frequency)
is_year_end	Logical indicating if last day of year (defined by frequency)
is_leap_year	Logical indicating if the date belongs to a leap year

Note
You can use `DatetimeIndex.isocalendar().week` to access week of year date information.

Furthermore, if you have a `Series` with datetimelike values, then you can access these properties via the `.dt` accessor, as detailed in the section on .dt accessors.

You may obtain the year, week and day components of the ISO year from the ISO 8601 standard:

```
idx = pd.date_range(start="2019-12-29", freq="D", periods=4)
idx.isocalendar()
idx.to_series().dt.isocalendar()
```

DateOffset objects

In the preceding examples, frequency strings (e.g. `'D'`) were used to specify a frequency that defined:

- how the date times in `DatetimeIndex` were spaced when using `date_range`
- the frequency of a `Period` or `PeriodIndex`

These frequency strings map to a `DateOffset` object and its subclasses. A `DateOffset` is similar to a `Timedelta` that represents a duration of time but follows specific calendar duration rules. For example, a `Timedelta` day will always increment `datetimes` by 24 hours, while a `DateOffset` day will increment `datetimes` to the same time the next day whether a day represents 23, 24 or 25 hours due to daylight savings time. However, all `DateOffset` subclasses that are an hour or smaller (`Hour`, `Minute`, `Second`, `Milli`, `Micro`, `Nano`) behave like `Timedelta` and respect absolute time.

The basic `DateOffset` acts similar to `dateutil.relativedelta` (relativedelta documentation) that shifts a date time by the corresponding calendar duration specified. The arithmetic operator (`+`) can be used to perform the shift.

```
# This particular day contains a day light savings time transition
ts = pd.Timestamp("2016-10-30 00:00:00", tz="Europe/Helsinki")
# Respects absolute time
ts + pd.Timedelta(days=1)
# Respects calendar time
ts + pd.DateOffset(days=1)
friday = pd.Timestamp("2018-01-05")
```

```
friday.day_name()
# Add 2 business days (Friday --> Tuesday)
two_business_days = 2 * pd.offsets.BDay()
friday + two_business_days
(friday + two_business_days).day_name()
```

Most `DateOffsets` have associated frequencies strings, or offset aliases, that can be passed into `freq` keyword arguments. The available date offsets and associated frequency strings can be found below:

Date Offset Frequency String Description ————- ———————— ————-

`DateOffsets` additionally have `rollforward` and `rollback` methods for moving a date forward or backward respectively to a valid offset date relative to the offset. For example, business offsets will roll dates that land on the weekends (Saturday and Sunday) forward to Monday since business offsets operate on the weekdays.

```
ts = pd.Timestamp("2018-01-06 00:00:00")
ts.day_name()
# BusinessHour's valid offset dates are Monday through Friday
offset = pd.offsets.BusinessHour(start="09:00")
# Bring the date to the closest offset date (Monday)
offset.rollforward(ts)
# Date is brought to the closest offset date first and then the hour is added
ts + offset
```

Both methods roll *conditionally*: a date that is already a valid offset date is returned unchanged, so `rollforward` and `rollback` give the same answer there. Use `is_on_offset` to check whether a date is already valid, and add or subtract the offset to move unconditionally.

```
# The last business day of July 2021, so already a valid BMonthEnd date
ts = pd.Timestamp("2021-07-30")
offset = pd.offsets.BMonthEnd()
offset.is_on_offset(ts)
offset.rollback(ts)
offset.rollforward(ts)
# Subtract the offset to always move to the previous business month end
ts - offset
```

These operations preserve time (hour, minute, etc) information by default. To reset time to midnight, use `normalize` before or after applying the operation (depending on whether you want the time information included in the operation).

```
ts = pd.Timestamp("2014-01-01 09:00")
day = pd.offsets.Day()
day + ts
(day + ts).normalize()

ts = pd.Timestamp("2014-01-01 22:00")
hour = pd.offsets.Hour()
hour + ts
(hour + ts).normalize()
(hour + pd.Timestamp("2014-01-01 23:30")).normalize()
```

Parametric offsets

Some of the offsets can be ”parameterized” when created to result in different behaviors. For example, the `Week` offset for generating weekly data accepts a `weekday` parameter which results in the generated dates always lying on a particular day of the week:

```
d = datetime.datetime(2008, 8, 18, 9, 0)
d
d + pd.offsets.Week()
d + pd.offsets.Week(weekday=4)
(d + pd.offsets.Week(weekday=4)).weekday()

d - pd.offsets.Week()
```

The `normalize` option will be effective for addition and subtraction.

```
d + pd.offsets.Week(normalize=True)
d - pd.offsets.Week(normalize=True)
```

Another example is parameterizing `YearEnd` with the specific ending month:

```
d + pd.offsets.YearEnd()
d + pd.offsets.YearEnd(month=6)
```

Using offsets with `Series` / `DatetimeIndex`

Offsets can be used with either a `Series` or `DatetimeIndex` to apply the offset to each element.

```
rng = pd.date_range("2012-01-01", "2012-01-03")
s = pd.Series(rng)
rng
rng + pd.DateOffset(months=2)
s + pd.DateOffset(months=2)
s - pd.DateOffset(months=2)
```

If the offset class maps directly to a `Timedelta` (`Hour`, `Minute`, `Second`, `Micro`, `Milli`, `Nano`) it can be used exactly like a `Timedelta` - see the Timedelta section for more examples.

```
s - pd.offsets.Day(2)
td = s - pd.Series(pd.date_range("2011-12-29", "2011-12-31"))
td
td + pd.offsets.Minute(15)
```

Note that some offsets (such as `BQuarterEnd`) do not have a vectorized implementation. They can still be used but may calculate significantly slower and will show a `PerformanceWarning`

```
rng + pd.offsets.BQuarterEnd()
```

Custom business days

The `CDay` or `CustomBusinessDay` class provides a parametric `BusinessDay` class which can be used to create customized business day calendars which account for local holidays and local weekend conventions.

As an interesting example, let's look at Egypt where a Friday-Saturday weekend is observed.

```
weekmask_egypt = "Sun Mon Tue Wed Thu"

# They also observe International Workers' Day so let's
# add that for a couple of years

holidays = [
    "2012-05-01",
    datetime.datetime(2013, 5, 1),
    np.datetime64("2014-05-01"),
```

```
]
bday_egypt = pd.offsets.CustomBusinessDay(
    holidays=holidays,
    weekmask=weekmask_egypt,
)
dt = datetime.datetime(2013, 4, 30)
dt + 2 * bday_egypt
```

Let's map to the weekday names:

```
dts = pd.date_range(dt, periods=5, freq=bday_egypt)

pd.Series(dts.day_of_week, dts).map(pd.Series("Mon Tue Wed Thu Fri Sat Sun".split()))
```

Holiday calendars can be used to provide the list of holidays. See the holiday calendar section for more information.

```
from pandas.tseries.holiday import USFederalHolidayCalendar

bday_us = pd.offsets.CustomBusinessDay(calendar=USFederalHolidayCalendar())

# Friday before MLK Day
dt = datetime.datetime(2014, 1, 17)

# Tuesday after MLK Day (Monday is skipped because it's a holiday)
dt + bday_us
```

Monthly offsets that respect a certain holiday calendar can be defined in the usual way.

```
bmth_us = pd.offsets.CustomBusinessMonthBegin(calendar=USFederalHolidayCalendar())

# Skip new years
dt = datetime.datetime(2013, 12, 17)
dt + bmth_us

# Define date index with custom offset
pd.date_range(start="20100101", end="20120101", freq=bmth_us)
```

> Note
> The frequency string 'C' is used to indicate that a CustomBusinessDay DateOffset is used, it is important to note that since CustomBusinessDay is a parameterised type, instances of CustomBusinessDay may differ and this is not detectable from the 'C' frequency string. The user therefore needs to ensure that the 'C' frequency string is used consistently within the user's application.

Business hour

The `BusinessHour` class provides a business hour representation on `BusinessDay`, allowing to use specific start and end times.

By default, `BusinessHour` uses 9:00 - 17:00 as business hours. Adding `BusinessHour` will increment `Timestamp` by hourly frequency. If target `Timestamp` is out of business hours, move to the next business hour then increment it. If the result exceeds the business hours end, the remaining hours are added to the next business day.

```
bh = pd.offsets.BusinessHour()
bh
```

```
# 2014-08-01 is Friday
pd.Timestamp("2014-08-01 10:00").weekday()
pd.Timestamp("2014-08-01 10:00") + bh

# Below example is the same as: pd.Timestamp('2014-08-01 09:00') + bh
pd.Timestamp("2014-08-01 08:00") + bh

# If the results is on the end time, move to the next business day
pd.Timestamp("2014-08-01 16:00") + bh

# Remainings are added to the next day
pd.Timestamp("2014-08-01 16:30") + bh

# Adding 2 business hours
pd.Timestamp("2014-08-01 10:00") + pd.offsets.BusinessHour(2)

# Subtracting 3 business hours
pd.Timestamp("2014-08-01 10:00") + pd.offsets.BusinessHour(-3)
```

You can also specify `start` and `end` time by keywords. The argument must be a `str` with an `hour:minute` representation or a `datetime.time` instance. Specifying seconds, microseconds and nanoseconds as business hour results in `ValueError`.

```
bh = pd.offsets.BusinessHour(start="11:00", end=datetime.time(20, 0))
bh

pd.Timestamp("2014-08-01 13:00") + bh
pd.Timestamp("2014-08-01 09:00") + bh
pd.Timestamp("2014-08-01 18:00") + bh
```

Passing `start` time later than `end` represents midnight business hour. In this case, business hour exceeds midnight and overlap to the next day. Valid business hours are distinguished by whether it started from valid `BusinessDay`.

```
bh = pd.offsets.BusinessHour(start="17:00", end="09:00")
bh

pd.Timestamp("2014-08-01 17:00") + bh
pd.Timestamp("2014-08-01 23:00") + bh

# Although 2014-08-02 is Saturday,
# it is valid because it starts from 08-01 (Friday).
pd.Timestamp("2014-08-02 04:00") + bh

# Although 2014-08-04 is Monday,
# it is out of business hours because it starts from 08-03 (Sunday).
pd.Timestamp("2014-08-04 04:00") + bh
```

Applying `BusinessHour.rollforward` and `rollback` to out of business hours results in the next business hour start or previous day's end. Different from other offsets, `BusinessHour.rollforward` may output different results from `apply` by definition.

This is because one day's business hour end is equal to next day's business hour start. For example, under the default business hours (9:00 - 17:00), there is no gap (0 minutes) between `2014-08-01 17:00` and `2014-08-04 09:00`.

```
# This adjusts a Timestamp to business hour edge
pd.offsets.BusinessHour().rollback(pd.Timestamp("2014-08-02 15:00"))
pd.offsets.BusinessHour().rollforward(pd.Timestamp("2014-08-02 15:00"))
```

```
# It is the same as BusinessHour() + pd.Timestamp('2014-08-01 17:00').
# And it is the same as BusinessHour() + pd.Timestamp('2014-08-04 09:00')
pd.offsets.BusinessHour() + pd.Timestamp("2014-08-02 15:00")

# BusinessDay results (for reference)
pd.offsets.BusinessHour().rollforward(pd.Timestamp("2014-08-02"))

# It is the same as BusinessDay() + pd.Timestamp('2014-08-01')
# The result is the same as rollworward because BusinessDay never overlap.
pd.offsets.BusinessHour() + pd.Timestamp("2014-08-02")
```

`BusinessHour` regards Saturday and Sunday as holidays. To use arbitrary holidays, you can use `CustomBusinessHour` offset, as explained in the following subsection.

Custom business hour

The `CustomBusinessHour` is a mixture of `BusinessHour` and `CustomBusinessDay` which allows you to specify arbitrary holidays. `CustomBusinessHour` works as the same as `BusinessHour` except that it skips specified custom holidays.

```
from pandas.tseries.holiday import USFederalHolidayCalendar

bhour_us = pd.offsets.CustomBusinessHour(calendar=USFederalHolidayCalendar())
# Friday before MLK Day
dt = datetime.datetime(2014, 1, 17, 15)

dt + bhour_us

# Tuesday after MLK Day (Monday is skipped because it's a holiday)
dt + bhour_us * 2
```

You can use keyword arguments supported by either `BusinessHour` and `CustomBusinessDay`.

```
bhour_mon = pd.offsets.CustomBusinessHour(start="10:00", weekmask="Tue Wed Thu Fri")

# Monday is skipped because it's a holiday, business hour starts from 10:00
dt + bhour_mon * 2
```

Offset aliases

A number of string aliases are given to useful common time series frequencies. We will refer to these aliases as *offset aliases*.

Alias	Description
B	business day frequency
C	custom business day frequency
D	calendar day frequency
W	weekly frequency (default anchored to Sunday, i.e. W-SUN)
ME	month end frequency
SME	semi-month end frequency (15th and end of month)
BME	business month end frequency
CBME	custom business month end frequency
MS	month start frequency
SMS	semi-month start frequency (1st and 15th)
BMS	business month start frequency
CBMS	custom business month start frequency

Alias	Description
QE	quarter end frequency
BQE	business quarter end frequency
QS	quarter start frequency
BQS	business quarter start frequency
YE	year end frequency
BYE	business year end frequency
YS	year start frequency
BYS	business year start frequency
h	hourly frequency
bh	business hour frequency
cbh	custom business hour frequency
min	minutely frequency
s	secondly frequency
ms	milliseconds
us	microseconds
ns	nanoseconds

Note
When using the offset aliases above, it should be noted that functions such as `date_range`, `bdate_range`, will only return timestamps that are in the interval defined by `start_date` and `end_date`. If the `start_date` does not correspond to the frequency, the returned timestamps will start at the next valid timestamp, same for `end_date`, the returned timestamps will stop at the previous valid timestamp.
For example, for the offset `MS`, if the `start_date` is not the first of the month, the returned timestamps will start with the first day of the next month. If `end_date` is not the first day of a month, the last returned timestamp will be the first day of the corresponding month.

```
dates_lst_1 = pd.date_range("2020-01-06", "2020-04-03", freq="MS")
dates_lst_1

dates_lst_2 = pd.date_range("2020-01-01", "2020-04-01", freq="MS")
dates_lst_2
```

We can see in the above example `date_range` and `start_date` and `end_date`. If these are not valid timestamps for the given frequency it will roll to the next value for `start_date` (respectively previous for the `end_date`)

Period aliases

A number of string aliases are given to useful common time series frequencies. We will refer to these aliases as *period aliases*.

Alias	Description
B	business day frequency
D	calendar day frequency
W	weekly frequency (default anchored to Sunday, i.e. W-SUN)
M	monthly frequency
Q	quarterly frequency
Y	yearly frequency
h	hourly frequency
min	minutely frequency
s	secondly frequency

Alias	Description
ms	milliseconds
us	microseconds
ns	nanoseconds

Combining aliases

As we have seen previously, the alias and the offset instance are fungible in most functions:

```
pd.date_range(start, periods=5, freq="B")

pd.date_range(start, periods=5, freq=pd.offsets.BDay())
```

You can combine together day and intraday offsets:

```
pd.date_range(start, periods=10, freq="2h20min")

pd.date_range(start, periods=10, freq="1D10us")
```

Anchored offsets

For some frequencies you can specify an anchoring suffix:

Alias	Description
W-SUN	weekly frequency, week ends on Sunday. Same as 'W'
W-MON	weekly frequency, week ends on Monday
W-TUE	weekly frequency, week ends on Tuesday
W-WED	weekly frequency, week ends on Wednesday
W-THU	weekly frequency, week ends on Thursday
W-FRI	weekly frequency, week ends on Friday
W-SAT	weekly frequency, week ends on Saturday
(B)QE-DEC	quarterly frequency, year ends in December. Same as 'QE'
(B)QE-JAN	quarterly frequency, year ends in January
(B)QE-FEB	quarterly frequency, year ends in February
(B)QE-MAR	quarterly frequency, year ends in March
(B)QE-APR	quarterly frequency, year ends in April
(B)QE-MAY	quarterly frequency, year ends in May
(B)QE-JUN	quarterly frequency, year ends in June
(B)QE-JUL	quarterly frequency, year ends in July
(B)QE-AUG	quarterly frequency, year ends in August
(B)QE-SEP	quarterly frequency, year ends in September
(B)QE-OCT	quarterly frequency, year ends in October
(B)QE-NOV	quarterly frequency, year ends in November
(B)QS-JAN	quarterly frequency, year starts in January. Same as 'QS'
(B)QS-FEB	quarterly frequency, year starts in February
(B)QS-MAR	quarterly frequency, year starts in March
(B)QS-APR	quarterly frequency, year starts in April
(B)QS-MAY	quarterly frequency, year starts in May
(B)QS-JUN	quarterly frequency, year starts in June
(B)QS-JUL	quarterly frequency, year starts in July
(B)QS-AUG	quarterly frequency, year starts in August
(B)QS-SEP	quarterly frequency, year starts in September
(B)QS-OCT	quarterly frequency, year starts in October
(B)QS-NOV	quarterly frequency, year starts in November

Alias	Description
(B)QS-DEC	quarterly frequency, year starts in December
(B)YE-DEC	annual frequency, anchored end of December. Same as 'YE'
(B)YE-JAN	annual frequency, anchored end of January
(B)YE-FEB	annual frequency, anchored end of February
(B)YE-MAR	annual frequency, anchored end of March
(B)YE-APR	annual frequency, anchored end of April
(B)YE-MAY	annual frequency, anchored end of May
(B)YE-JUN	annual frequency, anchored end of June
(B)YE-JUL	annual frequency, anchored end of July
(B)YE-AUG	annual frequency, anchored end of August
(B)YE-SEP	annual frequency, anchored end of September
(B)YE-OCT	annual frequency, anchored end of October
(B)YE-NOV	annual frequency, anchored end of November
(B)YS-JAN	annual frequency, anchored start of January. Same as 'YS'
(B)YS-FEB	annual frequency, anchored start of February
(B)YS-MAR	annual frequency, anchored start of March
(B)YS-APR	annual frequency, anchored start of April
(B)YS-MAY	annual frequency, anchored start of May
(B)YS-JUN	annual frequency, anchored start of June
(B)YS-JUL	annual frequency, anchored start of July
(B)YS-AUG	annual frequency, anchored start of August
(B)YS-SEP	annual frequency, anchored start of September
(B)YS-OCT	annual frequency, anchored start of October
(B)YS-NOV	annual frequency, anchored start of November
(B)YS-DEC	annual frequency, anchored start of December

These can be used as arguments to `date_range`, `bdate_range`, constructors for `DatetimeIndex`, as well as various other timeseries-related functions in pandas.

For a calendar step that is not anchored, such as yearly from an arbitrary date, pass a `DateOffset` like `freq=pd.DateOffset(years=1)`.

Anchored offset semantics

For those offsets that are anchored to the start or end of specific frequency (`MonthEnd`, `MonthBegin`, `WeekEnd`, etc), the following rules apply to rolling forward and backwards.

When `n` is not 0, if the given date is not on an anchor point, it snapped to the next(previous) anchor point, and moved `|n|-1` additional steps forwards or backwards.

```
pd.Timestamp("2014-01-02") + pd.offsets.MonthBegin(n=1)
pd.Timestamp("2014-01-02") + pd.offsets.MonthEnd(n=1)

pd.Timestamp("2014-01-02") - pd.offsets.MonthBegin(n=1)
pd.Timestamp("2014-01-02") - pd.offsets.MonthEnd(n=1)

pd.Timestamp("2014-01-02") + pd.offsets.MonthBegin(n=4)
pd.Timestamp("2014-01-02") - pd.offsets.MonthBegin(n=4)
```

If the given date *is* on an anchor point, it is moved `|n|` points forwards or backwards.

```
pd.Timestamp("2014-01-01") + pd.offsets.MonthBegin(n=1)
pd.Timestamp("2014-01-31") + pd.offsets.MonthEnd(n=1)

pd.Timestamp("2014-01-01") - pd.offsets.MonthBegin(n=1)
```

```
pd.Timestamp("2014-01-31") - pd.offsets.MonthEnd(n=1)

pd.Timestamp("2014-01-01") + pd.offsets.MonthBegin(n=4)
pd.Timestamp("2014-01-31") - pd.offsets.MonthBegin(n=4)
```

For the case when `n=0`, the date is not moved if on an anchor point, otherwise it is rolled forward to the next anchor point.

```
pd.Timestamp("2014-01-02") + pd.offsets.MonthBegin(n=0)
pd.Timestamp("2014-01-02") + pd.offsets.MonthEnd(n=0)

pd.Timestamp("2014-01-01") + pd.offsets.MonthBegin(n=0)
pd.Timestamp("2014-01-31") + pd.offsets.MonthEnd(n=0)
```

Holidays / holiday calendars

Holidays and calendars provide a simple way to define holiday rules to be used with `CustomBusinessDay` or in other analysis that requires a predefined set of holidays. The `AbstractHolidayCalendar` class provides all the necessary methods to return a list of holidays and only `rules` need to be defined in a specific holiday calendar class. Furthermore, the `start_date` and `end_date` class attributes determine over what date range holidays are generated. These should be overwritten on the `AbstractHolidayCalendar` class to have the range apply to all calendar subclasses. `USFederalHolidayCalendar` is the only calendar that exists and primarily serves as an example for developing other calendars.

For holidays that occur on fixed dates (e.g., US Memorial Day or July 4th) an observance rule determines when that holiday is observed if it falls on a weekend or some other non-observed day. Defined observance rules are:

Rule	Description
next_workday	move to the next weekday, always advancing by at least one day
previous_workday	move to the previous weekday, always moving back by at least one day
nearest_workday	move Saturday to Friday and Sunday to Monday
before_nearest_workday	apply `nearest_workday` and then move to previous workday before that day
after_nearest_workday	apply `nearest_workday` and then move to next workday after that day
sunday_to_monday	move Sunday to following Monday
next_monday_or_tuesday	move Saturday to Monday and Sunday/Monday to Tuesday
previous_friday	move Saturday and Sunday to previous Friday
next_monday	move Saturday and Sunday to following Monday
weekend_to_monday	same as `next_monday`

An example of how holidays and holiday calendars are defined:

```
from pandas.tseries.holiday import (
    Holiday,
    USMemorialDay,
    AbstractHolidayCalendar,
    nearest_workday,
    MO,
)

class ExampleCalendar(AbstractHolidayCalendar):
    rules = [
        USMemorialDay,
        Holiday("July 4th", month=7, day=4, observance=nearest_workday),
```

```
        Holiday(
            "Columbus Day",
            month=10,
            day=1,
            offset=pd.DateOffset(weekday=MO(2)),
        ),
    ]

cal = ExampleCalendar()
cal.holidays(datetime.datetime(2012, 1, 1), datetime.datetime(2012, 12, 31))
```

hint **weekday=MO(2)** is same as **2 * Week(weekday=2)**

Using this calendar, creating an index or doing offset arithmetic skips weekends and holidays (i.e., Memorial Day/July 4th). For example, the below defines a custom business day offset using the `ExampleCalendar`. Like any other offset, it can be used to create a `DatetimeIndex` or added to `datetime` or `Timestamp` objects.

```
pd.date_range(
    start="7/1/2012", end="7/10/2012", freq=pd.offsets.CDay(calendar=cal)
).to_pydatetime()
offset = pd.offsets.CustomBusinessDay(calendar=cal)
datetime.datetime(2012, 5, 25) + offset
datetime.datetime(2012, 7, 3) + offset
datetime.datetime(2012, 7, 3) + 2 * offset
datetime.datetime(2012, 7, 6) + offset
```

Ranges are defined by the `start_date` and `end_date` class attributes of `AbstractHolidayCalendar`. The defaults are shown below.

```
AbstractHolidayCalendar.start_date
AbstractHolidayCalendar.end_date
```

These dates can be overwritten by setting the attributes as datetime/Timestamp/string.

```
AbstractHolidayCalendar.start_date = datetime.datetime(2012, 1, 1)
AbstractHolidayCalendar.end_date = datetime.datetime(2012, 12, 31)
cal.holidays()
```

Every calendar class is accessible by name using the `get_calendar` function which returns a holiday class instance. Any imported calendar class will automatically be available by this function. Also, `HolidayCalendarFactory` provides an easy interface to create calendars that are combinations of calendars or calendars with additional rules.

```
from pandas.tseries.holiday import get_calendar, HolidayCalendarFactory, USLaborDay

cal = get_calendar("ExampleCalendar")
cal.rules
new_cal = HolidayCalendarFactory("NewExampleCalendar", cal, USLaborDay)
new_cal.rules
```

Time Series-related instance methods

Shifting / lagging

One may want to *shift* or *lag* the values in a time series back and forward in time. The method for this is `shift`, which is available on all of the pandas objects.

```
ts = pd.Series(range(len(rng)), index=rng)
ts = ts[:5]
ts.shift(1)
```

The `shift` method accepts a `freq` argument which can accept a `DateOffset` class or other `timedelta`-like object or also an offset alias.

When `freq` is specified, `shift` method changes all the dates in the index rather than changing the alignment of the data and the index:

```
ts.shift(5, freq="D")
ts.shift(5, freq=pd.offsets.BDay())
ts.shift(5, freq="BME")
```

Note that with when `freq` is specified, the leading entry is no longer NaN because the data is not being realigned.

Frequency conversion

The primary function for changing frequencies is the `asfreq` method. For a `DatetimeIndex`, this is basically just a thin, but convenient wrapper around `reindex` which generates a `date_range` and calls `reindex`.

```
dr = pd.date_range("1/1/2010", periods=3, freq=3 * pd.offsets.BDay())
ts = pd.Series(np.random.randn(3), index=dr)
ts
ts.asfreq(pd.offsets.BDay())
```

`asfreq` provides a further convenience so you can specify an interpolation method for any gaps that may appear after the frequency conversion.

```
ts.asfreq(pd.offsets.BDay(), method="pad")
```

Filling forward / backward

Related to `asfreq` and `reindex` is `fillna`, which is documented in the missing data section.

Converting to Python datetimes

`DatetimeIndex` can be converted to an array of Python native `datetime.datetime` objects using the `to_pydatetime` method.

Resampling

pandas has a simple, powerful, and efficient functionality for performing resampling operations during frequency conversion (e.g., converting secondly data into 5-minutely data). This is extremely common in, but not limited to, financial applications.

`resample` is a time-based groupby, followed by a reduction method on each of its groups. See some cookbook examples for some advanced strategies.

The `resample()` method can be used directly from `DataFrameGroupBy` objects, see the groupby docs.

Basics

```
rng = pd.date_range("1/1/2012", periods=100, freq="s")

ts = pd.Series(np.random.randint(0, 500, len(rng)), index=rng)

ts.resample("5Min").sum()
```

The `resample` function is very flexible and allows you to specify many different parameters to control the frequency conversion and resampling operation.

Any built-in method available via GroupBy is available as a method of the returned object, including `sum`, `mean`, `std`, `sem`, `max`, `min`, `median`, `first`, `last`, `ohlc`:

```
ts.resample("5Min").mean()

ts.resample("5Min").ohlc()

ts.resample("5Min").max()
```

For downsampling, `closed` can be set to 'left' or 'right' to specify which end of the interval is closed:

```
ts.resample("5Min", closed="right").mean()

ts.resample("5Min", closed="left").mean()
```

Parameters like `label` are used to manipulate the resulting labels. `label` specifies whether the result is labeled with the beginning or the end of the interval.

```
ts.resample("5Min").mean()  # by default label='left'

ts.resample("5Min", label="left").mean()
```

Warning
The default values for `label` and `closed` is '**left**' for all frequency offsets except for 'ME', 'YE', 'QE', 'BME', 'BYE', 'BQE', and 'W' which all have a default of 'right'.
This might unintendedly lead to looking ahead, where the value for a later time is pulled back to a previous time as in the following example with the `BusinessDay` frequency:

```
s = pd.date_range("2000-01-01", "2000-01-05").to_series()
s.iloc[2] = pd.NaT
s.dt.day_name()

# default: label='left', closed='left'
s.resample("B").last().dt.day_name()
```

Notice how the value for Sunday got pulled back to the previous Friday. To get the behavior where the value for Sunday is pushed to Monday, use instead

```
s.resample("B", label="right", closed="right").last().dt.day_name()
```

The `axis` parameter can be set to 0 or 1 and allows you to resample the specified axis for a `DataFrame`.

`kind` can be set to 'timestamp' or 'period' to convert the resulting index to/from timestamp and time span representations. By default `resample` retains the input representation.

`convention` can be set to 'start' or 'end' when resampling period data (detail below). It specifies how low frequency periods are converted to higher frequency periods.

Upsampling

For upsampling, you can specify a way to upsample and the `limit` parameter to interpolate over the gaps that are created:

```
# from secondly to every 250 milliseconds

ts[:2].resample("250ms").asfreq()

ts[:2].resample("250ms").ffill()

ts[:2].resample("250ms").ffill(limit=2)
```

Sparse resampling

Sparse timeseries are the ones where you have a lot fewer points relative to the amount of time you are looking to resample. Naively upsampling a sparse series can potentially generate lots of intermediate values. When you don't want to use a method to fill these values, e.g. `fill_method` is `None`, then intermediate values will be filled with `NaN`.

Since `resample` is a time-based groupby, the following is a method to efficiently resample only the groups that are not all `NaN`.

```
rng = pd.date_range("2014-1-1", periods=100, freq="D") + pd.Timedelta("1s")
ts = pd.Series(range(100), index=rng)
```

If we want to resample to the full range of the series:

```
ts.resample("3min").sum()
```

We can instead only resample those groups where we have points as follows:

```
from functools import partial
from pandas.tseries.frequencies import to_offset

def round(t, freq):
    # round a Timestamp to a specified freq
    freq = to_offset(freq)
    td = pd.Timedelta(freq)
    return pd.Timestamp((t.value // td.value) * td.value)

ts.groupby(partial(round, freq="3min")).sum()
```

Aggregation

The `resample()` method returns a `pandas.api.typing.Resampler` instance. Similar to the aggregating API, groupby API, and the window API, a `Resampler` can be selectively resampled.

Resampling a `DataFrame`, the default will be to act on all columns with the same function.

```
df = pd.DataFrame(
    np.random.randn(1000, 3),
    index=pd.date_range("1/1/2012", freq="s", periods=1000),
    columns=["A", "B", "C"],
)
r = df.resample("3min")
r.mean()
```

We can select a specific column or columns using standard getitem.

```
r["A"].mean()

r[["A", "B"]].mean()
```

You can pass a list or dict of functions to do aggregation with, outputting a `DataFrame`:

```
r["A"].agg(["sum", "mean", "std"])
```

On a resampled `DataFrame`, you can pass a list of functions to apply to each column, which produces an aggregated result with a hierarchical index:

```
r.agg(["sum", "mean"])
```

By passing a dict to `aggregate` you can apply a different aggregation to the columns of a `DataFrame`:

```
r.agg({"A": "sum", "B": lambda x: np.std(x, ddof=1)})
```

The function names can also be strings. In order for a string to be valid it must be implemented on the resampled object:

```
r.agg({"A": "sum", "B": "std"})
```

Furthermore, you can also specify multiple aggregation functions for each column separately.

```
r.agg({"A": ["sum", "std"], "B": ["mean", "std"]})
```

If a `DataFrame` does not have a datetimelike index, but instead you want to resample based on datetimelike column in the frame, it can passed to the `on` keyword.

```
df = pd.DataFrame(
    {"date": pd.date_range("2015-01-01", freq="W", periods=5), "a": np.arange(5)},
    index=pd.MultiIndex.from_arrays(
        [[1, 2, 3, 4, 5], pd.date_range("2015-01-01", freq="W", periods=5)],
        names=["v", "d"],
    ),
)
df
df.resample("MS", on="date")[["a"]].sum()
```

Similarly, if you instead want to resample by a datetimelike level of `MultiIndex`, its name or location can be passed to the `level` keyword.

```
df.resample("MS", level="d")[["a"]].sum()
```

Iterating through groups

With the `Resampler` object in hand, iterating through the grouped data is very natural and functions similarly to `itertools.groupby`:

```
small = pd.Series(
    range(6),
    index=pd.to_datetime(
        [
            "2017-01-01T00:00:00",
            "2017-01-01T00:30:00",
            "2017-01-01T00:31:00",
            "2017-01-01T01:00:00",
            "2017-01-01T03:00:00",
            "2017-01-01T03:05:00",
        ]
```

```
    ),
)
resampled = small.resample("h")

for name, group in resampled:
    print("Group: ", name)
    print("-" * 27)
    print(group, end="\n\n")
```

See groupby.iterating-label or `Resampler.__iter__` for more.

Use `origin` or `offset` to adjust the start of the bins

The bins of the grouping are adjusted based on the beginning of the day of the time series starting point. This works well with frequencies that are multiples of a day (like `30D`) or that divide a day evenly (like `90s` or `1min`). This can create inconsistencies with some frequencies that do not meet this criteria. To change this behavior you can specify a fixed Timestamp with the argument `origin`.

For example:

```
start, end = "2000-10-01 23:30:00", "2000-10-02 00:30:00"
middle = "2000-10-02 00:00:00"
rng = pd.date_range(start, end, freq="7min")
ts = pd.Series(np.arange(len(rng)) * 3, index=rng)
ts
```

Here we can see that, when using `origin` with its default value (`'start_day'`), the result after `'2000-10-02 00:00:00'` are not identical depending on the start of time series:

```
ts.resample("17min", origin="start_day").sum()
ts[middle:end].resample("17min", origin="start_day").sum()
```

Here we can see that, when setting `origin` to `'epoch'`, the result after `'2000-10-02 00:00:00'` are identical depending on the start of time series:

```
ts.resample("17min", origin="epoch").sum()
ts[middle:end].resample("17min", origin="epoch").sum()
```

If needed you can use a custom timestamp for `origin`:

```
ts.resample("17min", origin="2001-01-01").sum()
ts[middle:end].resample("17min", origin=pd.Timestamp("2001-01-01")).sum()
```

If needed you can just adjust the bins with an `offset` Timedelta that would be added to the default `origin`. Those two examples are equivalent for this time series:

```
ts.resample("17min", origin="start").sum()
ts.resample("17min", offset="23h30min").sum()
```

Note the use of `'start'` for `origin` on the last example. In that case, `origin` will be set to the first value of the timeseries.

Backward resample

Instead of adjusting the beginning of bins, sometimes we need to fix the end of the bins to make a backward resample with a given `freq`. The backward resample sets `closed` to `'right'` by default since the last value should be considered as the edge point for the last bin.

We can set `origin` to `'end'`. The value for a specific `Timestamp` index stands for the resample result from the current `Timestamp` minus `freq` to the current `Timestamp` with a right close.

```
ts.resample('17min', origin='end').sum()
```

Besides, in contrast with the `'start_day'` option, `end_day` is supported. This will set the origin as the ceiling midnight of the largest `Timestamp`.

```
ts.resample('17min', origin='end_day').sum()
```

The above result uses `2000-10-02 00:29:00` as the last bin's right edge since the following computation.

```
ceil_mid = rng.max().ceil('D')
freq = pd.offsets.Minute(17)
bin_res = ceil_mid - freq * ((ceil_mid - rng.max()) // freq)
bin_res
```

Time span representation

Regular intervals of time are represented by `Period` objects in pandas while sequences of `Period` objects are collected in a `PeriodIndex`, which can be created with the convenience function `period_range`.

Period

A `Period` represents a span of time (e.g., a day, a month, a quarter, etc). You can specify the span via `freq` keyword using a frequency alias like below. Because `freq` represents a span of `Period`, it cannot be negative like "-3D".

```
pd.Period("2012", freq="Y-DEC")

pd.Period("2012-1-1", freq="D")

pd.Period("2012-1-1 19:00", freq="h")

pd.Period("2012-1-1 19:00", freq="5h")
```

Adding and subtracting integers from periods shifts the period by its own frequency. Arithmetic is not allowed between `Period` with different `freq` (span).

```
p = pd.Period("2012", freq="Y-DEC")
p + 1
p - 3
p = pd.Period("2012-01", freq="2M")
p + 2
p - 1
p == pd.Period("2012-01", freq="3M")
```

If `Period` freq is daily or higher (`D`, `h`, `min`, `s`, `ms`, `us`, and `ns`), `offsets` and `timedelta`-like can be added if the result can have the same freq. Otherwise, `ValueError` will be raised.

```
p = pd.Period("2014-07-01 09:00", freq="h")
p + pd.offsets.Hour(2)
p + datetime.timedelta(minutes=120)
p + np.timedelta64(7200, "s")
```

```
p + pd.offsets.Minute(5)
```

If `Period` has other frequencies, only the same `offsets` can be added. Otherwise, `ValueError` will be raised.

```
p = pd.Period("2014-07", freq="M")
p + pd.offsets.MonthEnd(3)
```

```
p + pd.offsets.MonthBegin(3)
```

Taking the difference of `Period` instances with the same frequency will return the number of frequency units between them:

```
pd.Period("2012", freq="Y-DEC") - pd.Period("2002", freq="Y-DEC")
```

PeriodIndex and period_range

Regular sequences of `Period` objects can be collected in a `PeriodIndex`, which can be constructed using the `period_range` convenience function:

```
prng = pd.period_range("1/1/2011", "1/1/2012", freq="M")
prng
```

The `PeriodIndex` constructor can also be used directly:

```
pd.PeriodIndex(["2011-1", "2011-2", "2011-3"], freq="M")
```

Passing multiplied frequency outputs a sequence of `Period` which has multiplied span.

```
pd.period_range(start="2014-01", freq="3M", periods=4)
```

If `start` or `end` are `Period` objects, they will be used as anchor endpoints for a `PeriodIndex` with frequency matching that of the `PeriodIndex` constructor.

```
pd.period_range(
    start=pd.Period("2017Q1", freq="Q"), end=pd.Period("2017Q2", freq="Q"), freq="M"
)
```

Just like `DatetimeIndex`, a `PeriodIndex` can also be used to index pandas objects:

```
ps = pd.Series(np.random.randn(len(prng)), prng)
ps
```

`PeriodIndex` supports addition and subtraction with the same rule as `Period`.

```
idx = pd.period_range("2014-07-01 09:00", periods=5, freq="h")
idx
idx + pd.offsets.Hour(2)

idx = pd.period_range("2014-07", periods=5, freq="M")
idx
idx + pd.offsets.MonthEnd(3)
```

`PeriodIndex` has its own dtype named `period`, refer to Period Dtypes.

Period dtypes

`PeriodIndex` has a custom `period` dtype. This is a pandas extension dtype similar to the timezone aware dtype (`datetime64[ns, tz]`).

The `period` dtype holds the `freq` attribute and is represented with `period[freq]` like `period[D]` or `period[M]`, using frequency strings.

```
pi = pd.period_range("2016-01-01", periods=3, freq="M")
pi
pi.dtype
```

The `period` dtype can be used in `.astype(...)`. It allows one to change the `freq` of a `PeriodIndex` like `.asfreq()` and convert a `DatetimeIndex` to `PeriodIndex` like `to_period()`:

```
# change monthly freq to daily freq
pi.astype("period[D]")

# convert to DatetimeIndex
pi.astype("datetime64[ns]")

# convert to PeriodIndex
dti = pd.date_range("2011-01-01", freq="ME", periods=3)
dti
dti.astype("period[M]")
```

PeriodIndex partial string indexing

PeriodIndex now supports partial string slicing with non-monotonic indexes.

You can pass in dates and strings to `Series` and `DataFrame` with `PeriodIndex`, in the same manner as `DatetimeIndex`. For details, refer to DatetimeIndex Partial String Indexing.

```
ps["2011-01"]

ps[datetime.datetime(2011, 12, 25):]

ps["10/31/2011":"12/31/2011"]
```

Passing a string representing a lower frequency than `PeriodIndex` returns partial sliced data.

```
ps["2011"]

dfp = pd.DataFrame(
    np.random.randn(600, 1),
    columns=["A"],
    index=pd.period_range("2013-01-01 9:00", periods=600, freq="min"),
)
dfp
dfp.loc["2013-01-01 10h"]
```

As with `DatetimeIndex`, the endpoints will be included in the result. The example below slices data starting from 10:00 to 11:59.

```
dfp["2013-01-01 10h":"2013-01-01 11h"]
```

Frequency conversion and resampling with PeriodIndex

The frequency of `Period` and `PeriodIndex` can be converted via the `asfreq` method. Let's start with the fiscal year 2011, ending in December:

```
p = pd.Period("2011", freq="Y-DEC")
p
```

We can convert it to a monthly frequency. Using the `how` parameter, we can specify whether to return the starting or ending month:

```
p.asfreq("M", how="start")

p.asfreq("M", how="end")
```

The shorthands 's' and 'e' are provided for convenience:

```
p.asfreq("M", "s")
p.asfreq("M", "e")
```

Converting to a "super-period" (e.g., annual frequency is a super-period of quarterly frequency) automatically returns the super-period that includes the input period:

```
p = pd.Period("2011-12", freq="M")

p.asfreq("Y-NOV")
```

Note that since we converted to an annual frequency that ends the year in November, the monthly period of December 2011 is actually in the 2012 Y-NOV period.

Period conversions with anchored frequencies are particularly useful for working with various quarterly data common to economics, business, and other fields. Many organizations define quarters relative to the month in which their fiscal year starts and ends. Thus, first quarter of 2011 could start in 2010 or a few months into 2011. Via anchored frequencies, pandas works for all quarterly frequencies `Q-JAN` through `Q-DEC`.

`Q-DEC` define regular calendar quarters:

```
p = pd.Period("2012Q1", freq="Q-DEC")

p.asfreq("D", "s")

p.asfreq("D", "e")
```

`Q-MAR` defines fiscal year end in March:

```
p = pd.Period("2011Q4", freq="Q-MAR")

p.asfreq("D", "s")

p.asfreq("D", "e")
```

Converting between representations

Timestamped data can be converted to PeriodIndex-ed data using `to_period` and vice-versa using `to_timestamp`:

```
rng = pd.date_range("1/1/2012", periods=5, freq="ME")

ts = pd.Series(np.random.randn(len(rng)), index=rng)

ts

ps = ts.to_period()

ps

ps.to_timestamp()
```

Remember that 's' and 'e' can be used to return the timestamps at the start or end of the period:

```
ps.to_timestamp("D", how="s")
```

Converting between period and timestamp enables some convenient arithmetic functions to be used. In the following example, we convert a quarterly frequency with year ending in November to 9am of the end of the month following the quarter end:

```
prng = pd.period_range("1990Q1", "2000Q4", freq="Q-NOV")

ts = pd.Series(np.random.randn(len(prng)), prng)

ts.index = (prng.asfreq("M", "e") + 1).asfreq("h", "s") + 9

ts.head()
```

Representing out-of-bounds spans

If you have data that is outside of the `Timestamp` bounds, see Timestamp limitations, then you can use a `PeriodIndex` and/or `Series` of `Periods` to do computations.

```
span = pd.period_range("1215-01-01", "1381-01-01", freq="D")
span
```

To convert from an `int64` based YYYYMMDD representation.

```
s = pd.Series([20121231, 20141130, 99991231])
s

def conv(x):
    return pd.Period(year=x // 10000, month=x // 100 % 100, day=x % 100, freq="D")

s.apply(conv)
s.apply(conv)[2]
```

These can easily be converted to a `PeriodIndex`:

```
span = pd.PeriodIndex(s.apply(conv))
span
```

Time zone handling

pandas provides rich support for working with timestamps in different time zones using the `zoneinfo`, `pytz` and `dateutil` libraries or `datetime.timezone` objects from the standard library.

Working with time zones

By default, pandas objects are time zone unaware:

```
rng = pd.date_range("3/6/2012 00:00", periods=15, freq="D")
rng.tz is None
```

To localize these dates to a time zone (assign a particular time zone to a naive date), you can use the `tz_localize` method or the `tz` keyword argument in `date_range`, `Timestamp`, or `DatetimeIndex`. You can either pass `zoneinfo`, `pytz` or `dateutil` time zone objects or Olson time zone database

strings. Olson time zone strings will return `zoneinfo` time zone objects by default. To return `dateutil` time zone objects, append `dateutil/` before the string.

- For `zoneinfo`, a list of available timezones are available from `zoneinfo.available_timezones`.
- If `pytz` is installed (optional), you can find a list of common (and less common) time zones using `pytz.all_timezones`.
- `dateutil` uses the OS time zones so there isn't a fixed list available. For common zones, the names are the same as `zoneinfo`.

```
import dateutil
from zoneinfo import ZoneInfo

# zoneinfo (default when using Olson strings)
rng_zi = pd.date_range("3/6/2012 00:00", periods=3, freq="D", tz="Europe/London")
rng_zi.tz

# dateutil
rng_dateutil = pd.date_range("3/6/2012 00:00", periods=3, freq="D")
rng_dateutil = rng_dateutil.tz_localize("dateutil/Europe/London")
rng_dateutil.tz

# dateutil - utc special case
rng_utc = pd.date_range(
    "3/6/2012 00:00",
    periods=3,
    freq="D",
    tz=dateutil.tz.tzutc(),
)
rng_utc.tz
```

```
# datetime.timezone
rng_utc = pd.date_range(
    "3/6/2012 00:00",
    periods=3,
    freq="D",
    tz=datetime.timezone.utc,
)
rng_utc.tz
```

Note that the `UTC` time zone is a special case in `dateutil` and should be constructed explicitly as an instance of `dateutil.tz.tzutc`. You can also construct other time zones objects explicitly first.

```
# zoneinfo
tz_zi = ZoneInfo("Europe/London")
rng_zi = pd.date_range("3/6/2012 00:00", periods=3, freq="D")
rng_zi = rng_zi.tz_localize(tz_zi)
rng_zi.tz == tz_zi

# dateutil
tz_dateutil = dateutil.tz.gettz("Europe/London")
rng_dateutil = pd.date_range("3/6/2012 00:00", periods=3, freq="D", tz=tz_dateutil)
rng_dateutil.tz == tz_dateutil
```

To convert a time zone aware pandas object from one time zone to another, you can use the `tz_convert` method.

```
rng_zi.tz_convert("US/Eastern")
```

Warning
Be wary of conversions between time zone libraries. For some time zones, different libraries may have different definitions of the zone. This is more of a problem for unusual time zones than for 'standard' zones like `US/Eastern`.

Warning
Be aware that a time zone definition across versions of time zone libraries may not be considered equal. This may cause problems when working with stored data that is localized using one version and operated on with a different version. See here for how to handle such a situation.

Warning
Be aware that for times in the future, correct conversion between time zones (and UTC) cannot be guaranteed by any time zone library because a timezone's offset from UTC may be changed by the respective government.

Under the hood, all timestamps are stored in UTC. Values from a time zone aware `DatetimeIndex` or `Timestamp` will have their fields (day, hour, minute, etc.) localized to the time zone. However, timestamps with the same UTC value are still considered to be equal even if they are in different time zones:

```
rng_eastern = rng_utc.tz_convert("US/Eastern")
rng_berlin = rng_utc.tz_convert("Europe/Berlin")

rng_eastern[2]
rng_berlin[2]
rng_eastern[2] == rng_berlin[2]
```

Operations between `Series` in different time zones will yield UTC `Series`, aligning the data on the UTC timestamps:

```
ts_utc = pd.Series(range(3), pd.date_range("20130101", periods=3, tz="UTC"))
eastern = ts_utc.tz_convert("US/Eastern")
berlin = ts_utc.tz_convert("Europe/Berlin")
result = eastern + berlin
result
result.index
```

To remove time zone information, use `tz_localize(None)` or `tz_convert(None)`. `tz_localize(None)` will remove the time zone yielding the local time representation. `tz_convert(None)` will remove the time zone after converting to UTC time.

```
didx = pd.date_range(start="2014-08-01 09:00", freq="h", periods=3, tz="US/Eastern")
didx
didx.tz_localize(None)
didx.tz_convert(None)

# tz_convert(None) is identical to tz_convert('UTC').tz_localize(None)
didx.tz_convert("UTC").tz_localize(None)
```

Fold

For ambiguous times, pandas supports explicitly specifying the keyword-only fold argument. Due to daylight saving time, one wall clock time can occur twice when shifting from summer to winter time; fold describes whether the datetime-like corresponds to the first (0) or the second time (1) the wall clock hits the ambiguous time. Fold is supported only for constructing from naive

`datetime.datetime` (see datetime documentation for details) or from `Timestamp` or for constructing from components (see below). Fold is supported with `zoneinfo` and `dateutil` timezones. We recommend using `Timestamp.tz_localize` when localizing ambiguous datetimes if you need direct control over how they are handled.

```
pd.Timestamp(
    datetime.datetime(2019, 10, 27, 1, 30, 0, 0),
    tz="dateutil/Europe/London",
    fold=0,
)
pd.Timestamp(
    year=2019,
    month=10,
    day=27,
    hour=1,
    minute=30,
    tz="dateutil/Europe/London",
    fold=1,
)
```

Ambiguous times when localizing

`tz_localize` may not be able to determine the UTC offset of a timestamp because daylight savings time (DST) in a local time zone causes some times to occur twice within one day ("clocks fall back"). The following options are available:

- `'raise'`: Raises a `ValueError` (the default behavior)
- `'infer'`: Attempt to determine the correct offset based on the monotonicity of the timestamps
- `'NaT'`: Replaces ambiguous times with `NaT`
- `bool`: `True` represents a DST time, `False` represents non-DST time. An array-like of `bool` values is supported for a sequence of times.

```
rng_hourly = pd.DatetimeIndex(
    ["11/06/2011 00:00", "11/06/2011 01:00", "11/06/2011 01:00", "11/06/2011 02:00"]
)
```

This will fail as there are ambiguous times (`'11/06/2011 01:00'`)

```
rng_hourly.tz_localize('US/Eastern')
```

Handle these ambiguous times by specifying the following.

```
rng_hourly.tz_localize("US/Eastern", ambiguous="infer")
rng_hourly.tz_localize("US/Eastern", ambiguous="NaT")
rng_hourly.tz_localize("US/Eastern", ambiguous=[True, True, False, False])
```

Nonexistent times when localizing

A DST transition may also shift the local time ahead by 1 hour creating nonexistent local times ("clocks spring forward"). The behavior of localizing a timeseries with nonexistent times can be controlled by the `nonexistent` argument. The following options are available:

- `'raise'`: Raises a `ValueError` (the default behavior)
- `'NaT'`: Replaces nonexistent times with `NaT`
- `'shift_forward'`: Shifts nonexistent times forward to the closest real time
- `'shift_backward'`: Shifts nonexistent times backward to the closest real time
- timedelta object: Shifts nonexistent times by the timedelta duration

```
dti = pd.date_range(start="2015-03-29 02:30:00", periods=3, freq="h")
# 2:30 is a nonexistent time
```

Localization of nonexistent times will raise an error by default.

```
dti.tz_localize('Europe/Warsaw')
```

Transform nonexistent times to `NaT` or shift the times.

```
dti
dti.tz_localize("Europe/Warsaw", nonexistent="shift_forward")
dti.tz_localize("Europe/Warsaw", nonexistent="shift_backward")
dti.tz_localize("Europe/Warsaw", nonexistent=pd.Timedelta(1, unit="h"))
dti.tz_localize("Europe/Warsaw", nonexistent="NaT")
```

Time zone Series operations

A `Series` with time zone **naive** values is represented with a dtype of `datetime64[us]`.

```
s_naive = pd.Series(pd.date_range("20130101", periods=3))
s_naive
```

A `Series` with a time zone **aware** values is represented with a dtype of `datetime64[us, tz]` where `tz` is the time zone

```
s_aware = pd.Series(pd.date_range("20130101", periods=3, tz="US/Eastern"))
s_aware
```

Both of these `Series` time zone information can be manipulated via the `.dt` accessor, see the dt accessor section.

For example, to localize and convert a naive stamp to time zone aware.

```
s_naive.dt.tz_localize("UTC").dt.tz_convert("US/Eastern")
```

Time zone information can also be manipulated using the `astype` method. This method can convert between different timezone-aware dtypes.

```
# convert to a new time zone
s_aware.astype("datetime64[us, CET]")
```

Note
Using `Series.to_numpy` on a `Series`, returns a NumPy array of the data. NumPy does not currently support time zones (even though it is *printing* in the local time zone!), therefore an object array of Timestamps is returned for time zone aware data:

```
s_naive.to_numpy()
s_aware.to_numpy()
```

This object-dtype default is significantly slower and more memory-intensive than a native `datetime64` array. For better performance, pass an explicit `dtype`:

```
s_aware.to_numpy(dtype="datetime64[us]")
```

This converts to UTC and drops the timezone, returning a native `datetime64` array. If you need to preserve per-element timezone information (e.g. when converting back to a Series), use `dtype=object` explicitly:

```
pd.Series(s_aware.to_numpy(dtype=object))
```

API quick reference

Generated from the docstrings of the installed pandas 3.1.0 package, which are the text the official API reference pages are built from. Full reference: https://pandas.pydata.org/docs/reference/index.html

Input / output

pd.read_csv

```
pd.read_csv(filepath_or_buffer: 'FilePath | ReadCsvBuffer[bytes] | ReadCsvBuffer[str]', *, sep: 'str |
↪  lib.NoDefault | None' = <no_default>, delimiter: 'str | lib.NoDefault | None' = <no_default>, header:
↪  "int | Sequence[int] | Literal['infer'] | None" = 'infer', names: 'Sequence[Hashable] | lib.NoDefault
↪  | None' = <no_default>, index_col: 'IndexLabel | Literal[False] | None' = None, usecols:
↪  'UsecolsArgType ...)
```

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for IO Tools.

Parameters

`filepath_or_buffer` : ***str, path object or file-like object*** Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: file://localhost/path/to/table.csv.

Certain URL schemes may require additional packages. For example, S3 URLs require the `s3fs` library. See install.optional_dependencies for a full list.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method that accepts an optional size argument, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

`sep` : ***str, default*** **','** Character or regex pattern to treat as the delimiter. `sep=None` detects the separator from the first valid row of the file with Python's builtin sniffer tool, `csv.Sniffer`; it is supported only by the Python parsing engine, which will be used automatically. In addition, separators longer than 1 character and different from `'\s+'` will be interpreted as regular expressions and will force the use of the Python parsing engine. Note that regex delimiters are prone to ignoring quoted data. Regex example: `'\r\t'`.

`delimiter` : ***str, optional*** Alias for `sep`.

`header` : ***int, Sequence of int, 'infer' or None, default 'infer'*** Row index or indices in the file to use as DataFrame column labels. Indexing starts at 0 and counts only non-blank, non-commented lines when `skip_blank_lines=True`, so `header=0` denotes the first line of data rather than the first line of the file. Valid arguments are:

- `int`: line index at which column labels are read.
- Sequence of `int`: line indices at which column labels are read and combined into a `MultiIndex` on the columns, e.g. `[0, 1, 3]`. Intervening rows that are not specified will be skipped (e.g. row 2 in this example).
- `None`: no row in the file is interpreted as column labels. Columns are labelled by integer position, or by the values passed to `names` when provided. Use this for files with no

header row. If the file has a header row that should be overridden by `names`, pass `header=0` instead.
- `'infer'` (default): equivalent to `header=0` when no `names` are passed, otherwise equivalent to `header=None`.

When inferred from the file contents, headers are kept distinct from each other by renaming duplicate names with a numeric suffix of the form `".{count}"` starting from 1, e.g. `"foo"` and `"foo.1"`. Empty headers are named `"Unnamed: {i}"` or `"Unnamed: {i}_level_{level}"` in the case of MultiIndex columns.

names : *Sequence of Hashable, optional* Sequence of column labels to apply. If the file contains a header row, then you should explicitly pass `header=0` to override the column names. Duplicates in this list are not allowed.

index_col : *Hashable, Sequence of Hashable or False, optional* Column(s) to use as row label(s), denoted either by column labels or column indices. If a sequence of labels or indices is given, `MultiIndex` will be formed for the row labels.

Note: `index_col=False` can be used to force pandas to *not* use the first column as the index, e.g., when you have a malformed file with delimiters at the end of each line.

usecols : *Sequence of Hashable or Callable, optional* Subset of columns to select, denoted either by column labels or column indices. If list-like, all elements must either be positional (i.e. integer indices into the document columns) or strings that correspond to column names provided either by the user in `names` or inferred from the document header row(s). If `names` are given, the document header row(s) are not taken into account. For example, a valid list-like `usecols` parameter would be `[0, 1, 2]` or `['foo', 'bar', 'baz']`. Element order is ignored, so `usecols=[0, 1]` is the same as `[1, 0]`. To instantiate a `DataFrame` from `data` with element order preserved use `pd.read_csv(data, usecols=['foo', 'bar'])[['foo', 'bar']]` for columns in `['foo', 'bar']` order or `pd.read_csv(data, usecols=['foo', 'bar'])[['bar', 'foo']]` for `['bar', 'foo']` order.

If callable, the callable function will be evaluated against the column names, returning names where the callable function evaluates to `True`. An example of a valid callable argument would be `lambda x: x.upper() in ['AAA', 'BBB', 'DDD']`. Using this parameter results in much faster parsing time and lower memory usage.

dtype : *dtype or dict of {Hashable : dtype}, optional* Data type(s) to apply to either the whole dataset or individual columns. E.g., `{'a': np.float64, 'b': np.int32, 'c': 'Int64'}` Use `str` or `object` together with suitable `na_values` settings to preserve and not interpret `dtype`. Specifying a `dtype` does not change how missing values are parsed; values recognized as missing (see `na_values` and `keep_default_na`) are still read as `NaN` (or the dtype's NA value) rather than cast to the requested `dtype`. If `converters` are specified, they will be applied INSTEAD of `dtype` conversion. Specify a `defaultdict` as input where the default determines the `dtype` of the columns which are not explicitly listed.

engine : *{'c', 'python', 'pyarrow'}, optional* Parser engine to use. The C and pyarrow engines are faster, while the python engine is currently more feature-complete. The pyarrow engine is multithreaded, and the C engine reads sufficiently large files in parallel. Some features of the ”pyarrow” engine are unsupported or may not work correctly.

converters : *dict of {Hashable : Callable}, optional* Functions for converting values in specified columns. Keys can either be column labels or column indices. The function is applied to the raw text read from the file, so an empty field is passed as an empty string `''`; `na_values` and `keep_default_na` are then applied to the value the function returns.

true_values : *list, optional* Values to consider as `True` in addition to case-insensitive variants of 'True'.

false_values : *list, optional* Values to consider as `False` in addition to case-insensitive variants of 'False'.

skipinitialspace : *bool, default False* Skip spaces after delimiter.

skiprows : *int, list of int or Callable, optional* Line numbers to skip (0-indexed) or number of lines to skip (`int`) at the start of the file.

If callable, the callable function will be evaluated against the row indices, returning `True` if the row should be skipped and `False` otherwise. An example of a valid callable argument would be `lambda x: x in [0, 2]`.

skipfooter : *int, default 0* Number of lines at bottom of file to skip (Unsupported with `engine='c'`).

nrows : *int, optional* Number of rows of file to read. Useful for reading pieces of large files. Refers to the number of data rows in the returned DataFrame, excluding:

- The header row containing column names.
- Rows before the header row, if `header=1` or larger.

Example usage:

- To read the first 999,999 (non-header) rows: `read_csv(..., nrows=999999)`
- To read rows 1,000,000 through 1,999,999: `read_csv(..., skiprows=1000000, nrows=999999)`

na_values : *Hashable, Iterable of Hashable or dict of {Hashable : Iterable},* optional Additional strings to recognize as `NA`/`NaN`. If `dict` passed, specific per-column `NA` values. By default the following values are interpreted as `NaN`: empty string, ”NaN”, ”N/A”, ”NULL”, and other common representations of missing data.

keep_default_na : *bool, default True* Whether or not to include the default `NaN` values when parsing the data. Depending on whether `na_values` is passed in, the behavior is as follows:

- If `keep_default_na` is `True`, and `na_values` are specified, `na_values` is appended to the default `NaN` values used for parsing.
- If `keep_default_na` is `True`, and `na_values` are not specified, only the default `NaN` values are used for parsing.
- If `keep_default_na` is `False`, and `na_values` are specified, only the `NaN` values specified `na_values` are used for parsing.
- If `keep_default_na` is `False`, and `na_values` are not specified, no strings will be parsed as `NaN`.

Note that if `na_filter` is passed in as `False`, the `keep_default_na` and `na_values` parameters will be ignored.

na_filter : *bool, default True* Detect missing value markers (empty strings and the value of `na_values`). In data without any `NA` values, passing `na_filter=False` can improve the performance of reading a large file.

skip_blank_lines : *bool, default True* If `True`, skip over blank lines rather than interpreting as `NaN` values.

parse_dates : *bool, None, list of Hashable, default None* The behavior is as follows:

- `bool`. If `True` -> try parsing the index.
- `None`. Behaves like `True` if `date_format` is specified.
- `list` of `int` or names. e.g. If `[1, 2, 3]` -> try parsing columns 1, 2, 3 each as a separate date column.

If a column or index cannot be represented as an array of `datetime`, say because of an unparsable value or a mixture of timezones, the column or index will be returned unaltered as an `object` data type. For non-standard `datetime` parsing, use `to_datetime` after `read_csv`.

Note: A fast-path exists for iso8601-formatted dates.

date_format : *str or dict of column -> format, optional* Format to use for parsing dates and/or times when used in conjunction with `parse_dates`. The strftime to parse time, e.g. `"%d/%m/%Y"`. See strftime documentation <https://docs.python.org/3/library/datetime.html #strftime-and-strptime-behavior> for more information on choices, though note that `"%f"` will parse all the way up to nanoseconds. You can also pass:

- "ISO8601", to parse any ISO8601 time string (not necessarily in exactly the same format);
- "mixed", to infer the format for each element individually. This is risky, and you should probably use it along with dayfirst.

New in version 2.0.0.

dayfirst : *bool, default False* DD/MM format dates, international and European format.

cache_dates : *bool, default True* If `True`, use a cache of unique, converted dates to apply the `datetime` conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets.

iterator : *bool, default False* Return `TextFileReader` object for iteration or getting chunks with `get_chunk()`.

chunksize : *int, optional* Number of lines to read from the file per chunk. Passing a value will cause the function to return a `TextFileReader` object for iteration. See the IO Tools docs for more information on `iterator` and `chunksize`.

compression : *str or dict, default 'infer'* For on-the-fly decompression of on-disk data. If 'infer' and 'filepath_or_buffer' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). If using 'zip' or 'tar', the ZIP file must contain only one data file to be read in. Set to `None` for no decompression. Can also be a dict with key `'method'` set to one of {`'zip'`, `'gzip'`, `'bz2'`, `'zstd'`, `'xz'`, `'tar'`} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdDecompressor`, `lzma.LZMAFile` or `tarfile.TarFile`, respectively. As an example, the following could be passed for Zstandard decompression using a custom compression dictionary: `compression={'method': 'zstd', 'dict_data': my_compression_dict}`.

thousands : *str (length 1), optional* Character acting as the thousands separator in numerical values.

decimal : *str (length 1), default '.'* Character to recognize as decimal point (e.g., use ',' for European data).

lineterminator : *str (length 1), optional* Character used to denote a line break. Only valid with C parser.

quotechar : *str (length 1), optional* Character used to denote the start and end of a quoted item. Quoted items can include the `delimiter` and it will be ignored.

quoting : *{0, 1, 2, 3}, default csv.QUOTE_MINIMAL* Control field quoting behavior per `csv.QUOTE_*` constants. Use one of `0` or `csv.QUOTE_MINIMAL`, `1` or `csv.QUOTE_ALL`, `2` or `csv.QUOTE_NONNUMERIC`, or `3` or `csv.QUOTE_NONE`. Default is `csv.QUOTE_MINIMAL` (i.e., 0) which implies that only fields containing special characters are quoted (e.g., characters defined in `quotechar`, `delimiter`, or `lineterminator`.

doublequote : *bool, default True* When `quotechar` is specified and `quoting` is not `QUOTE_NONE`, indicate whether or not to interpret two consecutive `quotechar` elements INSIDE a field as a single `quotechar` element.

escapechar : *str (length 1), optional* Character used to escape other characters.

comment : *str (length 1), optional* Character indicating that the remainder of line should not be parsed. If found at the beginning of a line, the line will be ignored altogether. This parameter must be a single character. Like empty lines (as long as `skip_blank_lines=True`), fully commented lines are ignored by the parameter `header` but not by `skiprows`. For example, if `comment='#'`, parsing `#empty\na,b,c\n1,2,3` with `header=0` will result in `'a,b,c'` being treated as the header.

encoding : *str, optional, default 'utf-8'* Encoding to use for UTF when reading/writing (ex. `'utf-8'`). List of Python standard encodings .

encoding_errors : *str, optional, default 'strict'* How encoding errors are treated. List of possible values .

dialect : *str or csv.Dialect, optional* If provided, this parameter will override values (default or not) for the following parameters: `delimiter`, `doublequote`, `escapechar`, `skipinitialspace`, `quotechar`, and `quoting`. If it is necessary to override values, a `ParserWarning` will be issued. See `csv.Dialect` documentation for more details.

on_bad_lines : *{'error', 'warn', 'skip'} or Callable, default 'error'* Specifies what to do upon encountering a bad line (a line with too many fields). Allowed values are:

- `'error'`, raise an Exception when a bad line is encountered.
- `'warn'`, raise a warning when a bad line is encountered and skip that line.
- `'skip'`, skip bad lines without raising or warning when they are encountered.
- **Callable, function that will process a single bad line.**
  - With `engine='python'`, function with signature `(bad_line: list[str]) -> list[str] | None`. `bad_line` is a list of strings split by the `sep`. If the function returns `None`, the bad line will be ignored. If the function returns a new `list` of strings with more elements than expected, a `ParserWarning` will be emitted while dropping extra elements.
  - With `engine='pyarrow'`, function with signature as described in pyarrow documentation: invalid_row_handler <https://arrow.apache.org/docs/python /generated/pyarrow.csv.ParseOptions.html #pyarrow.csv.ParseOptions.invalid_row_handler>.

Changed in version 2.2.0. Callable for `engine='pyarrow'`

low_memory : *bool, default True* Internally process the file in chunks, resulting in lower memory use while parsing, but possibly mixed type inference. Because each chunk is type-inferred independently, the same literal can be parsed as an `int` in one chunk and a `str` in another; after concatenation the column has `object` dtype and contains both representations, so comparisons such as `df["col"] == 12345` or `df["col"] == "12345"` will match only a subset of the rows holding that value. A `DtypeWarning` is emitted when this occurs. To ensure no mixed types either set `False`, or specify the type with the `dtype` parameter. Note that the entire file is read into a single `DataFrame` regardless, use the `chunksize` or `iterator` parameter to return the data in chunks. (Only valid with C parser).

memory_map : *bool, default False* If a filepath is provided for `filepath_or_buffer`, map the file object directly onto memory and access the data directly from there. Using this option can improve performance because there is no longer any I/O overhead.

float_precision : *{'high', 'legacy', 'round_trip'}, optional* Specifies which converter the C engine should use for floating-point values. The options are `None` or `'high'` for the ordinary converter, `'legacy'` for the original lower precision pandas converter, and `'round_trip'` for the round-trip converter.

Deprecated since version 3.1.0. All float precision modes now use the same converter. The `float_precision` argument will be removed in a future version.

storage_options : *dict, optional* Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer here <https://pandas.pydata.org/docs/user_guide/io.html? highlight=storage_options#reading-writing-remote-files>.

dtype_backend : *{'numpy_nullable', 'pyarrow'}* Back-end data type applied to the resultant `DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame`
- `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`

New in version 2.0.

Returns

DataFrame or TextFileReader A comma-separated values (csv) file is returned as two-dimensional data structure with labeled axes.

See also: `DataFrame.to_csv`, `read_table`, `read_fwf`

Notes

Column labels read from a header row are always strings. To use column labels of another type, pass them explicitly with `names` and set `header=0` when the file contains a header row.

Sufficiently large local uncompressed files read with the C engine may be parsed by multiple threads in parallel. Use the `mode.max_threads` option to cap or disable this; see io.csv.parallel for details.

Examples

```
>>> pd.read_csv("data.csv")
   Name  Value
0   foo      1
1   bar      2
2  #baz      3
```

Index and header can be specified via the index_col and header arguments.

```
>>> pd.read_csv("data.csv", header=None)
      0      1
0  Name  Value
1   foo      1
2   bar      2
3  #baz      3
```

```
>>> pd.read_csv("data.csv", index_col="Value")
       Name
Value
1       foo
2       bar
3      #baz
...
```

pd.read_parquet

```
pd.read_parquet(path: 'FilePath | ReadBuffer[bytes]', engine: 'str | lib.NoDefault' = <no_default>,
↪  columns: 'list[str] | None' = None, storage_options: 'StorageOptions | None' = None, dtype_backend:
↪  'DtypeBackend | lib.NoDefault' = <no_default>, filesystem: 'Any' = None, filters: 'list[tuple[Any,
↪  ...]] | list[list[tuple[Any, ...]]] | None' = None, to_pandas_kwargs: 'dict[str, Any] | None' = None,
↪  **kwargs) -> ' ...)
```

Load a parquet object from the file path, returning a DataFrame.

This function requires the pyarrow library.

The function automatically handles reading the data from a parquet file and creates a DataFrame with the appropriate structure.

Parameters

path : *str, path object or file-like object* String, path object (implementing `os.PathLike[str]`), or file-like object implementing a binary `read()` function. The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.parquet`.

Certain URL schemes may require additional packages. For example, S3 URLs require the `s3fs` library. See install.optional_dependencies for a full list.

A file URL can also be a path to a directory that contains multiple partitioned parquet files. Both pyarrow and fastparquet support paths to directories as well as file URLs. A directory path could be: `file://localhost/path/to/tables` or `s3://bucket/partition_dir`.

engine : *{'auto', 'pyarrow', 'fastparquet'}, default 'auto'* Parquet library to use. If 'auto', then the option `io.parquet.engine` is used. The default `io.parquet.engine` behavior is to try 'pyarrow', falling back to 'fastparquet' if 'pyarrow' is unavailable.

When using the `'pyarrow'` engine and no storage options are provided and a filesystem is implemented by both `pyarrow.fs` and `fsspec` (e.g. ”s3://”), then the `pyarrow.fs` filesystem is attempted first. Use the filesystem keyword with an instantiated fsspec filesystem if you wish to use its implementation.

Deprecated since version 3.1.0. The `'fastparquet'` and `'auto'` engine options are deprecated. Use `'pyarrow'` or do not pass `engine` to use the default.

columns : *list, default=None* If not None, only these columns will be read from the file.

storage_options : *dict, optional* Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with ”s3://”, and ”gcs://”) the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer here <https://pandas.pydata.org/docs/user_guide/io.html? highlight=storage_options#reading-writing-remote-files>.

dtype_backend : *{'numpy_nullable', 'pyarrow'}* Back-end data type applied to the resultant `DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame`
- `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`

New in version 2.0.

filesystem : *fsspec or pyarrow filesystem, default None* Filesystem object to use when reading the parquet file. Only implemented for `engine="pyarrow"`.

New in version 2.1.0.

filters : *List[Tuple] or List[List[Tuple]], default None* To filter out data. Filter syntax: [[(column, op, val), ...],...] where op is [==, =, >, >=, <, <=, !=, in, not in] The innermost tuples are transposed into a set of filters applied through an AND operation. The outer list combines these sets of filters through an OR operation. A single list of tuples can also be used, meaning that no OR operation between set of filters is to be conducted.

Using this argument will NOT result in row-wise filtering of the final partitions unless `engine="pyarrow"` is also specified. For other engines, filtering is only performed at the partition level, that is, to prevent the loading of some row-groups and/or files.

New in version 2.1.0.

`to_pandas_kwargs` : ***dict | None, default None*** Keyword arguments to pass through to `pyarrow.Table.to_pandas` when `engine="pyarrow"`.

New in version 3.0.0.

`**kwargs` Additional keyword arguments passed to the engine:

- For `engine="pyarrow"`: passed to `pyarrow.parquet.read_table`
- For `engine="fastparquet"`: passed to `fastparquet.ParquetFile.to_pandas`

Returns

DataFrame DataFrame based on parquet file.

See also: `DataFrame.to_parquet`

Examples

```
>>> original_df = pd.DataFrame({"foo": range(5), "bar": range(5, 10)})
>>> original_df
   foo  bar
0    0    5
1    1    6
2    2    7
3    3    8
4    4    9
>>> df_parquet_bytes = original_df.to_parquet()
>>> from io import BytesIO
>>> restored_df = pd.read_parquet(BytesIO(df_parquet_bytes))
>>> restored_df
   foo  bar
0    0    5
1    1    6
2    2    7
3    3    8
4    4    9
>>> restored_df.equals(original_df)
True
>>> restored_bar = pd.read_parquet(BytesIO(df_parquet_bytes), columns=["bar"])
>>> restored_bar
    bar
0    5
...
```

pd.read_excel

```
pd.read_excel(io, sheet_name: 'str | int | list[IntStrT] | None' = 0, *, header: 'int | Sequence[int] |
↪  None' = 0, names: 'SequenceNotStr[Hashable] | range | None' = None, index_col: 'int | str |
↪  Sequence[int] | None' = None, usecols: 'int | str | Sequence[int] | Sequence[str] |
↪  Callable[[HashableT], bool] | None' = None, dtype: 'DtypeArg | None' = None, engine: "Literal['xlrd',
↪  'openpyxl', 'odf', 'pyxlsb', 'c ...)
```

Read an Excel file into a `DataFrame`.

Supports xls, xlsx, xlsm, xlsb, odf, ods and odt file extensions read from a local filesystem or URL. Supports an option to read a single sheet or a list of sheets.

This function requires an external library depending on the file format; see the `engine` parameter below.

Parameters

`io` : *str, ExcelFile, workbook object, path object, or file-like object* Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.xlsx`.

Certain URL schemes may require additional packages. For example, S3 URLs require the `s3fs` library. See install.optional_dependencies for a full list.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

A workbook object from a supported engine's library, such as an `openpyxl.Workbook`, is read with that engine.

`sheet_name` : *str, int, list, or None, default 0* Strings are used for sheet names. Integers are used in zero-indexed sheet positions (chart sheets do not count as a sheet position). Lists of strings/integers are used to request multiple sheets. When `None`, will return a dictionary containing DataFrames for each sheet.

Available cases:

- Defaults to `0`: 1st sheet as a DataFrame
- `1`: 2nd sheet as a DataFrame
- `"Sheet1"`: Load sheet with name ”Sheet1”
- `[0, 1, "Sheet5"]`: Load first, second and sheet named ”Sheet5” as a dict of DataFrame
- `None`: Returns a dictionary containing DataFrames for each sheet.

`header` : *int, list of int, default 0* Row (0-indexed) to use for the column labels of the parsed DataFrame. If a list of integers is passed those row positions will be combined into a `MultiIndex`. Use None if there is no header.

`names` : *array-like, default None* List of column names to use. If file contains no header row, then you should explicitly pass header=None.

`index_col` : *int, str, list of int, default None* Column (0-indexed) to use as the row labels of the DataFrame. Pass None if there is no such column. If a list is passed, those columns will be combined into a `MultiIndex`. If a subset of data is selected with `usecols`, index_col is based on the subset.

Missing values will be forward filled to allow roundtripping with `to_excel` for `merged_cells=True`. To avoid forward filling the missing values use `set_index` after reading the data instead of `index_col`.

When `header` is a list of two or more rows (`MultiIndex` columns), the row after the last header row is expected to hold the index name(s) — blank if the index is unnamed — as written by `to_excel`. A file lacking this separator row can lose its first data row to the index name(s); see `the user guide <io.excel.reading_multiindex>` for details.

`usecols` : *str, list-like, or callable, default None*

- If None, then parse all columns.
- If str, then indicates comma separated list of Excel column letters and column ranges (e.g. ”A:E” or ”A,C,E:F”). Ranges are inclusive of both sides.
- If list of int, then indicates list of column numbers to be parsed (0-indexed).
- If list of string, then indicates list of column names to be parsed.
- If callable, then evaluate each column name against it and parse the column if the callable returns `True`.

Returns a subset of the columns according to behavior above. Element order is ignored, so `usecols=[0, 1]` is the same as `[1, 0]`. To instantiate a `DataFrame` with element order preserved use `pd.read_excel(path, usecols=['foo', 'bar'])[['foo', 'bar']]` for columns in `['foo', 'bar']` order or `pd.read_excel(path, usecols=['foo', 'bar'])[['bar', 'foo']]` for `['bar', 'foo']` order.

`dtype` : *Type name or dict of column -> type, default None* Data type for data or columns. E.g. {'a': np.float64, 'b': np.int32} Use `object` to preserve data as stored in Excel and not interpret dtype, which will necessarily result in `object` dtype. Specifying a `dtype` does not change how missing values are parsed; values recognized as missing (see `na_values` and `keep_default_na`) are still read as `NaN` (or the dtype's NA value) rather than cast to the requested `dtype`. If converters are specified, they will be applied INSTEAD of dtype conversion. If you use `None`, it will infer the dtype of each column based on the data.

`engine` : *{'openpyxl', 'calamine', 'odf', 'pyxlsb', 'xlrd'}, default None* Engine compatibility :

- `openpyxl` supports newer Excel file formats.
- `calamine` supports Excel (.xls, .xlsx, .xlsm, .xlsb) and OpenDocument (.ods) file formats.
- `odf` supports OpenDocument file formats (.odf, .ods, .odt).
- `pyxlsb` supports Binary Excel files.
- `xlrd` supports old-style Excel files (.xls).

When `engine=None`, the following logic will be used to determine the engine:

- If `io` is a workbook object, the engine that created it will be used.
- Otherwise if `path_or_buffer` is an OpenDocument format (.odf, .ods, .odt), then odf will be used.
- Otherwise if `path_or_buffer` is an xls format, `xlrd` will be used.
- Otherwise if `path_or_buffer` is in xlsb format, `pyxlsb` will be used.
- Otherwise `openpyxl` will be used.

Deprecated since version 3.1.0. For xlsx/xlsm files the default engine will change from `openpyxl` to `calamine` in a future version. Pass `engine` explicitly, or set the `io.excel.xlsx.reader` option (xlsm files are format-detected as xlsx), to keep the current behavior.

`converters` : *dict, default None* Dict of functions for converting values in certain columns. Keys can either be integers or column labels, values are functions that take one input argument, the Excel cell content, and return the transformed content.

`true_values` : *list, default None* Values to consider as True.

`false_values` : *list, default None* Values to consider as False.

`skiprows` : *list-like, int, or callable, optional* Line numbers to skip (0-indexed) or number of lines to skip (int) at the start of the file. If callable, the callable function will be evaluated against the row indices, returning True if the row should be skipped and False otherwise. An example of a valid callable argument would be `lambda x: x in [0, 2]`.

`nrows` : *int, default None* Number of rows to parse. Does not include header rows.

`na_values` : *scalar, str, list-like, or dict, default None* Additional strings to recognize as NA/-NaN. If dict passed, specific per-column NA values. By default the following values are interpreted as NaN: '', '#N/A', '#N/A N/A', '#NA', '-1.#IND', '-1.#QNAN', '-NaN', '-nan', '1.#IND', '1.#QNAN', '<NA>', 'N/A', 'NA', 'NULL', 'NaN', 'None', 'n/a', 'nan', 'null'.

`keep_default_na` : *bool, default True* Whether or not to include the default NaN values when parsing the data. Depending on whether `na_values` is passed in, the behavior is as follows:

- If `keep_default_na` is True, and `na_values` are specified, `na_values` is appended to the default NaN values used for parsing.
- If `keep_default_na` is True, and `na_values` are not specified, only the default NaN values are used for parsing.

- If `keep_default_na` is False, and `na_values` are specified, only the NaN values specified `na_values` are used for parsing.
- If `keep_default_na` is False, and `na_values` are not specified, no strings will be parsed as NaN.

Note that if na_filter is passed in as False, the `keep_default_na` and `na_values` parameters will be ignored.

`na_filter` : *bool, default True* Detect missing value markers (empty strings and the value of na_values). In data without any NAs, passing `na_filter=False` can improve the performance of reading a large file.

`verbose` : *bool, default False* Indicate number of NA values placed in non-numeric columns.

`parse_dates` : *bool, list-like, or dict, default False* The behavior is as follows:

- `bool`. If True -> try parsing the index.
- `list` of int or names. e.g. If [1, 2, 3] -> try parsing columns 1, 2, 3 each as a separate date column.
- `list` of lists. e.g. If [[1, 3]] -> combine columns 1 and 3 and parse as a single date column.
- `dict`, e.g. {'foo' : [1, 3]} -> parse columns 1, 3 as date and call result 'foo'

If a column or index contains an unparsable date, the entire column or index will be returned unaltered as an object data type. If you don‘t want to parse some cells as date just change their type in Excel to ”Text”. For non-standard datetime parsing, use `pd.to_datetime` after `pd.read_excel`.

Note: A fast-path exists for iso8601-formatted dates.

`date_format` : *str or dict of column -> format, default “None”* If used in conjunction with `parse_dates`, will parse dates according to this format. For anything more complex, please read in as `object` and then apply `to_datetime` as-needed.

New in version 2.0.0.

`thousands` : *str, default None* Thousands separator for parsing string columns to numeric. Note that this parameter is only necessary for columns stored as TEXT in Excel, any numeric columns will automatically be parsed, regardless of display format.

`decimal` : *str, default* '.' Character to recognize as decimal point for parsing string columns to numeric. Note that this parameter is only necessary for columns stored as TEXT in Excel, any numeric columns will automatically be parsed, regardless of display format.(e.g. use ',' for European data).

`comment` : *str, default None* Comments out remainder of line. Pass a character or characters to this argument to indicate comments in the input file. Any data between the comment string and the end of the current line is ignored.

`skipfooter` : *int, default 0* Rows at the end to skip (0-indexed).

`storage_options` : *dict, optional* Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with ”s3://”, and ”gcs://”) the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer here <https://pandas.pydata.org/docs/user_guide/io.html? highlight=storage_options#reading-writing-remote-files>.

`dtype_backend` : *{'numpy_nullable', 'pyarrow'}* Back-end data type applied to the resultant `DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame`
- `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`

New in version 2.0.

`engine_kwargs` : *dict, optional* Arbitrary keyword arguments passed to excel engine.

Returns

DataFrame or dict of DataFrames DataFrame from the passed in Excel file. See notes in sheet_name argument for more information on when a dict of DataFrames is returned.

See also: `DataFrame.to_excel`, `DataFrame.to_csv`, `read_csv`, `read_fwf`

Notes

For specific information on the methods used for each Excel engine, refer to the pandas user guide

Examples

The file can be read using the file name as string or an open file object:

```
>>> pd.read_excel("tmp.xlsx", index_col=0)
       Name  Value
0   string1      1
1   string2      2
2  #Comment      3
```

```
>>> pd.read_excel(open("tmp.xlsx", "rb"), sheet_name="Sheet3")
   Unnamed: 0      Name  Value
0           0   string1      1
1           1   string2      2
2           2  #Comment      3
```

Index and header can be specified via the index_col and header arguments

```
>>> pd.read_excel("tmp.xlsx", index_col=None, header=None)
     0         1      2
0  NaN      Name  Value
1  0.0   string1      1
2  1.0   string2      2
3  2.0  #Comment      3
...
```

pd.read_json

```
pd.read_json(path_or_buf: 'FilePath | ReadBuffer[str] | ReadBuffer[bytes]', *, orient: 'str | None' =
↪  None, typ: "Literal['frame', 'series']" = 'frame', dtype: 'DtypeArg | None' = None, convert_axes:
↪  'bool | None' = None, convert_dates: 'bool | list[str] | lib.NoDefault' = <no_default>,
↪  keep_default_dates: 'bool | lib.NoDefault' = <no_default>, precise_float: 'bool' = False, date_unit:
↪  'str | None' = None, en ...)
```

Convert a JSON string to pandas object.

This method reads JSON files or JSON-like data and converts them into pandas objects. It supports a variety of input formats, including line-delimited JSON, compressed files, and various data representations (table, records, index-based, etc.). When chunksize is specified, an iterator is returned instead of loading the entire data into memory.

Parameters

path_or_buf : *a str path, path object or file-like object* Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.json`.

Certain URL schemes may require additional packages. For example, S3 URLs require the `s3fs` library. See install.optional_dependencies for a full list.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handle (e.g. via builtin `open` function) or `StringIO`.

orient : *str, optional* Indication of expected JSON string format. Compatible JSON strings can be produced by `to_json()` with a corresponding orient value. When `typ == 'frame'`, the set of possible orients is:

- `'split'` : dict like `{index -> [index], columns -> [columns], data -> [values]}`
- `'records'` : list like `[{column -> value}, ... , {column -> value}]`
- `'index'` : dict like `{index -> {column -> value}}`
- `'columns'` : dict like `{column -> {index -> value}}`
- `'values'` : just the values array
- `'table'` : dict like `{'schema': {schema}, 'data': {data}}`

When `typ == 'series'`, the shapes differ, since a Series has no columns:

- `'split'` : dict like `{name -> name, index -> [index], data -> [values]}`
- `'records'` : list like `[value, ... , value]`
- `'index'` : dict like `{index -> value}`

The allowed and default values depend on the value of the typ parameter.

- when `typ == 'series'`,
  - allowed orients are `{'split','records','index'}`
  - default is `'index'`
  - The Series index must be unique for orient `'index'`.
- when `typ == 'frame'`,
  - allowed orients are `{'split','records','index', 'columns','values', 'table'}`
  - default is `'columns'`
  - The DataFrame index must be unique for orients `'index'` and `'columns'`.
  - The DataFrame columns must be unique for orients `'index'`, `'columns'`, and `'records'`.

typ : *{'frame', 'series'}, default 'frame'* The type of object to recover.

dtype : *bool or dict, default None* If True, infer dtypes; if a dict of column to dtype, then use those; if False, then don't infer dtypes at all, applies only to the data.

For all `orient` values except `'table'`, default is True.

convert_axes : *bool, default None* Try to convert the axes to the proper dtypes.

For all `orient` values except `'table'`, default is True.

convert_dates : *bool or list of str, default True* If True then default datelike columns may be converted (depending on keep_default_dates). If False, no dates will be converted. If a list of column names, then those columns will be converted and default datelike columns may also be converted (depending on keep_default_dates).

Deprecated since version 3.1.0. Pass `dtype=False` to disable type conversion, or parse date columns with `to_datetime` after reading.

keep_default_dates : *bool, default True* If parsing dates (convert_dates is not False), then try to parse the default datelike columns. A column label is datelike if

- it ends with `'_at'`,
- it ends with `'_time'`,
- it begins with `'timestamp'`,
- it is `'modified'`, or
- it is `'date'`.

Deprecated since version 3.1.0. Pass `dtype=False` to disable type conversion, or parse date columns with `to_datetime` after reading.

precise_float : *bool, default False* Set to enable usage of higher precision (strtod) function when decoding string to double values. Default (False) is to use fast but less precise builtin functionality.

date_unit : *str, default None* The timestamp unit to detect if converting dates. The default behaviour is to try and detect the correct precision, but if this is not desired then pass one of 's', 'ms', 'us' or 'ns' to force parsing only seconds, milliseconds, microseconds or nanoseconds respectively.

encoding : *str, default is 'utf-8'* The encoding to use to decode py3 bytes.

encoding_errors : *str, optional, default "strict"* How encoding errors are treated. List of possible values .

lines : *bool, default False* Read the file as a json object per line.

chunksize : *int, optional* Return JsonReader object for iteration. See the line-delimited json docs for more information on `chunksize`. This can only be passed if lines=True. If this is None, the file will be read into memory all at once.

compression : *str or dict, default 'infer'* For on-the-fly decompression of on-disk data. If 'infer' and 'path_or_buf' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). If using 'zip' or 'tar', the ZIP file must contain only one data file to be read in. Set to `None` for no decompression. Can also be a dict with key `'method'` set to one of {`'zip'`, `'gzip'`, `'bz2'`, `'zstd'`, `'xz'`, `'tar'`} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdDecompressor`, `lzma.LZMAFile` or `tarfile.TarFile`, respectively. As an example, the following could be passed for Zstandard decompression using a custom compression dictionary: `compression={'method': 'zstd', 'dict_data': my_compression_dict}`.

nrows : *int, optional* The number of lines from the line-delimited jsonfile that has to be read. This can only be passed if lines=True. If this is None, all the rows will be returned.

storage_options : *dict, optional* Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer here <https://pandas.pydata.org/docs/user_guide/io.html? highlight=storage_options#reading-writing-remote-files>.

dtype_backend : *{'numpy_nullable', 'pyarrow'}* Back-end data type applied to the resultant `DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame`
- `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`

New in version 2.0.

engine : *{"ujson", "pyarrow"}, default "ujson"* Parser engine to use. The `"pyarrow"` engine is only available when `lines=True`, and only supports `typ="frame"` with default parsing op-

tions; passing an option it cannot apply (for example `convert_dates`, `date_unit`, `precise_float`, `encoding`, or a non-`ArrowDtype` `dtype`) raises `ValueError`.

New in version 2.0.

Returns

Series, DataFrame, or pandas.api.typing.JsonReader A JsonReader is returned when `chunksize` is not `0` or `None`. Otherwise, the type returned depends on the value of `typ`.

See also: `DataFrame.to_json`, `Series.to_json`, `json_normalize`

Notes

Specific to `orient='table'`, if a `DataFrame` with a literal `Index` name of index gets written with `to_json`, the subsequent read operation will incorrectly set the `Index` name to `None`. This is because index is also used by `DataFrame.to_json` to denote a missing `Index` name, and the subsequent `read_json` operation cannot distinguish between the two. The same limitation is encountered with a `MultiIndex` and any names beginning with `'level_'`.

Examples

```
>>> from io import StringIO
>>> df = pd.DataFrame(
...     [["a", "b"], ["c", "d"]],
...     index=["row 1", "row 2"],
...     columns=["col 1", "col 2"],
... )
```

Encoding/decoding a Dataframe using `'split'` formatted JSON:

```
>>> df.to_json(orient="split")
'{"columns":["col 1","col 2"],"index":["row 1","row 2"],"data":[["a","b"],["c","d"]]}'
```

```
>>> pd.read_json(StringIO(_), orient="split")  # noqa: F821
      col 1 col 2
row 1     a     b
row 2     c     d
...
```

pd.read_sql

```
pd.read_sql(sql, con, index_col: 'str | list[str] | None' = None, coerce_float: 'bool' = True,
 ↪  params=None, parse_dates=None, columns: 'list[str] | None' = None, chunksize: 'int | None' = None,
 ↪  dtype_backend: 'DtypeBackend | lib.NoDefault' = <no_default>, dtype: 'DtypeArg | None' = None) ->
 ↪  'DataFrame | Iterator[DataFrame]'
```

Read SQL query or database table into a DataFrame.

This function is a convenience wrapper around `read_sql_table` and `read_sql_query` (for backward compatibility). It will delegate to the specific function depending on the provided input. A SQL query will be routed to `read_sql_query`, while a database table name will be routed to `read_sql_table`. Note that the delegated function might have more specific notes about their functionality not listed here.

Parameters

`sql` : ***str or SQLAlchemy Selectable (select or text object)*** SQL query to be executed or a table name.

`con` : ***ADBC Connection, SQLAlchemy connectable, str, or sqlite3 connection*** ADBC provides high performance I/O with native type support, where available. Using SQLAlchemy

makes it possible to use any DB supported by that library. If a DBAPI2 object, only sqlite3 is supported. The user is responsible for engine disposal and connection closure for the ADBC connection and SQLAlchemy connectable; str connections are closed automatically. See here.

New in version 2.2.0. Support for ADBC drivers.

index_col : *str or list of str, optional, default: None* Column(s) to set as index(MultiIndex).

coerce_float : *bool, default True* Attempts to convert values of non-string, non-numeric objects (like decimal.Decimal) to floating point, useful for SQL result sets. This can lose precision: an integral `decimal.Decimal` larger than `2**53` has no exact `float64` representation, so a long identifier can be silently rounded. Pass `False` to leave such values as Python objects in an `object`-dtype column.

params : *list, tuple or dict, optional, default: None* List of parameters to pass to execute method. The syntax used to pass parameters is database driver dependent. Check your database driver documentation for which of the five syntax styles, described in PEP 249's paramstyle, is supported. Eg. for psycopg2, uses %(name)s so use params={'name' : 'value'}.

parse_dates : *list or dict, default: None*

- List of column names to parse as dates.
- Dict of `{column_name: format string}` where format string is strftime compatible in case of parsing string times, or is one of (D, s, ns, ms, us) in case of parsing integer timestamps.
- Dict of `{column_name: arg dict}`, where the arg dict corresponds to the keyword arguments of `pandas.to_datetime` Especially useful with databases without native Datetime support, such as SQLite.

columns : *list, default: None* List of column names to select from SQL table (only used when reading a table).

chunksize : *int, default None* If specified, return an iterator where chunksize is the number of rows to include in each chunk. By itself this typically does not reduce peak memory usage, as most drivers buffer the full result set unless a server-side cursor is used; see the user guide on streaming results.

dtype_backend : *{'numpy_nullable', 'pyarrow'}* Back-end data type applied to the resultant `DataFrame` (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- `"numpy_nullable"`: returns nullable-dtype-backed `DataFrame`
- `"pyarrow"`: returns pyarrow-backed nullable `ArrowDtype` `DataFrame`

New in version 2.0.

dtype : *Type name or dict of columns* Data type for data or columns. E.g. np.float64 or {'a': np.float64, 'b': np.int32, 'c': 'Int64'}. The argument is ignored if a table is passed instead of a query.

New in version 2.0.0.

Returns

DataFrame or Iterator[DataFrame] Returns a DataFrame object that contains the result set of the executed SQL query or an SQL Table based on the provided input, in relation to the specified database connection.

See also: `read_sql_table`, `read_sql_query`

Notes

`pandas` does not attempt to sanitize SQL statements; instead it simply forwards the statement you are executing to the underlying driver, which may or may not sanitize from there. Please

refer to the underlying driver documentation for any details. Generally, be wary when accepting statements from arbitrary sources.

Examples

Read data from SQL via either a SQL query or a SQL tablename. When using a SQLite database only SQL queries are accepted, providing only the SQL tablename will result in an error.

```
>>> from sqlite3 import connect
>>> conn = connect(":memory:")
>>> df = pd.DataFrame(
...     data=[[0, "10/11/12"], [1, "12/11/10"]],
...     columns=["int_column", "date_column"],
... )
>>> df.to_sql(name="test_data", con=conn)
2
```

```
>>> pd.read_sql("SELECT int_column, date_column FROM test_data", conn)
   int_column date_column
0           0    10/11/12
1           1    12/11/10
...
```

pd.DataFrame.to_csv

```
pd.DataFrame.to_csv(self, path_or_buf: 'FilePath | WriteBuffer[bytes] | WriteBuffer[str] | None' = None,
↪  *, sep: 'str' = ',', na_rep: 'str' = '', float_format: 'str | Callable | None' = None, columns:
↪  'Sequence[Hashable] | None' = None, header: 'bool | list[str]' = True, index: 'bool' = True,
↪  index_label: 'IndexLabel | None' = None, mode: 'str' = 'w', encoding: 'str | None' = None,
↪  compression: 'CompressionOptions'  ...)
```

Write object to a comma-separated values (csv) file.

By default, the resulting file includes row index and column headers. Supports customization of delimiter, encoding, compression, and more. The output can be written to a file path, file-like buffer, or returned as a string.

Parameters

`path_or_buf` : *str, path object, file-like object, or None, default None* String, path object (implementing os.PathLike[str]), or file-like object implementing a write() function. If None, the result is returned as a string. If a non-binary file object is passed, it should be opened with newline='', disabling universal newlines. If a binary file object is passed, mode might need to contain a 'b'.

`sep` : *str, default* ',' String of length 1. Field delimiter for the output file.

`na_rep` : *str, default* '' Missing data representation.

`float_format` : *str, Callable, default None* Format string for floating point numbers. If a Callable is given, it takes precedence over other numeric formatting parameters, like decimal.

`columns` : *sequence, optional* Columns to write.

`header` : *bool or list of str, default True* Write out the column names. If a list of strings is given it is assumed to be aliases for the column names.

`index` : *bool, default True* Write row names (index).

`index_label` : *str or sequence, or False, default None* Column label for index column(s) if desired. If None is given, and header and index are True, then the index names are used. A

sequence should be given if the object uses MultiIndex. If False do not print fields for index names. Use index_label=False for easier importing in R.

mode : *{'w', 'x', 'a'}, default 'w'* Forwarded to either open(mode=) or fsspec.open(mode=) to control the file opening. Typical values include:

- 'w', truncate the file first.
- 'x', exclusive creation, failing if the file already exists.
- 'a', append to the end of file if it exists.

encoding : *str, optional* A string representing the encoding to use in the output file, defaults to 'utf-8'. encoding is not supported if path_or_buf is a non-binary file object.

compression : *str or dict, default 'infer'* For on-the-fly compression of the output data. If 'infer' and 'path_or_buf' is path-like, then detect compression from the following extensions: '.gz', '.bz2', '.zip', '.xz', '.zst', '.tar', '.tar.gz', '.tar.xz' or '.tar.bz2' (otherwise no compression). Set to `None` for no compression. Can also be a dict with key `'method'` set to one of {`'zip'`, `'gzip'`, `'bz2'`, `'zstd'`, `'xz'`, `'tar'`} and other key-value pairs are forwarded to `zipfile.ZipFile`, `gzip.GzipFile`, `bz2.BZ2File`, `zstandard.ZstdCompressor`, `lzma.LZMAFile` or `tarfile.TarFile`, respectively. As an example, the following could be passed for faster compression and to create a reproducible gzip archive: `compression={'method': 'gzip', 'compresslevel': 1, 'mtime': 1}`.

> May be a dict with key 'method' as compression mode and other entries as additional compression options if compression mode is 'zip'.
> Passing compression options as keys in dict is supported for compression modes 'gzip', 'bz2', 'zstd', and 'zip'.

quoting : *optional constant from csv module* Defaults to csv.QUOTE_MINIMAL. If you have set a float_format then floats are converted to strings and thus csv.QUOTE_NONNUMERIC will treat them as non-numeric.

quotechar : *str, default '"'* String of length 1. Character used to quote fields.

lineterminator : *str, optional* The newline character or character sequence to use in the output file. Defaults to os.linesep, which depends on the OS in which this method is called ('\n' for linux, '\r\n' for Windows, i.e.).

chunksize : *int or None* Rows to write at a time.

date_format : *str, default None* Format string for datetime objects.

doublequote : *bool, default True* Control quoting of quotechar inside a field.

escapechar : *str, default None* String of length 1. Character used to escape sep and quotechar when appropriate.

decimal : *str, default '.'* Character recognized as decimal separator. E.g. use ',' for European data.

errors : *str, default 'strict'* Specifies how encoding and decoding errors are to be handled. See the errors argument for `open` for a full list of options.

storage_options : *dict, optional* Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer here <https://pandas.pydata.org/docs/user_guide/io.html? highlight=storage_options#reading-writing-remote-files>.

Returns

None or str If path_or_buf is None, returns the resulting csv format as a string. Otherwise returns None.

See also: `read_csv`, `to_excel`

Examples

Create 'out.csv' containing 'df' without indices

```
>>> df = pd.DataFrame(
...     [["Raphael", "red", "sai"], ["Donatello", "purple", "bo staff"]],
...     columns=["name", "mask", "weapon"],
... )
>>> df.to_csv("out.csv", index=False)
```

Create 'out.zip' containing 'out.csv'

```
>>> df.to_csv(index=False)
'name,mask,weapon\nRaphael,red,sai\nDonatello,purple,bo staff\n'
>>> compression_opts = dict(
...     method="zip", archive_name="out.csv"
... )
>>> df.to_csv(
...     "out.zip", index=False, compression=compression_opts
... )
...
```

pd.DataFrame.to_parquet

```
pd.DataFrame.to_parquet(self, path: 'FilePath | WriteBuffer[bytes] | None' = None, *, engine:
↪  "Literal['auto', 'pyarrow', 'fastparquet'] | lib.NoDefault" = <no_default>, compression:
↪  'ParquetCompressionOptions' = 'snappy', index: 'bool | None' = None, partition_cols: 'list[str] |
↪  None' = None, storage_options: 'StorageOptions | None' = None, filesystem: 'Any' = None, **kwargs) ->
↪  'bytes | None'
```

Write a DataFrame to the binary parquet format.

This function writes the dataframe as a parquet file. You can choose different parquet backends, and have the option of compression. See the user guide for more details.

Parameters

`path` **: *str, path object, file-like object, or None, default None*** String, path object (implementing `os.PathLike[str]`), or file-like object implementing a binary `write()` function. If None, the result is returned as bytes. If a string or path, it will be used as the root directory path when writing a partitioned dataset.

The string could be a URL. Valid URL schemes include http, ftp, s3, gs, and file. For file URLs, a host is expected. A local file could be: `file://localhost/path/to/table.parquet`. A remote example could be: `s3://bucket/path/to/table.parquet`.

Certain URL schemes may require additional packages. For example, S3 URLs require the `s3fs` library. See install.optional_dependencies for a full list.

`engine` **: *{'auto', 'pyarrow', 'fastparquet'}, default 'auto'*** Parquet library to use. If 'auto', then the option `io.parquet.engine` is used. The default `io.parquet.engine` behavior is to try 'pyarrow', falling back to 'fastparquet' if 'pyarrow' is unavailable.

Deprecated since version 3.1.0. The `'fastparquet'` and `'auto'` engine options are deprecated. Use `'pyarrow'` or do not pass `engine` to use the default.

compression : *str or None, default 'snappy'* Name of the compression to use. Use `None` for no compression. Supported options: 'snappy', 'gzip', 'brotli', 'lz4', 'zstd'.

index : *bool, default None* If `True`, include the dataframe's index(es) in the file output. If `False`, they will not be written to the file. If `None`, similar to `True` the dataframe's index(es) will be saved. However, instead of being saved as values, the RangeIndex will be stored as a range in the metadata so it doesn't require much space and is faster. Other indexes will be included as columns in the file output.

partition_cols : *list, optional, default None* Column names by which to partition the dataset. Columns are partitioned in the order they are given. Must be None if path is not a string.

storage_options : *dict, optional* Extra options that make sense for a particular storage connection, e.g. host, port, username, password, etc. For HTTP(S) URLs the key-value pairs are forwarded to `urllib.request.Request` as header options. For other URLs (e.g. starting with "s3://", and "gcs://") the key-value pairs are forwarded to `fsspec.open`. Please see `fsspec` and `urllib` for more details, and for more examples on storage options refer here <https://pandas.pydata.org/docs/user_guide/io.html? highlight=storage_options#reading-writing-remote-files>.

filesystem : *fsspec or pyarrow filesystem, default None* Filesystem object to use when reading the parquet file. Only implemented for `engine="pyarrow"`.

New in version 2.1.0.

****kwargs** Additional arguments passed to the parquet library. See pandas io for more details.

Returns

bytes if no path argument is provided else None Returns the DataFrame converted to the binary parquet format as bytes if no path argument. Returns None and writes the DataFrame to the specified location in the Parquet format if the path argument is provided.

See also: `read_parquet`, `DataFrame.to_orc`, `DataFrame.to_csv`, `DataFrame.to_sql`, `DataFrame.to_hdf`

Notes

- This function requires either the fastparquet or pyarrow library.
- When saving a DataFrame with categorical columns to parquet, the file size may increase due to the inclusion of all possible categories, not just those present in the data. This behavior is expected and consistent with pandas' handling of categorical data. To manage file size and ensure a more predictable roundtrip process, consider using `Categorical.remove_unused_categories` on the DataFrame before saving.

Examples

```
>>> df = pd.DataFrame(data={"col1": [1, 2], "col2": [3, 4]})
>>> df.to_parquet("df.parquet.gzip", compression="gzip")
>>> pd.read_parquet("df.parquet.gzip")
   col1  col2
0     1     3
1     2     4
```

If you want to get a buffer to the parquet content you can use a io.BytesIO object, as long as you don't use partition_cols, which creates multiple files.

```
>>> f = io.BytesIO()
>>> df.to_parquet(f)
>>> f.seek(0)
0
>>> content = f.read()
```

Core objects

pd.DataFrame

```
pd.DataFrame(data=None, index: 'Axes | None' = None, columns: 'Axes | None' = None, dtype: 'Dtype | None'
↪  = None, copy: 'bool | None' = None) -> 'None'
```

Two-dimensional, size-mutable, potentially heterogeneous tabular data.

Data structure also contains labeled axes (rows and columns). Arithmetic operations align on both row and column labels. Can be thought of as a dict-like container for Series objects. The primary pandas data structure.

Parameters

`data` : ***ndarray (structured or homogeneous), Iterable, dict, or DataFrame*** Dict can contain Series, arrays, constants, dataclass or list-like objects. If data is a dict, column order follows insertion-order. If a dict contains Series which have an index defined, it is aligned by its index. This alignment also occurs if data is a Series or a DataFrame itself. Alignment is done on Series/DataFrame inputs.

If data is a list of dicts, column order follows insertion-order.

`index` : ***Index or array-like*** Index to use for resulting frame. Will default to RangeIndex if no indexing information part of input data and no index provided.

`columns` : ***Index or array-like*** Column labels to use for resulting frame when data does not have them, defaulting to RangeIndex(0, 1, 2, ..., n). If data contains column labels, will perform column selection instead.

`dtype` : ***dtype, default None*** Data type to force. Only a single dtype is allowed. If None, infer. If `data` is DataFrame then is ignored. Missing values in `data` are not cast: for example, with `dtype=str` a `NaN` entry stays missing rather than becoming the string `"nan"`.

`copy` : ***bool or None, default None*** Copy data from inputs. For dict data, the default of None behaves like `copy=True`. For DataFrame or 2d ndarray input, the default of None behaves like `copy=False`. If data is a dict containing one or more Series (possibly of different dtypes), `copy=False` will ensure that these inputs are not copied.

See also: `DataFrame.from_records`, `DataFrame.from_dict`, `read_csv`, `read_table`, `read_clipboard`

Notes

Please reference the User Guide for more information.

Examples

Constructing DataFrame from a dictionary.

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df
   col1  col2
0     1     3
1     2     4
```

Notice that the inferred dtype is int64.

```
>>> df.dtypes
col1    int64
col2    int64
dtype: object
```

To enforce a single dtype: ...

pd.Series

```
pd.Series(data=None, index=None, dtype: 'Dtype | None' = None, name=None, copy: 'bool | None' = None) ->
↪  'None'
```

One-dimensional ndarray with axis labels (including time series).

Labels need not be unique but must be a hashable type. The object supports both integer- and label-based indexing and provides a host of methods for performing operations involving the index. Statistical methods from ndarray have been overridden to automatically exclude missing data (currently represented as NaN).

Operations between Series (+, -, /, *, **) align values based on their associated index values-- they need not be the same length. The result index will be the sorted union of the two indexes.

Parameters

`data` : ***array-like, Iterable, dict, or scalar value*** Contains data stored in Series. If data is a dict, argument order is maintained. Unordered sets are not supported.

`index` : ***array-like or Index (1d)*** Values must be hashable and have the same length as data. Non-unique index values are allowed. Will default to RangeIndex (0, 1, 2, ..., n) if not provided. If data is dict-like and index is None, then the keys in the data are used as the index. If the index is not None, the resulting Series is reindexed with the index values.

`dtype` : ***str, numpy.dtype, or ExtensionDtype, optional*** Data type for the output Series. If not specified, this will be inferred from data. See the user guide for more usages.

`name` : ***Hashable, default None*** The name to give to the Series.

`copy` : ***bool, default None*** Whether to copy input data, only relevant for array, Series, and Index inputs (for other input, e.g. a list, a new array is created anyway). Defaults to True for array input and False for Index/Series. Even when False for Index/Series, a shallow copy of the data is made. Set to False to avoid copying array input at your own risk (if you know the input data won't be modified elsewhere). Set to True to force copying Series/Index input up front.

See also: `DataFrame, Index`

Notes

Please reference the User Guide for more information.

NumPy arrays with `dtype=object` may be inferred to a more specific dtype (e.g. strings or date-times) rather than keeping object dtype, similar to passing a Python list. Pass `dtype=object` when you need object dtype. The same inference rules apply when building an `Index` from such an array.

Examples

Constructing Series from a dictionary with an Index specified

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> ser = pd.Series(data=d, index=["a", "b", "c"])
>>> ser
a   1
b   2
c   3
dtype: int64
```

The keys of the dictionary match with the Index values, hence the Index values have no effect.

```
>>> d = {"a": 1, "b": 2, "c": 3}
>>> ser = pd.Series(data=d, index=["x", "y", "z"])
>>> ser
x   NaN
y   NaN
z   NaN
dtype: float64
...
```

Inspecting and selecting

pd.DataFrame.head

```
pd.DataFrame.head(self, n: 'int' = 5) -> 'Self'
```

Return the first n rows.

This function exhibits the same behavior as `df[:n]`, returning the first `n` rows based on position. It is useful for quickly checking if your object has the right type of data in it.

When `n` is positive, it returns the first `n` rows. For `n` equal to 0, it returns an empty object. When `n` is negative, it returns all rows except the last `|n|` rows, mirroring the behavior of `df[:n]`.

If `n` is larger than the number of rows, this function returns all rows.

Parameters

`n` : *int, default 5* Number of rows to select.

Returns

same type as caller The first n rows of the caller object.

See also: `DataFrame.tail`

Examples

```
>>> df = pd.DataFrame(
...     {
...         "animal": [
...             "alligator",
...             "bee",
...             "falcon",
...             "lion",
...             "monkey",
...             "parrot",
...             "shark",
...             "whale",
...             "zebra",
...         ]
...     }
... )
>>> df
      animal
0  alligator
1        bee
2     falcon
3       lion
4     monkey
5     parrot
```

```
6      shark
...
```

pd.DataFrame.info

```
pd.DataFrame.info(self, verbose: 'bool | None' = None, buf: 'WriteBuffer[str] | None' = None, max_cols:
↪  'int | None' = None, memory_usage: 'bool | str | None' = None, show_counts: 'bool | None' = None) ->
↪  'None'
```

Print a concise summary of a DataFrame.

This method prints information about a DataFrame including the index dtype and columns, non-NA values and memory usage.

Parameters

`verbose` : ***bool, optional*** Whether to print the full summary. By default, the setting in `pandas.options.display.max_info_columns` is followed.

`buf` : ***writable buffer, defaults to sys.stdout*** Where to send the output. By default, the output is printed to sys.stdout. Pass a writable buffer if you need to further process the output.

`max_cols` : ***int, optional*** When to switch from the verbose to the truncated output. If the DataFrame has more than max_cols columns, the truncated output is used. By default, the setting in `pandas.options.display.max_info_columns` is used.

`memory_usage` : ***bool, str, optional*** Specifies whether total memory usage of the DataFrame elements (including the index) should be displayed. By default, this follows the `pandas.options.display.memory_usage` setting.

True always show memory usage. False never shows memory usage. A value of 'deep' is equivalent to "True with deep introspection". Memory usage is shown in human-readable units (base-2 representation). Without deep introspection a memory estimation is made based in column dtype and number of rows assuming values consume the same memory amount for corresponding dtypes. With deep memory introspection, a real memory usage calculation is performed at the cost of computational resources. See the Frequently Asked Questions for more details.

`show_counts` : ***bool, optional*** Whether to show the non-null counts. By default, this is shown only if the DataFrame is smaller than `pandas.options.display.max_info_rows` and `pandas.options.display.max_info_columns`. A value of True always shows the counts, and False never shows the counts.

Returns

None This method prints a summary of a DataFrame and returns None.

See also: `DataFrame.describe`, `DataFrame.memory_usage`

Examples

```
>>> int_values = [1, 2, 3, 4, 5]
>>> text_values = ["alpha", "beta", "gamma", "delta", "epsilon"]
>>> float_values = [0.0, 0.25, 0.5, 0.75, 1.0]
>>> df = pd.DataFrame(
...     {
...         "int_col": int_values,
...         "text_col": text_values,
...         "float_col": float_values,
...     }
... )
```

```
>>> df
    int_col text_col  float_col
0         1    alpha       0.00
1         2     beta       0.25
2         3    gamma       0.50
3         4    delta       0.75
4         5  epsilon       1.00
...
```

pd.DataFrame.describe

```
pd.DataFrame.describe(self, percentiles=None, include=None, exclude=None) -> 'DataFrame'
```

Generate descriptive statistics.

Summarize the central tendency, dispersion, and shape of each analyzed column's distribution, excluding `NaN` values. By default only numeric columns are analyzed; pass `include` to also analyze non-numeric columns (or `exclude` to omit columns by dtype).

Parameters

`percentiles` : *list-like of numbers, optional* The percentiles to include in the output. All should fall between 0 and 1. The default, `None`, returns the 25th, 50th, and 75th percentiles.

`include` : *'all', list-like of dtypes or None (default), optional* Which column dtypes to include. Options:

- `'all'` : Include all columns, including non-numeric ones.
- list-like of dtypes : Limit the result to columns of the given dtypes, in the style of `DataFrame.select_dtypes` (e.g. `include=[np.number]` or `include=["category"]`).
- `None` (default) : Include only numeric columns, falling back to object and categorical columns if there are no numeric columns.

`exclude` : *list-like of dtypes or None (default), optional* Column dtypes to omit from the result, in the style of `DataFrame.select_dtypes`. `None` (default) excludes nothing.

Returns

DataFrame Summary statistics of the DataFrame's columns.

See also: `Series.describe`, `DataFrame.count`, `DataFrame.max`, `DataFrame.min`, `DataFrame.mean`, `DataFrame.std`, `DataFrame.select_dtypes`

Notes

For numeric columns, the result's index includes `count`, `mean`, `std`, `min`, `max`, and the requested percentiles. By default the lower percentile is `25` and the upper is `75`; the `50` percentile is the same as the median.

For object columns, the result's index includes `count`, `unique`, `top`, and `freq`. The `top` is the most common value and `freq` is its count. If multiple values tie for the highest count, `top` is chosen arbitrarily from among them.

With `include='all'`, the result's index is the union of the per-dtype indices, with `NaN` for statistics that do not apply to a given column's dtype.

Examples

By default, only numeric columns are analyzed.

```
>>> df = pd.DataFrame(
...     {
```

```
...         "categorical": pd.Categorical(["d", "e", "f"]),
...         "numeric": [1, 2, 3],
...         "object": ["a", "b", "c"],
...     }
... )
>>> df.describe()
       numeric
count      3.0
mean       2.0
std        1.0
min        1.0
25%        1.5
50%        2.0
75%        2.5
max        3.0
...
```

pd.DataFrame.query

```
pd.DataFrame.query(self, expr: 'str', *, parser: "Literal['pandas', 'python']" = 'pandas', engine:
↪  "Literal['python', 'numexpr'] | None" = None, local_dict: 'dict[str, Any] | None' = None,
↪  global_dict: 'dict[str, Any] | None' = None, resolvers: 'list[Mapping] | None' = None, level: 'int' =
↪  0, inplace: 'bool | lib.NoDefault' = <no_default>) -> 'DataFrame | None'
```

Query the columns of a DataFrame with a boolean expression.

> Warning
> This method can run arbitrary code which can make you vulnerable to code injection if you pass user input to this function.

Parameters

`expr` : *str* The query string to evaluate.

See the documentation for `eval` for details of supported operations and functions in the query string.

See the documentation for `DataFrame.eval` for details on referring to column names and variables in the query string.

`parser` : *{'pandas', 'python'}, default 'pandas'* The parser to use to construct the syntax tree from the expression. The default of `'pandas'` parses code slightly different than standard Python. Alternatively, you can parse an expression using the `'python'` parser to retain strict Python semantics. See the enhancing performance documentation for more details.

`engine` : *{'python', 'numexpr'}, default 'numexpr'* The engine used to evaluate the expression. Supported engines are

- None : tries to use `numexpr`, falls back to `python`
- `'numexpr'` : This default engine evaluates pandas objects using numexpr for large speed ups in complex expressions with large frames.
- `'python'` : Performs operations as if you had `eval`'d in top level python. This engine is generally not that useful.

More backends may be available in the future.

`local_dict` : *dict or None, optional* A dictionary of local variables, taken from locals() by default.

global_dict : *dict or None, optional* A dictionary of global variables, taken from globals() by default.

resolvers : *list of dict-like or None, optional* A list of objects implementing the `__getitem__` special method that you can use to inject an additional collection of namespaces to use for variable lookup. For example, this is used in the `query` method to inject the `DataFrame.index` and `DataFrame.columns` variables that refer to their respective `DataFrame` instance attributes.

level : *int, optional* The number of prior stack frames to traverse and add to the current scope. Most users will **not** need to change this parameter.

inplace : *bool* Whether to modify the DataFrame rather than creating a new one.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

Returns

DataFrame or None DataFrame resulting from the provided query expression or None if `inplace=True`.

Raises

ValueError If `expr` refers to a column label that is not unique and still evaluates to a DataFrame, since there is then no row mask to select with. Reducing those columns back to one dimension, as in `df.query("a.max(axis=1) > 4")`, is fine.

See also: `eval`, `DataFrame.eval`

Notes

The result of the evaluation of this expression is first passed to `DataFrame.loc` and if that fails because of a multidimensional key (e.g., a DataFrame) then the result will be passed to `DataFrame.__getitem__`.

This method uses the top-level `eval` function to evaluate the passed query.

The `query` method uses a slightly modified Python syntax by default. For example, the `&` and `|` (bitwise) operators have the precedence of their boolean cousins, `and` and `or`. This *is* syntactically valid Python, however the semantics are different.

(Notes truncated; see the full reference page.)

Examples

```
>>> df = pd.DataFrame(
...     {"A": range(1, 6), "B": range(10, 0, -2), "C&C": range(10, 5, -1)}
... )
>>> df
   A   B  C&C
0  1  10   10
1  2   8    9
2  3   6    8
3  4   4    7
4  5   2    6
>>> df.query("A > B")
   A  B  C&C
4  5  2    6
```

The previous expression is equivalent to

```
>>> df[df.A > df.B]
   A  B  C&C
4  5  2    6
...
```

pd.DataFrame.set_index

```
pd.DataFrame.set_index(self, keys, *, drop: 'bool' = True, append: 'bool' = False, inplace: 'bool |
↪  lib.NoDefault' = <no_default>, verify_integrity: 'bool | lib.NoDefault' = <no_default>) -> 'DataFrame
↪  | None'
```

Set the DataFrame index using existing columns.

Set the DataFrame index (row labels) using one or more existing columns or arrays (of the correct length). The index can replace the existing index or expand on it.

Parameters

keys : *label or array-like or list of labels/arrays* This parameter can be either a single column key, a single array of the same length as the calling DataFrame, or a list containing an arbitrary combination of column keys and arrays. Here, "array" encompasses `Series`, `Index`, `np.ndarray`, and instances of `Iterator`.

drop : *bool, default True* Delete columns to be used as the new index.

append : *bool, default False* Whether to append columns to existing index. Setting to True will add the new columns to existing index. When set to False, the current index will be dropped from the DataFrame.

inplace : *bool, default False* Whether to modify the DataFrame rather than creating a new one.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

verify_integrity : *bool, default False* Check the new index for duplicates. Otherwise defer the check until necessary. Setting to False will improve the performance of this method.

Deprecated since version 3.0.0. The `verify_integrity` keyword is deprecated and will be removed in a future version. Check `result.index.is_unique` directly instead.

Returns

DataFrame or None Changed row labels or None if `inplace=True`.

See also: `DataFrame.reset_index`, `DataFrame.reindex`, `DataFrame.reindex_like`

Notes

Columns and Series do not store a `freq`, so a `DatetimeIndex` or `TimedeltaIndex` created from a column has `freq=None`; set `.index.freq` on the result to restore it. Passing the index directly as `keys` keeps its `freq`.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "month": [1, 4, 7, 10],
...         "year": [2012, 2014, 2013, 2014],
...         "sale": [55, 40, 84, 31],
...     }
... )
>>> df
   month  year  sale
0      1  2012    55
1      4  2014    40
2      7  2013    84
3     10  2014    31
```

Set the index to become the 'month' column:

```
>>> df.set_index("month")
       year  sale
month
1      2012    55
4      2014    40
7      2013    84
10     2014    31
...
```

pd.DataFrame.reset_index

```
pd.DataFrame.reset_index(self, level: 'IndexLabel | None' = None, *, drop: 'bool' = False, inplace: 'bool
↪  | lib.NoDefault' = <no_default>, col_level: 'Hashable' = 0, col_fill: 'Hashable' = '',
↪  allow_duplicates: 'bool' = False, names: 'Hashable | Sequence[Hashable] | None' = None) -> 'DataFrame
↪  | None'
```

Reset the index, or a level of it.

Reset the index of the DataFrame, and use the default one instead. If the DataFrame has a MultiIndex, this method can remove one or more levels.

Parameters

`level` : ***int, str, tuple, or list, default None*** Only remove the given levels from the index. Removes all levels by default.

`drop` : ***bool, default False*** Do not try to insert index into dataframe columns. This resets the index to the default integer index.

`inplace` : ***bool, default False*** Whether to modify the DataFrame rather than creating a new one.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

`col_level` : ***int or str, default 0*** If the columns have multiple levels, determines which level the labels are inserted into. By default it is inserted into the first level.

`col_fill` : ***object, default ''*** If the columns have multiple levels, determines how the other levels are named. If None then the index name is repeated.

`allow_duplicates` : ***bool, default False*** Allow duplicate column labels to be created.

`names` : ***int, str or 1-dimensional list, default None*** Using the given string, rename the DataFrame column which contains the index data. If the DataFrame has a MultiIndex, this has to be a list with length equal to the number of levels.

Returns

DataFrame or None DataFrame with the new index or None if `inplace=True`.

See also: `DataFrame.set_index`, `DataFrame.reindex`, `DataFrame.reindex_like`

Notes

Columns do not store a `freq`, so the `freq` of a `DatetimeIndex` or `TimedeltaIndex` is lost when it is inserted as a column, and is not restored by a subsequent `DataFrame.set_index`.

Examples

```
>>> df = pd.DataFrame(
...     [("bird", 389.0), ("bird", 24.0), ("mammal", 80.5), ("mammal", np.nan)],
...     index=["falcon", "parrot", "lion", "monkey"],
...     columns=("class", "max_speed"),
... )
>>> df
         class  max_speed
falcon    bird      389.0
parrot    bird       24.0
lion    mammal       80.5
monkey  mammal        NaN
```

When we reset the index, the old index is added as a column, and a new sequential index is used:

```
>>> df.reset_index()
    index   class  max_speed
0  falcon    bird      389.0
1  parrot    bird       24.0
2    lion  mammal       80.5
3  monkey  mammal        NaN
...
```

pd.DataFrame.sort_values

```
pd.DataFrame.sort_values(self, by: 'IndexLabel', *, axis: 'Axis' = 0, ascending: 'bool | list[bool] |
↪  tuple[bool, ...]' = True, inplace: 'bool | lib.NoDefault' = <no_default>, kind: 'SortKind' =
↪  'quicksort', na_position: 'str' = 'last', ignore_index: 'bool' = False, key: 'ValueKeyFunc | None' =
↪  None) -> 'DataFrame | None'
```

Sort by the values along either axis.

This method sorts the DataFrame by the values in one or more columns or by index/column labels.

Parameters

by : *str or list of str* Name or list of names to sort by.

- if axis is 0 or 'index' then by may contain index levels and/or column labels.
- if axis is 1 or 'columns' then by may contain column levels and/or index labels.

axis : *"{0 or 'index', 1 or 'columns'}", default 0* Axis to be sorted.

ascending : *bool or list of bool, default True* Sort ascending vs. descending. Specify list for multiple sort orders. If this is a list of bools, must match the length of the by.

inplace : *bool, default False* If True, perform operation in-place.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

kind : *{'quicksort', 'mergesort', 'heapsort', 'stable'}, default 'quicksort'* Choice of sorting algorithm. See also `numpy.sort` for more information. The sort order is deterministic for a given input. mergesort and stable are the only stable algorithms, which preserve the relative order of equal keys. For DataFrames, this option is only applied when sorting on a single column or label.

na_position : *{'first', 'last'}, default 'last'* Puts NaNs at the beginning if first; last puts NaNs at the end.

ignore_index : *bool, default False* If True, the resulting axis will be labeled 0, 1, …, n - 1.

key : *callable, optional* Apply the key function to the values before sorting. This is similar to the key argument in the builtin `sorted` function, with the notable difference that this key function should be *vectorized*. It should expect a `Series` and return a Series with the same shape as the input. It will be applied to each column in by independently. The values in the returned Series will be used as the keys for sorting.

Returns

DataFrame or None DataFrame with sorted values or None if `inplace=True`.

See also: `DataFrame.sort_index`, `Series.sort_values`

Examples

```
>>> df = pd.DataFrame(
...     {
...         "col1": ["A", "A", "B", np.nan, "D", "C"],
...         "col2": [2, 1, 9, 8, 7, 4],
...         "col3": [0, 1, 9, 4, 2, 3],
...         "col4": ["a", "B", "c", "D", "e", "F"],
...     }
... )
>>> df
  col1  col2  col3 col4
0    A     2     0    a
1    A     1     1    B
2    B     9     9    c
3  NaN     8     4    D
4    D     7     2    e
5    C     4     3    F
...
```

pd.Series.value_counts

```
pd.Series.value_counts(self, normalize: 'bool' = False, sort: 'bool' = True, ascending: 'bool' = False,
↪  bins=None, dropna: 'bool' = True) -> 'Series'
```

Return a Series containing counts of unique values.

The resulting object will be in descending order so that the first element is the most frequently-occurring element. Excludes NA values by default.

Parameters

normalize : *bool, default False* If True then the object returned will contain the relative frequencies of the unique values.

sort : *bool, default True* Stable sort by frequencies when True. Preserve the order of the data when False.

Changed in version 3.0.0. Prior to 3.0.0, the sort was unstable.

ascending : *bool, default False* Sort in ascending order.

bins : *int, optional* Rather than count values, group them into half-open bins, a convenience for `pd.cut`, only works with numeric data.

dropna : *bool, default True* Don't include counts of NaN.

Returns

Series Series containing counts of unique values.

See also: `Series.count`, `DataFrame.count`, `DataFrame.value_counts`

Examples

```
>>> index = pd.Index([3, 1, 2, 3, 4, np.nan])
>>> index.value_counts()
3.0    2
1.0    1
2.0    1
4.0    1
Name: count, dtype: int64
```

With normalize set to True, returns the relative frequency by dividing all values by the sum of values.

```
>>> s = pd.Series([3, 1, 2, 3, 4, np.nan])
>>> s.value_counts(normalize=True)
3.0    0.4
1.0    0.2
2.0    0.2
4.0    0.2
Name: proportion, dtype: float64
...
```

Cleaning

pd.DataFrame.isna

```
pd.DataFrame.isna(self) -> 'DataFrame'
```

Detect missing values.

Return a boolean same-sized object indicating if the values are NA. NA values, such as None or `numpy.NaN`, gets mapped to True values. Everything else gets mapped to False values. Characters such as empty strings `''` or `numpy.inf` are not considered NA values.

Returns

Series/DataFrame Mask of bool values for each element in Series/DataFrame that indicates whether an element is an NA value.

See also: `Series.isnull`, `DataFrame.isnull`, `Series.notna`, `DataFrame.notna`, `Series.dropna`, `DataFrame.dropna`, `isna`

Examples

Show which entries in a DataFrame are NA.

```
>>> df = pd.DataFrame(
...     dict(
...         age=[5, 6, np.nan],
...         born=[
...             pd.NaT,
...             pd.Timestamp("1939-05-27"),
...             pd.Timestamp("1940-04-25"),
...         ],
...         name=["Alfred", "Batman", ""],
...         toy=[None, "Batmobile", "Joker"],
...     )
... )
```

```
>>> df
   age       born    name        toy
0  5.0        NaT  Alfred        NaN
1  6.0 1939-05-27  Batman  Batmobile
2  NaN 1940-04-25              Joker
...
```

pd.DataFrame.dropna

```
pd.DataFrame.dropna(self, *, axis: 'Axis' = 0, how: 'AnyAll | lib.NoDefault' = <no_default>, thresh: 'int
↪  | lib.NoDefault' = <no_default>, subset: 'IndexLabel | AnyArrayLike | None' = None, inplace: 'bool |
↪  lib.NoDefault' = <no_default>, ignore_index: 'bool' = False) -> 'DataFrame | None'
```

Remove missing values.

See the User Guide for more on which values are considered missing, and how to work with missing data.

Parameters

axis : *{0 or 'index', 1 or 'columns'}, default 0* Determine if rows or columns which contain missing values are removed.

- 0, or 'index' : Drop rows which contain missing values.
- 1, or 'columns' : Drop columns which contain missing values.

Only a single axis is allowed.

how : *{'any', 'all'}, default 'any'* Determine if row or column is removed from DataFrame, when we have at least one NA or all NA.

- 'any' : If any NA values are present, drop that row or column.
- 'all' : If all values are NA, drop that row or column.

thresh : *int, optional* Require that many non-NA values. Cannot be combined with how.

subset : *column label or iterable of labels, optional* Labels along other axis to consider, e.g. if you are dropping rows these would be a list of columns to include.

inplace : *bool, default False* Whether to modify the DataFrame rather than creating a new one.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

ignore_index : *bool, default "False"* If `True`, the resulting axis will be labeled 0, 1, …, n - 1.

New in version 2.0.0.

Returns

DataFrame or None DataFrame with NA entries dropped from it or None if `inplace=True`.

See also: `DataFrame.isna`, `DataFrame.notna`, `DataFrame.fillna`, `Series.dropna`, `Index.dropna`

Examples

```
>>> df = pd.DataFrame(
...     {
...         "name": ["Alfred", "Batman", "Catwoman"],
...         "toy": [np.nan, "Batmobile", "Bullwhip"],
...         "born": [pd.NaT, pd.Timestamp("1940-04-25"), pd.NaT],
...     }
... )
>>> df
```

```
      name        toy       born
0    Alfred        NaN        NaT
1    Batman  Batmobile 1940-04-25
2  Catwoman   Bullwhip        NaT
```

Drop the rows where at least one element is missing.

```
>>> df.dropna()
     name        toy       born
1  Batman  Batmobile 1940-04-25
...
```

pd.DataFrame.fillna

```
pd.DataFrame.fillna(self, value: 'Hashable | Mapping | Series | DataFrame', *, axis: 'Axis | None' =
↪  None, inplace: 'bool' = False, limit: 'int | None' = None) -> 'Self'
```

Fill NA/NaN values with value.

This method replaces missing values with a specified value.

Parameters

`value` : ***scalar, dict, Series, or DataFrame*** Value to use to fill holes (e.g. 0), alternately a dict/Series/DataFrame of values specifying which value to use for each index (for a Series) or column (for a DataFrame). Values not in the dict/Series/DataFrame will not be filled. This value cannot be a list.

`axis` : ***{0 or 'index'} for Series, {0 or 'index', 1 or 'columns'} for DataFrame*** Axis along which to fill missing values. For Series this parameter is unused and defaults to 0.

`inplace` : ***bool, default False*** If True, fill in-place. Note: this will modify any other views on this object (e.g., a no-copy slice for a column in a DataFrame).

`limit` : ***int, default None*** This is the maximum number of entries along the entire axis where NaNs will be filled. Must be greater than 0 if not None.

Returns

Series/DataFrame Object with missing values filled.

See also: `ffill`, `bfill`, `interpolate`, `reindex`, `asfreq`

Notes

For non-object dtype, `value=None` will use the NA value of the dtype. See more details in the Filling missing data section.

Examples

```
>>> df = pd.DataFrame(
...     [
...         [np.nan, 2, np.nan, 0],
...         [3, 4, np.nan, 1],
...         [np.nan, np.nan, np.nan, np.nan],
...         [np.nan, 3, np.nan, 4],
...     ],
...     columns=list("ABCD"),
... )
>>> df
     A    B   C    D
0  NaN  2.0 NaN  0.0
```

```
1  3.0  4.0 NaN  1.0
2  NaN  NaN NaN  NaN
3  NaN  3.0 NaN  4.0
```

Replace all NaN elements with 0s. ...

pd.DataFrame.drop_duplicates

```
pd.DataFrame.drop_duplicates(self, subset: 'Hashable | Iterable[Hashable] | None' = None, *, keep:
↪  'DropKeep' = 'first', inplace: 'bool | lib.NoDefault' = <no_default>, ignore_index: 'bool' = False)
↪  -> 'DataFrame | None'
```

Return DataFrame with duplicate rows removed.

Considering certain columns is optional. Indexes, including time indexes are ignored.

Parameters

`subset` : ***column label or iterable of labels, optional*** Only consider certain columns for identifying duplicates, by default use all of the columns.

`keep` : ***{'first', 'last', "False"}, default 'first'*** Determines which duplicates (if any) to keep.

- 'first' : Drop duplicates except for the first occurrence.
- 'last' : Drop duplicates except for the last occurrence.
- `False` : Drop all duplicates.

`inplace` : ***bool, default "False"*** Whether to modify the DataFrame rather than creating a new one.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

`ignore_index` : ***bool, default "False"*** If `True`, the resulting axis will be labeled 0, 1, …, n - 1.

Returns

DataFrame or None DataFrame with duplicates removed or None if `inplace=True`.

See also: `DataFrame.value_counts`, `Index.duplicated`

Notes

This method requires columns specified by `subset` to be of hashable type. Passing unhashable columns will raise a `TypeError`.

Examples

Consider dataset containing ramen rating.

```
>>> df = pd.DataFrame(
...     {
...         "brand": ["Yum Yum", "Yum Yum", "Indomie", "Indomie", "Indomie"],
...         "style": ["cup", "cup", "cup", "pack", "pack"],
...         "rating": [4, 4, 3.5, 15, 5],
...     }
... )
>>> df
    brand style  rating
0  Yum Yum   cup     4.0
1  Yum Yum   cup     4.0
2  Indomie   cup     3.5
3  Indomie  pack    15.0
```

```
4  Indomie  pack     5.0
...
```

pd.DataFrame.astype

```
pd.DataFrame.astype(self, dtype, copy: 'bool | lib.NoDefault' = <no_default>, errors: 'IgnoreRaise' =
↪  'raise') -> 'Self'
```

Cast a pandas object to a specified dtype `dtype`.

This method allows the conversion of the data types of pandas objects, including DataFrames and Series, to the specified dtype. It supports casting entire objects to a single data type or applying different data types to individual columns using a mapping.

Parameters

`dtype` : *str, data type, Series or Mapping of column name -> data type* Use a str, numpy.dtype, pandas.ExtensionDtype or Python type to cast entire pandas object to the same type. Alternatively, use a mapping, e.g. {col: dtype, ...}, where col is a column label and dtype is a numpy.dtype or Python type to cast one or more of the DataFrame's columns to column-specific types. The mapping may be a `dict` or a `Series` indexed by column label; columns whose labels are absent from the mapping are left unchanged. A Python type (e.g. `int`, `float`, `str`, `bool`) is mapped to the corresponding dtype; a type with no such mapping, such as `datetime.datetime`, raises `TypeError`.

`copy` : *bool, default False* This keyword is now ignored; changing its value will have no impact on the method.

Deprecated since version 3.0.0. This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the user guide on Copy-on-Write for more details.

`errors` : *{'raise', 'ignore'}, default 'raise'* Control raising of exceptions on invalid data for provided dtype.

- `raise` : allow exceptions to be raised
- `ignore` : suppress exceptions. On error return original object.

This does not apply to keys in a `dtype` mapping that are not column labels (or, for a Series, not its name); those always raise `KeyError`.

Returns

same type as caller The pandas object casted to the specified `dtype`.

See also: `to_datetime`, `to_timedelta`, `to_numeric`, `numpy.ndarray.astype`

Notes

Changed in version 2.0.0. Using `astype` to convert from timezone-naive dtype to timezone-aware dtype will raise an exception. Use `Series.dt.tz_localize` instead.

Examples

Create a DataFrame:

```
>>> d = {"col1": [1, 2], "col2": [3, 4]}
>>> df = pd.DataFrame(data=d)
>>> df.dtypes
col1    int64
```

```
col2    int64
dtype: object
```

Cast all columns to int32:

```
>>> df.astype("int32").dtypes
col1    int32
col2    int32
dtype: object
```

Cast col1 to int32 using a dictionary: ...

pd.DataFrame.rename

```
pd.DataFrame.rename(self, mapper: 'Renamer | None' = None, *, index: 'Renamer | None' = None, columns:
↪  'Renamer | None' = None, axis: 'Axis | None' = None, copy: 'bool | lib.NoDefault' = <no_default>,
↪  inplace: 'bool | lib.NoDefault' = <no_default>, level: 'Level | None' = None, errors: 'IgnoreRaise' =
↪  'ignore') -> 'DataFrame | None'
```

Rename columns or index labels.

Function / dict values must be unique (1-to-1). Labels not contained in a dict / Series will be left as-is. Extra labels listed don't throw an error.

See the user guide for more.

Parameters

`mapper` : *dict-like or function* Dict-like or function transformations to apply to that axis' values. Use either `mapper` and `axis` to specify the axis to target with `mapper`, or `index` and `columns`.

`index` : *dict-like or function* Alternative to specifying axis (`mapper, axis=0` is equivalent to `index=mapper`).

`columns` : *dict-like or function* Alternative to specifying axis (`mapper, axis=1` is equivalent to `columns=mapper`).

`axis` : *{0 or 'index', 1 or 'columns'}, default 0* Axis to target with `mapper`. Can be either the axis name ('index', 'columns') or number (0, 1). The default is 'index'.

`copy` : *bool, default False* This keyword is now ignored; changing its value will have no impact on the method.

Deprecated since version 3.0.0. This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the user guide on Copy-on-Write for more details.

`inplace` : *bool, default False* Whether to modify the DataFrame rather than creating a new one. If True then value of copy is ignored.

Deprecated since version 3.1.0. This keyword is deprecated and will be removed in pandas 4.0. See the whatsnew note on PDEP-8 for more details.

`level` : *int or level name, default None* In case of a MultiIndex, only rename labels in the specified level.

`errors` : *{'ignore', 'raise'}, default 'ignore'* If 'raise', raise a KeyError when a dict-like mapper, index, or columns contains labels that are not present in the Index being transformed. If 'ignore', existing keys will be renamed and extra keys will be ignored.

Returns

DataFrame or None DataFrame with the renamed axis labels or None if `inplace=True`.

Raises

KeyError If any of the labels is not found in the selected axis and "errors='raise'".

See also: `DataFrame.rename_axis`

Examples

`DataFrame.rename` supports two calling conventions

- `(index=index_mapper, columns=columns_mapper, ...)`
- `(mapper, axis={'index', 'columns'}, ...)`

We *highly* recommend using keyword arguments to clarify your intent.

Rename columns using a mapping:

```
>>> df = pd.DataFrame({"A": [1, 2, 3], "B": [4, 5, 6]})
>>> df.rename(columns={"A": "a", "B": "c"})
   a  c
0  1  4
1  2  5
2  3  6
...
```

pd.DataFrame.replace

```
pd.DataFrame.replace(self, to_replace=None, value=<no_default>, *, inplace: 'bool' = False, regex: 'bool'
↪  = False) -> 'Self'
```

Replace values given in to_replace with value.

Values of the Series/DataFrame are replaced with other values dynamically. This differs from updating with `.loc` or `.iloc`, which require you to specify a location to update with some value.

Parameters

`to_replace` **: *str, regex, list, dict, Series, int, float, or None*** How to find the values that will be replaced.

- numeric, str or regex:

  - numeric: numeric values equal to to_replace will be replaced with value
  - str: string exactly matching to_replace will be replaced with value
  - regex: regexes matching to_replace will be replaced with value

- list of str, regex, or numeric:

  - First, if to_replace and value are both lists, they **must** be the same length.
  - Second, if `regex=True` then all of the strings in **both** lists will be interpreted as regexes otherwise they will match directly. This doesn't matter much for value since there are only a few possible substitution regexes you can use.
  - str, regex and numeric rules apply as above.

- dict:

  - Dicts can be used to specify different replacement values for different existing values. For example, `{'a': 'b', 'y': 'z'}` replaces the value 'a' with 'b' and 'y'

with 'z'. To use a dict in this way, the optional value parameter should not be given.
- For a DataFrame a dict can specify that different values should be replaced in different columns. For example, `{'a': 1, 'b': 'z'}` looks for the value 1 in column 'a' and the value 'z' in column 'b' and replaces these values with whatever is specified in value. The value parameter should not be `None` in this case. You can treat this as a special case of passing two lists except that you are specifying the column to search in.
- For a DataFrame nested dictionaries, e.g., `{'a': {'b': np.nan}}`, are read as follows: look in column 'a' for the value 'b' and replace it with NaN. The optional value parameter should not be specified to use a nested dict in this way. You can nest regular expressions as well. Note that column names (the top-level dictionary keys in a nested dictionary) **cannot** be regular expressions.

- None:

  - This means that the regex argument must be a string, compiled regular expression, or list, dict, ndarray or Series of such elements. If value is also `None` then this **must** be a nested dictionary or Series.

See the examples section for examples of each of these.

`value` : *scalar, dict, list, str, regex, default None* Value to replace any values matching to_replace with. For a DataFrame a dict of values can be used to specify which value to use for each column (columns not in the dict will not be filled). Regular expressions, strings and lists or dicts of such objects are also allowed.

`inplace` : *bool, default False* If True, performs operation inplace.

`regex` : *bool or same types as 'to_replace', default False* Whether to interpret to_replace and/or value as regular expressions. Alternatively, this could be a regular expression or a list, dict, or array of regular expressions in which case to_replace must be `None`. Patterns may be passed either as strings or as compiled regex objects (`re.compile(...)`); use a compiled object when you need to set flags such as `re.IGNORECASE`, since `replace` does not accept a separate `flags` argument.

Returns

Series/DataFrame Object after replacement.

Raises

AssertionError
- If regex is not a `bool` and to_replace is not `None`.

TypeError
- If to_replace is not a scalar, array-like, `dict`, or `None`
- If to_replace is a `dict` and value is not a `list`, `dict`, `ndarray`, or `Series`
- If to_replace is `None` and regex is not compilable into a regular expression or is a list, dict, ndarray, or Series.
- When replacing multiple `bool` or `datetime64` objects and the arguments to to_replace does not match the type of the value being replaced

ValueError
- If a `list` or an `ndarray` is passed to to_replace and value but they are not the same length.

See also: `Series.fillna`, `DataFrame.fillna`, `Series.where`, `DataFrame.where`, `DataFrame.map`, `Series.map`, `Series.str.replace`

Notes

(Notes truncated; see the full reference page.)

Examples

Scalar ‘to_replace‘ and ‘value‘

```
>>> s = pd.Series([1, 2, 3, 4, 5])
>>> s.replace(1, 5)
0    5
1    2
2    3
3    4
4    5
dtype: int64
```

```
>>> df = pd.DataFrame(
...     {
...         "A": [0, 1, 2, 3, 4],
...         "B": [5, 6, 7, 8, 9],
...         "C": ["a", "b", "c", "d", "e"],
...     }
... )
>>> df.replace(0, 5)
    A  B  C
0  5  5  a
1  1  6  b
2  2  7  c
3  3  8  d
...
```

pd.to_datetime

```
pd.to_datetime(arg: 'DatetimeScalarOrArrayConvertible | DictConvertible', errors: 'DateTimeErrorChoices'
↪  = 'raise', dayfirst: 'bool' = False, yearfirst: 'bool' = False, utc: 'bool' = False, format: 'str |
↪  None' = None, exact: 'bool | lib.NoDefault' = <no_default>, unit: 'str | None' = None, origin: 'str' =
↪  'unix', cache: 'bool' = True) -> 'DatetimeIndex | Series | DatetimeScalar | NaTType'
```

Convert argument to datetime.

This function converts a scalar, array-like, `Series` or `DataFrame`/dict-like to a pandas datetime object.

Parameters

`arg` : ***int, float, str, datetime, list, tuple, 1-d array, Series, DataFrame/dict-like*** The object to convert to a datetime. If a `DataFrame` is provided, the method expects minimally the following columns: `"year"`, `"month"`, `"day"`. The column ”year” must be specified in 4-digit format.

`errors` : ***{'raise', 'coerce'}, default 'raise'***

- If `'raise'`, then invalid parsing will raise an exception.
- If `'coerce'`, then invalid parsing will be set as `NaT`.

`dayfirst` : ***bool, default False*** Specify a date parse order if arg is str or is list-like. If `True`, parses dates with the day first, e.g. `"10/11/12"` is parsed as `2012-11-10`.

> Warning
> `dayfirst=True` is not strict, but will prefer to parse with day first.

`yearfirst` : ***bool, default False*** Specify a date parse order if arg is str or is list-like.

- If `True` parses dates with the year first, e.g. `"10/11/12"` is parsed as `2010-11-12`.
- If both dayfirst and yearfirst are `True`, yearfirst is preceded (same as `dateutil`).

> Warning
> `yearfirst=True` is not strict, but will prefer to parse with year first.

`utc` : ***bool, default False*** Control timezone-related parsing, localization and conversion.

- If `True`, the function *always* returns a timezone-aware UTC-localized `Timestamp`, `Series` or `DatetimeIndex`. To do this, timezone-naive inputs are *localized* as UTC, while timezone-aware inputs are *converted* to UTC.
- If `False` (default), inputs will not be coerced to UTC. Timezone-naive inputs will remain naive, while timezone-aware ones will keep their time offsets. Limitations exist for mixed offsets (typically, daylight savings), see `Examples <to_datetime_tz_examples>` section for details.

See also: pandas general documentation about timezone conversion and localization <https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html #time-zone-handling>.

`format` : ***str, default None*** The strftime to parse time, e.g. `"%d/%m/%Y"`. See strftime documentation <https://docs.python.org/3/library/datetime.html #strftime-and-strptime-behavior> for more information on choices, though note that `"%f"` will parse all the way up to nanoseconds. You can also pass:

- "ISO8601", to parse any ISO8601 time string (not necessarily in exactly the same format);
- "mixed", to infer the format for each element individually. This is risky, and you should probably use it along with dayfirst.

Deprecated since version 3.1.0. Passing integer or float values together with `format` is deprecated; cast them to strings first. In a future version numeric values will be interpreted as epochs via `unit` instead, and `format` will only apply to string input.

> Note
> If a `DataFrame` is passed, then format has no effect.

> Note
> When using `format`, any datetime components not present in the format string default to `1900-01-01 00:00:00`, consistent with Python's `datetime.datetime.strptime` behavior. For example, `to_datetime(["1", "2"], format="%d")` returns dates in January 1900. The `origin` parameter does not affect string parsing via `format`; it only applies to numeric input interpreted through `unit`.

`exact` : ***bool, default True*** Control how format is used:

- If `True`, require an exact format match.
- If `False`, allow the format to match anywhere in the target string.

Cannot be used alongside `format='ISO8601'` or `format='mixed'`.

`unit` : ***str, default 'ns'*** The unit of the numeric arg (Y, M, W, D, h, m, s, ms, us, ns, ps, fs, as). Specifies the unit of the input values when arg is numeric (int or float), interpreted relative to `origin`. For example, with `unit='ms'` and `origin='unix'`, the input values are treated as millisecond offsets from the Unix epoch (1970-01-01).

This does not truncate or round datetime-like inputs to the given unit. To change the resolution of the result, use `Series.dt.as_unit`. To truncate datetime values, use `Series.dt.floor` or `Series.dt.normalize`.

Only applicable to numeric input; has no effect on datetime-like input or when `format` is specified.

`origin` : *scalar, default 'unix'* Define the reference date. The numeric values would be parsed as number of units (defined by `unit`) since this reference date.

- If `'unix'` (or POSIX) time; origin is set to 1970-01-01.
- If `'julian'`, unit must be `'D'`, and origin is set to beginning of Julian Calendar. Julian day number `0` is assigned to the day starting at noon on January 1, 4713 BC.
- If Timestamp convertible (Timestamp, dt.datetime, np.datetime64 or date string), origin is set to Timestamp identified by origin.
- If a float or integer, origin is the difference (in units determined by the `unit` argument) relative to 1970-01-01.

> Note
> This parameter only affects numeric input used with `unit`. It does not affect string parsing via `format`. See the `format` parameter for how defaults are handled during string parsing.

`cache` : *bool, default True* If `True`, use a cache of unique, converted dates to apply the datetime conversion. May produce significant speed-up when parsing duplicate date strings, especially ones with timezone offsets. The cache is only used when there are at least 50 values. The presence of out-of-bounds values will render the cache unusable and may slow down parsing.

Returns

datetime If parsing succeeded. Return type depends on input (types in parenthesis correspond to fallback in case of unsuccessful timezone or out-of-range timestamp parsing):

- scalar: `Timestamp` (or `datetime.datetime`)
- array-like: `DatetimeIndex` (or `Series` with `object` dtype containing `datetime.datetime`)
- Series: `Series` of `datetime64` dtype (or `Series` of `object` dtype containing `datetime.datetime`)
- DataFrame: `Series` of `datetime64` dtype (or `Series` of `object` dtype containing `datetime.datetime`)

Raises

ParserError When parsing a date from string fails.

ValueError When another datetime conversion error happens. For example when one of 'year', 'month', day' columns is missing in a `DataFrame`, or when a Timezone-aware `datetime.datetime` is found in an array-like of mixed time offsets, and `utc=False`, or when parsing datetimes with mixed time zones unless `utc=True`. If parsing datetimes with mixed time zones, please specify `utc=True`.

See also: `DataFrame.astype`, `to_timedelta`, `convert_dtypes`

Notes

Many input types are supported, and lead to different output types:

- **scalars** can be int, float, str, datetime object (from stdlib `datetime` module or `numpy`). They are converted to `Timestamp` when possible, otherwise they are converted to `datetime.datetime`. None/NaN/null scalars are converted to `NaT`.
- **array-like** can contain int, float, str, datetime objects. They are converted to `DatetimeIndex` when possible, otherwise they are converted to `Index` with `object` dtype, containing `datetime.datetime`. None/NaN/null entries are converted to `NaT` in both cases.

(Notes truncated; see the full reference page.)

Examples

Handling various input formats

Assembling a datetime from multiple columns of a `DataFrame`. The keys can be common abbreviations like ['year', 'month', 'day', 'minute', 'second', 'ms', 'us', 'ns']) or plurals of the same

```
>>> df = pd.DataFrame({"year": [2015, 2016], "month": [2, 3], "day": [4, 5]})
>>> pd.to_datetime(df)
0   2015-02-04
1   2016-03-05
dtype: datetime64[us]
```

Using a unix epoch time

```
>>> pd.to_datetime(1490195805, unit="s")
Timestamp('2017-03-22 15:16:45')
>>> pd.to_datetime(1490195805433502912, unit="ns")
Timestamp('2017-03-22 15:16:45.433502912')
...
```

pd.to_numeric

```
pd.to_numeric(arg, errors: 'DateTimeErrorChoices' = 'raise', downcast: "Literal['integer', 'signed',
↪  'unsigned', 'float'] | None" = None, dtype_backend: 'DtypeBackend | lib.NoDefault' = <no_default>)
```

Convert argument to a numeric type.

If the input is already of a numeric dtype, the dtype will be preserved. For non-numeric inputs, the default return dtype is float64 or int64 depending on the data supplied. Use the downcast parameter to obtain other dtypes.

Please note that precision loss may occur if really large numbers are passed in. Due to the internal limitations of ndarray, if numbers smaller than -9223372036854775808 (np.iinfo(np.int64).min) or larger than 18446744073709551615 (np.iinfo(np.uint64).max) are passed in, it is very likely they will be converted to float so that they can be stored in an ndarray. These warnings apply similarly to Series since it internally leverages ndarray.

Parameters

`arg` : ***scalar, list, tuple, 1-d array, Index, or Series*** Argument to be converted.

`errors` : ***{'raise', 'coerce'}, default 'raise'***

- If 'raise', then invalid parsing will raise an exception.
- If 'coerce', then invalid parsing will be set as NaN.

`downcast` : ***str, default None*** Can be 'integer', 'signed', 'unsigned', or 'float'. If not None, and if the data has been successfully cast to a numerical dtype (or if the data was numeric to begin with), downcast that resulting data to the smallest numerical dtype possible according to the following rules:

- 'integer' or 'signed': smallest signed int dtype (min.: np.int8)
- 'unsigned': smallest unsigned int dtype (min.: np.uint8)
- 'float': smallest float dtype (min.: np.float32)

As this behaviour is separate from the core conversion to numeric values, any errors raised during the downcasting will be surfaced regardless of the value of the 'errors' input.

In addition, downcasting will only occur if the size of the resulting data's dtype is strictly larger than the dtype it is to be cast to, so if none of the dtypes checked satisfy that specification, no downcasting will be performed on the data.

dtype_backend : *{'numpy_nullable', 'pyarrow'}* Back-end data type applied to the result (still experimental). If not specified, the default behavior is to not use nullable data types. If specified, the behavior is as follows:

- `"numpy_nullable"`: returns nullable-dtype-backed object
- `"pyarrow"`: returns with pyarrow-backed nullable object

New in version 2.0.

Returns

scalar, Series, Index, numpy.ndarray, or ExtensionArray Numeric scalars are returned unchanged. Other scalars are returned as a NumPy scalar: `np.int64` if possible (`np.uint64` for positive values above the `int64` maximum), otherwise `np.float64`, before any `downcast` is applied. With `dtype_backend="pyarrow"`, a Python scalar is returned instead. For 1-d: `Series` if Series, `Index` if Index, otherwise an array as specified by `dtype_backend`.

If `errors='coerce'`, un-parseable values become `NaN` (or `pd.NA` if `dtype_backend` is not the default). If `errors='raise'` (default), a `ValueError` is raised for invalid values.

Raises

ValueError If the input contains non-numeric values and errors='raise'.

TypeError If the input is not list-like, 1D, or scalar convertible to numeric, such as nested lists or unsupported input types (e.g., dict).

See also: `DataFrame.astype`, `to_datetime`, `to_timedelta`, `numpy.ndarray.astype`, `DataFrame.convert_dtypes`

Examples

Take separate series and convert to numeric, coercing when told to

```
>>> s = pd.Series(["1.0", "2", -3])
>>> pd.to_numeric(s)
0    1.0
1    2.0
2   -3.0
dtype: float64
>>> pd.to_numeric(s, downcast="float")
0    1.0
1    2.0
2   -3.0
dtype: float32
>>> pd.to_numeric(s, downcast="signed")
0    1
1    2
2   -3
dtype: int8
>>> s = pd.Series(["apple", "1.0", "2", -3])
>>> pd.to_numeric(s, errors="coerce")
0    NaN
1    1.0
2    2.0
3   -3.0
...
```

Transforming

pd.DataFrame.assign

```
pd.DataFrame.assign(self, **kwargs) -> 'DataFrame'
```

Assign new columns to a DataFrame.

Returns a new object with all original columns in addition to new ones. Existing columns that are re-assigned will be overwritten.

Parameters

`**kwargs` : *callable, Series, scalar, array-like, or dict* The column names are keywords. If the values are callable, they are computed on the DataFrame and assigned to the new columns. The callable must not change input DataFrame (though pandas doesn't check it). If the values are not callable (e.g. a Series, scalar, array, or dict), they are simply assigned. See the Notes section for details on alignment and broadcasting.

Returns

DataFrame A new DataFrame with the new columns in addition to all the existing columns.

See also: `DataFrame.loc`, `DataFrame.iloc`

Notes

Assigning multiple columns within the same `assign` is possible. Later items in '**kwargs' may refer to newly created or modified columns in 'df'; items are computed and assigned into 'df' in order. Non-callable values (Series, arrays, scalars) follow the same alignment and broadcasting rules as `DataFrame.__setitem__`. See that method's documentation for details.

Examples

```
>>> df = pd.DataFrame({"temp_c": [17.0, 25.0]}, index=["Portland", "Berkeley"])
>>> df
          temp_c
Portland    17.0
Berkeley    25.0
```

Where the value is a callable, evaluated on ‘df‘:

```
>>> df.assign(temp_f=lambda x: x.temp_c * 9 / 5 + 32)
          temp_c  temp_f
Portland    17.0    62.6
Berkeley    25.0    77.0
```

Alternatively, the same behavior can be achieved by directly referencing an existing Series or sequence:

```
>>> df.assign(temp_f=df["temp_c"] * 9 / 5 + 32)
          temp_c  temp_f
Portland    17.0    62.6
Berkeley    25.0    77.0
...
```

pd.DataFrame.apply

```
pd.DataFrame.apply(self, func: 'AggFuncType', axis: 'Axis' = 0, raw: 'bool' = False, result_type:
↪  "Literal['expand', 'reduce', 'broadcast'] | None" = None, args=(), by_row: "Literal[False, 'compat']"
↪  = 'compat', engine: "Callable | Literal['python', 'numba'] | None" = None, engine_kwargs: 'dict[str,
↪  bool] | None' = None, **kwargs)
```

Apply a function along an axis of the DataFrame.

Objects passed to the function are Series objects whose index is either the DataFrame's index (`axis=0`) or the DataFrame's columns (`axis=1`); the `name` attribute of each passed Series is the label of the column (`axis=0`) or row (`axis=1`) currently being processed. However, by default (`by_row="compat"`), if `func` is a list-like or dict-like of functions, each function is first applied to the individual values of the Series rather than the Series itself; if this fails, pandas retries by passing the entire Series. By default (`result_type=None`), the final return type is inferred from the return type of the applied function. Otherwise, it depends on the result_type argument. The return type of the applied function is inferred based on the first computed result obtained after applying the function to a Series object.

Parameters

`func` : *function* Function to apply to each column or row.

`axis` : *{0 or 'index', 1 or 'columns'}, default 0* Axis along which the function is applied:

- 0 or 'index': apply function to each column.
- 1 or 'columns': apply function to each row.

`raw` : *bool, default False* Determines if row or column is passed as a Series or ndarray object:

- `False` : passes each row or column as a Series to the function.
- `True` : the passed function will receive ndarray objects instead. If you are just applying a NumPy reduction function this will achieve much better performance.

Note
When `raw=True`, the result dtype is inferred from the **first** returned value.

`result_type` : *{'expand', 'reduce', 'broadcast', None}, default None* How to interpret list-like results from ‘func‘:

- 'expand' : list-like results will be turned into columns. Whether a result is list-like is inferred from the first computed result; if that result is not list-like, `'expand'` has no effect and a Series is returned.
- 'reduce' : returns a Series if possible rather than expanding list-like results. This is the opposite of 'expand'.
- 'broadcast' : results will be broadcast to the original shape of the DataFrame, the original index and columns will be retained.

The default behaviour (None) depends on the return value of the applied function: list-like results will be returned as a Series of those. However if the apply function returns a Series these are expanded to columns.

Note
`result_type` has no effect when `func` is a NumPy universal function (e.g. `np.sqrt`). In that case the ufunc is applied directly to the underlying values and the result has the same shape as the input, regardless of `axis` or `result_type`. To use `result_type` with a ufunc, wrap it in a Python function (e.g. `lambda x: np.sqrt(x)`).

`args` : *tuple* Positional arguments to pass to func in addition to the array/series.

`by_row` : *False or ”compat”, default ”compat”* Only has an effect when `func` is a listlike or dict-like of funcs and the func isn't a string. If ”compat”, will if possible first translate the func into pandas methods (e.g. `Series().apply(np.sum)` will be translated to `Series().sum()`). If

that doesn't work, will try call to apply again with `by_row=True` and if that fails, will call apply again with `by_row=False` (backward compatible). If False, the funcs will be passed the whole Series at once.

New in version 2.1.0.

`engine` : *decorator or {'python', 'numba'}, optional* Choose the execution engine to use. If not provided the function will be executed by the regular Python interpreter.

Other options include JIT compilers such as Numba and Bodo, which in some cases can speed up the execution. To use an executor you can provide the decorators `numba.jit`, `numba.njit` or `bodo.jit`. You can also provide the decorator with parameters, like `numba.jit(nogil=True)`.

Not all functions can be executed with all execution engines. In general, JIT compilers will require type stability in the function (no variable should change data type during the execution). And not all pandas and NumPy APIs are supported. Check the engine documentation and for limitations.

> Warning
> String parameters will stop being supported in a future pandas version.

New in version 2.2.0.

`engine_kwargs` : *dict* Pass keyword arguments to the engine. This is currently only used by the numba engine, see the documentation for the engine argument for more information.

`**kwargs` Additional keyword arguments to pass as keywords arguments to func.

Returns

Series or DataFrame Result of applying `func` along the given axis of the DataFrame.

See also: `DataFrame.map`, `DataFrame.aggregate`, `DataFrame.transform`

Notes

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See gotchas.udf-mutation for more details.

Examples

```
>>> df = pd.DataFrame([[4, 9]] * 3, columns=["A", "B"])
>>> df
   A  B
0  4  9
1  4  9
2  4  9
```

Using a numpy universal function (in this case the same as `np.sqrt(df)`):

```
>>> df.apply(np.sqrt)
     A    B
0  2.0  3.0
1  2.0  3.0
2  2.0  3.0
```

Using a reducing function on either axis ...

pd.Series.map

```
pd.Series.map(self, func: 'Callable | Mapping | Series | None' = None, na_action: "Literal['ignore'] |
↪  None" = None, engine: 'Callable | None' = None, **kwargs) -> 'Series'
```

Map values of Series according to an input mapping or function.

Used for substituting each value in a Series with another value, that may be derived from a function, a `dict` or a `Series`.

Parameters

`func` : ***function, collections.abc.Mapping subclass or Series*** Function or mapping correspondence.

`na_action` : ***{None, 'ignore'}, default None*** If 'ignore', propagate NaN values, without passing them to the mapping correspondence.

`engine` : ***decorator, optional*** Choose the execution engine to use to run the function. Only used for functions. If `map` is called with a mapping or `Series`, an exception will be raised. If `engine` is not provided the function will be executed by the regular Python interpreter.

Options include JIT compilers such as Numba, Bodo or Blosc2, which in some cases can speed up the execution. To use an executor you can provide the decorators `numba.jit`, `numba.njit`, `bodo.jit` or `blosc2.jit`. You can also provide the decorator with parameters, like `numba.jit(nogil=True)`.

Not all functions can be executed with all execution engines. In general, JIT compilers will require type stability in the function (no variable should change data type during the execution). And not all pandas and NumPy APIs are supported. Check the engine documentation for limitations.

New in version 3.0.0.

`**kwargs` Additional keyword arguments to pass as keywords arguments to `func`.

New in version 3.0.0.

Returns

Series Same index as caller.

See also: `Series.apply`, `Series.replace`, `DataFrame.apply`, `DataFrame.map`

Notes

When `func` is a dictionary, values in Series that are not in the dictionary (as keys) are converted to `NaN`. However, if the dictionary is a `dict` subclass that defines `__missing__` (i.e. provides a method for default values), then this default is used rather than `NaN`.

Examples

```
>>> s = pd.Series(["cat", "dog", np.nan, "rabbit"])
>>> s
0      cat
1      dog
2      NaN
3   rabbit
dtype: str
```

`map` accepts a `dict` or a `Series`. Values that are not found in the `dict` are converted to `NaN`, unless the dict has a default value (e.g. `defaultdict`):

```
>>> s.map({"cat": "kitten", "dog": "puppy"})
0   kitten
1    puppy
2      NaN
3      NaN
dtype: str
...
```

pd.DataFrame.agg

```
pd.DataFrame.agg(self, func=None, axis: 'Axis' = 0, *args, **kwargs) -> 'DataFrame | Series'
```

Aggregate using one or more operations over the specified axis.

This method allows combining multiple aggregation functions at once, such as `sum`, `mean`, and `min`, and can apply them either per-column or per-row. It accepts functions as strings, callables, lists, or dictionaries mapping column labels to the desired aggregation(s).

Parameters

`func` : ***function, str, list or dict*** Function to use for aggregating the data. If a function, must either work when passed a DataFrame or when passed to DataFrame.apply.

Accepted combinations are:

- function
- string function name
- list of functions and/or function names, e.g. `[np.sum, 'mean']`
- dict of axis labels -> functions, function names or list of such.

`axis` : ***{0 or 'index', 1 or 'columns'}, default 0*** If 0 or 'index': apply function to each column. If 1 or 'columns': apply function to each row.

`*args` Positional arguments to pass to func.

`**kwargs` Keyword arguments to pass to func.

Returns

scalar, Series or DataFrame The return can be:

- scalar : when Series.agg is called with single function
- Series : when DataFrame.agg is called with a single function
- DataFrame : when DataFrame.agg is called with several functions

See also: `DataFrame.apply`, `DataFrame.transform`, `DataFrame.groupby`, `DataFrame.resample`, `DataFrame.rolling`, `DataFrame.expanding`, `core.window.ewm.ExponentialMovingWindow`

Notes

The aggregation operations are always performed over an axis, either the index (default) or the column axis. This behavior is different from numpy aggregation functions (mean, median, prod, sum, std, var), where the default is to compute the aggregation of the flattened array, e.g., `numpy.mean(arr_2d)` as opposed to `numpy.mean(arr_2d, axis=0)`.

agg is an alias for aggregate. Use the alias.

Functions that mutate the passed object can produce unexpected behavior or errors and are not supported. See gotchas.udf-mutation for more details.

(Notes truncated; see the full reference page.)

Examples

```
>>> df = pd.DataFrame(
...     [[1, 2, 3], [4, 5, 6], [7, 8, 9], [np.nan, np.nan, np.nan]],
...     columns=["A", "B", "C"],
... )
```

Aggregate these functions over the rows.

```
>>> df.agg(["sum", "min"])
        A     B     C
sum  12.0  15.0  18.0
min   1.0   2.0   3.0
```

Different aggregations per column.

```
>>> df.agg({"A": ["sum", "min"], "B": ["min", "max"]})
        A    B
sum  12.0  NaN
min   1.0  2.0
max   NaN  8.0
...
```

pd.cut

```
pd.cut(x, bins, right: 'bool' = True, labels=None, retbins: 'bool' = False, precision: 'int' = 3,
↪  include_lowest: 'bool' = False, duplicates: 'str' = 'raise', ordered: 'bool' = True)
```

Bin values into discrete intervals.

Use cut when you need to segment and sort data values into bins. This function is also useful for going from a continuous variable to a categorical variable. For example, cut could convert ages to groups of age ranges. Supports binning into an equal number of bins, or a pre-specified array of bins.

Parameters

`x` : ***1d ndarray or Series*** The input array to be binned. Must be 1-dimensional.

`bins` : ***int, sequence of scalars, or IntervalIndex*** The criteria to bin by.

- int : Defines the number of equal-width bins in the range of x. The range of x is extended by .1% on each side to include the minimum and maximum values of x.
- sequence of scalars : Defines the bin edges allowing for non-uniform width. No extension of the range of x is done.
- IntervalIndex : Defines the exact bins to be used. Note that IntervalIndex for bins must be non-overlapping.

`right` : ***bool, default True*** Indicates whether bins includes the rightmost edge or not. If `right == True` (the default), then the bins `[1, 2, 3, 4]` indicate (1,2], (2,3], (3,4]. This argument is ignored when bins is an IntervalIndex.

`labels` : ***array or False, default None*** Specifies the labels for the returned bins. Must be the same length as the resulting bins. If False, returns only integer indicators of the bins. This affects the type of the output container (see below). This argument is ignored when bins is an IntervalIndex. If True, raises an error. When ordered=False, labels must be provided.

`retbins` : ***bool, default False*** Whether to return the bins or not. Useful when bins is provided as a scalar.

`precision` : ***int, default 3*** The precision at which to store and display the bins labels.

include_lowest : *bool, default False* Whether values equal to the first bin edge are placed in the first bin. Because the returned intervals must all be closed on the same side, the left edge of the first default label is lowered slightly, so integer bins get float labels. Values between the lowered edge and the first bin edge are still NA. No effect when `right=False` or when bins is an IntervalIndex.

duplicates : *{'raise', 'drop'}, default 'raise'* If bin edges are not unique, raise ValueError or drop non-uniques.

ordered : *bool, default True* Whether the labels are ordered or not. Applies to returned types Categorical and Series (with Categorical dtype). If True, the resulting categorical will be ordered. If False, the resulting categorical will be unordered (labels must be provided).

Returns

out : *Categorical, Series, or ndarray* An array-like object representing the respective bin for each value of x. The type depends on the value of labels.

- None (default) : returns a Series for Series x or a Categorical for all other inputs. The values stored within are Interval dtype.
- sequence of scalars : returns a Series for Series x or a Categorical for all other inputs. The values stored within are whatever the type in the sequence is.
- False : returns a 1d ndarray or Series of integers.

bins : *numpy.ndarray or IntervalIndex.* The computed or specified bins. Only returned when retbins=True. For scalar or sequence bins, this is an ndarray with the computed bins. If set duplicates=drop, bins will drop non-unique bin. For an IntervalIndex bins, this is equal to bins.

See also: `qcut`, `Categorical`, `Series`, `IntervalIndex`, `numpy.histogram_bin_edges`

Notes

Any NA values will be NA in the result. Values that fall outside the range of bins (or, when bins is a non-contiguous IntervalIndex, between two bins) are not assigned to any bin and will be NA in the resulting Series or Categorical object.

`numpy.histogram_bin_edges` can be used along with cut to calculate bins according to some predefined methods.

See the user guide for more examples.

Examples

Discretize into three equal-sized bins.

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3)
...
[(0.994, 3.0], (5.0, 7.0], (3.0, 5.0], (3.0, 5.0], (5.0, 7.0], ...
Categories (3, interval[float64, right]): [(0.994, 3.0] < (3.0, 5.0] ...
```

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3, retbins=True)
...
([(0.994, 3.0], (5.0, 7.0], (3.0, 5.0], (3.0, 5.0], (5.0, 7.0], ...
Categories (3, interval[float64, right]): [(0.994, 3.0] < (3.0, 5.0] ...
array([0.994, 3.   , 5.   , 7.   ]))
```

Discovers the same bins, but assign them specific labels. Notice that the returned Categorical's categories are labels and is ordered.

```
>>> pd.cut(np.array([1, 7, 5, 4, 6, 3]), 3, labels=["bad", "medium", "good"])
['bad', 'good', 'medium', 'medium', 'good', 'bad']
```

```
Categories (3, str): ['bad' < 'medium' < 'good']
...
```

pd.qcut

```
pd.qcut(x, q, labels=None, retbins: 'bool' = False, precision: 'int' = 3, duplicates: 'str' = 'raise')
```

Quantile-based discretization function.

Discretize variable into equal-sized buckets based on rank or based on sample quantiles. For example 1000 values for 10 quantiles would produce a Categorical object indicating quantile membership for each data point.

Parameters

x : *1d ndarray or Series* Input Numpy array or pandas Series object to be discretized.

q : *int or list-like of float* Number of quantiles. 10 for deciles, 4 for quartiles, etc. Alternately array of quantiles, e.g. [0, .25, .5, .75, 1.] for quartiles.

labels : *array or False, default None* Used as labels for the resulting bins. Must be of the same length as the resulting bins. If False, return only integer indicators of the bins. If True, raises an error.

retbins : *bool, optional* Whether to return the (bins, labels) or not. Can be useful if bins is given as a scalar.

precision : *int, optional* The precision at which to store and display the bins labels.

duplicates : *{default 'raise', 'drop'}, optional* If bin edges are not unique, raise ValueError or drop non-uniques.

Returns

out : *Categorical or Series or array of integers if labels is False* The return type (Categorical or Series) depends on the input: a Series of type category if input is a Series else Categorical. Bins are represented as categories when categorical data is returned.

bins : *ndarray of floats* Returned only if retbins is True.

See also: `cut`, `Series.quantile`

Notes

Out of bounds values will be NA in the resulting Categorical object. See the user guide for more examples.

Examples

```
>>> pd.qcut(range(5), 4)
...
[(-0.001, 1.0], (-0.001, 1.0], (1.0, 2.0], (2.0, 3.0], (3.0, 4.0]]
Categories (4, interval[float64, right]): [(-0.001, 1.0] < (1.0, 2.0] ...
```

```
>>> pd.qcut(range(5), 3, labels=["good", "medium", "bad"])
...
[good, good, medium, bad, bad]
Categories (3, str): [good < medium < bad]
```

```
>>> pd.qcut(range(5), 4, labels=False)
array([0, 0, 1, 2, 3])
```

pd.get_dummies

```
pd.get_dummies(data, prefix=None, prefix_sep: 'str | Iterable[str] | dict[str, str]' = '_', dummy_na:
↪  'bool' = False, columns=None, sparse: 'bool' = False, drop_first: 'bool' = False, dtype: 'NpDtype |
↪  None' = None) -> 'DataFrame'
```

Convert categorical variable into dummy/indicator variables.

Each variable is converted in as many 0/1 variables as there are different values. Columns in the output are each named after a value; if the input is a DataFrame, the name of the original variable is prepended to the value.

Parameters

`data` : ***array-like, Series, or DataFrame*** Data of which to get dummy indicators.

`prefix` : ***str, list of str, or dict of str, default None*** A string to be prepended to DataFrame column names. Pass a list with length equal to the number of columns when calling get_dummies on a DataFrame. Alternatively, prefix can be a dictionary mapping column names to prefixes.

`prefix_sep` : ***str, list of str, or dict of str, default*** **`'_'`** Should you choose to prepend DataFrame column names with a prefix, this is the separator/delimiter to use between the two. Alternatively, prefix_sep can be a list with length equal to the number of columns, or a dictionary mapping column names to separators.

`dummy_na` : ***bool, default False*** If True, a NaN indicator column will be added even if no NaN values are present. If False, NA values are encoded as all zero.

`columns` : ***list-like, default None*** Column names in the DataFrame to be encoded. If columns is None then all the columns with object, string, or category dtype will be converted.

`sparse` : ***bool, default False*** Whether the dummy-encoded columns should be backed by a `SparseArray` (True) or a regular NumPy array (False).

`drop_first` : ***bool, default False*** Whether to get k-1 dummies out of k categorical levels by removing the first level.

`dtype` : ***dtype, default bool*** Data type for new columns. Only a single dtype is allowed.

Returns

DataFrame Dummy-coded data. If data contains other columns than the dummy-coded one(s), these will be prepended, unaltered, to the result.

See also: `Series.str.get_dummies`, `~pandas.from_dummies`

Notes

Reference the user guide for more examples.

Examples

```
>>> s = pd.Series(list("abca"))
```

```
>>> pd.get_dummies(s)
       a      b      c
0   True  False  False
1  False   True  False
2  False  False   True
3   True  False  False
```

```
>>> s1 = ["a", "b", np.nan]
```

```
>>> pd.get_dummies(s1)
       a      b
0   True  False
1  False   True
2  False  False
...
```

Grouping and combining

pd.DataFrame.groupby

```
pd.DataFrame.groupby(self, by=None, level: 'IndexLabel | None' = None, *, as_index: 'bool' = True, sort:
↪  'bool' = True, group_keys: 'bool' = True, observed: 'bool' = True, dropna: 'bool' = True) ->
↪  'DataFrameGroupBy'
```

Group DataFrame using a mapper or by a Series of columns.

A groupby operation involves some combination of splitting the object, applying a function, and combining the results. This can be used to group large amounts of data and compute operations on these groups.

Parameters

by : *mapping, function, label, pd.Grouper or list of such* Used to determine the groups for the groupby. If `by` is a function, it's called on each value of the object's index. If a dict or Series is passed, the Series or dict VALUES will be used to determine the groups (the Series' values are first aligned; see `.align()` method). If a list or ndarray of length equal to the number of rows is passed (see the groupby user guide), the values are used as-is to determine the groups. A label or list of labels may be passed to group by the columns in `self`. Notice that a tuple is interpreted as a (single) key.

level : *int, level name, or sequence of such, default None* If the axis is a MultiIndex (hierarchical), group by a particular level or levels. Do not specify both `by` and `level`.

as_index : *bool, default True* Return object with group labels as the index. Only relevant for DataFrame input. as_index=False is effectively "SQL-style" grouped output. This argument has no effect on filtrations (see the filtrations in the user guide), such as `head()`, `tail()`, `nth()` and in transformations (see the transformations in the user guide).

sort : *bool, default True* Sort group keys. Get better performance by turning this off. Note this does not influence the order of observations within each group. Groupby preserves the order of rows within each group. If False, the groups will appear in the same order as they did in the original DataFrame. This argument has no effect on filtrations (see the filtrations in the user guide), such as `head()`, `tail()`, `nth()` and in transformations (see the transformations in the user guide).

Changed in version 2.0.0. Specifying `sort=False` with an ordered categorical grouper will no longer sort the values.

group_keys : *bool, default True* When calling apply and the `by` argument produces a like-indexed (i.e. a transform) result, add group keys to index to identify pieces. By default group keys are not included when the result's index (and column) labels match the inputs, and are included otherwise.

Changed in version 2.0.0. `group_keys` now defaults to `True`.

observed : *bool, default True* This only applies if any of the groupers are Categoricals. If True: only show observed values for categorical groupers. If False: show all values for categorical

groupers.

Changed in version 3.0.0. The default value is now `True`.

`dropna` : *bool, default True* If True, and if group keys contain NA values, NA values together with row/column will be dropped. If False, NA values will also be treated as the key in groups.

Returns

pandas.api.typing.DataFrameGroupBy Returns a groupby object that contains information about the groups.

See also: `resample`

Notes

See the user guide for more detailed usage and examples, including splitting an object into groups, iterating through groups, selecting a group, aggregation, and more.

The implementation of groupby is hash-based, meaning in particular that objects that compare as equal will be considered to be in the same group. An exception to this is that pandas has special handling of NA values: any NA values will be collapsed to a single group, regardless of how they compare. See the user guide linked above for more details.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "Animal": ["Falcon", "Falcon", "Parrot", "Parrot"],
...         "Max Speed": [380.0, 370.0, 24.0, 26.0],
...     }
... )
>>> df
   Animal  Max Speed
0  Falcon      380.0
1  Falcon      370.0
2  Parrot       24.0
3  Parrot       26.0
>>> df.groupby(["Animal"]).mean()
        Max Speed
Animal
Falcon      375.0
Parrot       25.0
...
```

pd.merge

```
pd.merge(left: 'DataFrame | Series', right: 'DataFrame | Series', how: 'MergeHow' = 'inner', on:
 ↪  'IndexLabel | AnyArrayLike | None' = None, left_on: 'IndexLabel | AnyArrayLike | None' = None,
 ↪  right_on: 'IndexLabel | AnyArrayLike | None' = None, left_index: 'bool' = False, right_index: 'bool'
 ↪  = False, sort: 'bool' = False, suffixes: 'Suffixes' = ('_x', '_y'), copy: 'bool | lib.NoDefault' =
 ↪  <no_default>, in ...)
```

Merge DataFrame or named Series objects with a database-style join.

A named Series object is treated as a DataFrame with a single named column.

The join is done on columns or indexes. If joining columns on columns, the DataFrame indexes *will be ignored*. Otherwise if joining indexes on indexes or indexes on a column or columns, the index will be passed on. When performing a cross merge, no column specifications to merge on are allowed.

Warning
If both key columns contain rows where the key is a null value, those rows will be matched against each other. This is different from usual SQL join behaviour and can lead to unexpected results.

Parameters

left : *DataFrame or named Series* First pandas object to merge.

right : *DataFrame or named Series* Second pandas object to merge.

how : *{'left', 'right', 'outer', 'inner', 'cross', 'left_anti', 'right_anti},* default 'inner' Type of merge to be performed.

- left: use only keys from left frame, similar to a SQL left outer join; preserve key order.
- right: use only keys from right frame, similar to a SQL right outer join; preserve key order.
- outer: use union of keys from both frames, similar to a SQL full outer join; sort keys lexicographically.
- inner: use intersection of keys from both frames, similar to a SQL inner join; preserve the order of the left keys.
- cross: creates the cartesian product from both frames, preserves the order of the left keys.
- left_anti: use only keys from left frame that are not in right frame, similar to SQL left anti join; preserve key order.
- right_anti: use only keys from right frame that are not in left frame, similar to SQL right anti join; preserve key order.

on : *Hashable or a sequence of the previous* Column or index level names to join on. These must be found in both DataFrames. If on is None and not merging on indexes then this defaults to the intersection of the columns in both DataFrames.

left_on : *Hashable or a sequence of the previous, or array-like* Column or index level names to join on in the left DataFrame. Can also be an array or list of arrays of the length of the left DataFrame. These arrays are treated as if they are columns.

right_on : *Hashable or a sequence of the previous, or array-like* Column or index level names to join on in the right DataFrame. Can also be an array or list of arrays of the length of the right DataFrame. These arrays are treated as if they are columns.

left_index : *bool, default False* Use the index from the left DataFrame as the join key(s). If it is a MultiIndex, the number of keys in the other DataFrame (either the index or a number of columns) must match the number of levels.

right_index : *bool, default False* Use the index from the right DataFrame as the join key. Same caveats as left_index.

sort : *bool, default False* Sort the join keys lexicographically in the result DataFrame. If False, the order of the join keys depends on the join type (how keyword).

suffixes : *list-like, default is ("_x", "_y")* A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in left and right respectively. Pass a value of None instead of a string to indicate that the column name from left or right should be left as-is, with no suffix. At least one of the values must not be None.

copy : *bool, default False* This keyword is now ignored; changing its value will have no impact on the method.

Deprecated since version 3.0.0. This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism

that defers copies until necessary (Copy-on-Write). See the user guide on Copy-on-Write for more details.

indicator : *bool or str, default False* If True, adds a column to the output DataFrame called "_merge" with information on the source of each row. The column can be given a different name by providing a string argument. The column will have a Categorical type with the value of "left_only" for observations whose merge key only appears in the left DataFrame, "right_only" for observations whose merge key only appears in the right DataFrame, and "both" if the observation's merge key is found in both DataFrames.

validate : *str, optional* If specified, checks if merge is of specified type.

- "one_to_one" or "1:1": check if merge keys are unique in both left and right datasets.
- "one_to_many" or "1:m": check if merge keys are unique in left dataset.
- "many_to_one" or "m:1": check if merge keys are unique in right dataset.
- "many_to_many" or "m:m": allowed, but does not result in checks.

Returns

DataFrame A DataFrame of the two merged objects.

See also: `merge_ordered`, `merge_asof`, `DataFrame.join`

Examples

```
>>> df1 = pd.DataFrame(
...     {"lkey": ["foo", "bar", "baz", "foo"], "value": [1, 2, 3, 5]}
... )
>>> df2 = pd.DataFrame(
...     {"rkey": ["foo", "bar", "baz", "foo"], "value": [5, 6, 7, 8]}
... )
>>> df1
    lkey value
0   foo      1
1   bar      2
2   baz      3
3   foo      5
>>> df2
    rkey value
0   foo      5
1   bar      6
2   baz      7
3   foo      8
...
```

pd.concat

```
pd.concat(objs: 'Iterable[Series | DataFrame] | Mapping[HashableT, Series | DataFrame]', *, axis: 'Axis'
↪  = 0, join: 'str' = 'outer', ignore_index: 'bool' = False, keys: 'Iterable[Hashable] | None' = None,
↪  levels=None, names: 'list[HashableT] | None' = None, verify_integrity: 'bool' = False, sort: 'bool |
↪  lib.NoDefault' = <no_default>, copy: 'bool | lib.NoDefault' = <no_default>, union_categories: 'bool'
↪  =  ...)
```

Concatenate pandas objects along a particular axis.

Allows optional set logic along the other axes.

Can also add a layer of hierarchical indexing on the concatenation axis, which may be useful if the labels are the same (or overlapping) on the passed axis number.

Parameters

objs : *an iterable or mapping of Series or DataFrame objects* If a mapping is passed, the keys will be used as the keys argument, unless it is passed, in which case the values will be selected (see below). Any None objects will be dropped silently unless they are all None in which case a ValueError will be raised.

axis : *{0/'index', 1/'columns'}, default 0* The axis to concatenate along.

join : *{'inner', 'outer'}, default 'outer'* How to handle indexes on other axis (or axes).

ignore_index : *bool, default False* If True, do not use the index values along the concatenation axis. The resulting axis will be labeled 0, ..., n - 1. This is useful if you are concatenating objects where the concatenation axis does not have meaningful indexing information. Note the index values on the other axes are still respected in the join.

keys : *sequence, default None* If multiple levels passed, should contain tuples. Construct hierarchical index using the passed keys as the outermost level.

levels : *list of sequences, default None* Specific levels (unique values) to use for constructing a MultiIndex. Otherwise they will be inferred from the keys.

names : *list, default None* Names for the levels in the resulting hierarchical index.

verify_integrity : *bool, default False* Check whether the new concatenated axis contains duplicates. This can be very expensive relative to the actual data concatenation.

sort : *bool, default False* Sort non-concatenation axis. One exception to this is when the non-concatenation axis is a DatetimeIndex and join='outer' and the axis is not already aligned. In that case, the non-concatenation axis is always sorted lexicographically.

copy : *bool, default False* This keyword is now ignored; changing its value will have no impact on the method.

Deprecated since version 3.0.0. This keyword is ignored and will be removed in pandas 4.0. Since pandas 3.0, this method always returns a new object using a lazy copy mechanism that defers copies until necessary (Copy-on-Write). See the user guide on Copy-on-Write for more details.

union_categories : *bool, default False* If True and all values for a given column have categorical dtype, the resulting column will preserve categorical dtype with the union of the categories, rather than casting to the underlying dtype. Categories whose dtypes differ are cast to a common dtype. The result is unordered unless every input is ordered with the same categories (after any casting to a common dtype) in the same order, in which case the result is ordered. Has no effect when concatenating mixed categorical and non-categorical data.

New in version 3.1.0.

Returns

object, type of objs When concatenating all `Series` along the index (axis=0), a `Series` is returned. When `objs` contains at least one `DataFrame`, a `DataFrame` is returned. When concatenating along the columns (axis=1), a `DataFrame` is returned.

See also: `DataFrame.join`, `DataFrame.merge`

Notes

The keys, levels, and names arguments are all optional.

A walkthrough of how this method fits in with other tools for combining pandas objects can be found here.

It is not recommended to build DataFrames by adding single rows in a for loop. Build a list of rows and make a DataFrame in a single concat.

Examples

Combine two `Series`.

```
>>> s1 = pd.Series(["a", "b"])
>>> s2 = pd.Series(["c", "d"])
>>> pd.concat([s1, s2])
0    a
1    b
0    c
1    d
dtype: str
```

Clear the existing index and reset it in the result by setting the `ignore_index` option to `True`.

```
>>> pd.concat([s1, s2], ignore_index=True)
0    a
1    b
2    c
3    d
dtype: str
...
```

pd.DataFrame.join

```
pd.DataFrame.join(self, other: 'DataFrame | Series | Iterable[DataFrame | Series]', on: 'IndexLabel |
↪  None' = None, how: 'MergeHow' = 'left', lsuffix: 'str | lib.NoDefault' = <no_default>, rsuffix: 'str
↪  | lib.NoDefault' = <no_default>, sort: 'bool' = False, validate: 'JoinValidate | None' = None, *,
↪  suffixes: 'Suffixes | lib.NoDefault' = <no_default>) -> 'DataFrame'
```

Join columns of another DataFrame.

Join columns with other DataFrame either on index or on a key column. Efficiently join multiple DataFrame objects by index at once by passing a list.

Parameters

`other` : ***DataFrame, Series, or a list containing any combination of them*** Index should be similar to one of the columns in the caller. If a Series is passed, its name attribute must be set, and that will be used as the column name in the resulting joined DataFrame.

`on` : ***str, list of str, or array-like, optional*** Column or index level name(s) in the caller to join on the index in other, otherwise joins index-on-index. If multiple values given, the other DataFrame must have a MultiIndex. Can pass an array as the join key if it is not already contained in the calling DataFrame. Like an Excel VLOOKUP operation.

`how` : ***{'left', 'right', 'outer', 'inner', 'cross', 'left_anti', 'right_anti'},*** default 'left' How to handle the operation of the two objects.

- left: use calling frame's index (or column if on is specified)
- right: use other's index.
- outer: form union of calling frame's index (or column if on is specified) with other's index, and sort it lexicographically.
- inner: form intersection of calling frame's index (or column if on is specified) with other's index, preserving the order of the calling's one.
- cross: creates the cartesian product from both frames, preserves the order of the left keys.
- left_anti: use set difference of calling frame's index and other's index.
- right_anti: use set difference of other's index and calling frame's index.

lsuffix : *str, default ''* Suffix to use from left frame's overlapping columns.

Deprecated since version 3.1.0. Use `suffixes` instead.

rsuffix : *str, default ''* Suffix to use from right frame's overlapping columns.

Deprecated since version 3.1.0. Use `suffixes` instead.

sort : *bool, default False* Order result DataFrame lexicographically by the join key. If False, the order of the join key depends on the join type (how keyword).

validate : *str, optional* If specified, checks if join is of specified type.

- "one_to_one" or "1:1": check if join keys are unique in both left and right datasets.
- "one_to_many" or "1:m": check if join keys are unique in left dataset.
- "many_to_one" or "m:1": check if join keys are unique in right dataset.
- "many_to_many" or "m:m": allowed, but does not result in checks.

suffixes : *tuple of (str, str), default ("", "")* A length-2 sequence where each element is optionally a string indicating the suffix to add to overlapping column names in the caller and `other` respectively. Pass a value of `None` instead of a string to indicate that the column name from the caller or `other` should be left as-is, with no suffix. At least one of the values must not be None.

Returns

DataFrame A dataframe containing columns from both the caller and other.

See also: `DataFrame.merge`

Notes

Parameters on and suffixes are not supported when passing a list of DataFrame objects.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "key": ["K0", "K1", "K2", "K3", "K4", "K5"],
...         "A": ["A0", "A1", "A2", "A3", "A4", "A5"],
...     }
... )
```

```
>>> df
  key   A
0  K0  A0
1  K1  A1
2  K2  A2
3  K3  A3
4  K4  A4
5  K5  A5
```

```
>>> other = pd.DataFrame({"key": ["K0", "K1", "K2"], "B": ["B0", "B1", "B2"]})
...
```

pd.pivot_table

```
pd.pivot_table(data: 'DataFrame', values=None, index=None, columns=None, aggfunc: 'AggFuncType' = 'mean',
↪  fill_value=None, margins: 'bool' = False, dropna: 'bool' = True, margins_name: 'Hashable' = 'All',
↪  observed: 'bool' = True, sort: 'bool' = True, **kwargs) -> 'DataFrame'
```

Create a spreadsheet-style pivot table as a DataFrame.

The levels in the pivot table will be stored in MultiIndex objects (hierarchical indexes) on the index and columns of the result DataFrame.

Parameters

`data` : *DataFrame* Input pandas DataFrame object.

`values` : *list-like or scalar, optional* Column or columns to aggregate.

`index` : *column, Grouper, array, or sequence of the previous* Keys to group by on the pivot table index. If a list is passed, it can contain any of the other types (except list). If an array is passed, it must be the same length as the data and will be used in the same manner as column values.

`columns` : *column, Grouper, array, or sequence of the previous* Keys to group by on the pivot table column. If a list is passed, it can contain any of the other types (except list). If an array is passed, it must be the same length as the data and will be used in the same manner as column values.

`aggfunc` : *function, list of functions, dict, default "mean"* If a list of functions is passed, the resulting pivot table will have hierarchical columns whose top level are the function names (inferred from the function objects themselves). If a dict is passed, the key is column to aggregate and the value is function or list of functions. If `margins=True`, aggfunc will be used to calculate the partial aggregates.

`fill_value` : *scalar, default None* Value to replace missing values with (in the resulting pivot table, after aggregation).

`margins` : *bool, default False* If `margins=True`, special `All` columns and rows will be added with partial group aggregates across the categories on the rows and columns.

`dropna` : *bool, default True* Do not include columns whose entries are all NaN. If True,

- rows with an NA value in any column will be omitted before computing margins,
- index/column keys containing NA values will be dropped (see `dropna` parameter in `DataFrame.groupby`).

`margins_name` : *str, default 'All'* Name of the row / column that will contain the totals when margins is True.

`observed` : *bool, default True* This only applies if any of the groupers are Categoricals. If True: only show observed values for categorical groupers. If False: show all values for categorical groupers.

Changed in version 3.0.0. The default value is now `True`.

`sort` : *bool, default True* Specifies if the result should be sorted.

`**kwargs` : *dict* Optional keyword arguments to pass to `aggfunc`.

New in version 3.0.0.

Returns

DataFrame An Excel style pivot table.

See also: `DataFrame.pivot`, `DataFrame.melt`, `wide_to_long`

Notes

Reference the user guide for more examples.

Changed in version 3.1.0. When `values` is empty, which is the case when all columns of `data` are either in `index` or `columns` arguments, aggregation will act on a Series of all NA values.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "A": ["foo", "foo", "foo", "foo", "foo", "bar", "bar", "bar", "bar"],
...         "B": ["one", "one", "one", "two", "two", "one", "one", "two", "two"],
...         "C": [
...             "small",
...             "large",
...             "large",
...             "small",
...             "small",
...             "large",
...             "small",
...             "small",
...             "large",
...         ],
...         "D": [1, 2, 2, 3, 3, 4, 5, 6, 7],
...         "E": [2, 4, 5, 5, 6, 6, 8, 9, 9],
...     }
... )
>>> df
     A    B      C  D  E
0  foo  one  small  1  2
1  foo  one  large  2  4
2  foo  one  large  2  5
...
```

pd.crosstab

```
pd.crosstab(index, columns, values=None, rownames=None, colnames=None, aggfunc=None, margins: 'bool' =
↪  False, margins_name: 'Hashable' = 'All', dropna: 'bool' = True, normalize: "bool | Literal[0, 1,
↪  'all', 'index', 'columns']" = False) -> 'DataFrame'
```

Compute a simple cross tabulation of two (or more) factors.

By default, computes a frequency table of the factors unless an array of values and an aggregation function are passed.

Parameters

index : *array-like, Series, or list of arrays/Series* Values to group by in the rows.

columns : *array-like, Series, or list of arrays/Series* Values to group by in the columns.

values : *array-like, optional* Array of values to aggregate according to the factors. Requires aggfunc be specified.

rownames : *sequence, default None* If passed, must match number of row arrays passed.

colnames : *sequence, default None* If passed, must match number of column arrays passed.

aggfunc : *function, optional* If specified, requires values be specified as well.

margins : *bool, default False* Add row/column margins (subtotals).

margins_name : *str, default 'All'* Name of the row/column that will contain the totals when margins is True.

dropna : *bool, default True* Do not include columns whose entries are all NaN.

normalize : *bool, {'all', 'index', 'columns'}, or {0,1}, default False* Normalize by dividing all values by the sum of values.

- If passed 'all' or True, will normalize over all values.
- If passed 'index' will normalize over each row.
- If passed 'columns' will normalize over each column.
- If margins is True, will also normalize margin values.

Returns

DataFrame Cross tabulation of the data.

See also: `DataFrame.pivot`, `pivot_table`

Notes

Any Series passed will have their name attributes used unless row or column names for the cross-tabulation are specified.

With the default `dropna=True`, categories without any observed instances in the input are dropped from the cross-tabulation. When `dropna=False`, all categories of any Categorical input are included, even if the actual data does not contain any instances of a particular category.

Series arguments are aligned on their index before tabulating; rows where any value is missing after alignment (e.g. index labels not present in all Series) are dropped. In the event that there aren't overlapping indexes an empty DataFrame will be returned.

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array(
...     [
...         "foo",
...         "foo",
...         "foo",
...         "foo",
...         "bar",
...         "bar",
...         "bar",
...         "bar",
...         "foo",
...         "foo",
...         "foo",
...     ],
...     dtype=object,
... )
>>> b = np.array(
...     [
...         "one",
...         "one",
...         "one",
...         "two",
...         "one",
...         "one",
...
```

pd.melt

```
pd.melt(frame: 'DataFrame', id_vars=None, value_vars=None, var_name=None, value_name: 'Hashable' =
↪  'value', col_level=None, ignore_index: 'bool' = True) -> 'DataFrame'
```

Unpivot a DataFrame from wide to long format, optionally leaving identifiers set.

This function is useful to reshape a DataFrame into a format where one or more columns are identifier variables (id_vars), while all other columns are considered measured variables (value_vars), and are "unpivoted" to the row axis, leaving just two non-identifier columns, 'variable' and 'value'.

Parameters

`frame` : ***DataFrame*** The DataFrame to unpivot.

`id_vars` : ***scalar, tuple, list, or ndarray, optional*** Column(s) to use as identifier variables.

`value_vars` : ***scalar, tuple, list, or ndarray, optional*** Column(s) to unpivot. If not specified, uses all columns that are not set as id_vars.

`var_name` : ***scalar, tuple, list, or ndarray, optional*** Name to use for the 'variable' column. If None it uses `frame.columns.name` or 'variable'. Must be a scalar if columns are a MultiIndex.

`value_name` : ***scalar, default 'value'*** Name to use for the 'value' column, can't be an existing column label.

`col_level` : ***scalar, optional*** If columns are a MultiIndex then use this level to melt.

`ignore_index` : ***bool, default True*** If True, original index is ignored. If False, the original index is retained. Index labels will be repeated as necessary.

Returns

DataFrame Unpivoted DataFrame.

See also: `DataFrame.melt`, `pivot_table`, `DataFrame.pivot`, `DataFrame.explode`

Notes

Reference the user guide for more examples.

Examples

```
>>> df = pd.DataFrame(
...     {
...         "A": {0: "a", 1: "b", 2: "c"},
...         "B": {0: 1, 1: 3, 2: 5},
...         "C": {0: 2, 1: 4, 2: 6},
...     }
... )
>>> df
   A  B  C
0  a  1  2
1  b  3  4
2  c  5  6
```

```
>>> pd.melt(df, id_vars=["A"], value_vars=["B"])
   A variable  value
0  a        B      1
1  b        B      3
2  c        B      5
...
```

Time series and windows

pd.date_range

```
pd.date_range(start=None, end=None, periods=None, freq=None, tz=None, normalize: 'bool' = False, name:
↪  'Hashable | None' = None, inclusive: 'IntervalClosedType' = 'both', *, unit: 'TimeUnit | None' =
↪  None, ambiguous: 'TimeAmbiguous' = 'raise', nonexistent: 'TimeNonexistent' = 'raise', **kwargs) ->
↪  'DatetimeIndex'
```

Return a fixed frequency DatetimeIndex.

Returns the range of equally spaced time points (where the difference between any two adjacent points is specified by the given frequency) such that they fall in the range [start, end] , where the first one and the last one are, resp., the first and last time points in that range that fall on the boundary of `freq` (if given as a frequency string) or that are valid for `freq` (if given as a `pandas.tseries.offsets.DateOffset`). If `freq` is positive, the points satisfy start <[=] x <[=] end, and if `freq` is negative, the points satisfy end <[=] x <[=] start. (If exactly one of `start`, `end`, or `freq` is *not* specified, this missing parameter can be computed given `periods`, the number of timesteps in the range. See the note below.)

Parameters

`start` : ***str or datetime-like, optional*** Left bound for generating dates.

`end` : ***str or datetime-like, optional*** Right bound for generating dates.

`periods` : ***int, optional*** Number of periods to generate.

`freq` : ***str, Timedelta, datetime.timedelta, or DateOffset, default 'D'*** Frequency strings can have multiples, e.g. '5h'. See here for a list of frequency aliases.

`tz` : ***str or tzinfo, optional*** Time zone name for returning localized DatetimeIndex, for example 'Asia/Hong_Kong'. By default, the resulting DatetimeIndex is timezone-naive unless timezone-aware datetime-likes are passed.

`normalize` : ***bool, default False*** Normalize start/end dates to midnight before generating date range.

`name` : ***Hashable, default None*** Name of the resulting DatetimeIndex.

`inclusive` : ***{"both", "neither", "left", "right"}, default "both"*** Include boundaries; Whether to set each bound as closed or open.

`unit` : ***{'s', 'ms', 'us', 'ns', None}, default None*** Specify the desired resolution of the result. If not specified, this is inferred from the 'start', 'end', and 'freq' using the same inference as `Timestamp` taking the highest resolution of the three that are provided.

New in version 2.0.0.

`ambiguous` : ***'infer', bool-ndarray, 'NaT', default 'raise'*** When clocks moved backward due to DST, ambiguous times may arise. For example in Central European Time (UTC+01), when going from 03:00 DST to 02:00 non-DST, 02:30:00 local time occurs both at 00:30:00 UTC and at 01:30:00 UTC. In such a situation, the ambiguous parameter dictates how ambiguous times should be handled.

- 'infer' will attempt to infer fall dst-transition hours based on order. Requires that the timestamps are monotonically increasing.
- bool-ndarray where True signifies a DST time, False signifies a non-DST time (note that this flag is only applicable for ambiguous times)
- 'NaT' will return NaT where there are ambiguous times
- 'raise' will raise a ValueError if there are ambiguous times.

nonexistent : 'shift_forward', 'shift_backward', 'NaT', timedelta, default 'raise' A nonexistent time does not exist in a particular timezone where clocks moved forward due to DST.

- 'shift_forward' will shift the nonexistent time forward to the closest existing time
- 'shift_backward' will shift the nonexistent time backward to the closest existing time
- 'NaT' will return NaT where there are nonexistent times
- timedelta objects will shift nonexistent times by the timedelta
- 'raise' will raise a ValueError if there are nonexistent times.

****kwargs** For compatibility. Has no effect on the result.

Returns

DatetimeIndex A DatetimeIndex object of the generated dates.

See also: `DatetimeIndex`, `timedelta_range`, `period_range`, `interval_range`

Notes

Of the four parameters `start`, `end`, `periods`, and `freq`, a maximum of three can be specified at once. Of the three parameters `start`, `end`, and `periods`, at least two must be specified. If `freq` is omitted, the resulting `DatetimeIndex` will have `periods` linearly spaced elements between `start` and `end` (closed on both sides).

To learn more about the frequency strings, please see this link.

Examples

Specifying the values

The next four examples generate the same DatetimeIndex, but vary the combination of start, end and periods.

Specify start and end, with the default daily frequency.

```
>>> pd.date_range(start="1/1/2018", end="1/08/2018")
DatetimeIndex(['2018-01-01', '2018-01-02', '2018-01-03', '2018-01-04',
               '2018-01-05', '2018-01-06', '2018-01-07', '2018-01-08'],
              dtype='datetime64[us]', freq='D')
```

Specify timezone-aware start and end, with the default daily frequency.

```
>>> pd.date_range(
...     start=pd.to_datetime("1/1/2018").tz_localize("Europe/Berlin"),
...     end=pd.to_datetime("1/08/2018").tz_localize("Europe/Berlin"),
... )
DatetimeIndex(['2018-01-01 00:00:00+01:00', '2018-01-02 00:00:00+01:00',
               '2018-01-03 00:00:00+01:00', '2018-01-04 00:00:00+01:00',
               '2018-01-05 00:00:00+01:00', '2018-01-06 00:00:00+01:00',
               '2018-01-07 00:00:00+01:00', '2018-01-08 00:00:00+01:00'],
              dtype='datetime64[us, Europe/Berlin]', freq='D')
...
```

pd.DataFrame.resample

```
pd.DataFrame.resample(self, rule, closed: "Literal['right', 'left'] | None" = None, label:
↪  "Literal['right', 'left'] | None" = None, convention: "Literal['start', 'end', 's', 'e']" = 'start',
↪  on: 'Level | None' = None, level: 'Level | None' = None, origin: 'str | TimestampConvertibleTypes' =
↪  'start_day', offset: 'TimedeltaConvertibleTypes | None' = None, group_keys: 'bool' = False) ->
↪  'Resampler'
```

Resample time-series data.

Convenience method for frequency conversion and resampling of time series. The object must have a datetime-like index (DatetimeIndex, PeriodIndex, or TimedeltaIndex), or the caller must pass the label of a datetime-like series/index to the `on`/`level` keyword parameter.

Parameters

`rule` : *DateOffset, Timedelta or str* The offset string or object representing target conversion.

`closed` : *{'right', 'left'}, default None* Which side of bin interval is closed. The default is 'left' for all frequency offsets except for 'ME', 'YE', 'QE', 'BME', 'BA', 'BQE', and 'W' which all have a default of 'right'.

`label` : *{'right', 'left'}, default None* Which bin edge label to label bucket with. The default is 'left' for all frequency offsets except for 'ME', 'YE', 'QE', 'BME', 'BA', 'BQE', and 'W' which all have a default of 'right'.

`convention` : *{'start', 'end', 's', 'e'}, default 'start'* For PeriodIndex only, controls whether to use the start or end of rule.

`on` : *str, optional* For a DataFrame, column to use instead of index for resampling. Column must be datetime-like.

`level` : *str or int, optional* For a MultiIndex, level (name or number) to use for resampling. level must be datetime-like.

`origin` : *Timestamp or str, default 'start_day'* The timestamp on which to adjust the grouping. The timezone of origin must match the timezone of the index. If string, must be Timestamp convertible or one of the following:

- 'epoch': origin is 1970-01-01
- 'start': origin is the first value of the timeseries
- 'start_day': origin is the first day at midnight of the timeseries
- 'end': origin is the last value of the timeseries
- 'end_day': origin is the ceiling midnight of the last day

> Note
> Only takes effect for Tick-frequencies (i.e. fixed frequencies like hours and minutes) and, on a timezone-naive index, for day frequencies; not for months or quarters.

`offset` : *Timedelta or str, default is None* An offset timedelta added to the origin.

`group_keys` : *bool, default False* Whether to include the group keys in the result index when using `.apply()` on the resampled object.

Changed in version 2.0.0. `group_keys` now defaults to `False`.

Returns

pandas.api.typing.Resampler `Resampler` object.

See also: `Series.resample`, `DataFrame.resample`, `groupby`, `asfreq`

Notes

See the user guide for more.

To learn more about the offset strings, please see this link.

Examples

Start by creating a series with 9 one minute timestamps.

```
>>> index = pd.date_range("1/1/2000", periods=9, freq="min")
>>> series = pd.Series(range(9), index=index)
>>> series
```

```
2000-01-01 00:00:00    0
2000-01-01 00:01:00    1
2000-01-01 00:02:00    2
2000-01-01 00:03:00    3
2000-01-01 00:04:00    4
2000-01-01 00:05:00    5
2000-01-01 00:06:00    6
2000-01-01 00:07:00    7
2000-01-01 00:08:00    8
Freq: min, dtype: int64
```

Downsample the series into 3 minute bins and sum the values of the timestamps falling into a bin. ...

pd.DataFrame.rolling

```
pd.DataFrame.rolling(self, window: 'int | dt.timedelta | str | BaseOffset | BaseIndexer', min_periods:
↪  'int | None' = None, center: 'bool' = False, win_type: 'str | None' = None, on: 'str | None' = None,
↪  closed: 'IntervalClosedType | None' = None, step: 'int | None' = None, method: 'str' = 'single') ->
↪  'Window | Rolling'
```

Provide rolling window calculations.

This method returns a rolling window object, enabling aggregation, transformation, and other operations over a sliding window of a specified size.

Parameters

`window` : *int, timedelta, str, offset, or BaseIndexer subclass* Interval of the moving window.

If an integer, the delta between the start and end of each window. The number of points in the window depends on the `closed` argument.

If a timedelta, str, or offset, the time period of each window. Each window will be a variable sized based on the observations included in the time-period. This is only valid for datetime-like indexes. The offset must correspond to a fixed frequency (for example, `'2D'` or `'1h'`); non-fixed frequencies such as `'B'` (business day) or `'ME'` (month end) are not supported and raise `ValueError`. To learn more about the offsets & frequency strings, please see this link.

If a BaseIndexer subclass, the window boundaries based on the defined `get_window_bounds` method. Additional rolling keyword arguments, namely `min_periods`, `center`, `closed` and `step` will be passed to `get_window_bounds`.

`min_periods` : *int, default None* Minimum number of observations in window required to have a value; otherwise, result is `np.nan`.

For a window that is specified by an offset, `min_periods` will default to 1.

For a window that is specified by an integer, `min_periods` will default to the size of the window.

`center` : *bool, default False* If False, set the window labels as the right edge of the window index.

If True, set the window labels as the center of the window index.

`win_type` : *str, default None* If `None`, all points are evenly weighted.

If a string, it must be a valid scipy.signal window function.

Certain Scipy window types require additional parameters to be passed in the aggregation function. The additional parameters must match the keywords specified in the Scipy window type method signature.

on : *str, optional* For a DataFrame, a column label or Index level on which to calculate the rolling window, rather than the DataFrame's index.

For integer `window` values, the window bounds are based on the number of observations and are not calculated using the values of the `on` column. The `on` column is excluded from the aggregation, but is included in the result when its values differ from the object's index.

When `on` is specified, the values of that column also become the index of the `Series` passed to `Rolling.apply` when `raw=False`, in place of the original `DataFrame` index.

closed : *str, default None* Determines the inclusivity of points in the window

If `'right'`, uses the window (first, last] meaning the last point is included in the calculations.

If `'left'`, uses the window [first, last) meaning the first point is included in the calculations.

If `'both'`, uses the window [first, last] meaning all points in the window are included in the calculations.

If `'neither'`, uses the window (first, last) meaning the first and last points in the window are excluded from calculations.

() and [] are referencing open and closed set notation respetively.

Default `None` (`'right'`).

step : *int, default None* Evaluate the window at every `step` result, equivalent to slicing as `[::step]`. `window` must be an integer. Using a step argument other than None or 1 will produce a result with a different shape than the input.

method : *str {'single', 'table'}, default 'single'* Execute the rolling operation per single column or row (`'single'`) or over the entire object (`'table'`).

This argument is only implemented when specifying `engine='numba'` in the method call.

Returns

pandas.api.typing.Window or pandas.api.typing.Rolling An instance of Window is returned if `win_type` is passed. Otherwise, an instance of Rolling is returned.

See also: `expanding`, `ewm`

Notes

See Windowing Operations for further usage details and examples.

Examples

```
>>> df = pd.DataFrame({"B": [0, 1, 2, np.nan, 4]})
>>> df
     B
0  0.0
1  1.0
2  2.0
3  NaN
4  4.0
```

window

Rolling sum with a window length of 2 observations.

```
>>> df.rolling(2).sum()
     B
0  NaN
1  1.0
2  3.0
3  NaN
```

```
4  NaN
...
```

pd.DataFrame.shift

```
pd.DataFrame.shift(self, periods: 'int | Sequence[int]' = 1, freq: 'Frequency | None' = None, axis:
↪  'Axis' = 0, fill_value: 'Hashable' = <no_default>, suffix: 'str | None' = None) -> 'DataFrame'
```

Shift index by desired number of periods with an optional time freq.

When freq is not passed, shift the index without realigning the data. If freq is passed (in this case, the index must be date or datetime, or it will raise a NotImplementedError), the index will be increased using the periods and the freq. freq can be inferred when specified as ”infer” as long as either freq or inferred_freq attribute is set in the index.

Parameters

periods : *int or Sequence* Number of periods to shift. Can be positive or negative. If an iterable of ints, the data will be shifted once by each int. This is equivalent to shifting by one value at a time and concatenating all resulting frames. The resulting columns will have the shift suffixed to their column names. For multiple periods, axis must not be 1.

freq : *DateOffset, tseries.offsets, timedelta, or str, optional* Offset to use from the tseries module or time rule (e.g. 'EOM'). If freq is specified then the index values are shifted but the data is not realigned. That is, use freq if you would like to extend the index when shifting and preserve the original data. If freq is specified as ”infer” then it will be inferred from the freq or inferred_freq attributes of the index. If neither of those attributes exist, a ValueError is thrown.

axis : *{0 or 'index', 1 or 'columns', None}, default None* Shift direction. For Series this parameter is unused and defaults to 0.

fill_value : *object, optional* The scalar value to use for newly introduced missing values. the default depends on the dtype of self. For Boolean and numeric NumPy data types, `np.nan` is used. For datetime, timedelta, or period data, etc. `NaT` is used. For extension dtypes, `self.dtype.na_value` is used.

suffix : *str, optional* If str and periods is an iterable, this is added after the column name and before the shift value for each shifted column name.

Returns

DataFrame Copy of input object, shifted.

See also: `Index.shift`, `DatetimeIndex.shift`, `PeriodIndex.shift`

Examples

```
>>> df = pd.DataFrame(
...     [[10, 13, 17], [20, 23, 27], [15, 18, 22], [30, 33, 37], [45, 48, 52]],
...     columns=["Col1", "Col2", "Col3"],
...     index=pd.date_range("2020-01-01", "2020-01-05"),
... )
>>> df
            Col1  Col2  Col3
2020-01-01    10    13    17
2020-01-02    20    23    27
2020-01-03    15    18    22
2020-01-04    30    33    37
2020-01-05    45    48    52
```

```
>>> df.shift(periods=3)
            Col1  Col2  Col3
2020-01-01   NaN   NaN   NaN
2020-01-02   NaN   NaN   NaN
2020-01-03   NaN   NaN   NaN
2020-01-04  10.0  13.0  17.0
2020-01-05  20.0  23.0  27.0
...
```

pd.DataFrame.pct_change

```
pd.DataFrame.pct_change(self, periods: 'int' = 1, fill_method: 'None' = None, freq=None, **kwargs) ->
↪  'Self'
```

Fractional change between the current and a prior element.

Computes the fractional change from the immediately previous row by default. This is useful in comparing the fraction of change in a time series of elements.

> Note
> Despite the name of this method, it calculates fractional change (also known as per unit change or relative change) and not percentage change. If you need the percentage change, multiply these values by 100.

Parameters

`periods` : ***int, default 1*** Periods to shift for forming percent change.

`fill_method` : ***None*** Must be None. This argument will be removed in a future version of pandas.

`freq` : ***DateOffset, timedelta, or str, optional*** Increment to use from time series API (e.g. 'ME' or BDay()).

`**kwargs` Additional keyword arguments are passed into DataFrame.shift or Series.shift.

Returns

Series or DataFrame The same type as the calling object.

See also: `Series.diff`, `DataFrame.diff`, `Series.shift`, `DataFrame.shift`

Examples

Series

```
>>> s = pd.Series([90, 91, 85])
>>> s
0    90
1    91
2    85
dtype: int64
```

```
>>> s.pct_change()
0         NaN
1    0.011111
2   -0.065934
dtype: float64
```

```
>>> s.pct_change(periods=2)
0         NaN
1         NaN
```

```
2   -0.055556
dtype: float64
...
```